

UretimV4 (CepPatronERP) - Sayfa/Islem Analizi

Feza Muhendislik · Uretim Takip Ekosistemi · Otomatik analiz dokumani

WinForms masaustu ERP. Ofis/yonetim: uretim emri, recete, istasyon, operasyon, mal kabul, kontrol, raporlama. DB'ye dogrudan baglanir (UretimV3_FEZA + MikroDB_V16_FEZA24).

Bu dokuman otomatik analizle uretilmistir. Her sayfa/islem icin: ne yapar, oncesinde ne olmalı (onkosul), sonrasında ne olur, butonlar & kisayollar, cagirdigi manager/servis/SQL/endpoint, istasyon sirasiyla iliskisi.

Icindekiler

- Istasyon Sirasi Nasil Olusur?
- Modul: UretimModule
- Modul: UretimIstasyonModule
- Modul: UretimOperasyonModule
- Modul: OperasyonModul
- Modul: IstasyonModul
- Modul: IstasyonHareketlerModul
- Modul: ReceteModul
- Modul: ReceteIstasyonGrupModul
- Modul: MalKabul
- Modul: UretimKontroller
- Modul: UretimTalepler
- Modul: MikroModul
- Modul: SiparisModule
- Modul: KullaniciModul
- Modul: PersonelModul
- Modul: AyarVeGenelModul
- Modul: MailModul
- Modul: SmsModule
- Modul: Aciklamalar
- Modul: Raporlar
- Modul: HizliUretimModule
- Modul: Ana Kabuk, Login, Yetki ve DB Yonetimi

Istasyon Sirasi Nasıl Olur?

Bu bolum, UretimV4 (WinForms ERP) + ApiFeza + WebUretim TabletV2 ekosisteminde "bir uretim emrindeki operasyon/istasyon sirasinin nasıl olustugunu, ilk adimin nasıl acildigini, bir adim bitince sonrakinin nasıl tetiklendigini ve bu akisin UI'ya nasıl yansidigini" UC PROJE acisindan, koddan kanitli (dosya:satir) olarak belgeler. Ortak veritabanı `UretimV3_FEZA` ; miktar/akis motoru OPERASYON-Sira bazli SQL prosedurleridir (`Uretim_MiktarGuncelle` , `Uretim_PlanlananGuncelle` , `Uretim_SonrakiIstasyonaGonder`).

Kavram haritasi (tablolar / entity'ler)

Sira ve akis su zincir uzerinde tasinir:

- **Recete (sablon) tarafı:**
- `ReceteOperasyon` — operasyon sablonu. `Sira` (int) operasyonun receteyici sirasini tutar. Kanit: `My\Entities\Receteler\ReceteOperasyon.cs:32` (`public int Sira { get; set; }`), `:45` (`RcAId` = bagli recete). Ctor'da varsayilan `Sira=0` (`:16`).
- `ReceteIstasyon` — bir operasyona bagli istasyon sablonu. `RcOId` ile operasyona, `RcAId` ile receteye baglanir; istasyonun kendi `Sira` alanı YOKTUR. Kanit: `My\Entities\ReceteIstasyonlar\ReceteIstasyon.cs:33-34` (`RcAId` , `RcOId`), `:26-29` (`OperasyonKodu/IstasyonKodu`).
- `ReceteIstasyonGrupOperasyon` — istasyon gruplama eslestirme tablosu: `GrupKodu + OperasyonKodu -> IstasyonKodu/IstasyonAdi` . Kanit: `My\Entities\ReceteIstasyonGruplar\ReceteIstasyonGrupOperasyon.cs:11-15` (`GrupKodu` , `OperasyonKodu` , `IstasyonKodu` , `IstasyonAdi`). Bu tabloda da `Sira` YOKTUR; eslestirme operasyon koduna gore yapilir.
- **Uretim (canli) tarafı:**
- `UretimOperasyon` — uretim emrine kopyalanan operasyon. `Sira` (int) buradaki gercek akis sirasidir. Kanit: `My\Entities\UretimOperasyonlar\UretimOperasyon.cs:30` (`public int Sira { get; set; }`), `:35-39` (`UrId` , `RcAId` , `RcOId` , `SipId` , `SipHId`).
- `UretimOperasyonHareket` — operasyonun "baslatilmis" hareketi (motorun isledigi satir). `Sira` alanini tasir (INSERT'lerde `Sira` set edilir; bkz. `_ProcedureListCreates.cs:145-146`).
- `UretimOperasyonHareketDetay` — bir operasyon hareketinin acilis/giris detayi.
- `UretimIstasyon` — operasyon hareketine bagli somut istasyon satiri. `Sira` alanı YOKTUR; `RcOId` / `RcIstId` ile sablona, `UrOHId` / `UrOHDId` ile harekete baglanir. Kanit: `My\Entities\UretimIstasyonlar\UretimIstasyon.cs:36-43` (`UrId` , `UrOId` , `UrOHId` , `UrOHDId` , `RcAId` , `RcOId` , `RcIstId` , `SipId`). Istasyonda `Sira` olmamasi, "tek operasyon altinda cok istasyon" durumunda akis bozulmasinin temel sebebidir (asagida).

Ozet: Sira tek bir yerde, OPERASYON seviyesinde yasar (`ReceteOperasyon.Sira -> UretimOperasyon.Sira -> UretimOperasyonHareket.Sira`). Istasyonlari kendi sirasi yoktur; istasyonlar daima icinde bulunduklari operasyonun sirasina tabidir.

1) Uretim emrinde operasyon/istasyon sirasi nasıl belirlenir?

Operasyon sirasi (ReceteOperasyon.Sira -> UretimOperasyon.Sira): Uretim emri siparisten oluşturulurken, recetenin operasyonlari `UretimOperasyon` satirlarina kopyalanir ve `Sira`, sablondaki `ReceteOperasyon.Sira` degerinden birebir tasinir.

Kanit: `MyUI\UretimModule\FrmUretimEmriED.cs:199-232` (`OperasyonlarOlustur`): - `:207` `foreach (var oprs in oprmod.Operasyonlar)` — recete operasyonlari uzerinde doner. - `:218-219` `RcOId = oprs.Id , RcAId = oprs.RcAId` — sablon baglantilari. - `:226` `Sira = oprs.Sira` — **operasyon sirasi sablondan kopyalanir.**

Operasyon listesi (sablon) `UretimEmriManager.GetOperasyon` ile yuklenir:

`My\Business\Manager\UretimEmriManager.cs:543` (`_db.ReceteOperasyon.SelectList(c => c.RcAId == rcAId)`) ve istasyonlar `:546` (`_db.ReceteIstasyon.SelectList(c => c.RcAId == rcAId)`).

Istasyon sirasi (operasyona baglilik): Istasyonlariin bagimsiz sirasi yoktur; bir operasyon hareketinin istasyonlari, o operasyonun `RcOId` 'sine eslesen `ReceteIstasyon` kayitlarindan uretilir.

Kanit: `MyUI\UretimIstasyonModule\FrmUretimIstasyonED.cs:284-318` (`IstasyonlarOlustur`): - `:286` `if (opr.Id == opm.RcOId)` — yalnızca gelen operasyon hareketinin operasyonu icin. - `:290` `if (opr.Id == ist.RcOId)` — istasyon o operasyona bagliysa. - `:292-316` bu sablon istasyondan bir `UretimIstasyon` satiri kurulur (`RcOId = ist.RcOId , RcIstId = ist.Id`).

Istasyon gruplama (alternatif istasyon belirleme: GrupKodu + OperasyonKodu -> IstasyonKodu):

Recetede `IstasyonGruplamaKullan = 1` ise, istasyonlar `ReceteIstasyon` yerine `ReceteIstasyonGrupOperasyon` eslestirmesinden gelir. Uretim emrine secilen `IstasyonGrupKodu` (UI'da `TxtIstasyonGrubu`) ile operasyon kodu eslestirilerek istasyon belirlenir. Bu eslestirme SQL motorunda yapilir.

Kanit: `My.DataBaseSettings\DatabaseCreate\UretimCreateModule\_ProcedureListCreates.cs:250` (`Uretim_SonrakiIstasyonaGonder` icindeki JOIN):

```
LEFT OUTER JOIN ReceteIstasyonGrupOperasyon GROU
ON GROU.GrupKodu = UR.IstasyonGrupKodu AND GROU.OperasyonKodu = URO.OperasyonKodu
```

- `:243-245` secilen alanlar: `UR.IstasyonGrupKodu` , `URO.OperasyonKodu` , `GROU.IstasyonKodu` , `GROU.IstasyonAdi` .
- `:262` `WHERE RCA.IstasyonGruplamaKullan = 1` — bu otomatik istasyon olusturma yalnızca gruplama acik recetelerde calisir.

UI tarafinda grup kodu secimi: `FrmUretimEmriED.cs:421` (`_mdl.UretimEmri.IstasyonGrupKodu = TxtIstasyonGrubu.Text`) ve secim ekranı `:714-724` (`TxtIstasyonGrubu_ButtonClick -> FrmReceteIstasyonGrupIstasyonEslestir`).

2) Uretim emri kaydedilince ilk operasyon/hareket/istasyon nasıl yaratilir?

Adim A — Kayit (operasyon satirlari + miktar motoru): `UretimEmriKaydetBySiparis` tek transaction'da: uretim emrini + operasyonlari + stoklari yazar, ardindan `Uretim_MiktarGuncelle` calistirir. Kanit:

`My\Business\Manager\UretimEmriManager.cs:48`
`( _db.UretimOperasyon.InsertOrUpdate(mdl.UretimOperasyonlar, trs) ), :63`
`( exec[Uretim_MiktarGuncelle] '...' ).`

Uretim_MiktarGuncelle icinde sıra mantigi: bir sonraki operasyonun planlanan miktarı, bir öncekinin (Sıra-1) üretim miktarından türetilir (MERGE). Kanit: **_ProcedureListCreates.cs:59-65** :

```
USING(SELECT UR.Id, coalesce((SELECT UretimMiktari FROM UretimOperasyon
WHERE SipHid=UR.SipHid AND Sira = UR.Sira-1), UR.PlanlananMiktar) AS PlanlananMiktar
FROM dbo.UretimOperasyon UR WHERE UR.Sira > 1 AND UR.UrId=@urid)
```

Yani **Sıra>1** operasyonlar, aynı **SipHid** içindeki **Sıra-1** operasyonun çıktı miktarını "planlanan" olarak alır.

Adım B — Uretimi baslatma (ilk operasyon hareketi): UI'da "Uretime Basla" akisinda yalnızca **Sıra <= 1** operasyonlar için **UretimOperasyonHareket** yaratılır (yani ilk operasyon başlatılır). Kanit:

FrmUretimEmriED.cs:233-304 (**UretimBaslat**): - **:241** **foreach (var oprs in**
_mdl.UretimOperasyonlar.Where(c => c.Sira <= 1)) — **sadece ilk sıra başlıyor.** - **:243-257** ilk operasyona
ait **UretimOperasyonHareket** oluşturulur (**Sira = oprs.Sira**). - **:293**
_mng.UretimOperasyonHareketKaydet(...) .

UretimOperasyonHareketKaydet , hareketi yazıp arkasından iki motoru çağırır: **Uretim_MiktarGuncelle** +
Uretim_PlanlananGuncelle . Kanit: **UretimEmriManager.cs:156** (**exec[Uretim_MiktarGuncelle]**), **:162**
(**exec[Uretim_PlanlananGuncelle]**).

Adım C — Hareket/istasyon detaylarının SQL ile açılması (Uretim_PlanlananGuncelle):

Uretim_PlanlananGuncelle iki cursor ile çalışır. Kanit: **_ProcedureListCreates.cs:121-191** .

- **curs (ilk açilis):** Her **UretimOperasyon** için, hareketi yoksa oluşturur.
- **:135-139** **curs** tanımı + ilk FETCH (**Sira** dahil okunur).
- **:143-147** **if(@sira=1)** -> hareket yoksa **planlanan miktarli** **UretimOperasyonHareket** INSERT eder (ilk operasyon dolu baslar). **:145-146** 'da **Sira** , **RcAId** , **RcOId** aktarılır.
- **:148-153** **if(@sira>1)** -> hareket yoksa **sifir miktarli** hareket INSERT eder (sonraki operasyonlar bos/bekleyen baslar). **:152** 'de tüm miktarlar **0** .

Bu, ilk operasyonun aktif (planlanan>0), sonraki tüm operasyonların "0 planlanan" bekleyen olarak yaratılmasını sağlar.

- **curs2 (sıra N -> N+1 zincirleme planlama):** Bir operasyonun üretimi olunca bir sonraki sıra operasyonun planlanan miktarını günceller.
- **:162-165** **curs2** tanımı + FETCH (**SipHid** , **Sira** dahil).
- **:169-173** **if(@UretimMiktari > 0)** -> aynı **RcAId** + **SipHid** içinde **Sira = @sira + 1** operasyon hareketinin **PlanlananMiktar / KalanMiktar** 'i, bir önceki operasyonun üretim miktarına eşitlenir.
- **:174-178** **if(@UretimMiktari <= 0)** -> sonraki sıra sıfırlanır.

Adım D — İstasyon oluşturma (gruplama açikken otomatik): **Uretim_PlanlananGuncelle** sonunda **Uretim_SonrakiİstasyonaGonder** çağırılır; bu prosedür **İstasyonGruplamaKullan=1** reçetelerde otomatik olarak **UretimOperasyonHareketDetay** ("Acilis") + **Uretimİstasyon** satırları INSERT eder ve hareketi "işlemde" yapar. Kanit: **_ProcedureListCreates.cs:278-294** (HareketDetay "Acilis" INSERT), **:297-315** (**Uretimİstasyon** INSERT, istasyon kodu/adi **RequeteİstasyonGrupOperasyon** 'dan gelir), **:317** (**UPDATE UretimOperasyonHareket SET İşlemdekiMiktar = @planlananMiktar**).

Gruplama KAPALI ise istasyon manuel açilir: İstasyonlar otomatik değil, kullanıcı tarafından

FrmUretimİstasyonED ile (aşağıda UI bölümü) eklenir. Bu durumda istasyon satırları **İstasyonlarOlustur**

(`FrmUretimIstasyonED.cs:272-346`) ile sablon `ReceteIstasyon` 'dan kurulur ve `UretimOperasyonManager.KaydetNormal` ile yazilir.

3) Bir adım bitince sonraki adım nasıl acilir?

Uc farkli mekanizma vardır; hangisinin çalıştığı receteye/UI'ya göre değişir.

(a) SQL motorunda zincirleme planlama (Sıra N -> N+1): Yukarıda Adım C/curs2'de açıklandı. Bir operasyonun `UretimMiktari` artınca, aynı `RcAId + SipHid` içindeki `Sıra+1` operasyon hareketinin planlanan miktarı otomatik dolar. Kanıt: `_ProcedureListCreates.cs:169-173`. Bu motor her üretim girişinde tetiklenir (çünkü girişler `Uretim_MiktarGuncelle` + `Uretim_PlanlananGuncelle` çağırır; orn.

`UretimEmriManager.cs:156-163`, `UretimOperasyonManager.cs:80-89`, `UretimIstasyonManager.cs:45-52`).

(b) İstasyon gruplama otomatik sevki (`Uretim_SonrakiIstasyonaGonder`): `IstasyonGruplamaKullan=1` recetelerde, planlanan>0 ve "henüz işlemde olmayan" her operasyon hareketi için (curs) yeni açılış detayı + istasyon satırı yaratır, operasyonun istasyon kodu `ReceteIstasyonGrupOperasyon` eşleştirmesinden gelir. Kanıt: `_ProcedureListCreates.cs:225-265` (cursor + JOIN'lar), filtre `:262-265`:

```
WHERE RCA.IstasyonGruplamaKullan = 1
AND UROH.PlanlananMiktar > 0
AND UROH.PlanlananMiktar > UROH.IslemdekiMiktar
AND UROH.UrId = @UrId;
```

Boylece (a) sonraki operasyonun planlananını doldurur, (b) bu yeni dolan operasyonu otomatik doğru istasyona "gonderir" (acar).

(c) Manuel "Sonraki Operasyona Gecir" (WinForms UI): Gruplama yokken kullanıcı, bir operasyon hareketinden sonraki operasyona elle geçirir. `FrmUretimIstasyonED` `SonrakiOperasyon` modunda çalışır. Kanıt: `FrmUretimIstasyonED.cs:142-231` (`SonrakiOperasyonBaslat`) - `:151-157` bu operasyon hareketinin operasyonu (`OncekiOperasyon`) bulunur. - `:158` `int siras = OncekiOperasyon.Sira + 1;` — **sonraki sıra hesabi.** - `:159` `_srvOperasyon.SelectFirst(c => c.UrId==... && c.SipHid==OncekiOperasyon.SipHid && c.Sira==sira)` — aynı `SipHid` içinde `Sıra+1` operasyon. - `:172-192` sonraki operasyon için yeni `UretimOperasyonHareket` (`Sira = SonrakiOperasyon.Sira`). - `:208` `HareketYeni.PlanlananMiktar = oncedenUretilen - operasyondaolan;` — devredilecek miktar. - Kayıt: `KaydetSonraki` -> `UretimOperasyonManager.KaydetSonraki` (`UretimOperasyonManager.cs:109-179`); yeni hareketi Insert eder, detay/istasyonları yeni harekete bağlar, `Uretim_MiktarGuncelle` çalıştırır (`:159`).

(d) TabletV2 istasyon sevki (`IstasyonSevkManager`): TabletV2 (Blazor saha tableti) tarafında "sonraki istasyona gönderme", operasyon sıra motoru ile DEĞİL, doğrudan `UretimIstasyon.IstasyonKodu` değiştirilerek yapılır; is emri elle seçilen hedef istasyona tasınır. Kanıt: `WebUretim` `TabletV2\My\Business\Managers\IstasyonSevkManager.cs:63-90` (`IstasyonSevkKaydet`) - `:77-79` mevcut takip hareketi hedef istasyona göre guncellenir, `Durumu="Durduruldu"` . - `:82` `update UretimIstasyon set IstasyonKodu=@IstasyonKodu, IstasyonAdi=@IstasyonAdi where Id=@Id` . - `:84-88` tek transaction'da: `INSERT(IstasyonTakipHareket) + UPDATE(UretimIstasyon) + INSERT(log)`.

Bekleyen (takip baslamamış) is emri için: `:43-58` (`IstasyonBekleyenSevkKaydet`) — yalnızca `UretimIstasyon` istasyon kodu `UPDATE + log INSERT` (`:50-55`). Bu modulde üretim/miktar motoru (`Uretim_*` prosedürleri) çağırılmaz; sadece istasyon ataması değişir (bkz. TabletV2 docs `_sections\01_IsEmirleri.md:3`, `:55`).

Önemli fark: (a)/(b) otomatik sıra tabanlı akistir; (c) operasyon sıra tabanlı manuel akistir; (d) sıra motoru ile ilgisiz, istasyondan istasyona elle tasimadır.

4) Tek operasyon altında çok istasyon olursa akis sorunu ("YOK uyarisi" / kisitlama)

Sıra yalnızca OPERASYON seviyesinde tutulduğu, İSTASYON'da `Sıra` alanı olmadığı (`UretimIstasyon.cs` içinde `Sıra` yok) için, "aynı operasyon altında birden çok istasyonu sırayla çalıştırma" otomatik akisi YOKTUR. Sistem bunu şu kısıtla onler: istasyon gruplama kullanılacaksa üretim emrine TEK reçete girilmesini zorunlu kılar.

Kanıt: `MyUI\UretimModule\FrmUretimEmriED.cs:471-486` (`TextLeriKontrolEt`):

```
foreach (var itm in _mdl.ReceteModeller) {
    if (itm.Recete.IstasyonGruplamaKullan) {
        if (rcaid == Guid.Empty) rcaid = itm.Recete.Id;
        else if (rcaid != itm.Recete.Id) {
            MesajHata("İstasyon Gruplama Kullanılacaksa Tek Reçete Seçilebilir. Üretime Lütfen Tek Reçete Seçiniz.");
            return false;
        }
    }
}
```

Ek olarak `:489-499` gruplama açıkken `IstasyonGrupKodu` (grup kodu) boş bırakılmaz: `MesajHata("Lütfen İstasyon Grubu Seçiniz.")`.

`Uretim_SonrakiIstasyonaGonder` içindeki `ReceteIstasyonGrupOperasyon` JOIN'i `OperasyonKodu` üzerinden tek istasyon dondurmeye dayanır (`_ProcedureListCreates.cs:250`); aynı grup+operasyon için birden çok istasyon eşlesirse cursor satırları çöğalir ve otomatik akis belirsizlesir. Gruplama KAPALI ve bir operasyona birden çok `ReceteIstasyon` bağlıysa, istasyonlar paralel olarak aynı operasyon hareketine bağlanır (`IstasyonlarOlustur` , `FrmUretimIstasyonED.cs:288-318`) ancak aralarında otomatik bir sıra/gecis tetiklemesi tanımlı değildir; gecis yalnızca OPERASYON değişince (Sıra+1) olur. Yani "tek operasyon - çok istasyon" senaryosunda istasyonlar arası otomatik sevk akisi yoktur; pratikte her istasyon adımı ayrı bir OPERASYON olarak modellenmelidir.

5) Bilinen hata: `_ProcedureListCreates.cs:186` dinamik EXEC (varchar + uniqueidentifier)

`Uretim_PlanlananGuncelle` sonunda `Uretim_SonrakiIstasyonaGonder` çağılırken dinamik EXEC kullanılmış ve string birleştirme hatalıdır.

Kanıt: `_ProcedureListCreates.cs:186` :

```
begin EXEC ('EXEC [Uretim_SonrakiIstasyonaGonder] ''' + @uridsi + '''; END
```

Hemen altında `:187` doğru (statik) çağırının yorum satırına alındığı görülür:

```
/* BEGIN EXEC [Uretim_SonrakiIstasyonaGonder] @uridsi ; END*/
```

Sorunlar: 1. **Tip hatası (varchar + uniqueidentifier):** `@uridsi` `uniqueidentifier` (`:125`) tipindedir; `'...' + @uridsi` ifadesi string ile `uniqueidentifier`'i doğrudan `+` ile birleştirmeye çalışır. SQL Server bunu implicit

cast etmez, "Conversion failed / operand type clash" benzeri hata üretir. Doğru kullanım `CAST(@uridsi AS varchar(36))` olmalıydı. 2. **Tirnak yerlesimi bozuk:** Uretilmek istenen metin muhtemelen `EXEC [Uretim_SonrakiIstasyonaGonder] '<guid>';` olmalı; ancak `'''+@uridsi+' ;'''` ifadesi tırnakları yanlış yerleştirir — kapanış tırnağı GUID'den sonra değil ; ifadesinden sonra gelir, yani oluşan komut hem tip olarak hem sözdizimi olarak geçersizdir.

Etki: Bu dinamik EXEC çalıştırılmasında `Uretim_PlanlananGuncelle` içinden `Uretim_SonrakiIstasyonaGonder` otomatik tetiklenmesi başarısız olur (hata fırlatır veya — çağırılan katmanda yutuluyorsa — sessizce çalışmaz). Yorumdaki statik çağrı (:187) parametre tipi açısından doğru olan biçimdir; dinamik EXEC yerine onun kullanılması gerekir. Bu satır, "istasyon gruplama otomatik sevki" akisinin (Bolum 3-b) beklenen şekilde çalışmamasının doğrudan kaynağıdır.

6) İstasyon sırası UI'da kullanıcıya nasıl yansır?

UretimV4 (WinForms):

- **Operasyon listesi + sıra:** `FrmUretimEmriED` ana grid'i operasyonları (`UretimOperasyon`) gösterir; `Sıra` kolonu görünür (gizlenen kolonlar yalnızca Id/FK'lardır: `FrmUretimEmriED.cs:326-335`). Operasyon durumu renkle yansır: "Hazır" koyu yeşil, "Uretimde" indigo (:693-712 , `GridView_RowStyle`). Bu `Durumu` , `Uretim_MiktarGuncelle` / `Uretim_DurumGuncelle` içinde hesaplanır (`_ProcedureListCreates.cs:69-75` , :99-105 : `Hazir` / `Uretimde` / `Beklemede`).
- **İlk adım kisiti:** Sonraki bir operasyona istasyon eklenmek istendiğinde, önceki operasyon başlatılmamışsa engellenir: `MyUI\UretimModule\FrmOperasyonTakipDetaylar.cs:157` ve :252 `MesajBilgi("Operasyon Baslatılmamış İşlem Yapılamaz Bir önceki operasyondan işlem yapınız")` . Bu, sıra dışı istasyon işlemini önler.
- **İstasyon ekranı (`FrmUretimIstasyonED`):** Uc mod ile açılır — `OperasyonTuruEnum` = `SonrakiOperasyon` / `IstasyonEkle` / `Degistir` (`MyUI\UretimOperasyonModule\OperasyonTuruEnums.cs:9-14`). `IstasyonEkle` ile gelir (`FrmOperasyonTakipDetaylar.cs:164` , :259), istasyonlar `IstasyonlarOlustur` ile operasyona bağlı sablondan listelenir (`FrmUretimIstasyonED.cs:272-346`). "Sonraki operasyona gecir" sıra+1 hesabıyla yapılır (:158-159) ve devreden miktar kullanıcıya planlanan olarak gösterilir (:242 , `cnt.Miktar = HareketYeni.PlanlananMiktar`).
- **Miktar/durum geri bildirimi:** Her kayıt sonrası motorlar çağrıldığı için (`Uretim_MiktarGuncelle` / `Uretim_PlanlananGuncelle`) operasyon/istasyon/uretim emri/siparis durumları üst seviyeye doğru güncellenir (`_ProcedureListCreates.cs:77-84` `UretimEmri` ve `Siparis` durumu).

TabletV2 (Blazor saha tableti):

- İş emirleri OPERASYON sırasına göre değil, atandıkları `IstasyonKodu` 'na göre gruplanıp kart panosunda gösterilir; her kart bir istasyondur. Kanıt: TabletV2 docs `_sections\01_IsEmirleri.md:22` ("İş emirleri `IstasyonKodu` alanına göre gruplanır; her kart bir istasyonu temsil eder. Operasyon sıra/akı motoru burada tetiklenmez").
- Her istasyon kartında iş emirleri durum renkleriyle gösterilir: Aktif (yeşil), Durdurulan (kirmizi), Bekleyen (mavi). Kanıt: `_sections\01_IsEmirleri.md:26` .
- Kullanıcı "Sevk" ile iş emrini elle başka istasyona taşıyabilir (sıra motoru devreye girmez); hedef istasyon combo'dan seçilir. Kanıt: `_sections\01_IsEmirleri.md:41` , :55 ve kod `IstasyonSevkManager.cs:63-90` .
- Bu modulde üretim/miktar motoru çağrılmaz; istasyon sırası UI'da yalnızca "iş emrinin şu an hangi istasyonda olduğu" olarak yansır, otomatik sonraki-istasyon gecisi (gruplama akisi) WinForms+SQL tarafında kalır. Kanıt: `_sections\01_IsEmirleri.md:3` .

Kanit ozeti (dosya:satir)

- Operasyon sirasi sablonu: `My\Entities\Receteler\ReceteOperasyon.cs:32` , `:45` .
- Istasyon-operasyon baglantis (sira yok): `My\Entities\ReceteIstasyonlar\ReceteIstasyon.cs:33-34` ; `My\Entities\UretimIstasyonlar\UretimIstasyon.cs:36-43` .
- Grup eslestirme: `My\Entities\ReceteIstasyonGruplar\ReceteIstasyonGrupOperasyon.cs:11-15` .
- Uretim operasyon sirasi: `My\Entities\UretimOperasyonlar\UretimOperasyon.cs:30` .
- Sira kopyalama (sablon->uretim): `MyUI\UretimModule\FrmUretimEmriED.cs:218-226` .
- Ilk operasyon baslatma (Sira<=1): `MyUI\UretimModule\FrmUretimEmriED.cs:241-257` .
- Tek-recete kisiti / grup kodu zorunlu: `MyUI\UretimModule\FrmUretimEmriED.cs:474-499` .
- Ilk acilis cursor'u (Sira=1 dolu / Sira>1 sifir): `_ProcedureListCreates.cs:143-153` .
- Sira N->N+1 planlama (curs2): `_ProcedureListCreates.cs:169-178` .
- Sonraki operasyon planlamasi (MERGE Sira-1): `_ProcedureListCreates.cs:59-65` .
- Otomatik istasyon sevki (gruplama) + grup JOIN: `_ProcedureListCreates.cs:250` , `:262-265` , `:278-317` .
- Dinamik EXEC hatasi: `_ProcedureListCreates.cs:186` (yorumlu dogru cagri `:187`).
- Manuel sonraki operasyon (Sira+1) UI: `MyUI\UretimIstasyonModule\FrmUretimIstasyonED.cs:158-159` , `:172-208` .
- KaydetSonraki/KaydetNormal motorlari: `My\Business\Manager\UretimOperasyonManager.cs:80-89` , `:109-167` .
- Motor cagirlari (her giriste): `UretimEmriManager.cs:63` , `:156-163` ; `UretimIstasyonManager.cs:45-52` .
- TabletV2 elle istasyon sevki: `WebUretim TabletV2\My\Business\Managers\IstasyonSevkManager.cs:43-58` , `:63-90` .
- TabletV2 UI yansimasi: `WebUretim TabletV2\docs\_sections\01_IsEmirleri.md:3` , `:22` , `:26` , `:41` , `:55` .

Modul: UretimModule

UretimV4 (WinForms masaustu ERP) icindeki **UretimModule** klasoru, uretim emirlerinin (is emri) olusturulmasi, baslatilmasi, takip edilmesi ve operasyon bazli uretim girisinin yonlendirilmesinden sorumludur. Klasor 4 formdan olusur: uretim emirlerinin listelendigi ve filtrelendigi **FrmUretimEmriListesi**, tek bir uretim emrinin acilis/duzenleme + uretim baslatma ekranı **FrmUretimEmriED**, operasyonlari kart (kanban) gorunumunde gosteren canli takip ekranı **FrmOperasyonTakip**, ve secilen operasyonun bekleyen/uretimdeki satirlarini grid olarak listeleyip uretim girişi veya istasyona gonderme islemine kopru kuran **FrmOperasyonTakipDetaylar**.

Modulun is mantigi cogunlukla **UretimEmriManager** (kayit/sil/baslat orkestrasyonu) ve **UretimTakipManagerV2** (canli takip sorgulari) üzerinden yurur. Tum miktar/akis hesapları **Uretim_MiktarGuncelle**, **Uretim_PlanlananGuncelle**, **Uretim_DurumGuncelle** ve **Uretim_SonrakiIstasyonaGonder** stored procedure'lari ile yapilir; bu prosedurler operasyon Sira'sina gore miktarlari yukari toplar, bir sonraki operasyonun planlanana tasir ve istasyon gruplama acikken otomatik UretimIstasyon kaydi olusturur.

Not (tum formlar icin): Base form siniflari (**MyFrmKayitFull**, **MyFrmListe**, **MyFrmSadeFull** — **My.Kontrol.Formlar** namespace'i) derlenmis bir kutuphanededir, kaynak kod bu projede yoktur. Bu nedenle base butonlar (BtnKaydet/BtnSil/BtnKapat/BtnYeni/BtnDuzenle/BtnAra/BtnTemizle/BtnYazdir/navigasyon) ve Enter=Kaydet / Esc=Kapat gibi global kisayollar kodla DOGRULANAMADI; Designer dosyalarinda sadece bu butonlari gorunum/konum ayarlari mevcuttur. F-tusu kisayolu (ShortcutKeys) hicbir formda tanimli degildir.

Uretim Emri Listesi (**FrmUretimEmriListesi.cs** / **.Designer.cs**)

Ne ise yarar: Uretim emirlerini (is emri) tarih, kod, cari, tur, recete ve duruma gore filtreleyip listeler. Ust grid uretim emrini, alt grid o emre ait operasyonlari gosterir. Durum sekmesi butonlari (Hepsi/Beklemede/Uretimde/Hazir) ile hizli filtre yapilir. Yeni uretim emri olusturmaya ve secili emri duzenlemeye giris noktasidir. **Once ne olmalı (onkosul):** Sistemde siparis ve recete tanimlari olmalı (Turu/Durumu/ReceteAdi combolari **GenelService.GrupListesi** ile UretimEmri tablosundan beslenir). Form **SiparisKodundanFiltrele=true** ve **SiparisKodu** set edilerek de acilabilir (siparis ekranindan tek bir siparise drill-down). **Sonra ne olur:** Satira cift tiklayinca (SecimIcinAcilmadiysa) **FrmUretimEmriED** acilir (**IdGuid** ile edit). Yeni kayit butonlari **FrmUretimEmriED** 'yi **UretimTuru="Siparis"** veya **"Recete"** ile acar. Yazdir secili emri DevExpress rapor (**UretimEmriListesi**) olarak basar. Bu form dogrudan tablo degistirme (salt okuma + navigasyon); kayit/silme islemleri ED formunda yapilir. **Butonlar & kisayollar:** - **BtnAra** — **Bagla()** ile filtreye gore **UretimEmri** listesini ceker (Frm_Load sonunda **PerformClick** ile otomatik aranir). - **BtnTemizle** — tum filtre alanlarini (kod, ad, cari, tarihler, combolar) sifirlar. - **BtnYazdir** — secili emir icin **Yazdir()** : **_list** DataSet'e cevriyip **ds.Yaz("UretimEmriListesi", false)** ile basilir. - **BtnEkleSiparisen** — **FrmUretimEmriED** (UretimTuru="Siparis") acar. - **BtnEkleReceteden** — **FrmUretimEmriED** (UretimTuru="Recete") acar. - **BtnDurumuHepsi** / **BtnDurumuBeklemede** / **BtnDurumuUretimde** / **BtnDurumuHazir** — **DurumuAyarla(...)** ile **durumu** filtresini set edip butonu renklendirir, sonra **BtnAra.PerformClick()** . (Acilis varsayilani "Uretimde"). - Grid cift tiklama/Enter (**MyView1_MyEventDoubleClickEnter**) — secim modunda satir secip kapatir; normal modda **FrmUretimEmriED** acar. **Agirdigi katmanlar:** - Service: **IUretimEmriService.SelectListWhere(sor)** — filtreli uretim emri listesi. - Service: **IUretimOperasyonService.SelectListWhere("where UrId=...")** — secili emrin operasyonlari (alt

grid, `MyView1_FocusedRowChanged` ve ilk satirda). - Service:

`IGenelService.GrupListesi("UretimEmri","Turu"/"Durumu"/"ReceteAdi")` — combo kaynaklari. - Manager: `SiparisManager.GetSiparis(itm.Id)` — Yazdir oncesi siparis modeli (yazdirmada dogrulama amaclı cagilir). - Manager: `UretimEmriManager.UretimDurumGuncelle(id)` — `DurumGuncelle` metodu (bu formda buton bagli degil; `exec Uretim_DurumGuncelle` calistirir). - SQL: filtre WHERE'leri string ile uretilir (SiparisKodu/SiparisCariKodu/SiparisCariUnvani LIKE, Turu/Durumu/ReceteAdi, Baslangic/Bitis tarih araliklari). **Istasyon sirasiyla iliskisi:** Dogrudan yok; sadece listeleme. Operasyon Sira mantigi ED ve takip formlarında devreye girer. **Notlar:** `CmbDurumu` ve `myLabel7` (Durumu) Designer'da `Visible=false` — durum filtresi artık sekme butonlariyla yönetiliyor. `BtnDizayn` gizli. Alt operasyon gridinin grid yerlesim adi "UretimEmriListesi".

Uretim Emri Acilis/Duzenleme (`FrmUretimEmriED.cs` / `.Designer.cs`)

Ne ise yarar: Tek bir uretim emrinin (is emrinin) olusturuldugu/duzenlendiği ana ekran. Siparis veya receteden operasyonlari, sarf stoklarini ve baslatilmis operasyon hareketlerini (3 sekmeli grid: Operasyonlar / Stoklar / Istasyon-Hareketler) yönetir. "Uretime Kaydet ve Baslat" ile uretim akisini fiilen baslatir. Istasyon gruplama secimini de burada yapar. **Once ne olmalı (onkosul):** `IdGuid` (mevcut emir) veya `SipId` set edilmiş olmalı; ikisi de bossa form acilisinda `FrmSiparisListesi` secim modunda acilir ve bir siparis secilmek ZORUNDADIR (secilmezse form kapanir). Secilen siparise zaten bir UretimEmri varsa edit, yoksa yeni acilis yapilir. Yeni acilista operasyonlar siparis hareketleri x recete operasyonlarından otomatik uretilir (`OperasyonlarOlustur`). **Sonra ne olur:** - **Kaydet:** `UretimEmriManager.UretimEmriKaydetBySiparis` tek transaction'da `UretimEmri` (InsertOrUpdate), eski `UretimOperasyon` ve `UretimStok` kayitlarini siler, yenilerini yazar, ardından `exec Uretim_MiktarGuncelle` calistirir. Kayit sonrası `ActionAktar` (liste yenileme) tetiklenir ve edit'e gecilir. - **Uretime Kaydet ve Baslat:** once `Kaydet`, sonra `UretimBaslat()` Sira<=1 operasyonlar için `UretimOperasyonHareket` olusturup `UretimOperasyonHareketKaydet` ile yazar — bu da `exec Uretim_MiktarGuncelle` + `exec Uretim_PlanlananGuncelle` (sonuncu icinde `Uretim_SonrakiIstasyonaGonder`) calistirir. Form kapanir. - **Sil:** baslatilmis hareket varsa engellenir; yoksa `UretimEmriManager.UretimEmriSil` ana+operasyon+stok siler, `exec Uretim_MiktarGuncelle` ve `Siparis.Durumu='YeniKayit'` yapar. - **Baslatilmis Uretimleri Sil / Uretime Ait Tum Kayitlari Sil:** ilgili hareket/istasyon/takip tablolarini temizler. - Cikis (Frm_FormClosing): yeni kayit edilip uretim baslatilmamissa kapanis engellenir / sorulur. **Butonlar & kisayollar:** - `BtnKaydet` — `Kaydet()` (base buton; Text Designer'da yok). - `BtnSil` — `Sil()`. - `BtnUretimeBasla` ("Üretime Kaydet \r\nve Başlat") — `KaydetBaslat()` -> kayit + `UretimBaslat()`. - `BtnUretimeSil` ("Başlatılmış \r\nÜretimleri Sil") — `OperasyonHareketSil()` (her hareket için önce `UretimOperasyonHareketKayitVarMi` kontrolü). - `BtnUretimeAitTumKayitlariSil` ("Üretime Ait \r\nTüm Kayitlari Sil") — `OperasyonaAitTumKayitlariSil()` -> `UretimeBagliTumHareketleriSil`. - `BtnCariTemizle` — cari kodu/unvani textlerini temizler. - `TxtSiparisKodu` (button edit) — Frm_Load'da `TxtIsEmriNo.ButtonClick` bagli (siparis kodu button click ayrıca `BaglaSiparisden` ile yönetilir). - `TxtIsEmriNo` (button edit) `ButtonClick` — `EvrakNoAl()` ile `GenelService.GetEvrakNo("UretimEmri")` cagilir (varsa önce degistirme onayi sorar). - `TxtIstasyonGrubu` (button edit) `ButtonClick` — `FrmReceteIstasyonGrupIstasyonEslestir` secim modunda acar, secileni `TxtIstasyonGrubu` 'na yazar. - `TxtCariKodu` (button edit) `ButtonClick` — `FrmMikroCariListesi` secim modunda acar, MikroCari secip kod+unvani doldurur. - `ChckKapandi` — emrin kapandi durumu (kaydedilirken modele aktarilir). - Grid satir stili (`GridView_RowStyle`) — Durumu "Hazir" yesil, "Uretimde" mor renklerinir. **Cagirdigi katmanlar:** - Manager: `UretimEmriManager.GetUretimSiparisNew(sipId)` — yeni acilis modeli (siparis hareketleri, recete/operasyon modelleri, sarf stoklari fire yuzdesiyle hesaplanir). - Manager: `UretimEmriManager.GetUretimSiparisEdit(urId)` — mevcut emrin tum alt verilerini (operasyon, hareket view'i, stok, recete/operasyon modelleri) yukler. - Manager:

`UretimEmriManager.UretimEmriKaydetBySiparis(md1, yenikayit)` — transaction'li kayıt + `Uretim_MiktarGuncelle` . - Manager: `UretimEmriManager.UretimOperasyonHareketKaydet(list)` — Sira<=1 hareketleri yazar + `Uretim_MiktarGuncelle` + `Uretim_PlanlananGuncelle` . - Manager: `UretimEmriManager.UretimEmriSil / UretimOperasyonHareketSil / UretimOperasyonHareketKayitVarmi / UretimeBagliTumHareketleriSil` . - Service: `IReceteAnaService.SelectFind(id)` — istasyon gruplama kullanan recete kontrolu (`TextLeriKontrolEt` : gruplama varsa tek recete + istasyon grubu zorunlu). - Service: `IGenelService.GetEvrakNo("UretimEmri")` — is emri no. - SQL/Prosedur: `Uretim_MiktarGuncelle` (kayit/sil/hareket sonrası), `Uretim_PlanlananGuncelle` (baslat sonrası, içinde `Uretim_SonrakiIstasyonaGonder`). **Istasyon sirasiyla iliskisi:** Cekirdek. Operasyonlar `Sira` ile uretilir; baslatma yalnızca Sira<=1 operasyonlar için hareket acar. `Uretim_PlanlananGuncelle` Sira N uretimini Sira N+1 planlananına tasir. `IstasyonGruplamaKullan=1` recetelerde istasyon grubu (TxtIstasyonGrubu) secilirse `Uretim_SonrakiIstasyonaGonder` her operasyona TEK `UretimIstasyon` olusturur. **Notlar:** `BtnUretimeBasla/BtnUretimiSil/BtnUretimeAitTumKayitlariSil` click event'leri Designer'da bagli (kod içinde tekrar baglama yorum satirina alinmis — çift tetikleme önlemek için). Uyarı etiketi: "Istasyon Gruplama Tek Ürün Üretim Emrinde Kullanılabilir". UretimTuru combosu: Siparis / Recete / MikroSiparis.

Operasyon Takip (Kart/Kanban) (`FrmOperasyonTakip.cs` / `.Designer.cs`)

Ne ise yarar: Tüm aktif uretimi operasyon kodu bazında canlı kart (kanban) görünümünde gösterir. Her operasyon kartı (`OperasyonCardControlV2`) o operasyonun Bekleyen ve Üretimdeki iş satırlarını özetler. 60 saniyede bir otomatik yenilenir. Saha/yonetim için "neyin nerede olduğu" panosu. **Once ne olmalı (onkosul):** En az bir üretim emri baslatılmış (UretimOperasyonHareket kaydı oluşmuş), durumu "Hazır" olmayan, kapanmamış (Ur.Kapandi=0) ve KalanMiktar>0 olan operasyonlar bulunmalı — aksi halde kart çıkmaz. **Sonra ne olur:** Bu form veri degistirmez. Bir karttan Bekleyen detayına tıklayınca `Ortak.UretimTakipBekleyenAcV2(operasyon)` -> `FrmOperasyonTakipDetaylar` (Durumu="Beklemede"); Üretimde detayına tıklayınca `Ortak.UretimTakipUretimdeAcV2(operasyon, Action)` -> `FrmOperasyonTakipDetaylar` (Durumu="Üretimde"). `BtnTumOperasyonlar` ise `FrmOperasyonTakipDetaylar` 'i `TumunuBagla=true` ile acar. **Butonlar & kisayollar:** - `BtnYenile` ("Yenile") — `Bagla()` ile takip listesini yeniden cekir. - `BtnTumOperasyonlar` ("Tum\\n\\nOperasyonlar") — `FrmOperasyonTakipDetaylar` {`TumunuBagla=true`} acar. - `TimerYenile` (60000 ms) — `BtnYenile.PerformClick()` ile otomatik yenileme. - (Kart içindeki tıklamalar `OperasyonCardControlV2` içinde; detay acma yukarıdaki Ortak metodlarıyla.) **Çagirdigi katmanlar:** - Manager: `UretimTakipManagerV2.GetTakipList()` — operasyon kodu + recete + siparis bazında Planlanan/Üretim/İşlemdaki/Fire/Iptal/Kalan miktarlarını SUM'layan sorgu (UretimOperasyonHareket -> UretimOperasyon -> UretimEmri -> Siparis JOIN; `Durumu<>'Hazır' AND Kapandi=0 AND KalanMiktar>0`). - UI: `OperasyonCardControlV2` — her operasyon için kart; satırları Bekleyen (Planlanan-İşlemdaki>0) ve Üretimde (İşlemdaki>0) olarak ayırır. - Ortak: `UretimTakipBekleyenAcV2 / UretimTakipUretimdeAcV2` — detay formunu Durumu ile acar. **Istasyon sirasiyla iliskisi:** Dolaylı; kartlar operasyon (Sira) bazlı toplam miktarları yansıtır, ancak bu ekran Sira geçişlerini tetiklemez. **Notlar:** Designer'da 4 örnek kart tasarım zamanı placeholder; çalışmada `PnlOrta.Controls.Clear()` sonrası operasyon koduna göre dinamik kart eklenir. `BtnKaydet` gizli (bu salt-izleme formu).

Operasyon Takip Detaylar (`FrmOperasyonTakipDetaylar.cs` / `.Designer.cs`)

Ne ise yarar: Secilen operasyonun (veya tüm operasyonların) iş satırlarını grid olarak listeler ve üretim girişi / istasyona gönderme / detay hareket görüntüleme işlemlerine köprü kurar. Durumu="Üretimde" iken üretim girişi, Durumu="Beklemede" iken istasyona gönderme akisi açılır. **Once ne olmalı (onkosul):**

`FrmOperasyonTakip` (veya baska bir ekran) tarafından `Operasyon` + `Durumu` set edilerek acilmali; ya da `TumunuBagla=true` ile tum operasyonlar getirilir; ya da `SipIdDenBagla=true` + `SipId` ile siparise filtrelenir. Satirin `Id` 'si bos olmamali (operasyon baslatilmamis satirda islem yapilamaz — "bir onceki operasyondan islem yapiniz" uyarisi). **Sonra ne olur:** - Durumu="Uretimde" + Uretim Girişi: once `FrmUretimIstasyonHareketSec` (secim modu, UrOHdId=secili Id) ile istasyon hareketi secilir, sonra `FrmUretimIstasyonUretimGir` acilir; orada uretim girilince istasyon/operasyon hareket detay tablolari ve (prosedurler araciligiyla) UretimOperasyon/UretimEmri/Siparis miktar+durumlari guncellenir. - Durumu="Beklemede" + Istasyona Gonder: `FrmUretimIstasyonED` (OperasyonTuru=IstasyonEkle, OprId=secili Id) acilir; islem sonrasi `Action` (kart yenileme) tetiklenir. - Detay Hareketler: `FrmUretimOperasyonHareketDetayList` (DetayGoster=true, DetayId=secili Id) acilir. - Her islem sonrasi `this.Close()` ile detay formu kapanir. **Butonlar & kisayollar:** - `BtnUretimGirisi` — Designer Text'i "Üretim /İstasyon"; `Frm_Load`'da Durumu="Uretimde" iken "Uretim Giriş", "Beklemede" iken "İstasyona Gonder" olarak degisir. `BtnUretimGirisi_Click` Durumu'ya gore uretim girişi veya istasyona gonderme akisini calistirir. - Sağ tik (context menu, `ContexMenuyeEkle`) ogeleri: - "Üretim Girişi" (Durumu="Uretimde") — `UretimGirisi` (hareket sec -> uretim gir). - "İstasyona Gonder" (Durumu="Beklemede") — `UretimeGonder_Operasyon` (`FrmUretimIstasyonED` IstasyonEkle). - "Detay Hareketler" (her durumda) — `Detaylar` (`FrmUretimOperasyonHareketDetayList`). - Grid çift tik/Enter (`MyView1_MyEventDoubleClickEnter`) — secili satiri alır (govde bos; islem context menu/buton ile). **Cagirdigi katmanlar:** - Manager: `UretimTakipManagerV2.GetTakipDetayList(operasyon, durumu, sipid)` — Durumu="Uretimde" iken `UretimOperasyonHareketDetay` (Islemdeki>0 & Kalan>0), aksi halde `UretimOperasyonHareket` (Kalan>0) satirlarini Siparis/Operasyon ile JOIN'leyip getirir. - Manager: `UretimTakipManagerV2.GetTakipDetayListAll()` — `TumunuBagla` icin KalanMiktar>0 olan tum operasyonlari hareketleri. - UI kopru formlari: `FrmUretimIstasyonHareketSec`, `FrmUretimIstasyonUretimGir`, `FrmUretimIstasyonED`, `FrmUretimOperasyonHareketDetayList` (asil uretim girişi/istasyon islemleri bu formlarda yapilir; miktar guncellemeleri `Uretim_MiktarGuncelle` / `Uretim_PlanlananGuncelle` araciligiyla olur). **İstasyon sirasiyla iliskisi:** Dogrudan. "Beklemede" satir, bir onceki Sira operasyonu uretim yapilmadan baslatilmamis olabilir; bu yuzden `Id` bos satirda islem engellenir ve onceki operasyondan ilerlemesi istenir. Uretim girişi/istasyona gonderme, Sira N+1 operasyonunun planlanan miktarini besler. **Notlar:** Grid yerlesim adi "ReceteEkleDetaylar" (Designer kalintisi). Form basligi acilista "Üretim Takip Detay = Durumu : ... - Operasyon : ..." olarak set edilir.

Modul: UretimIstasyonModule

Bu modul, üretim emirlerinin OPERASYON-istasyon kiralimindeki saha takibini ve üretim/fire miktarı girişini sağlar. Bir üretim emrinin her operasyonu altında bir veya birden çok istasyon (UretimIstasyon) bulunur; bu modul (a) hangi istasyonda ne kadar planlandı/üretildi/fire verildi şeklinde görsel takip kartları (FrmUretimIstasyonTakip + IstasyonCardControl), (b) istasyon ve istasyon-hareket listeleri/filtreleri (FrmUretimIstasyonHareketList, FrmUretimIstasyonHareketDetayList), (c) tek bir istasyon hareketine üretim/fire miktarı girme ekranı (FrmUretimIstasyonUretimGir) ve (d) bir operasyona istasyon ekleme/değiştirme/sonraki operasyona geçirme ekranı (FrmUretimIstasyonED + OperasyonIstasyonControlV2) sunar. Miktar/akış motoru tamamen OPERASYON-Sıra bazlı SQL prosedurleriyle (Uretim_MiktarGuncelle -> Uretim_PlanlananGuncelle -> Uretim_SonrakIstasyonaGonder) yürür; bu modulün kayıt ekranları bu prosedurleri tetikler. Modul ofis/yonetim WinForms uygulamasıdır (CepPatronERP.exe), DB'ye doğrudan bağlanır (Ortak.DbPro = UretimV3_FEZA, Ortak.DbMikro = MikroDB_V16_FEZA24).

Istasyon Takip (FrmUretimIstasyonTakip.cs / .Designer.cs)

Ne ise yarar: Açık (UrO.KalanMiktar > 0) tüm istasyonların özet kartlarını gösterir. Her istasyon kodu için tek bir kart (IstasyonCardControl) çıkar; kartta istasyon kodu/adi, operasyon, fason durumu, planlanan/üretim/fire miktarları toplanmış olarak yer alır. Saha/yonetim için "hangi istasyonda ne durumda" panosudur. **Once ne olmalı (onkosul):** Üretim emri ve operasyonları oluşturulmuş, ilgili operasyona en az bir istasyon atanmış (UretimIstasyon kaydı var) ve operasyonun KalanMiktar > 0 olmalı. Aksi halde kartlar yerine boş placeholder kartlar gösterilir. **Sonra ne olur:** Bu ekran salt okunur özet panosudur; veritabanını değiştirmez. Bir kartın buyutec ikonuna tıklayınca o istasyonun detayı FrmUretimIstasyonTakipDetay ekranına geçer (Ortak.IstasyonKontrolDetayAc). **Butonlar & kısayollar:** - Yenile (BtnYenile) — Listeyi yeniden çek (Bagla() -> GetTakipList()). - Kart buyutec ikonu (IstasyonCardControl.pictureBox2) — Seçili istasyonun detayını açar (kısayol yok, mouse click). - Not: Designer'da BtnKaydet Visible=false (bu ekranda kayıt yok). **Çağırdığı katmanlar:** - Manager/Service: UretimIstasyonTakipManager.GetTakipList() — UretimIstasyon (Url) + UretimOperasyon (UrO) JOIN, UrO.KalanMiktar > 0 filtresi; istasyon kodu/operasyon/fason/cari bazında PlanlananMiktar/UretimMiktarı/FireMiktarı/IptalMiktarı SUM'lar. (IGenelService.Query ile ham SQL). - SQL/Prosedur: Bu ekranda prosedür tetiklenmez (sadece SELECT). - API: - **Istasyon sırasıyla ilişkisi:** Kartlar operasyon Sıra bilgisiyle gruplanmaz; istasyon kodu bazında gruplanır. Hangi operasyonda oldukları kartta OperasyonKodu-OperasyonAdi olarak gösterilir. **Notlar:** Frm_Load'da _mng = new UretimIstasyonTakipManager(Ortak.DbPro) . Kart sayısı 9'a tamamlanır (boş kartlar eklenir). FromAc.IstasyonTakip() ile MDI child olarak açılır. Manager'da kullanılmayan varyantlar: GetTakipListUretimKodlu() (IsEmri/Siparis kodu kiralimli).

Istasyon Card Control (IstasyonCardControl.cs / .Designer.cs)

Ne ise yarar: FrmUretimIstasyonTakip içindeki tek bir istasyon kartını render eden UserControl. UretimIstasyonTakipModel verisini etiketlere basar (istasyon kodu/adi, operasyon, fason, planlanan/üretim/fire miktarı, fason cari kodu/unvani). **Once ne olmalı (onkosul):** Model property'si bir UretimIstasyonTakipModel ile set edilmiş olmalı; null ise tüm etiketler gizlenir (boş placeholder kart). **Sonra ne olur:** Salt gösterim. Buyutec ikonuna tıklayınca Ortak.IstasyonKontrolDetayAc(IstasyonKodu, Fason) çağırılır -> FrmUretimIstasyonTakipDetay açılır. **Butonlar & kısayollar:** - Buyutec ikonu (pictureBox2) — Click: detay ekranını açar; MouseHover: arka plan Maroon; MouseLeave: transparent. **Çağırdığı katmanlar:** - Manager/Service: - (veri dışarıdan Model ile gelir). - SQL/Prosedur: - - API: - **Istasyon sırasıyla ilişkisi:** -

Notlar: Namespace `MyUI.MyControl` (dosya `UretimIstasyonModule` klasorunde). Fason etiketi true->"Evet", false->"Hayir".

Istasyon Takip Detay (`FrmUretimIstasyonTakipDetay.cs` / `.Designer.cs`)

Ne ise yarar: Tek bir istasyon kodu (ve fason ayrımı) icindeki tum acik istasyon hareketlerini grid olarak listeler. Siparis/IsEmri kodu, operasyon, recete, planlanan/uretim/fire/iptal miktarlari ve baslangic tarihiyle satir bazinda detay verir. **Once ne olmalı (onkosul):** `IstasyonKodu` ve `Fason` (0/1) property'leri set edilmiş olmalı (`FrmUretimIstasyonTakip` kartından `Ortak.IstasyonKontrolDetayAc` ile gelir). **Sonra ne olur:** Salt okunur grid; veritabanini degistirmez. **Butonlar & kisayollar:** - `MyFrmSadeFull` alt bar butonlari (Yazdir/Kopyala/Kapat) - bu formda ozel buton/handler yok; sadece grid gosterimi. **Agirdigi katmanlar:** - Manager/Service: `UretimIstasyonTakipManager.GetTakipDetayList(IstasyonKodu, Fason)` — Url + UrO + Siparis JOIN, `UrO.KalanMiktar > 0 AND UrI.IstasyonKodu = @kodu AND Coalesce(UrI.Fason,0) = @fason` filtreli detay SELECT. - SQL/Prosedur: Sadece SELECT. - API: - **Istasyon sirasiyla iliskisi:** Detayda her satir bir `UretimIstasyon` hareketidir; operasyon/recete bilgisi gosterilir. **Notlar:** BaslangicTarihi sutunu "dd.MM.yyyy HH:mm" formatlanir. Form basligi "IstasyonKodu : " olur. `SutunGizle()` bos.

Istasyon Hareket Listesi (`FrmUretimIstasyonHareketList.cs` / `.Designer.cs`)

Ne ise yarar: Tum `UretimIstasyon` (Url) kayitlarini ust grid'de, secili istasyonun istasyon-hareketlerini (`UretimIstasyonHareket`) alt grid'de gosteren master-detail liste. Istasyon kodu, recete adi ve tarih araliklarina gore filtrelenir. Ayrica baska ekranlardan "istasyon secimi" icin secim modunda (`SecimIcinAcildi`) acilir. **Once ne olmalı (onkosul):** `UretimIstasyon` kayitlari olusmus olmalı. Secim modunda (`FrmUretimIstasyonUretimGir`'den) calismasi icin `SecimIcinAcildi=true` set edilir. **Sonra ne olur:** Liste modunda salt okunur (satira cift tiklayınca eski `FrmUretimEmriED_V2` kodu commentli, islem yok). Secim modunda secilen `UretimIstasyon` `SecilenRow` olarak doner ve form kapanir. **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — Filtreleri uygular (`Bagla()`). - `Temizle` (`BtnTemizle`) — Filtre alanlarini sifirlar (`CmbIstasyonKodu`, `CmbReceteAdi`, 4 tarih). - Grid cift tik / Enter (`myView1.MyEventDoubleClickEnter`) — Secim modunda secip kapatir. - Filtreler: `CmbIstasyonKodu`, `CmbReceteAdi`, `TxtTarihi1-4` (baslangic $\geq$ / $\leq$ ve bitis $\geq$ / $\leq$ tarih araliklari). **Agirdigi katmanlar:** - Manager/Service: `IUretimIstasyonService.GetViewListWhere(where)` (`Ortak.DbPro.UretimIstasyon`) — Url view'i (UrO ile JOIN'li) where ile ceker. `IUretimIstasyonHareketService.GetViewListWhere(where)` — secili istasyonun Url'd'sine gore hareketler. `IGenelService.GrupListesi("UretimIstasyon", "IstasyonKodu")` ve `("UretimEmri", "ReceteAdi")` — filtre combobox'larini doldurur. - SQL/Prosedur: Sadece SELECT (where 1=1 + dinamik tarih/kod filtreleri). - API: - **Istasyon sirasiyla iliskisi:** Master grid Url, detail grid o istasyonun hareketleri; operasyon Sira ile dogrudan kullanim yok, tarih bazli filtre var. **Notlar:** `Frm_Load`'da `TxtTarihi1` = bugun-1 ay (varsayilan). SQL string birlestirme ile kurulur (parametresiz). `Frm_Load` icinde `BtnTemizle.Click` iki kez baglanir (kod tekrari). `MyFrmListe` turevi.

Istasyon Hareket Detay Listesi (`FrmUretimIstasyonHareketDetayList.cs` / `.Designer.cs`)

Ne ise yarar: Tum `UretimIstasyonHareket` (uretim/fire giris) kayitlarini tek bir grid'de, istasyon kodu / recete adi / tarih araligina gore listeler. Bu hareketlere yeni kayit eklemenin ve mevcut kaydi acmanin giris noktasidir. **Once ne olmalı (onkosul):** En az bir `UretimIstasyon` hareketi (uretim girisi) yapilmis olmalı; ekleme icin once bir istasyon secilebilir durumda olmalı. **Sonra ne olur:** Satira cift tik/Enter veya Ekle ile `FrmUretimIstasyonUretimGir` acilir; orada kaydet/sil yapilınca `Action=Bagla` ile bu liste yenilenir. `UretimGir` kaydi `Uretim_MiktarGuncelle` + `Uretim_PlanlananGuncelle` prosedurlerini tetikler (asagidaki ekrana bakiniz). **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — Filtreleri uygular (`Bagla()`). - `Temizle` (`BtnTemizle`) — Filtre alanlarini sifirlar. - `Ekle` (`BtnEkle`) — Bos `FrmUretimIstasyonUretimGir` acar (yeni uretim girisi). - Grid cift tik / Enter — Secili `UretimIstasyonHareket`'i `FrmUretimIstasyonUretimGir` ile duzenlemeye acar (`IdGuid = itm.Id`). -

Filtreler: CmbIstasyonKodu, CmbReceteAdi, TxtTarihi1/TxtTarihi2 (hareket tarihi UrlH.Tarih araligi). **Çagirdigi katmanlar:** - Manager/Service: `IUretimIstasyonHareketService.GetViewListWhere(where)` (Ortak.DbPro.UretimIstasyonHareket) — UrlH view'i (Url/UrO JOIN'li). `IGenelService.GrupListesi(...)` ile filtre listeleri. - SQL/Prosedur: Sadece SELECT (kayıt ismi UretimGir ekranında). - API: - **Istasyon sirasiyla ilişkisi:** Hareketler istasyon/operasyon bilgisini tasir; Sira ile dogrudan filtre yok. **Notlar:** Frm_Load'da TxtTarihi1 = bugun-1 ay. MyFrmListe turevi.

Istasyon Uretim Kayıt Gir (`FrmUretimIstasyonUretimGir.cs` / `.Designer.cs`)

Ne ise yarar: Tek bir istasyon hareketine (UretimIstasyon) uretim miktarı ve fire miktarı girisini yapan ana kayıt ekranıdır. Üst kismda istasyon/operasyon/recete/planlanan/mevcut uretim bilgileri (readonly), alt kismda uretim miktarı, fire miktarı, tarih, personel kodu/adi girilir. Saha uretim girisinin masastu karsiligidir. **Önce ne olmalı (onkosul):** Bir UretimIstasyon kaydı secilmis olmalı. IdGuid bos ve Mdllst null ise ekran acilisinda `FrmUretimIstasyonHareketList` secim modunda acilir ve kullanıcı bir istasyon secer (secmezse form kapanir). IdGuid dolu ise mevcut hareket duzenlemeye acilir. **Sonra ne olur:** Kaydet ->

`UretimIstasyonManager.UretimIstasyonHareketKaydet(_mdl)` -> UretimIstasyonHareket tablosuna Insert/Update, ardından `exec Uretim_MiktarGuncelle <UrId>` ve `exec Uretim_PlanlananGuncelle <UrId>` calisir (miktarlar yukari toplanir ve operasyon Sira N -> N+1 planlamasi guncellenir). Basarili olursa Action?.Invoke() ile cagiran liste yenilenir ve form kapanir. Sil benzer sekilde Delete + ayni iki prosedur. **Butonlar & kisayollar:** - **Kaydet** (BtnKaydet) — `Kaydet()` : dogrulama (uretim miktarı >= 1), miktar kontrolu (planlanani asamaz), kayıt + prosedur tetikleme. - **Sil** (BtnSil) — Onay sorar, `Sil()` ile kaydi siler + prosedurleri tetikler. - **Kapat** (BtnKapat) — Formu kapatir (MyFrmKayit alt bar). - Personel alanlari: CmbPersonelKodu / CmbPersonelAdi (LookupEdit) — biri secilince diğeri otomatik dolar (Leave event).

Çagirdigi katmanlar: - Manager/Service:

`UretimIstasyonManager.UretimIstasyonHareketKaydet(UretimIstasyonHareket)` — InsertOrUpdate + iki prosedur. `UretimIstasyonManager.UretimIstasyonHareketSil(...)` — Delete + iki prosedur. `UretimIstasyonManager.GetUretimIstasyonHareket(id)` / `GetUretimIstasyon(id)` — duzenleme verisini ceker. `IPersonelService.SelectListWhere()` — personel lookup. `IGenelService` (genel). - SQL/Prosedur: `Uretim_MiktarGuncelle <UrId>` — UretimIstasyonHareket->UretimIstasyon->UrOHD->UrOH->UrO miktar toplamlarini yukari yazar. `Uretim_PlanlananGuncelle <UrId>` — operasyon Sira N uretimini N+1 planlanana tasir; sonunda `Uretim_SonrakiIstasyonaGonder` cagrilabilir. - API: - **Istasyon sirasiyla ilişkisi:** Bu girisin sonucu, Uretim_PlanlananGuncelle/SonrakistasyonaGonder araciligıyla operasyon Sira'sina gore bir sonraki operasyonun istasyonlariinin planlanana yansir. (SonrakistasyonaGonder yalnızca ReceteAna.IstasyonGruplamaKullan=1 iken RecetelstasyonGrupOperasyon eslemesiyle calisir.) **Notlar:** Miktar kontrolu: yeni kayitta uretim miktarı (PlanlananMiktar - UretimMiktarı)'ni asamaz; duzenlemede _editMiktar (onceki deger) hesaba katilir. Kayit yeni mi kontrolu icin Audit alanlari (KayitEden/Degistiren) atanir (mantik ters gibi gorunse de mevcut kod boyle). `FromAc` icinde parametresiz UretimGir acan bir launcher da var.

Uretim Istasyon Kayıt / ED (`FrmUretimIstasyonED.cs` / `.Designer.cs`)

Ne ise yarar: Bir operasyon hareketine (UretimOperasyonHareket) bagli istasyonlari TANIMLAMA/DEGISTIRME ve "sonraki operasyona gecirme" ekranıdır. Operasyonun bekleyen miktarini istasyonlara planlanan miktar olarak dagitir, fason istasyonlar icin cari secer, istasyon ekler/siler. Ucuncu sekme niteliginde uc mod ile calisir: IstasyonEkle, SonrakiOperasyon, Degistir (OperasyonTuruEnum). **Önce ne olmalı (onkosul):** Gecerli bir UretimOperasyonHareket (OprId) olmalı. Acilista `UretimEmriManager.GetUretimOperasyonSiparisEdit(OprId)` ile siparis/operasyon modeli (UretimEmriKayitModelSiparis) yuklenir ve `IUretimOperasyonHareketService.GetViewListWhere(UrOH.Id=OprId)` ile Hareket bulunur. Degistir modunda IdGuid (UrOHD) gerekir; SonrakiOperasyon modunda bir onceki operasyonun bilgisinden Sira+1 operasyonu

bulunur. **Sonra ne olur:** Kaydet -> mod'a gore `UretimOperasyonManager.KaydetNormal(...)` veya `KaydetSonraki(...)`. KaydetNormal: UretimOperasyonHareketDetay Insert/Update + o detaya bagli UretimIstasyon kayitlarini sil/yeniden ekle + silinen istasyonlari IstasyonTakupHareket/Detay/Log/StokHareket kayitlarini temizle + `Uretim_MiktarGuncelle` + `Uretim_PlanlananGuncelle` (transaction icinde). KaydetSonraki: yeni UretimOperasyonHareket Insert + Detay + istasyonlari yeni harekete bagla + eski istasyonlari sil + `Uretim_MiktarGuncelle`. Sonrasinda Action?.Invoke() ile cagiran ekran yenilenir, form kapanir.

Butonlar & kisayollar: - `Kaydet` (BtnKaydet) — `Kaydet()` : istasyonlari toplar (IstasyonlarAl), planlanan/uretim/fason cari dogrulamalari, miktar tavan kontrolu, mod'a gore KaydetNormal/KaydetSonraki. - OperasyonIstasyonControlV2 ic butonlari (asagidaki bilesen): Fason Sec / Fason Temizle / Istasyon Ekle / Istasyon Sil. - MyFrmKayitFull alt bar: Kapat/Yazdir/navigasyon (Ilk/Onceki/Sonraki/Son) - bu formda nav butonlari pasif gorunum. **Cagirdigi katmanlar:** - Manager/Service: `UretimOperasyonManager.KaydetNormal(ist, Detay, silinenler)` / `KaydetSonraki(ist, Detay, HareketYeni)` — transaction'li kayit + prosedur: `UretimTakipManagerV2.GetSiparisKalanKontrol(UrId)` — sonraki operasyon planlanan miktar hesaplamasi icin. `UretimEmriManager.GetUretimOperasyonSiparisEdit(OprId)` — model. `IUretimIstasyonService`, `IUretimOperasyonHareketService`, `IUretimOperasyonHareketDetayService`, `IUretimOperasyonService` (SelectFirst/GetViewListWhere). Istasyon secimi icin `FrmIstasyonKartList` (BagliIstasyonSec, RcAld). - SQL/Prosedur: `Uretim_MiktarGuncelle <UrId>`, `Uretim_PlanlananGuncelle <UrId>` (KaydetNormal); `Uretim_MiktarGuncelle <UrId>` (KaydetSonraki). Silinen istasyonlarda IstasyonTakupHareket / IstasyonTakupHareketDetay / IstasyonTakupHareketLog / IstasyonTakupStokHareket Delete. - API: - **Istasyon sirasiyla iliskisi:** Cekirdek ekran. IstasyonlarOlustur, recete modelindeki operasyon (RcOld) ve ona bagli istasyonlari (RcIstId) eslestirerek operasyon icin istasyon hareketleri uretir. SonrakiOperasyon modu, OncekiOperasyon.Sira+1 operasyonu bulup once uretileni-operasyondaolani fark olarak yeni operasyonun PlanlananMiktar'ina yazar (operasyon Sira mantiginin UI tarafindaki yansimasi). **Notlar:** IstasyonlarAl: IstDurumu dolu (islem gormus) istasyonda planlanan miktar degistirilemez; planlanan < uretim/fire olamaz. Fason istasyonda FasonCariKodu zorunlu. Kaydedilecek istasyon yoksa onay sorar (uretim beklemeye alinir). MyFrmKayitFull turevi; SizeChanged ile ic control yeniden boyutlanir.

Operasyon Istasyon Control V2 (`OperasyonIstasyonControlV2.cs` / `.Designer.cs`)

Ne ise yarar: FrmUretimIstasyonED icinde calisan, bir operasyona ait istasyon hareketlerini duzenlenebilir grid ile gosteren UserControl. Ustte recete kodu/adi, operasyon kodu, miktar; gridde her istasyonun PlanlananMiktar, Fason, Baslangic/Bitis tarihi, FasonCariKodu/Unvani duzenlenir. **Once ne olmalı (onkosul):** FrmUretimIstasyonED tarafından property'leri (RcOld, ReceteKodu, ReceteAdi, Operasyon, Miktar, oprKayMdl, IstasyonHareketler) set edilip `Bagla()` cagirlmis olmalı. **Sonra ne olur:** Grid üzerindeki degisiklikler IstasyonHareketler listesine yansir; FrmUretimIstasyonED.Kaydet bunlari okur. Fason Sec ile secilen Mikro cari satira yazilir. **Butonlar & kisayollar:** - `Fason Sec` (BtnCariEkle) — `FrmMikroCariListesi` secim modunda acar; secilen cariye satira yazar, Fason=true yapar. - `Fason Temizle` (BtnCariTemizle) — Satirin fason cari bilgisini temizler, Fason=false. - `Istasyon Ekle` (BtnIstasyonEkle) — MyIstasyonEkleEvent tetikler; FrmUretimIstasyonED bunu yakalayip `FrmIstasyonKartList` ile yeni istasyon ekler. - `Istasyon Sil` (BtnIstasyonSil) — Uretim girilmis satir silinemez (uyari); aksi halde MyIstasyonSilEvent tetikler, FrmUretimIstasyonED satiri listeden cikarir. **Cagirdigi katmanlar:** - Manager/Service: Dogrudan yok; veri/eventler FrmUretimIstasyonED üzerinden yonetilir. Fason cari secimi icin `FrmMikroCariListesi` (Mikro DB). - SQL/Prosedur: - - API: - **Istasyon sirasiyla iliskisi:** Grid satirlari bir operasyonun istasyonlaridir; eklenen istasyon o operasyona (RcOld) baglanir. **Notlar:** Namespace `MyUI.UretimOperasyonModule` (dosya UretimIstasyonModule klasorunde). PlanlananMiktar/Fason/Baslangic/Bitis kolonlari yesil baslikli ve editable; FasonCari kolonlari buton edit (ColBtnCariKodu/ColBtnCariUnvani).

Uretim Istasyon Cari Sec (FrmUretimIstasyonCariSec.cs / .Designer.cs)

Ne ise yarar: Bir istasyona tanimli recete istasyon carilerini (RecetelIstasyonCari) listeleyip secim yaptiran basit secim ekranidir. **Once ne olmalı (onkosul):** Cariler (List) disaridan set edilmiş olmalı. **Sonra ne olur:** Satira cift tik/Enter ile secilen cari SecilenRow / SecilenKod (CariKodu) olarak doner, form kapanir. **Butonlar & kisayollar:** - Grid cift tik / Enter (myView1.MyEventDoubleClickEnter) — Secip kapatir. **Cagirdigi katmanlar:** - Manager/Service: - (veri Cariler property'siyle gelir). - SQL/Prosedur: - - API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** MyFrmSadeFull turevi. Id/RcAld/RcOld/RclstId sutunlari gizli.

Uretim Istasyon Hareket Sec (FrmUretimIstasyonHareketSec.cs / .Designer.cs)

Ne ise yarar: UretimIstasyon kayitlarini ya bir operasyon hareketi detayina (UrOHDId) ya da bir operasyon koduna (OperasyonKodu) gore listeleyip secim yaptiran secim ekranidir. **Once ne olmalı (onkosul):** UrOHDId (varsayilan mod) veya OperasyondanBagla=true + OperasyonKodu set edilmiş olmalı. Secim modu icin SecimIcinAcildi=true. **Sonra ne olur:** Satir secilince (cift tik/Enter veya Kaydet) secilen UretimIstasyon SecilenRow , SecilenKod =IstasyonKodu olarak doner ve form kapanir. **Butonlar & kisayollar:** - Kaydet (BtnKaydet) — Secili satiri secip kapatir. - Grid cift tik / Enter (myView1.MyEventDoubleClickEnter -> MyView1_MyEditAc) — Secip kapatir. **Cagirdigi katmanlar:** - Manager/Service: IUretimIstasyonService.GetViewListWhere(" where UrI.UrOHDId='...'") (Bagla) veya (" where UrO.OperasyonKodu='...'") (BaglaOperasyondan). Ortak.DbMikro ve IGenelService referanslari tutulur ancak bu formda aktif kullanim sade. - SQL/Prosedur: Sadece SELECT. - API: - **Istasyon sirasiyla iliskisi:** Operasyon koduna gore filtre secenegi ile operasyon-istasyon iliskisini gosterir. **Notlar:** MyFrmSade turevi. RowIndicator'da satir numarasi gosterimi (gridView1_CustomDrawRowIndicator). IndicatorWidth=30.

Modul: UretimOperasyonModule

Bu modul, bir üretim emrinin OPERASYON katmanını (recetedeki operasyon adımlarını) izlemek/raporlamak içindir. Veri modeli üç seviyelidir: **UretimOperasyon** (operasyon adımları, Sıra bazlı) -> **UretimOperasyonHareket** (operasyonun sipariş-hattı/parti bazlı toplam miktarları) -> **UretimOperasyonHareketDetay** (her üretim girişi/istasyon hareketi detayı). Modul cogenlukla READ-ONLY listeleme/raporlama yapar; yazma (miktar/akış) ise TabletV2 saha akışı ve **Uretim_MiktarGuncelle** / **Uretim_PlanlananGuncelle** prosedurleriyle gerçekleşir. Modulün tek yazma noktası **FrmUretimOperasyonList** içindeki "Durum Guncelle" sağ-tık komutudur (**Uretim_DurumGuncelle** prosedurunu çağırır). Modul ayrıca panodaki (**FrmOperasyonTakip**) operasyon kartlarını oluşturan iki UserControl içerir: **OperasyonCardControlV2** ve **OperasyonCardRowControlV2**.

Tüm Frm* formları **My.Kontrol.Formlar.MyFrmListe** taban sınıfından türer. Taban sınıf standart liste iskeletini sağlar: sol "Ara" sekmesi (**tabAra1**), üç grid (**myGrid1/2/3**), alt buton seridi (**BtnAra**, **BtnTemizle**, **BtnKapat**, **BtnYazdır**, **BtnDizayn**) ve seçim modu altyapısı (**SecimIcinAcildi**, **Secildi**, **SecilenRow**). Liste veri kaynağı her zaman entity'nin **GetSelectSqlCode(where)** ile üretilen view-SQL'idir; servisler sadece **GetViewListWhere(where)** çağırır (Dapper okuma). Tarih filtreleri **TxtTarihi1..4** ile (baş./bitiş alt-üst) kurulur, kod filtresi serbest metindir; bu yüzden where-cümleleri string birleştirme ile oluşur (parametrelili değil).

Operasyonlar Listesi (**FrmUretimOperasyonList.cs** / **FrmUretimOperasyonList.Designer.cs**)

Ne ise yarar: Üretim operasyon adımlarını (UretimOperasyon) üst grid'de listeler; seçilen operasyonun hareketlerini (UretimOperasyonHareket) orta grid'de, seçilen hareketin istasyon-detaylarını (UretimOperasyonHareketDetay) alt grid'de master-detail-detail olarak gösterir. Operasyon/recete/durum/tarih filtreleri ve durum-hızlı-filtre butonları (Hepsi/Beklemede/Uretimde/Hazır) içerir. **Once ne olmalı (onkosul):** Bir sipariş/is emri için üretim emri ve operasyonları oluşturulmuş olmalı (**UretimEmriManager** ile; **Uretim_PlanlananGuncelle** Sıra=1 için **UretimOperasyonHareket** satırlarını otomatik üretir). Siparisten gelindiye **SiparisKodundanFiltrele=true** ve **SiparisKodu** doludur. **Sonra ne olur:** Cogenlukla salt okuma. Tek yazma: üst grid satırında sağ-tık "Durum Guncelle" -> **Uretim_DurumGuncelle** prosedurunu çalıştırır; bu prosedür **UretimOperasyonHareket.KalanMiktar** ve **UretimOperasyon.KalanMiktar** alanlarını yeniden hesaplar, ardından **UretimOperasyon.Durumu** (Beklemede/Uretimde/Hazır), **UretimEmri.Durumu** ve **Siparis.Durumu** alanlarını günceller, ayrıca miktarı sıfır kalan **UretimOperasyonHareketDetay** satırlarını siler. Alt grid'de bir detaya çift tıklayınca **FrmUretimIstasyonED** (Degistir modu) açılır. **Butonlar & kısayollar:** - **Ara** (BtnAra) — filtrelerle yeniden sorgular (**Bagla()**); Frm_Load sonunda **BtnAra.PerformClick()** ile otomatik tetiklenir (sayfa açılır açılmaz dolar). - **Temizle** (BtnTemizle) — CmbDurumu/CmbReceteAdi ve TxtTarihi1..4 alanlarını boşaltır. - **Hepsi / Beklemede / Uretimde / Hazır** (BtnDurumuHepsi/Beklemede/Uretimde/Hazır) — durum hızlı filtresi; **DurumuAyarla(...)** ile **durumu** değişkenini set edip ilgili butonu renklendirir ve **BtnAra.PerformClick()** yapar. - **Sağ-tık > Durum Guncelle** (ContextMenu ogesi "DurumGuncelle") — seçili operasyonun **UrId** 'si için **UretimEmriManager.UretimDurumGuncelle()** çağırır, başarılıysa "Guncellendi." mesajı verir. - Üst grid çift-tık/Enter (**MyView1_MyEventDoubleClickEnter**) — seçim modunda (**SecimIcinAcildi**) satırı seçip formu kapatır; normal modda etkin değil (eski FrmUretimEmriED_V2 çağırışı yorum satırı). - **Kapat / Yazdır / Dizayn** (BtnKapat/BtnYazdır/BtnDizayn) — MyFrmListe taban davranışı (kapatma, grid yazdırma, grid dizayn/yerleşim). **Çağırdığı katmanlar:** - Service: **IUretimOperasyonService.GetViewListWhere(where)** — **select Ur0.* from UretimOperasyon Ur0 left outer join Siparis Sip on Sip.Id=Ur0.SipId where ...** (üst grid). - Service: **IUretimOperasyonHareketService.GetViewListWhere("where Ur0Id='...'")** — seçili operasyonun hareketleri

(orta grid). - Service: `IUretimOperasyonHareketDetayService.GetViewListWhere("where UrOHId='...')"` — secili hareketin detaylari (alt grid). - Service: `IGenelService.GrupListesi("UretimOperasyon", <kolon>)` — Durumu/OperasyonKodu/ReceteAdi combobox kaynaklari (distinct deger listesi). - Manager: `UretimEmriManager.UretimDurumGuncelle(Guid? urid)` — transaction icinde `exec [Uretim_DurumGuncelle] '<urid>'` calistirir. - SQL/Prosedur: `Uretim_DurumGuncelle` — KalanMiktar yeniden hesabi + UretimOperasyon/UretimEmri/Siparis.Durumu guncellemesi + sifir miktarli detay silme. - API: - **Istasyon sirasiyla iliskisi:** Operasyon `Sira`alani akisin omurgasidir. `Uretim_PlanlananGuncelle` (bu formdan dogrudan cagrilmaz ama veri bu mantikla olusur) `Sira=1` operasyonu icin hareket olusturur ve bir operasyonun uretim miktarini `Sira N+1` operasyonun planlananina tasir. Alt grid detayina cift-tik ile acilan `FrmUretimIstasyonED` operasyon-detayinin istasyona giden islemini gosterir/degistirir (myLabel5 notu: "Istasyona Giden Islemi Degistirmek Icin Operasyon Detaya Cift Tiklayin"). **Notlar:** Durum combobox'i ve etiketi gizli (`CmbDurumu.Visible=false` , `myLabel4.Visible=false`); durum filtresi sadece hizli-filtre butonlariyla yapilir. `FocusedRowChanged` ile ust grid satiri degisince orta/alt grid'ler yeniden baglanir. Where filtreleri parametresiz string birlestirmesidir (SQL injection acisindan dikkat). `SiparisFiltreAyarla()` siparis kodunu `TxtKodu` 'ya yazip baslangic tarihini yilin 1 Ocak'ina cekeer.

Operasyon Hareketler Listesi (`FrmUretimOperasyonHareketList.cs` /

`FrmUretimOperasyonHareketList.Designer.cs`)

Ne ise yarar: `UretimOperasyonHareket` kayitlarini (operasyon hareketlerinin toplam miktarlari: Planlanan/Islemdeki/Uretim/Fire/Iptal/Kalan) tek bir duz listede gosteren rapor ekranidir. Operasyon/durum/recete ve baslangic/bitis tarihi filtreleri icerir. **Once ne olmalı (onkosul):** En az bir uretim emri ve operasyonu olusturulmus, dolayisiyla `UretimOperasyonHareket` satirlari uretilmis olmalı. **Sonra ne olur:** Salt okuma raporu; herhangi bir tablo/prosedur degistirmez. Cift-tik/Enter sadece secim modunda satiri secip formu kapatir (normal modda is yapmaz). **Butonlar & kisayollar:** - `Ara` (BtnAra) — filtrelerle `Bagla()` calistirir. - `Temizle` (BtnTemizle) — combobox ve tarih alanlarini temizler. - Grid cift-tik/Enter (`MyView1_MyEventDoubleClickEnter`) — secim modunda satiri dondurur; normal modda pasif. - `Kapat / Yazdir / Dizayn` — MyFrmListe taban davranisi. **Cagirdigi katmanlar:** - Service: `IUretimOperasyonHareketService.GetViewListWhere(where)` — `select UrOH.*, UrO.IsEmriNo, UrO.OperasyonKodu, UrO.OperasyonAdi, UrO.ReceteKodu, UrO.ReceteAdi from UretimOperasyonHareket UrOH left outer join UretimOperasyon UrO on UrO.Id=UrOH.UrOHId where ...` . - Service: `IGenelService.GrupListesi("UretimOperasyon", <kolon>)` — Durumu/OperasyonKodu/ReceteAdi combobox kaynaklari. - Manager: - - SQL/Prosedur: - - API: - **Istasyon sirasiyla iliskisi:** Her hareket satiri bir operasyonun `Sira` 'sina baglidir (where filtreleri `UrO.OperasyonKodu` uzerinden, tarih filtreleri `UrOH.BaslangicTarihi/BitisTarihi` uzerinden). Akis motoru (`Uretim_MiktarGuncelle/PlanlananGuncelle`) bu satirlarin miktar alanlarini gunceller. **Notlar:** `BtnTemizle.Click += BtnTemizle_Click;` iki kez baglanmis (kucuk bir kod tekrari; islevsel etkisi temizleme isleminin iki kez calismasidir, sonuc ayni). Filtre where'leri parametresiz string birlestirmesidir.

Operasyon Hareket Detay Listesi (`FrmUretimOperasyonHareketDetayList.cs` /

`FrmUretimOperasyonHareketDetayList.Designer.cs`)

Ne ise yarar: `UretimOperasyonHareketDetay` kayitlarini (her uretim girisinin detayi: tarih, miktarlar) ust grid'de; secilen detaya ait `UretimIstasyon` kayitlarini alt grid'de master-detail gosterir. Iki modda calisir: (1) normal liste/filtreli, (2) belirli bir hareketin detaylarini gosteren odakli mod (`DetayGoster=true` + `DetayId`). **Once ne olmalı (onkosul):** Operasyon hareketleri uzerinde uretim girisi yapilmis olmalı (detay/istasyon kayitlari olusmus

olmalı). Odakli modda cagiran ekran `DetayGoster=true` ve `DetayId` 'yi (UrOH.Id) set etmelidir. **Sonra ne olur:** Salt okuma. Ust grid'de bir detaya cift-tik/Enter -> `FrmUretimIstasyonED` (OperasyonTuru=Degistir, IdGuid=detay.Id, Oprld=detay.UrOHId) acilir; istasyon hareketi orada degistirilir (yazma o formda gerceklesir, bu form degil). **Butonlar & kisayollar:** - `Ara` (BtnAra) — `Bagla()`; `DetayGoster=true` ise filtreleri yok sayar ve sadece `DetayId` icin sorgular. - `Temizle` (BtnTemizle) — combobox ve `TxtTarihi1/2` alanlarini temizler. - Ust grid cift-tik/Enter (`MyView1_MyEventDoubleClickEnter`) — secim modunda satiri secip kapatir; normal modda `FrmUretimIstasyonED` (Degistir) acar. - `Kapat / Yazdir / Dizayn` — MyFrmListe taban davranisi. **Cagirdigi katmanlar:** - Service: `IUretimOperasyonHareketDetayService.GetViewListWhere(where)` — `select UrOHD.*, UrO.IsEmriNo, UrO.OperasyonKodu, UrO.OperasyonAdi, UrO.ReceteKodu, UrO.ReceteAdi from UretimOperasyonHareketDetay UrOHD left outer join UretimOperasyonHareket UrOH on UrOH.Id=UrOHD.UrOHId left outer join UretimOperasyon UrO on UrO.Id=UrOHD.UrOId where ...`. Odakli modda where = `UrOH.Id=<DetayId>`. - Service: `IUretimIstasyonService.GetViewListWhere("where UrOHDId='...'")` — secili detayin istasyon kayitlari (alt grid). - Service: `IGenelService.GrupListesi("UretimOperasyon", <kolon>)` — combobox kaynaklari. - Manager: - - SQL/Prosedur: - - API: - **Istasyon sirasiyla iliskisi:** Bu ekran operasyon-detayi ile UretimIstasyon arasindaki koprudur; alt grid `UrOHDId` uzerinden istasyon kayitlarini cozer. Cift-tik ile acilan `FrmUretimIstasyonED`, detayin hangi istasyona gittigini/islendigini gosterir. **Notlar:** `BtnTemizle.Click += BtnTemizle_Click;` yine iki kez baglanmis. Sadece iki tarih filtresi var (`UrOHD.Tarih` alt/ust); diger formlardaki dort tarihten farklidir.

OperasyonCardControlV2 (`OperasyonCardControlV2.cs` / `OperasyonCardControlV2.Designer.cs`)

Ne ise yarar: Operasyon takip panosunda (`FrmOperasyonTakip`) tek bir OPERASYON icin kart gosteren UserControl'dur. Iki kolonlu (Bekleyen / Uretimde) bir kart cizer; her kolona o operasyonun is satirlarini (`OperasyonCardRowControlV2`) `FlowLayoutPanel` icine doldurur. Saglik tik menuleriyle detay/uretim girisi ekranlarina hizli gecis saglar. **Once ne olmalı (onkosul):** Cagiran (`FrmOperasyonTakip`) `OperasyonKodu`, `OperasyonAdi`, `Hareketler` (List) ve `Action` (yenile geri cagrisi) property'lerini set edip kontrolu panoya eklemeli. Hareket modelleri `UretimTakipManagerV2` benzeri bir kaynaktan dolu gelir. **Sonra ne olur:** `Bagla()` her hareket icin: `PlanlananMiktar - IslemdekiMiktar > 0` ise "Beklemede" satiri, `IslemdekiMiktar > 0` ise "Uretimde" satiri uretir. Satir cift-tiklaninca ilgili detay penceresi (`FrmOperasyonTakipDetaylar`) acilir. Menu komutlari ilgili liste/giris formlarini acar. **Butonlar & kisayollar (sag-tik ContextMenu ogeleri):** - Bekleyen panel `Detaylar > Istasyona Gonder` (detaylarToolStripMenuItem) — `BekleyenDetaylarAc()` -> `Ortak.UretimTakipBekleyenAcV2(OperasyonKodu)` -> `FrmOperasyonTakipDetaylar` (Durumu="Beklemede"). - Bekleyen panel `Operasyon Hareket Detaylari` (operasyonDetaylariToolStripMenuItem) — `FrmUretimOperasyonHareketDetayList` (ShowDialog). - Uretimde panel `Direk > Uretim Giris` (uretimGirisToolStripMenuItem) — once `FrmUretimIstasyonHareketSec` (secim) sonra secilen istasyon icin `FrmUretimIstasyonUretimGir` acar. - Uretimde panel `Detaylar > Uretim Giris` (toolStripMenuItem1) — `UretimdeDetaylarAc()` -> `Ortak.UretimTakipUretimdeAcV2(OperasyonKodu, Action)` -> `FrmOperasyonTakipDetaylar` (Durumu="Uretimde", yenile Action ile). - Uretimde panel `Istasyon Hareketler/Uretimler` (uretimlerToolStripMenuItem) — `FormAc.FormSec("IstasyonDetaylar")` -> `FrmUretimIstasyonHareketDetayList`. - Uretimde panel `Operasyon Hareket Detaylari` (operasyonDetaylariToolStripMenuItem1) — `FrmUretimOperasyonHareketDetayList` (ShowDialog). - Satir cift-tik (kart satirinda) — `OperasyonCardRowControlV2` icindeki `Action` (BekleyenDetaylarAc / UretimdeDetaylarAc) calisir. **Cagirdigi katmanlar:** - Helper: `Ortak.UretimTakipBekleyenAcV2 / UretimTakipUretimdeAcV2` — `FrmOperasyonTakipDetaylar` 'i ilgili durumla acar. - Helper: `FormAc.FormSec("IstasyonDetaylar")` — istasyon hareket detay listesini acar. - Form: `FrmUretimIstasyonHareketSec`, `FrmUretimIstasyonUretimGir`, `FrmUretimOperasyonHareketDetayList`. -

Service/Manager: dogrudan yok (veri disaridan **Hareketler** ile gelir). - SQL/Prosedur: - - API: - **Istasyon sirasiyla iliskisi:** Kart "Uretim Girişi" akisi saha uretim girisini tetikler (FrmUretimIstasyonUretimGir); uretim girişi sonrasında akis motoru (Uretim_MiktarGuncelle/Uretim_PlanlananGuncelle) miktarlari Sira boyunca yukari/ileri tasir. Kart panoyu **Action** (Bagla yenile) ile guncel tutar. **Notlar:** Bekleyen miktar = PlanlananMiktar - IslemdekiMiktar; Uretimde miktar = IslemdekiMiktar - UretimMiktari (satir kontrolunde hesaplanir). Renkler durum vurgusu icindir (Bekleyen lacivert, Uretimde yesil).

OperasyonCardRowControlV2 (**OperasyonCardRowControlV2.cs** / **OperasyonCardRowControlV2.Designer.cs**)

Ne ise yarar: **OperasyonCardControlV2** kartinin icindeki tek bir is-satirini (Recete adi, Miktar, Siparis Kodu, Cari Unvani) gosteren kucuk UserControl'dur. Tiklaninca secili duruma gecer (tek-secim mantigi), cift-tiklaninca kartin verdigi **Action** 'i (detay penceresi) calistirir. **Once ne olmalı (onkosul):** **OperasyonCardControlV2.Bagla()** tarafindan **Model** (UretimTakipModelV2, Turu="Beklemede"/"Uretimde" set edilmiş), **Id** ve **Action** verilerek olusturulur. **Sonra ne olur:** Yuklenince etiketleri doldurur ve **OperasyonCardManagerV2.SecildiEvent** 'e abone olur. Tiklaninca **Secildi()** -> statik **OperasyonCardManagerV2.SecildiTetikle(...)** ile diger satirlara "secilmedi" sinyali yayar ve kendini Teal renge boyar. Cift-tik -> **Action.Invoke()** (detay penceresi acar). **Butonlar & kisayollar:** - Etiketlere tek tik (LblRecete/LblMiktar/LblSiparisKodu/LblCariUnvani Click) — **Secildi()** (satiri sec/vurgula). - Etiketlere cift-tik (MouseDownDoubleClick) — **Action.Invoke()** (kart tarafindan verilen detay acma). - Enter (kontrol Enter eventi) — **LblRecete_Click** (secim) ile ayni davranis. **Cagirdigi katmanlar:** - Helper: **OperasyonCardManagerV2** (statik event yoneticisi) — **SecildiEvent** / **SecildiTetikle** ile satirlar arasi tek-secim koordinasyonu; **RowId** artan kimlik uretir. - Service/Manager: - - SQL/Prosedur: - - API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Modeli **UretimTakipModelV2** 'dir; Turu alanina gore "Uretimde" satirinda kalan = IslemdekiMiktar - UretimMiktari, "Beklemede" satirinda kalan = PlanlananMiktar - IslemdekiMiktar gosterilir.

Yardimci tipler (**OperasyonTuruEnums.cs** , **OperasyonCardRowEventArgs.cs**)

Ne ise yarar: Modulun yardimci tip tanimlaridir; ekran degildir. - **OperasyonTuruEnum** (SonrakiOperasyon, IstasyonEkle, Degistir) — **FrmUretimIstasyonED** acilirken islem turunu belirtmek icin kullanilir (detay cift-tik akislari **Degistir** gonderir). - **OperasyonCardRowEventArgsV2** + **OperasyonCardManagerV2** — kart satirlari arasi tek-secim icin EventArgs ve statik event-broker; **OperasyonCardRowControlV2** bunlari kullanir. **Once ne olmalı (onkosul):** - **Sonra ne olur:** - **Butonlar & kisayollar:** - **Cagirdigi katmanlar:** - **Istasyon sirasiyla iliskisi:** - **Notlar:** **OperasyonCardRowEventArgsV2** ve **OperasyonCardManagerV2** **MyUI.MyControl** namespace'inde (modul klasorunde olsalar da farkli namespace) tanimlidir.

Modul: OperasyonModul

OperasyonModul, üretim akisinin temel yapı taşı olan "operasyon" tanımlarının (örn. Kesim, Kaynak, Boya, Montaj) kart bazlı yönetimini sağlar. Klasörde tek bir ekran vardır: `FrmOperasyonKartlari`. Bu ekran operasyon kodu/adını ve her operasyona atanmış "varsayılan istasyon" bilgisini tutar. Operasyon kartları, reçete tarafında operasyon sırası (ReceteOperasyon) ve istasyon eşleştirme (ReceteIstasyon / ReceteIstasyonGrupOperasyon) kurulurken referans liste olarak kullanılır; dolayısıyla miktar/akış motorunun OPERASYON-Sıra bazlı çalışmasının sözlük/temel veri katmanıdır. Ekran iki moda çalışır: (1) ana menüden ShowDialog ile açılan klasik CRUD kartı, (2) reçete operasyon ekleme ekranından `SecimIcinAcildi=true` ile açılan seçim diyalogu.

Operasyon Kartları (`FrmOperasyonKartlari.cs` / `FrmOperasyonKartlari.Designer.cs`)

Ne ise yarar: Operasyon kartlarının (OperasyonKodu, OperasyonAdi ve VarsayilanIstasyonKodu/Adi) eklenmesi, düzenlenmesi, silinmesi ve listelenmesi. Ayrıca reçete operasyon editoründen çağrıldığında operasyon seçim diyalogu olarak görev yapar. **Once ne olmalı (onkosul):** İstasyon kartları önceden tanımlanmış olmalı; çünkü formun açılışında `IstasyonKarti` listesi çekilip "Varsayılan İstasyon Kodu/Adi" lookup'larına (CmbIstasyonKodu/CmbIstasyonAdi) bağlanıyor. Ekran ana menüden `FromAc.OperasyonKartlari()` ile `ShowDialog` olarak açılır. Veri tabanı bağlantısı (`Ortak.DbPro`) hazır olmalı. **Sonra ne olur:** - Kaydet: `OperasyonKarti` tablosuna kayıt eklenir/güncellenir (`InsertOrUpdate`). Eğer düzenleme sırasında OperasyonKodu değiştiyse aynı transaction içinde şu tablolarda eski kod yeni kodla (ve OperasyonAdi ile) toplu UPDATE edilir: `IstasyonKarti` (Operasyon, OperasyonAdi), `ReceteOperasyon`, `ReceteIstasyon`, `ReceteIstasyonGrupOperasyon`, `UretimOperasyon`, `IstasyonTakupHareket`. Liste yeniden yüklenir (`Bagla()`), text alanlar temizlenir. - Sil: Seçili `OperasyonKarti` satırı `OperasyonKarti` tablosundan silinir (`_srv.Delete`), liste yenilenir. - Seçim modunda (SecimIcinAcildi): satıra çift tıklayınca/Enter ile seçilen operasyon `SecilenRow/SecilenKod/SecilenId` ile çağırılan ekrana (örn. `FrmReceteOperasyonED`) döner ve form kapanır; orada ReceteOperasyon satırı olarak eklenir. - Ekran dışarı (kaydet/sil olduğunda) `KayitEdildi=true` set eder. **Butonlar & kısayollar:** - Kaydet (`BtnKaydet`) — `Kaydet()` çağırır: text doğrulaması (OperasyonKodu boş olamaz), modele aktarım, `OperasyonManager.Kaydet`. (Base form: alt buton serisinde, genelde Enter/F-tusu base davranışı.) - Yeni (`BtnYeni`) — `YeniKayit=true` yapar, text alanları temizler (`TemizleText`). - Düzenle (`BtnDuzenle`) — Grid'de seçili satırı (`MyGetCurrentItem<OperasyonKarti>`) klonlayıp text alanlara aktarır (`AktarTextlere`), `YeniKayit=false`. - Sil (`BtnSil`) — Onay sorar ("Kaydı silmek istiyormusunuz.."), seçili kayıt yoksa uyarır, sonra `Sil()`. - Kapat (`BtnKapat`) — Base form: formu kapatır. - Yazdır (`BtnYazdir`) — Base formdan gelir (bu ekranda özel kod bağlanmamış). - Navigasyon `BtnIlk` / `BtnOnceki` / `BtnSonraki` / `BtnSon` — Base form kayıt gezinme butonları (BindingSource `bs` üzerinde). - Grid çift tıklama / Enter (`myView1.MyEventDoubleClickEnter`) — Seçim modundaydı satırı seçip kapatır; değilse `BtnDuzenle.PerformClick()` tetikler. - Lookup `CmbIstasyonKodu/CmbIstasyonAdi` Leave — biri seçilince diğeri otomatik doldurulur (kod<->ad eşleme), `AcilisBittimi` bayrağıyla recursion engellenir. **Çağırdığı katmanlar:** - Manager/Service: `OperasyonManager.Kaydet(OperasyonKarti, bool yenikayit)` — yeni/güncelleme ayrımı yapar, `KodVarmi` ile aynı OperasyonKodu kontrolü yapar, transaction açar, `Service.InsertOrUpdate` ile ana kaydı yazar, kod değişikliğinde bağımlı 6 tabloyu UPDATE eder, commit eder. - Manager/Service: `OperasyonManager.KodVarmi<T>(...)` — tablo adı/anahtar kolonu reflection ile bulup `Select count(*)` ile aynı kodun varlığını denetler ("Aynı OperasyonKodu Kodla Kayıt Var"). - Manager/Service: `IOperasyonKartiService` (= `OperasyonKartiService : BaseService<OperasyonKarti>`) metotları: `SelectListWhere(" Order By OperasyonKodu")` (liste), `SelectFind(Id)` (eski kaydı bulma), `InsertOrUpdate`, `Delete`, `GetConnection`, `Execute`, `Query<int>`. - Manager/Service:

`IIstasyonKartiService.SelectListWhere(" Order By IstasyonKodu ")` — varsayilan istasyon lookup verisi. -
 Manager/Service: `IGeneIService ( _srvGeneI )` — enjekte edilmiş fakat bu ekranda aktif kullanım yok. -
 DataAccess: `OperasyonKartiDal : BaseDal<OperasyonKarti>` — `[Table("OperasyonKarti")]` POCO
 üzerinden Dapper CRUD. - SQL/Prosedur: Stored procedure çağrılmaz. Kod değişikliğinde inline UPDATE'ler
 (yukarıda listelenen 6 tablo). Sema: `OperasyonKartiCreates.OperasyonKartiCreate()` -> OperasyonKarti
 tablosu (OperasyonKodu varchar(50), OperasyonAdi varchar(150), VarsayilanIstasyonKodu varchar(50),
 VarsayilanIstasyonAdi varchar(150)). - API: - (UretimV4 doğrudan DB'ye bağlanır, API kullanmaz.) **Istasyon**
sirasiyla ilişkisi: Operasyon kartları, miktar/akis motorunun OPERASYON-Sira bazlı işleyişinde temel sözlüktür.
 Recetede operasyon sirası (ReceteOperasyon.Sira) ve operasyon->istasyon eşleştirmeleri (ReceteIstasyon,
 ReceteIstasyonGrupOperasyon) bu kartlardaki OperasyonKodu üzerine kurulur. Her operasyon için tanımlanan
 "VarsayilanIstasyonKodu", reçete operasyon ekleme sırasında otomatik istasyon eşleştirmesi önerisinde
 kullanılır (bkz. FrmReceteOperasyonED). OperasyonKodu değişince ReceteIstasyonGrupOperasyon ve
 UretimOperasyon dahil tüm akis tabloları güncellenerek sıra/akis butunluğu korunur. **Notlar:** -
 OperasyonManager.Kaydet'teki bağımlı tablo UPDATE'leri string interpolation ile kurulduğu için SQL injection
 ve OperasyonAdi/Kodu içindeki tek tırnak riski tasir (gerçek davranış; spekülasyon değil, koddan). - Kaydetme
 hatasında transaction `trs.Dispose()` ile bırakılıyor ancak bazı dallarda explicit `Rollback` çağrılmıyor
 (finally'de yine Dispose var). - Form base sınıfı `MyFrmKayit` (My.Kontrol.Formlar, harici DLL) — alt buton seridi,
 navigasyon, `bs` BindingSource,
`IdGuid/YeniKayit/KayitEdildi/AcilisBittimi/SecimIcinAcildi/Secildi/SecilenKod/SecilenRow/SecilenId`
 ve `MesajHata/MesajSor/MesajBilgi/MesajBilgi` mesaj yardımcılarını sağlar; bu ekranda
 Enter=Kaydet/Esc=Kapat gibi kısayollar Designer'da açıkça tanımlanmamış (base formdan gelir). -
`BtnDuzenle.Click` event'i `BtnDegistir_Click` metoduna bağlıdır (isim farkı sadece kozmetik). - Grid
 yerleşimi `myGrid1.MyGridKayitAdi = "OperasyonKartlariListesi"` adıyla saklanır/yüklenir; "Id" kolonu
 gizlenir (`SutunGizle`).

Modul: IstasyonModul

IstasyonModul, uretim akisinin temel yapi taslarindan biri olan **istasyon kartlarini** (uretim duraklari/operasyon noktaları), istasyonlara ait **bakim/ariza kayitlarini** ve saha tabletinde operatorun karsisina cikan **acilir aciklama listelerini** (baslatma hatasi, durdurma kodu, fire sebebi) yonetir. Tum veriler Uretim DB'sinde (UretimV3_FEZA) tutulur: **IstasyonKarti**, **IstasyonBakim**, **IstasyonBakimParca**, **IstasyonAciklama** tablolari. Istasyon kartlari bir **Operasyon** koduna baglanir; bu baglanti, recete-istasyon esleme (RecetelstasyonGrupOperasyon) ve uretim akisinin (UretimIstasyon olusturma) merkezindedir. Istasyon kart kodu degistirildiginde modul, kodu uretim zincirindeki tum bagli tablolara (Recetelstasyon, RecetelstasyonGrupOperasyon, UretimIstasyon, IstasyonTakipHareket, IstasyonKontrol) tek bir transaction icinde yayar. Modulde 6 form vardir: 3'u ana sayfa (Kart Listesi, Kart Kaydi, Bakim Listesi) ve 3'u yordimci/popup (Bakim Ekle, Bakim Parca Ekle, Aciklama Tanimlari). Formlar **My.Kontrol.Formlar** namespace'indeki base siniflardan turer: **MyFrmListe** (arama panelli liste ekranlari), **MyFrmKayit** (ust bilgi + grid'li kayit ekranlari, alt buton seridi Kaydet/Sil/Yeni/Duzenle/Kapat icerir) ve **MyFrmSade** (sade popup, Kaydet/Kapat). Grid'lerde cift tiklama/Enter ortak **MyEventDoubleClickEnter** olayini tetikler (**SecimIcinAcildi** ise satiri secip kapatir, degilse Duzenle'yi calistirir).

Istasyon Kart Listesi (**FrmIstasyonKartList.cs** / **FrmIstasyonKartList.Designer.cs**)

Ne ise yarar: Istasyon kartlarini operasyona gore filtreleyip listeleyen, esas olarak **secim diyalogu** olarak kullanimli liste ekrani. Recete-istasyon grubu operasyon eslestirme ve uretim istasyonu duzenleme ekranlarindan "istasyon sec" amaciyla acilir. **MyFrmListe** turevi (sol arama paneli + grid). **Once ne olmalı (onkosul):** Istasyon kartlari (**IstasyonKarti**) ve operasyon kartlari (**OperasyonKarti**) tanimli olmalı. Secim modunda acan ekran (orn. **FrmReceteIstasyonGrupOperasyonEslestir**, **FrmUretimIstasyonED**) **SecimIcinAcildi=true**, opsiyonel **Aranan** (operasyon kodu) ve **RcAId** (recete ana Id) atar. **Sonra ne olur:** Veri tabaninda DEGISIKLIK YAPMAZ (salt okuma/secim). Satira cift tiklaninca **SecilenKod=IstasyonKodu**, **SecilenRow=satir**, **Secildi=true** set edilip form kapanir; cagiran ekran secilen istasyonu alir. **Butonlar & kisayollar:** - **Ara** (**BtnAra**) — **Bagla()** cagirir; operasyon filtresine gore listeyi yeniden ceker. - **Temizle** (**BtnTemizle**) — **TemizleText()**; operasyon arama kutusunu bosaltir. - **Tümü** (**ChcTumu** onay kutusu) — isaretli ise tum istasyonlar; isaretsiz ve **RcAId** verilmissse yalnizca o receteye bagli istasyonlar (**ReceteyeBagliIstasyon.RcIId** ile eslesen) gosterilir. - **Operasyon** (**CmbOperasyon** lookup) — operasyon koduna gore filtre. - Grid cift tiklama / Enter — secim modunda satiri secip kapatir (**MyView1_MyEventDoubleClickEnter**). - **Kapat** (**BtnKapat**, base) — formu kapatir. **Cagirdigi katmanlar:** - Manager/Service: **IIstasyonKartiService.SelectListWhere(sor)** — operasyon filtresine gore istasyon kartlarini ceker (**_srv**, **Ortak.DbPro.IstasyonKarti**). - Manager/Service: **IReceteyeBagliIstasyonService.SelectListWhere("where RcAId=...")** — **RcAId** verildiyse receteye bagli istasyon Id listesini ceker (**_srvBIst**), "Tümü" isaretsizken filtre icin kullanimli. - Manager/Service: **IOperasyonKartiService.SelectListWhere(" Order By OperasyonKodu")** — operasyon combo'sunu doldurur (**_srvOpr**). - SQL/Prosedur: Dinamik **where 1=1 AND Operasyon = '<deger>'** (SorguAyarla) — NOT: tek tirnak kapanisi eksik, basit string birlestirme. - API: - **Istasyon sirasiyla iliskisi:** Dolayli. Secilen istasyon, recete operasyon-grup eslestirmesinde veya uretim istasyonu duzenlemede kullanimli; bu da uretim akisinin (operasyon Sira sirasi -> UretimIstasyon) kurulmasini etkiler. **Notlar:** Namespace **MyUI.IstasyonModul**. **BagliIstasyonSec** bayragi tanimli ama mantikta dogrudan kullanimliyor (cagiran taraf set ediyor). Grid duzeni **IstasyonKartlariSelectListesi** adiyla saklanir (**MyGridKayitAdi**). **Id** sutunu gizlenir.

Istasyon Kartlari (Kayit) (`FrmIstasyonKartlari.cs` / `FrmIstasyonKartlari.Designer.cs`)

Ne ise yarar: Istasyon kartlarinin tam CRUD ekrani (ekle/duzenle/sil). Her istasyon bir operasyona, opsiyonel olarak bir fason cariyeye baglanir; kalite kontrol, yazdirilmali, fason bayraklari tutulur. `MyFrmKayit` turevi (ust GroupControl form + alt grid). **Once ne olmalı (onkosul):** Operasyon kartlari (`OperasyonKarti`) tanimli olmalı (operasyon kodu/adi combo'lari icin). Fason cari secimi icin Mikro DB'de cari kartlari (`MikroCari`) bulunmalı.

Sonra ne olur: Kaydet -> `IstasyonManager.Kaydet()` tek transaction'da: (1) `IstasyonKarti` tablosuna InsertOrUpdate; (2) **istasyon kodu degistirilmisse** ayni transaction icinde `ReceteIstasyon` , `ReceteIstasyonGrupOperasyon` , `UretimIstasyon` , `IstasyonTakipHareket` , `IstasyonKontrol` tablolarindaki eski koda ait IstasyonKodu/IstasyonAdi alanlarini yeni degerle gunceller. Sil ->

`IIstasyonKartiService.Delete()` . Islem sonrasi liste `Bagla()` ile yenilenir; form acik kalir. **Butonlar & kisayollar:** - `Kaydet` (`BtnKaydet` , base) — `Kaydet()` -> dogrulama (kod zorunlu) -> `AktarModele()` -> `IstasyonManager.Kaydet(md1, YeniKayit)` -> `Bagla()` . - `Sil` (`BtnSil` , base) — onay sorar, secili kayıt yoksa uyarir, `Sil()` -> `_srv.Delete(_mdl)` . - `Yeni` (`BtnYeni` , base) — `YeniKayit=true` , `TemizleText()` ile formu temizler. - `Düzenle` (`BtnDuzenle` , base) — secili grid satirini klonlayip forma yukler (`AktarTextlere`) , `YeniKayit=false` . - `Bagla` (`BtnBagla`) — listeyi yeniden baglar (`Bagla()`) + sutun gizle + grid yerlesimi yukle). - Fason Cari Kodu/Adi (`TxtFasonCariKodu` / `TxtFasonCariAdi`) buton tiklamasi — `FrmMikroCariListesi` secim diyalogunu acar, secilen carinin kodu/unvani alanlara aktarilir. - `Operasyon` / `Operasyon Adi` (`CmbOperasyon` / `CmbOperasyonAdi` lookup) — biri secilince digeri otomatik dolar (Leave olaylari). - `Fason` / `KaliteKontrol` / `Yazdirilmali` (`ChcFason` / `ChcKaliteKontrol` / `myChcYazdir`) — istasyon ozellik bayraklari. - Grid cift tiklama / Enter — secim modunda satiri secip kapatir, degilse `BtnDuzenle.PerformClick()` . - `Kapat` (`BtnKapat` , base) — formu kapatir. **Cagirdigi katmanlar:** - Manager/Service: `IstasyonManager.Kaydet(IstasyonKarti mdl, bool yenikayit)` — kod tekillik kontrolu (`KodVarmi`) + transaction'li kayıt + kod degisikligini bagli uretim tablolarina yayma. - Manager/Service: `IstasyonManager.KodVarmi(...)` — `Select count(*) From IstasyonKarti ...` ile ayni IstasyonKodu var mi kontrolu. - Manager/Service: `IIstasyonKartiService` — `SelectListWhere` , `SelectFind` , `InsertOrUpdate` , `Delete` , `Execute` (DAL temel metotlari). - Manager/Service: `IOperasyonKartiService.SelectListWhere("Order By OperasyonKodu")` — operasyon combo'lari. - SQL/Prosedur: `UPDATE ReceteIstasyon / ReceteIstasyonGrupOperasyon / UretimIstasyon / IstasyonTakipHareket SET IstasyonKodu, IstasyonAdi ...` ve `UPDATE IstasyonKontrol SET IstasyonKodu ...` — istasyon kodu yeniden adlandirma yayilimi. - API:

- **Istasyon sirasiyla iliskisi:** Dogrudan ve kritik. Istasyon-operasyon eslesmesini burada kurulan kart belirler; uretim akisi operasyon `Sira` 'sina gore ilerlerken `ReceteIstasyonGrupOperasyon` (GrupKodu+OperasyonKodu->IstasyonKodu) ile her operasyona TEK `UretimIstasyon` olusturulur. Kart silme/yeniden adlandirma bu zinciri etkiledigi icin Manager kod degisimini tum bagli tablolara senkronlar. **Notlar:** Namespace `MyUI.UretimIstasyonModule` (dosya IstasyonModul klasorunde olsa da). Grid duzeni `IstasyonKartlariListesi` . `AcilisBittimi` bayragi combo Leave olaylarinin acilis sirasinda tetiklenmesini engeller. `BtnDegistir_Click` / `BtnYeni_Click` event'leri `EventlerBagla()` icinde kod ile baglanir (Designer'da degil).

Istasyon Aciklamalari (`FrmIstasyonAciklamalari.cs` / `FrmIstasyonAciklamalari.Designer.cs`)

Ne ise yarar: Saha tabletinde operatorun secebilecegi **kodlu aciklama listelerini** yonetir; tek form, `AciklamaModulTuru` enum'una gore 3 farkli amac icin acilir: **Istasyon Baslatma Hatasi**, **Istasyon Durdurma Kodu**, **Istasyon Fire Sebebi**. Her kayıt kod+deger, opsiyonel personel gorevi, SMS gonderim bayragi ve SMS sablon kodu icerir. `MyFrmKayit` turevi. **Once ne olmalı (onkosul):** Form, `AciklamaModulTuru` set edilerek acilmali (`FrmAna` menusunden: `IstasyonBaslatmaHata` / `IstasyonDurdurmaKodu` / `IstasyonFireSebeb`). Personel gorevi combo'su icin Personel tablosunda Gorevi degerleri bulunmalı. **Sonra ne olur:** Kaydet ->

`IIstasyonAciklamaService.InsertOrUpdate(md1)` ; `Modul` alani enum'un string karsiligi olarak yazilir. `Sil` -> `IIstasyonAciklamaService.Delete(md1)` . Liste yalnızca o modul turunun kayitlarini gosterir (`where Modul='<tur>' Order By Kodu`). Bu kayitlar saha tabletinde (TabletV2) durdurma/baslatma-hata/fire ekranlarında secim listesi olarak okunur. **Butonlar & kisayollar:** - `Kaydet` (`BtnKaydet` , base) — `Kaydet()` -> kod zorunlu dogrulaması -> `AktarModele()` (`Modul=enum.ToString()`) -> `InsertOrUpdate` -> `Bagla()` . - `Sil` (`BtnSil` , base) — onay sorar, `Sil()` -> `_srv.Delete(_mdl)` -> `Bagla()` . - `Yeni` (`BtnYeni` , base) — `TemizleText()` . - `Düzenle` (`BtnDuzenle` , base) — secili satiri klonlayip forma yukler. - `SmsGonder` (`ChcSmsGonder`) — bu aciklama secilince SMS gonderilsin mi. - `SmsKodu` (`TxtSmsKodu`) — SMS sablon kodu; `@IstasyonKodu` -> istasyon kodu, `@Personel` -> aktif kullanıcı ile degistirilir (form ustundeki bilgi etiketleri `myLabel5/myLabel6`). - `Per.Görevi` (`CmbGorevi`) — bu aciklamaya/SMS'e bagli personel gorevi. - Grid çift tıklama / Enter — secim modunda satiri secip kapatir, degilse `BtnDuzenle.PerformClick()` . - `Kapat` (`BtnKapat` , base) — formu kapatir. **Çagirdigi katmanlar:** - Manager/Service: `IIstasyonAciklamaService.SelectListWhere` , `InsertOrUpdate` , `Delete` (`_srv` , `Ortak.DbPro.IstasyonAciklama`) — aciklama CRUD. - Manager/Service: `IGenelService.GrupListesi("Personel", "Gorevi")` (`_srvGenel` , `Ortak.DbPro.GenelServis`) — gorevi combo'su için distinct deger listesi. - SQL/Prosedur: `where Modul='<IstasyonAciklamaModulTuru>' Order By Kodu` — liste filtresi. - API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Manager kullanmaz, dogrudan service ile `InsertOrUpdate` yapar (Kart/Bakim'in aksine). `AciklamaModulTuru` enum degerleri: `IstasyonBaslatmaHata` , `IstasyonDurdurmaKodu` , `IstasyonFireSebebi` (`My.Entities.IstasyonAciklamalar`). Form basligi ve `lblBaslik` calismada enum adina gore set edilir (Designer'daki "FrmlIstasyonBaslatmaHatalari" override edilir). Grid duzeni `IstasyonAciklamalariListesi1` ; `Id` ve `Modul` sutunlari gizlenir.

Istasyon Bakim Listesi (`FrmlIstasyonBakimList.cs` / `FrmlIstasyonBakimList.Designer.cs`)

Ne ise yarar: Istasyonlara ait bakım/ariza kayitlarini istasyon ve tarih araligina gore listeleyen ana ekran; ust grid bakım baslik kayitlarini, alt grid secili bakımın **degisen parçalarını** gosterir (master-detay). Buradan yeni bakım eklenir veya mevcut bakım duzenlenir. `MyFrmliste` turevi (master-detay split grid). **Once ne olmalı (onkosul):** Istasyon kartlari (`IstasyonKarti`) tanimli olmalı (istasyon filtre combo'su için). Form `FrmlAna` 'dan MDI çocuk olarak `Show()` ile acilir. **Sonra ne olur:** Bu ekran kayıt yapmaz; `Ekle` /çift tıklama -> `FrmlIstasyonBakimEkle` popup'i acar. Popup `KayitEdildi=true` ile kapanirsa `BtnAra` tetiklenir ve liste tazelenir, önceki satir secimi korunur. Ust grid satiri degisince alt parca grid'i `BaglaDetay()` ile yeniden yuklenir. **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — `Bagla()` ; istasyon kodu + tarih1/tarih2 araligina gore bakım listesini ceker. - `Ekle` (`BtnEkle`) — bos `FrmlIstasyonBakimEkle` acar (yeni bakım). - `Istasyon` (`CmbIstasyon` combo) — istasyon koduna gore filtre. - `Tarihi` (`TxtTarihi1` / `TxtTarihi2` tarih kutulari) — bakım tarihi alt/ust siniri (CAST DATE karsilastirmasi). - Ust grid çift tıklama / Enter (`MyView1.MyEventDoubleClickEnter`) — secili bakimi `FrmlIstasyonBakimEkle { IdGuid = itm.Id }` ile duzenlemeye acar. - Ust grid satir degisimi (`MyView1.FocusedRowChanged`) — alt parca grid'ini gunceller. - `Kapat` (`BtnKapat` , base) — formu kapatir. **Çagirdigi katmanlar:** - Manager/Service: `IIstasyonBakimService.SelectListWhere(sor)` (`_srv` , `Ortak.DbPro.IstasyonBakim`) — bakım baslik listesi (istasyon+tarih filtreli). - Manager/Service: `IIstasyonBakimParcaService.SelectListWhere(" where IstBakId='...'")` (`_srvParca`) — secili bakımın parca detaylari. - Manager/Service: `IIstasyonKartiService.SelectListWhere("")` (`_srvIst`) — istasyon filtre combo'sunu doldurur. - SQL/Prosedur: Dinamik `where 1=1 and IstasyonKodu='...' AND CAST(coalesce(Tarih,'1901-01-01') AS DATE) >= / <= CAST('...' AS DATE)` — istasyon + tarih araligi filtresi. - API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Tarih kutulari Designer'da "24.05.2023" sabit baslangic degeriyle gelir. Ust grid duzeni `IstasyonBakimList` , alt grid `IstasyonBakimParcaList` . `BtnEkle_Click` event'i Designer'da baglanir.

Istasyon Bakim Ekle (`FrmIstasyonBakimEkle.cs` / `FrmIstasyonBakimEkle.Designer.cs`)

Ne ise yarar: Tek bir bakım/ariza kaydini (istasyon, tarih, personel, islem turu, aciklama) ve ona ait degisen parca listesini olusturma/duzenleme popup'i. `MyFrmKayit` turevi; alt sekmede parca grid'i (Ekle/Sil panel butonlari) bulunur. **Once ne olmalı (onkosul):** Istasyon kartlari (`IstasyonKarti`) tanimli olmalı.

`FrmIstasyonBakimList` 'ten yeni icin bos, duzenleme icin `IdGuid` set edilerek acilir. **Sonra ne olur:** Kaydet -> `IstasyonBakimManager.Kaydet(md1, parcalar)` tek transaction'da: (1) `IstasyonBakim` baslik InsertOrUpdate; (2) bu bakima ait eski `IstasyonBakimParca` kayitlarini sil; (3) gecerli parca listesini InsertOrUpdate. Audit alanlari (KayitEden/KayitTarihi yeni kayitta, Degistiren/DegistirmeTarihi her zaman) `Ortak.KullaniciAdi` ile doldurulur. Sil -> `IstasyonBakimManager.Sil(md1)` (baslik + tum parcalar). Basariliysa `ActionAktar` callback'i tetiklenir, `KayitEdildi=true`, form kapanir; liste ekrani tazelenir. **Butonlar & kisayollar:** - `Kaydet` (`BtnKaydet`, base) — `Kaydet()` -> dogrulama (istasyon kodu zorunlu, tarih bossa now) -> `AktarModele()` -> `Manager.Kaydet()` -> kapan. - `Sil` (`BtnSil`, base) — onay sorar, `Manager.Sil()` -> kapan. - `Ekle` (`BtnStokEkle`, sag panel) — `FrmIstasyonBakimParcaEkle` popup'i (yeni parca) acar; donen parca listeye eklenir ve grid yenilenir. - `Sil` (`BtnStokSil`, sag panel) — onay sorar, secili parcayi listeden cikarir (DB kaydi Kaydet'te transaction ile guncellenir). - `IstasyonKodu` (`CmbIstasyonKodu` combo) — bakım yapılan istasyon. - `Personel` / `Bakim/Islem Turu` / `Açıklaması` / `Tarihi` (`TxtPersonel` / `TxtIslemTuru` / `TxtAciklama` / `TxtTarih`) — bakım baslik alanlari. - `Kapat` (`BtnKapat`, base) — formu kapatir. **Cagirdigi katmanlar:** - Manager/Service: `IstasyonBakimManager.Kaydet(IstasyonBakim md1, List<IstasyonBakimParca> parcalar)` — transaction'li baslik+parca kaydi (eski parcalari silip yeniden yazar). - Manager/Service: `IstasyonBakimManager.Sil(IstasyonBakim md1)` — transaction'li baslik+parca silme. - Manager/Service: `IIstasyonBakimService.SelectFirst(c => c.Id == IdGuid)` (`_mng.IstBakimService`) — duzenlemede baslik yukleme. - Manager/Service: `IIstasyonBakimParcaService.SelectList(c => c.IstBakId == IdGuid)` (`_mng.IstParcaService`) — duzenlemede parca yukleme. - Manager/Service: `IIstasyonKartiService.SelectListWhere("")` (`_mng.IstKartService`) — istasyon combo'sunu doldurur. - SQL/Prosedur: Manager icinde `IstasyonBakim` InsertOrUpdate + `IstasyonBakimParca` Delete (IstBakId) + InsertOrUpdate (Dapper DAL uretir). - API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Manager `DatabaseFactoryMikro` da alir (`_dbMikro`) ancak bu formda kullanilmaz (yalniz Pro DB'ye yazar). Parca grid'inde Parca/ParcaNo/EvrakNo/Aciklama/Garanti sutunlari inline duzenlenebilir (`SutunReadOnlyKapat`). `Id` / `IstBakId` gizlenir. Grid duzeni `IstasyonBakimEkleParcalar`. `ActionAktar` callback'i opsiyonel; cagiran liste ekrani genelde `KayitEdildi` bayragiyla yenileme yapar.

Istasyon Bakim Parca Ekle (`FrmIstasyonBakimParcaEkle.cs` /

`FrmIstasyonBakimParcaEkle.Designer.cs`)

Ne ise yarar: Bir bakım kaydina eklenecek tek bir degisen parcanin bilgilerini (parca adi, parca no, evrak no, aciklama, garanti) girmek icin kullanılan sade popup. `MyFrmSade` turevi. **Once ne olmalı (onkosul):**

`FrmIstasyonBakimEkle` icinden `Parca` ozelligi (yeni `IstasyonBakimParca`, Id atanmis) ve `YeniKayit=true` set edilerek acilir. **Sonra ne olur:** Kaydet -> parca adi bos degilse `AktarModele()` ile gelen `Parca` nesnesini doldurur, `KayitEdildi=true` set edip kapanir. DB'ye dogrudan YAZMAZ; donen parca cagiran

`FrmIstasyonBakimEkle` 'nin bellek listesine eklenir ve esas kayit oradaki Kaydet ile transaction icinde DB'ye yazilir. **Butonlar & kisayollar:** - `Kaydet` (`BtnKaydet`, base) — Parca bos ise uyarir; degilse `AktarModele()`, `KayitEdildi=true`, kapan. - `Parça` / `Parça No` / `Evrak No` / `Açıklama`

(`TxtParca` / `TxtParcaNo` / `TxtEvrakNo` / `TxtAciklama`) — parca alanlari. - `Garanti` (`ChcGaranti`) — garanti kapsaminda mi. - `Kapat` (`BtnKapat`, base) — kaydetmeden kapatir. **Cagirdigi katmanlar:** - Manager/Service: - (servis/Manager cagrisi yok; yalniz in-memory nesne doldurur) - SQL/Prosedur: - - API: - **Istasyon sirasiyla**

iliskisi: - **Notlar:** En basit form. Persist islemi tamamen cagiran `FrmIstasyonBakimEkle` + `IstasyonBakimManager.Kaydet()` sorumlulugunda. Tek zorunlu alan `TxtParca`.

Modul: IstasyonHareketlerModul

Bu modul, sahada (TabletV2) acilan istasyon takip kayitlarinin ofis/yonetim tarafından **listelenmesi, izlenmesi, duzeltilmesi ve Mikro ERP'ye fis aktarimini** kapsar. Ureticinin kendisi burada uretim girisi yapmaz; uretim girisi tablette **IstasyonTakipPage** akisinda yapilir ve buraya yalnız **sonuc kayitlari** (IstasyonTakipHareket = istasyon basligi/sarisi, IstasyonTakipHareketDetay = her mamul/fire/iptal hareketi, IstasyonTakipStokHareket = sarf edilen recete stoklari, IstasyonTakipHareketLog = baslat/durdur/bitir log'lari) yansir. Modul WinForms (UretimV4 / **CepPatronERP.exe**) icindedir, formlar **Frm*** deseni ile yazilmistir ve DB'ye dogrudan baglanir (**Ortak.DbPro** = Uretim DB, **Ortak.DbMikro** = Mikro DB). Tum servisler **IstasyonHareketManager** veya dogrudan **Ortak.DbPro.<Servis>** üzerinden cagrilir. Akis/miktar motoru OPERASYON-Sira bazlidir: bir detayda miktar duzeltilirse **Uretim_MiktarGuncelle** ve **Uretim_PlanlananGuncelle** prosedurleri yeniden cagrilir ve miktarlar UretimIstasyonHareket -> UretimIstasyon -> UrOHD/UrOH/UrO zincirinde yukari toplanir, bir sonraki operasyon Sira'sinin planlanani guncellenir.

Bu modulde 6 form vardir: 4 liste formu (Hareketler, Bekleyenler, Detaylar, Fis Listesi, Log Listesi) ve 1 kayit/duzenleme formu (Detay Guncelle).

Istasyon Hareketler (**FrmIstasyonHareketler.cs** / **.Designer.cs**)

Ne ise yarar: Acilmis istasyon takip basliklarini (IstasyonTakipHareket) listeler. Ust grid baslik kayitlari, alt sol grid o baslica ait hareket detaylari (IstasyonTakipHareketDetay), alt sag grid ilgili UretimIstasyon'a (UrIID) ait sarf stok hareketleri (IstasyonTakipStokHareket). Istasyon, durum (Aktif/Beklemede/Durduruldu/Bitti), is emri (siparis) kodu ve tarih araligina gore filtrelenir. **Once ne olmalı (onkosul):** Tablette ilgili istasyonda en az bir takip kaydi baslatilmis olmalı (UretimIstasyon + IstasyonTakipHareket olusmus olmalı). Form

SiparisKodundanFiltrele=true + **SiparisKodu** ile baska bir ekrandan (siparis/is emri) tek is emrine kilitli acilabilir. **Sonra ne olur:** Bu form yalnız okuma/goruntulemedir; veri degistirmez. Bir detay satirina cift tiklamak (yalnizca **SecimIcinAcildi** modunda) secim dondurur. Detay duzeltmesi bu formdan degil,

FrmIstasyonHareketDetaylar -> FrmIstasyonHareketDetayED üzerinden yapilir. **Butonlar & kisayollar:** -

BtnAra ("Ara") — **Bagla()** cagirir, filtreye gore ust gridi doldurur. Form acilisinda **BtnAra.PerformClick()** otomatik calisir. - **BtnTemizle** ("Temizle") — Tarih (son 1 ay) / istasyon / is emri kodu filtrelerini sifirlar. - **BtnDurumuHepsi** ("Hepsi") — durum filtresini bos birakir, otomatik arar. - **BtnDurumuAktif** ("Aktif") — Durumu=Aktif filtreler (acilista varsayilan secili). - **BtnDurumuBeklemede** ("Beklemede") — Durumu=Beklemede filtreler (Designer'da **Visible=false**, gizli). - **BtnDurumuDurduruldu** ("Durduruldu") — Durumu=Durduruldu filtreler. - **BtnDurumuBitti** ("Bitti") — Durumu=Bitti filtreler. - **CmbIstasyon** (combo) — istasyon kodu filtresi (IstasyonKarti listesinden dolar). - **TxtKodu** (Is Emri Kodu), **TxtTarihi1/TxtTarihi2** — like ve tarih araligi filtreleri. - Grid cift tik / Enter (**MyEventDoubleClickEnter**) — secim modunda kayit dondur. - **BtnKapat** (base) — formu kapatir. **Cagirdigi katmanlar:** - Manager/Service: **IstasyonHareketManager(Ortak.DbPro, Ortak.DbMikro)** — modul servislerini toplar. - Service: **IstHareketService.GetViewListWhere(where)** — IstasyonTakipHareket view'ini filtreyle ceker (ust grid). - Service: **IstHareketDetayService.GetViewListWhere("where IstHD.IstHrId='...'")** — secili baslica ait detaylar (alt sol grid). - Service:

IstHareketStokService.GetStokHareketByUrIID(UrIID) — secili UretimIstasyon'un sarf stok hareketleri (alt sag grid). - Service: **IstKartService.SelectListWhere("")** — istasyon combo'sunu doldurur. - SQL/Prosedur: yok (yalniz select). - API: yok (WinForms, dogrudan DB). **Istasyon sirasiyla iliskisi:** Durumu=Aktif satirlar o anda calisilan, Bitti satirlar bir sonraki istasyona sevk edilmiş/tamamlanmış operasyonlardir. Alt sag grid UrIID ile o operasyonun sarf recete stoklarini gosterir; operasyon Sira mantigini dogrudan degistirmez (okuma). **Notlar:**

`Ortak.PIKapat=true` ise Parti/Lot kolonlari gizlenir. SorguAyarla string concatenation ile yazilmistir (parametrik degil). "SiparisKodu" kolonu UI'da "IsEmriKodu" olarak gosterilir.

Istasyon Bekleyenler (`FrmlIstasyonBekleyenler.cs` / `.Designer.cs`)

Ne ise yarar: Henuz hicbir istasyon takip kaydi (IstasyonTakipHareket) olusturulmamis, yani sahada baslatilmayi bekleyen UretimIstasyon kuyrugunu gosterir. Her satir: kalan miktar, planlanan/uretilen/fire/iptal miktarlari, recete, operasyon, siparis (is emri), teslim tarihi, fason bilgisi vb. **Once ne olmalı (onkosul):** Uretim emri + receteden UretimOperasyon/UretimIstasyon kayitlari uretilmis (KalanMiktar > 0) ama o UretimIstasyon icin henuz IstasyonTakipHareket acilmamis olmalı. **Sonra ne olur:** Salt okuma listesidir. Cift tik islevleri kod icinde yorum satirina alinmis; herhangi bir kayit/degisiklik tetiklemez. Operatorun tablette hangi isleri baslatmasi gerektigini planlama amaclidir. **Butonlar & kisayollar:** - `BtnAra` ("Ara") — `Bagla()` cagirir; form acilisinda da otomatik `Bagla()` calisir. - `BtnTemizle` ("Temizle") — Designer'da var ancak bu formda click event'i baglanmamis (etkin degil). - `CmbIstasyon` (combo) — istasyon kodu filtresi (`UrI.IstasyonKodu`). - `TxtKodu` ("Is Emri Kodu") — siparis kodu like filtresi (`Sip.SiparisKodu`). - Grid cift tik / Enter (`MyEventDoubleClickEnter`) — aktif islev yok (yorumlu). - `BtnKapat` (base) — kapatir. **Cagirdigi katmanlar:** - Manager: `IstasyonHareketManager.GetBekleyenler(whereAnd)` — su SQL'i calistirir: `UretimIstasyon UrI + UretimOperasyon UrO + Siparis + SiparisHareket LEFT JOIN, IstasyonTakipHareket TH LEFT JOIN` ile `WHERE UrO.KalanMiktar > 0 AND TH.Id IS NULL` (yani henuz takip acilmamis); Url bazinda planlanan/uretilen/fire/iptal miktarlarini gruplayip toplar. (Benzer `GetBekleyenByIstKodu(Istasyon)` metodu da ayni mantikla tek istasyon icindir.) - Service: `IstKartService.SelectListWhere("")` — istasyon combo'su. - SQL/Prosedur: inline SELECT (ad yok), prosedur cagirmaz. - API: yok. **Istasyon sirasiyla iliskisi:** "Bekleyen" = `IstasyonTakipHareket` henuz yok; bir operasyonun Sira'sina ait UretimIstasyon olusmus ama saha basinda is alinmamistir. Tablette baslatildiginda bu satir listeden dusup FrmlIstasyonHareketler'e tasinir. **Notlar:** Tarih filtreleri (`SorguAyarlaTrh`) tamamen yorum satiridir, etkisizdir. PIKapat'ta Parti/Lot gizlenir.

Istasyon Hareket Detaylar (`FrmlIstasyonHareketDetaylar.cs` / `.Designer.cs`)

Ne ise yarar: Tum istasyon hareket **detaylarini** (IstasyonTakipHareketDetay) tek listede gosterir (mamul giris, fire mamul giris, uretim bitis, fire stok giris, uretim iptal turleri). Sarf/Fire **fis** turleri (SarfCikisFisi, FireGirisFisi) bu listeden HARIC tutulur (onlar FrmlIstasyonFisList'te). Detaya cift tiklanarak miktar/fire/iptal duzeltme ekrani acilir. **Once ne olmalı (onkosul):** Tablette uretim girisi yapilmis ve IstasyonTakipHareketDetay kayitlari olusmus olmalı. **Sonra ne olur:** Cift tik -> `FrmlIstasyonHareketDetayED` acilir; orada kaydedilirse miktarlar guncellenir ve dönülünce `Bagla()` ile liste tazelenir. **Butonlar & kisayollar:** - `BtnAra` ("Ara") — `Bagla()` . - `BtnTemizle` ("Temizle") — tarih (son 1 ay), saat (00:00:00 / 23:59:59), istasyon, is emri kodu filtrelerini sifirlar. - `CmbIstasyon` (combo) — `IstHr.IstasyonKodu` filtresi. - `CmbTuru` (combo) — `IstHD.Turu` filtresi; SarfCikisFisi ve FireGirisFisi turleri listeye eklenmez. - `TxtKodu` ("Is Emri Kodu") — `IstHr.SiparisKodu` like. - `TxtTarihi1/2` + `TxtSaat1/2` — `IstHD.Tarih` datetime araligi. - Grid cift tik / Enter (`MyEventDoubleClickEnter`) — secim modunda secim dondurur; degilse `FrmlIstasyonHareketDetayED` acar. - `BtnKapat` (base) — kapatir. **Cagirdigi katmanlar:** - Service: `Ortak.DbPro.IstasyonTakipHareketDetay.GetViewListWhere(where)` — detay view'i (SarfCikisFisi/FireGirisFisi haric SQL ile filtrelendir). - Service: `Ortak.DbPro.IstasyonKarti.SelectListWhere("")` — istasyon combo. - Service: `Ortak.DbPro.GenelServis.GrupListesi("IstasyonTakipHareketDetay", "Turu")` — Turu combo degerleri. - Acilan form: `FrmlIstasyonHareketDetayED` (Model=secili detay). - SQL/Prosedur: dogrudan prosedur cagirmaz (duzeltme FrmlIstasyonHareketDetayED'de yapilir). - API: yok. **Istasyon sirasiyla iliskisi:** Detay turleri (MamulGiris/UretimBitis/UretimIptal/FireMamulGiris) miktar motorunun yukari toplama

girdileridir; burada yapılan duzeltme bir sonraki operasyon Sira'sinin planlanani etkiler (ED formunda prosedur tetiklenir). **Notlar:** Sarf/Fire fisleri kasıtlı olarak haric. Saat alanlari TimeSpanEdit.

Istasyon Hareket Detay Guncelle (FrmIstasyonHareketDetayED.cs / .Designer.cs)

Ne ise yarar: Tek bir IstasyonTakupHareketDetay kaydinin Miktar / FireMiktar / IptalMiktar degerlerini duzeltir (ofiste hatali girilen miktari elle duzeltme). Stok/Recete/Operasyon kod-ad alanlari salt okunur gosterilir. **Once ne olmalı (onkosul):** FrmIstasyonHareketDetaylar listesinde bir detay satiri secilip cift tiklanmis (Model set edilmiş) olmalı. **Sonra ne olur (kaydet sonrası):**

IstasyonHareketManager.SaveIstasyonTakupHareketDetayUpdate(Model) calisir; tek transaction icinde su zincir isler ve form kapanir: 1. IstasyonTakupHareketDetay update (detay miktarlari yazilir), 2. UretimIstasyonHareket ayni Id ile UretimMiktar/FireMiktar/IptalMiktar guncellenir, 3. IstasyonTakupHareket baslik miktarlari (GetMiktarFireUpdateSqlCode) yeniden hesaplanir (detaylardan MamulGiris/FireMamulGiris/UretimBitis/UretimIptal toplanir; KalanMiktar = Planlanan - (Uretim+Iptal+Fire)), 4. UrId bazinda iptal miktarı stok hareketlerine oranlanir (GetIptalUpdateSqlCodeByUrId), 5. exec [Uretim_MiktarGuncelle] '<UrId>', 6. exec [Uretim_PlanlananGuncelle] '<UrId>'. **Butonlar & kisayollar:** - BtnKaydet ("Kaydet") — AktarRowa() + Kaydet() (yukaridaki update zincirini calistirir, sonra this.Close()). - BtnKapat (base) — kaydetmeden kapatir. - BtnSil (base) — Visible=false , gizli (silme yok). - BtnYeni/BtnDuzenle/BtnYazdir/BtnIlk/BtnOnceki/BtnSonraki/BtnSon (base nav) — bu formda islevsiz/gizli, tek kayit duzenleme. - Duzenlenebilir alanlar: TxtMiktar , TxtFireMiktar , TxtIptalMiktar (sayi). Diger alanlar ReadOnly. **Cagirdigi katmanlar:** - Manager:

IstasyonHareketManager.SaveIstasyonTakupHareketDetayUpdate(IstasyonTakupHareketDetay) — yukaridaki 6 adimli transaction. - SQL/Prosedur: Uretim_MiktarGuncelle — miktarlari UretimIstasyonHareket->UretimIstasyon->UrOHD/UrOH/UrO zincirinde yukari toplar. Uretim_PlanlananGuncelle — operasyon Sira N uretimini Sira N+1 planlanina tasir, sonunda Uretim_SonrakiIstasyonaGonder cagrilir (ReceteAna.IstasyonGruplamaKullan=1 ise yeni UretimIstasyon olusturur). - SQL kod (entity static): IstasyonTakupHareketDetay.GetUpdateSqlCode() , IstasyonTakupHareket.GetMiktarFireUpdateSqlCode(IstHrId) , IstasyonTakupHareket.GetIptalUpdateSqlCodeByUrId(UrId) . - API: yok. **Istasyon sirasiyla iliskisi:** Bu form akis motorunu **tetikleyen tek yazma noktasidir**. Bir detay miktarı degistiginde Uretim_PlanlananGuncelle -> Uretim_SonrakiIstasyonaGonder zinciriyle bir sonraki operasyon Sira'sinin planlanani/istasyon kaydi yeniden hesaplanir. **Notlar:** MyFrmKayit turevidir (kayit formu base'i). Transaction'da hata olursa rollback + ErrorResult; basaride form kapanir.

Istasyon Fis Listesi (FrmIstasyonFisList.cs / .Designer.cs)

Ne ise yarar: Yalnız SarfCikisFisi ve FireGirisFisi turundeki istasyon hareket detaylarini (Mikro'ya gonderilecek sarf cikisi / fire girisi fisleri) listeler ve secilenleri Mikro ERP'ye aktarir. "Mikroya Aktarildi" durumuna gore (Tumu / Aktarilan / Bekleyen) filtrelendir. **Once ne olmalı (onkosul):** Tablette/uretim girisinde sarf veya fire fisi turunde IstasyonTakupHareketDetay kayitlari olusmus olmalı. Aktarim oncesi Mikro entegre ayarlari (Ortak.MikroEntAyarlar icinde fis turu ayarlari) tanimli olmalı. **Sonra ne olur (Mikroya Gonder sonrası):** Secili fisler FrmMikroyaSarfFireKaydet e tasinir; orada Mikro STOK_HAREKETLERI kayitlari uretilip yazilir, basarida detay kayitlari Ent=true , EntSeri/EntSira/EntDate ile guncellenir (artik "Aktarilan" sekmesinde gorunur). **Butonlar & kisayollar:** - BtnAra ("Ara") — Bagla() ; acilista AktarimAyarla("Bekleyen") + otomatik BtnAra.PerformClick() . - BtnTemizle ("Temizle") — tarih filtrelerini bosaltir. - BtnAktarildiTumu ("Tumu") — Ent filtresi yok. - BtnAktarildiAktarilan ("Aktarilan") — coalesce(Ent,0)=1 . -

BtnAktarildiBekleyen ("Bekleyen") — **coalesce(Ent,0)=0** (acilista varsayilan). - **TxtTarihi1/2** — **IstHD.Tarih** araligi. - **Sec** kolonu — grid'de editlenebilir checkbox (aktarilacak satirlari isaretleme). - Sag tik context menu: **"Mikroya Gonder"** (**ToolStripMenuItem**) — secili (**Sec=true**) fisleri toplayip **FrmmikroyaSarfFireKaydet** acar. - Grid cift tik / Enter (**MyEventDoubleClickEnter**) — secim modunda secim dondurur; degilse islevsiz. - **BtnKapat** (base) — kapatir. **Cagirdigi katmanlar:** - Service: **Ortak.DbPro.IstasyonTakupHareketDetay.GetViewListWhere(where)** — yalnız **Turu=SarfCikisFisi OR Turu=FireGirisFisi** + Ent durumu filtresi. - Acilan form: **FrmmikroyaSarfFireKaydet** (MikroModul) — **FisList** ile. - **FrmmikroyaSarfFireKaydet** icinde: **MikroKayitManager** (**StokHareketIdKayitEdilmismi** , **StokHareketKaydet**), **MikroConvertManager** (**SetSarfCikisFisiTuruAyar** , **SetFireGirisFisiTuruAyar** , **ConvertStokVirmanFisi** , **ConvertSarfDepoCikis**), **IstasyonTakupHareketDetay** servisinin **InsertOrUpdate(FisList)** ile Ent guncellemesi. - SQL/Prosedur: dogrudan prosedur cagirmaz; aktarim Mikro tarafinda **StokHareketKaydet** ile yapilir. - API: yok. **Istasyon sirasiyla iliskisi:** - (Fis aktarimi miktar/Sira motorunu degil, Mikro ERP stok hareketlerini etkiler.) **Notlar:** Cift aktarimi engellemek icin **FrmmikroyaSarfFireKaydet** once **StokHareketIdKayitEdilmismi** ile onceden aktarilmis kayit kontrolu yapar; varsa Kaydet butonu gizlenir. **Sec** kolonu acilista ReadOnly kapatilip edit acilir.

Istasyon Hareket Log Listesi (**FrmmistasyonHareketLogList.cs** / **.Designer.cs**)

Ne ise yarar: Saha akisindaki baslat/durdur/bitir/sevk gibi olaylarin log kayitlarini (IstasyonTakupHareketLog) listeler. Operasyon, istasyon ve tarih-saat araligina gore filtrelendir. Denetim/izleme amaclidir. **Once ne olmalı (onkosul):** Tablette istasyon takip akisi calismis ve log kayitlari (**IstasyonTakupHareketLog**) olusmus olmalı. **Sonra ne olur:** Salt okuma; veri degistirmes. Cift tik islevi (uretim emri editorune gitme) kod icinde yorumludur, aktif degildir. **Butonlar & kisayollar:** - **BtnAra** ("Ara") — **Bagla()** ; form acilisinda da **Bagla()** otomatik. - **BtnTemizle** ("Temizle") — tarih bos, saat 00:00:00 / 23:59:59 yapar. - **CmbOperasyon** (MyLookupEdit) — **TH.OperasyonKodu** filtresi (OperasyonKarti listesinden). - **CmbIstasyon** (MyLookupEdit) — **TH.IstasyonKodu** filtresi (IstasyonKarti listesinden). - **TxtTarihi1/2** + **TxtSaat1/2** — **LG.Tarih** datetime araligi. - Grid cift tik / Enter (**MyEventDoubleClickEnter**) — secim modunda secim dondurur; degilse islevsiz (yorumlu). - **BtnKapat** (base) — kapatir. **Cagirdigi katmanlar:** - Service: **Ortak.DbPro.IstasyonTakupHareketLog.GetViewListWhere(where)** — log view'i. - Service: **Ortak.DbPro.OperasyonKarti.SelectListWhere(" Order By OperasyonKodu")** — operasyon combo. - Service: **Ortak.DbPro.IstasyonKarti.SelectListWhere(" Order By IstasyonKodu")** — istasyon combo. - SQL/Prosedur: yok (yalniz select). - API: yok. **Istasyon sirasiyla iliskisi:** Log kayitlari saha akisindaki adimlarin (baslat/durdur/bitir/sonraki istasyona sevk) zaman damgali izini tutar; Sira/akis motorunu degistirmes. **Notlar:** **IstasyonTakupHareketLogService** ayni zamanda **IstasyonHareketManager.IstHareketLogService** olarak da erisilebilir ama bu form servisi dogrudan **Ortak.DbPro** uzerinden alir.

Modul geneli notlar

- Tum liste formlari **MyFrmliste** , kayit formu (**FrmmistasyonHareketDetayED** , **FrmmikroyaSarfFireKaydet**) **MyFrmmKayit** base sinifindan turer (base siniflar **My.Kontrol.Formlar** namespace'inde, harici kontrol kutuphanesinde). **BtnAra/BtnTemizle/BtnKapat/BtnYazdir/BtnDizayn** ve kayit formundaki **BtnKaydet/BtnSil/BtnYeni/BtnIlk/BtnOnceki/BtnSonraki/BtnSon** base'ten gelir.
- Grid cift tik ve Enter, base'in **MyView.MyEventDoubleClickEnter** event'i ile ayni davranisi tetikler (cift tik = Enter).
- Filtre butonlarinin secili gorunumu **MyButton.FilterButonRenklendir(true/false)** ile renklendirilir.

- Tek **yazma** noktası **FrmIstasyonHareketDetayED** (miktar duzeltme + prosedur zinciri) ve **FrmIstasyonFisList** -> **FrmMikroyaSarfFireKaydet** (Mikro'ya fis aktarimi). Diger formlar salt okuma/listeleme.

Modul: ReceteModul

Recete (urun agaci / urun receteleri) modulu, uretilecek mamulun hangi hammadde/yari-mamul stoklardan (Detaylar -> Stoklar), hangi operasyonlardan (Operasyonlar -> Istasyonlar -> Cariler) ve hangi siralamayla olusacagini tanımlar. Bu modulde tanımlanan veriler dogrudan uretim akisinin temelidir: ReceteOperasyon.Sira degeri, uretim girisinde calisan **Uretim_PlanlananGuncelle** prosedurunun "operasyon Sira N -> Sira N+1" tasimasinin kaynagidir; **ReceteAna.IstasyonGruplamaKullan=1** ise **Uretim_SonrakiIstasyonaGonder** prosedurunun calisma sartidir. Modul WinForms (DevExpress) ekranlarından olusur: Recete Listesi (ana giris), Recete Ekle/Duzenle (kart), Recete Detay Ekle, Recete Stok Sec (renk/beden), Recete Operasyon Ekle/Duzenle, Recete Aciklamalari, iki Maliyet ekranı (Genel ve Stoklar), Recete Sec (secim diyalogu), Recete Grup Listesi ve Recete Grup Ekle/Duzenle. Veriler **UretimV3_FEZA** DB'sindeki ReceteAna / ReceteDetay / ReceteStok / ReceteStokRenkBeden / ReceteyeBagliIstasyon / ReceteOperasyon / Recetelstasyon / RecetelstasyonCari / ReceteGrup / ReceteGrupDetay tablolarında tutulur; maliyet ekranlari ayrıca Mikro DB'sindeki STOKLAR ve URUN_RECETELERI tablolarına baglanır.

Tum kayıt ekranlari **MyFrmKayit**, liste/secim ekranlari **MyFrmListe**, sade diyaloglar **MyFrmSade** / **XtraForm** base sinifından turer. Base siniflar (My.Kontrol DLL) standart alt buton seridi saglar: **BtnKaydet** (Kaydet), **BtnKapat** (Kapat/Esc), **BtnSil**, **BtnYazdir**, **BtnDuzenle**, navigasyon butonlari (BtnIlk/BtnOnceki/BtnSonraki/BtnSon) ve listede **BtnAra** / **BtnTemizle**. Grid'lerde satira cift tiklama veya Enter, **MyEventDoubleClickEnter** olayini tetikler (genelde duzenleme/secim). Designer dosyalarında ayrıca tanımlanmis F-tusu (ShortcutKeys) bulunmamaktadır; kisayollar base form davranisi ve grid Enter/cift-tik ile sinirlidir.

Recete Listesi (**FrmReceteListesi.cs** / **FrmReceteListesi.Designer.cs**)

Ne ise yarar: Tum recetelerin arandigi/listelendigi ana ekran. Ust grid recete basliklari (ReceteAna), alt iki grid secili recetenin Detaylari (ReceteDetay) ve Operasyonlarini (ReceteOperasyon) gosterir. Ayni form **SecimIcinAcildi=true** ile baska ekranlardan recete secmek icin de acilir. **Once ne olmalı (onkosul):** Mikro stok cinsi tanımlari okunabilmeli (BaglaStokCinsi). Liste filtresiz/filtreli acilir, otomatik arar (Frm_Load -> Bagla). **Sonra ne olur:** Bu ekran veriyi sadece okur; degisiklik alt ekranlardan yapilir. Satira cift-tik/Enter -> secim modunda secili receteyi dondurur ve kapanir; degilse **FrmReceteED** (duzenleme) acilir, kayıt sonrası liste yenilenir. **Butonlar & kisayollar:** - **Ara** (BtnAra) — filtre alanlarına (Kodu/Adi/Grup/Stok Cinsi/Ara) gore listeyi yeniden ceker (Bagla -> SorguAyarla LIKE/eq sorgusu). - **Temizle** (BtnTemizle) — Kodu/Adi/Grup filtre alanlarını bosaltir. - **Recete Ekle** (BtnReceteEkle) — bos **FrmReceteED** acar (yeni recete). - **Operasyon Ekle / Duzenle** (BtnOperasyonEkle) — secili recetenin **FrmReceteOperasyonED** ekranini acar; recete secili degilse uyarir. - **Operasyon Kopyala** (BtnOperasyonKopyala) — kaynak receteyi sectirir (**FrmReceteListesi** secim modu), kaynak operasyonlari hedef receteye kolonlamak icin **FrmReceteOperasyonED** 'i Kolonlanacak=true ile acar. - **Stok Maliyet** (BtnStokMaliyet) — secili recete icin **FrmReceteMaliyetStoklar** acar. - Grid cift-tik / Enter (myView1) — secim modunda receteyi dondurur, degilse **FrmReceteED** acar. - **Kapat** (BtnKapat) — formu kapatir. **Cagirdigi katmanlar:** - Service: **IReceteAnaService.GetListWhere(where)** — receteleri (StokCinsiAdi dahil view) filtreyle getirir. - Service: **IReceteDetayService.SelectList(c => c.RcAId == id)** — secili recetenin detaylarını getirir. - Service: **IReceteOperasyonService.SelectList(c => c.RcAId == id)** — secili recetenin operasyonlarını getirir. - Service: **IGenelService.GrupListesi("ReceteAna","Grubu")** — grup filtre comboboxunu doldurur. - Manager: **MikroStokCinsiManager.GetCinsListFull()** — Mikro stok cinsi listesi (Mamul vb. filtre). **Istasyon sirasiyla iliskisi:** Operasyon gridinde ReceteOperasyon.Sira goruntulenir; bu

sira uretim akisinin (Sira N -> N+1) belirleyicisidir. Burada salt okuma. **Notlar:** `ContextMenuyeEkle` (Uretim Emri Olustur context menu) kod icinde yorumlanmis, aktif degil. SorguAyarla string birlestirmeli LIKE filtresi kullanir.

Recete Ekle / Duzenle (`FrmReceteED.cs` / `FrmReceteED.Designer.cs`)

Ne ise yarar: Bir recetenin ana bilgileri (kod, ad, grup, entegre Mikro stok, raf omru, bayraklar) ile Detaylarini ve her detayin alternatif Stoklarini (renk/beden), ayrica Bagli Istasyonlarini tanımlamak/duzenlemek icin ana kart ekrani. Mikrodan recete ice aktarma da burada yapilir. **Once ne olmalı (onkosul):** Listedden recete secilmis (IdGuid dolu -> duzenleme) ya da yeni kayıt (IdGuid bos). `MikrodanAktar=true` ise gecerli `MikroReceteKodu` set edilmiş olmalı. **Sonra ne olur:** Kaydet -> transaction ile ReceteAna upsert + ReceteDetay/ReceteStok/ReceteyeBagliIstasyon/ReceteStokRenkBeden tablolari silinip yeniden yazilir (ReceteManager.ReceteKaydet). Ayni EntegreStokKodu baska recetede varsa kayıt engellenir. Kayit sonrasi `KayitEdildi=true` set edilir, form veriyi yeniden yukler; cagiran liste yenilenir. **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — TextLeriKontrolEt + AktarModele + Entegre stok kodu tekillik kontrolu + ReceteKaydet. - `Sil` (BtnSil) — once `ReceteSilKontrol` (Recete_Sil_Kontrol proc) ile bagli hareket var mi bakar, yoksa ReceteSil (operasyon bagliysa silmez). - `Kopyala Ctr+D` (BtnDuzenle -> BtnDegistir_Click) — gecerli receteyi yeni Id'lerle kopyalar (ReceteKodu bosaltilir, yeni kayıt olur). - `Detay Ekle` (BtnDetayEkle) — `FrmReceteDetayED` yeni detay ekler (Sira=receteSira+1). - `Detay Sil` (BtnDetaySil) — secili detayi ve ona bagli stok/renk-beden satirlarini listeden cikarir. - `Stok Ekle` (BtnStokEkle) — secili detaya Mikro stok secip (`FrmMikroStokListesi`) `FrmReceteStokSec` ile renk/beden tanımlar. - `Stok Sil` (BtnStokSil) — secili ReceteStok ve renk/beden satirlarini cikarir. - `Aciklamalar` (BtnAciklamalar) — kayitli recete icin `FrmReceteAciklamalar` acar (kaydedilmemisse uyarir). - `Operasyon Ekle / Duzenle` (BtnOperasyonEkle1) — kayitli recete icin `FrmReceteOperasyonED` acar. - `Operasyon Kopyala` (BtnOperasyonKopyala1) — kaynak receteyi secip operasyonlari bu receteye kolonlar. - `Stok Maliyet` (BtnStokMaliyet) — `FrmReceteMaliyetGenel` acar (gecerli model klonuyla). - `Istasyon Ekle / Istasyon Sil` (BtnIstasyonEkle/BtnIstasyonSil, "Bagli Istasyonlar" sekmesi) — ReceteyeBagliIstasyon listesine istasyon karti ekler/cikarir. - `Istasyon Gruplar` (BtnIstasyonGruplar) — `FrmReceteIstasyonGrupIstasyonEslestir` (grup-istasyon eslestirme) acar. - `Istasyon-Grup Ayarlar` (BtnIstasyonGrupAyarlar) — `FrmReceteIstasyonGrupOperasyonEslestir` acar. - `Stok Bilgi ^ Recete Aktar` (BtnRecetAdiAktar) — Entegre stok kodu/adini ReceteKodu/ReceteAdi alanlarina kopyalar. - TxtReceteKodu buton — `GetEvrakNo("Recete")` ile otomatik recete kodu uretir. - TxtEntegreStokKodu/Adi/Birim/ModelKodu buton — `FrmMikroStokListesi` secimi (entegre stok ata + cinsi getir). - Sekmeler: `Detaylar/Stoklar` (tabNavigationPage1), `Bagli Istasyonlar` (tabNavigationPage2). - Bayraklar: `Haziri Sonraki Istasyona Gonder` (Grupsuz), `Istasyon Gruplama Kullan`, `Aparat Zorunlu`, `Olcum Kullan` (Zorunlu) checkbox'lari. **Cagirdigi katmanlar:** - Manager: `ReceteManager.GetReceteKayit(id)` — recete + detay + stok + bagli istasyon + renk/beden modelini doldurur. - Manager: `ReceteManager.ReceteKaydet(model, yenikayit)` — transaction ile tum alt tablolari sil-yeniden yaz. - Manager: `ReceteManager.ReceteStokKoduDahaonceGirilmismi(id, entegreStokKodu)` — ayni entegre stok koduyla baska recete var mi kontrol eder. - Manager: `ReceteManager.ReceteSilKontrol(id)` / `ReceteManager.ReceteSil(model)` — Recete_Sil_Kontrol proc + iliskili tablolari sil. - Manager: `MikroReceteManager.GetMikroReceteList(wheres)` / `GetMikroReceteHareketler(receteKodu)` — Mikrodan aktarmada URUN_RECETELERI'nden recete basligi ve satirlarini ceker. - Service: `IMikroStokService.GetViewListWhere(...)` — entegre stok cinsini bulur (StokDanCinsiBagla). - Service: `IGenelService.GrupListesi/GetEvrakNo` — grup combobox + recete evrak no. - SQL/Prosedur: `Recete_Sil_Kontrol` — receteye bagli hareket adedini dondurur. **Istasyon sirasiyla iliskisi:** `IstasyonGruplamaKullan` bayragi burada set edilir; =1 ise uretim akisinda `Uretim_SonrakiIstasyonaGonder`

calisir. Detay ReceteSira (urun agaci satir sirasi) ve operasyon sirasi (ayri ekranda) buradaki receteye baglidir.

Notlar: Detay gridinde satir ici duzenlenebilen kolonlar yesil baslikli (ReceteSira, Miktar, Ebat, Gram, Olcu, FireYuzde). RafOmru gun bazli. Mikrodan aktarilan detaylar StokTuru="Sabit", Cinsi="MikroRecete" olarak gelir.

Recete Detay Ekle / Duzenle (FrmReceteDetayED.cs / FrmReceteDetayED.Designer.cs)

Ne ise yarar: Recetenin tek bir detay (urun agaci satiri) bilgilerini girer: cinsi (kategori), birim, miktar, fire %, recete sıra, varsayılan stok (kod/ad), renk/beden, operasyon maliyeti, stok turu (Stok/Grup/Sabit/Tumu), stok ana grup, açıklama ve bayraklar (StokKullan, SiparisdeGosterme). **Once ne olmalı (onkosul):** FrmReceteED 'den cagrilir; yeni icin `Detay=new ReceteDetay()` ve `Sira` set edilir, duzenleme icin mevcut `Detay` gecirilir. **Sonra ne olur:** Tamam -> AktarModele ile `Detay` nesnesi guncellenir, `KayitEdildi=true` ile kapanir; cagiran `FrmReceteED` detayi listesine ekler/gunceller (DB'ye yazma yok, kayit ana ekranin Kaydet'inde olur). **Butonlar & kisayollar:** - **Tamam** (BtnTamam) — zorunlu alan kontrolleri (Cinsi, Miktar>0, stok turune gore ana grup/varsayilan stok kodu) + AktarModele + kapat. - VarsayilanStokKodu/Adi buton (TxtVarsayilanStokKodu1/Adi1) — `FrmMikroStokListesi` ile Mikro stok secer, birim + renk/beden listelerini yukler. - StokTuru secimi: `Stok` / `Grup` / `Sabit` / `Tumu` radio (RadStok1/RadGrup1/RadSabit1/RadTumu1). - **Kapat** — diyalogu iptal eder. **Cagirdigi katmanlar:** - Service: `IMikroStokService.GetStokKategoriler()` — Cinsi comboboxunu doldurur (kategoriler). - Service: `IMikroStokService.GetRenkByStokKodu(kod)` / `GetBedenByStokKodu(kod)` — renk/beden comboboxlari. - Service: `IMikroGenelService.GrupListesi("STOKLAR", Ortak.MikroStokGrubu)` — Stok Ana Grup comboboxu. **Isstasyon sirasiyla iliskisi:** ReceteSira (detay/urun agaci sirasi) burada girilir; operasyon sirasindan farklıdır. **Notlar:** StokTuru "Grup" ise StokAnaGrup, "Sabit" ise VarsayilanStokKodu zorunludur. Yeni kayitta StokKullan ve SiparisdeGosterme varsayilan true gelir.

Recete Stok Sec (renk/beden) (FrmReceteStokSec.cs / FrmReceteStokSec.Designer.cs)

Ne ise yarar: Bir ReceteStok satiri icin stok kodu/ad, renk, beden, ebat, gram, olcu bilgilerini ve coklu renk/beden secim listesini (ReceteStokRenkBeden) belirler. **Once ne olmalı (onkosul):** FrmReceteED -> Stok Ekle (yeni: `MikroStok` gecirilir) veya stok satirina cift-tik (Edit=true, mevcut `ReceteStok` + secili renk/beden listesi gecirilir). **Sonra ne olur:** Kaydet -> `ReceteStok` guncellenir, `RenkBedenListSecilen` (Sec=true olanlar) doldurulur, `Secildi=true` ile kapanir; cagiran ekran model listelerine yansitir (DB'ye yazma ana ekranda). **Butonlar & kisayollar:** - **Kaydet** (BtnKaydet) — alanlari `ReceteStok` 'a aktarir, secili beden satirlarini RenkBedenListSecilen'e ekler, kapatir. - Beden grid (myView2) — Sec ve Miktar kolonlari duzenlenebilir. - **Kapat** — iptal. **Cagirdigi katmanlar:** - Service: `IMikroStokService.GetBedenByStokKodu(kod)` — beden listesi. - Service: `IReceteStokRenkBedenService.SelectListWhere(where)` — mevcut renk/beden kayitlarini (RcAld/RcDld/RcSTld) getirir. - Yardimci: `ReceteStokRenkBeden.GetBeden(receteStok, beden)` — eksik bedenler icin satir uretir. **Isstasyon sirasiyla iliskisi:** - **Notlar:** Edit modunda StokKodu kolonu gizlenir; Bedenler Mikro stoktan, secili olanlar mevcut kayitlardan birlesir.

Recete Operasyon Ekle / Duzenle (FrmReceteOperasyonED.cs / FrmReceteOperasyonED.Designer.cs)

Ne ise yarar: Bir recetenin operasyonlarini (ReceteOperasyon: Sira, uretim sure, maliyet, kullanilan aparat, olcum min/max 1-5), her operasyona bagli istasyonlari (ReceteIstasyon: fason, maliyet) ve her istasyona bagli carileri (ReceteIstasyonCari) tanımlar. Uretim akisinin operasyon sirasini belirleyen kritik ekran. **Once ne olmalı (onkosul):** `FrmReceteListesi` veya `FrmReceteED` uzerinden gecerli recete IdGuid'i ile acilir. Kopyalama icin

`Kolonlanacak=true` + `KolonRecete` (hedef recete) set edilir. **Sonra ne olur:** Kaydet -> sıra kontrolu (1'den başlayan tekil ardisik) sonrası `ReceteOperasyonManager.OperasyonKaydet` transaction ile `ReceteOperasyon/Recetelstasyon/RecetelstasyonCari` tablolarini sil-yeniden yazar; duzenlemede ayrıca `ReceteOperasyon_Tablo_AdGuncelle` ve `ReceteIstasyon_Tablo_AdGuncelle` proclari calistirilir. KayitEdildi=true ile kapanir. **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — `TextLeriKontrolEt` + `AktarModele` + `OperasyonSiraKontrol` + `OperasyonKaydet`. - `Sil` (BtnSil) — tum operasyonlar icin `Operasyon_Sil_Kontrol`, hepsi temizse `OperasyonSil`. - `Operasyon Ekle` (BtnOperasyonEkle) — `FrmOperasyonKartlari` secimi; operasyonu Sira artarak ekler, operasyon kartinin varsayilan istasyonu ve ona bagli istasyonlari da otomatik `Recetelstasyon` olarak ekler. - `Operasyon Sil` (BtnOperasyonSil) — `Operasyon_Sil_Kontrol` sonrası operasyonu, bagli istasyon ve carilerini listeden cikarir, siralari yeniden numaralandirir. - `Istasyon Ekle` (BtnIstasyonEkle) — secili operasyona `FrmIstasyonKartlari` ile istasyon ekler (operasyon koduna filtreli). - `Istasyon Sil` (BtnIstasyonSil) — `Istasyon_Sil_Kontrol` sonrası istasyonu ve bagli carileri cikarir. - `Cari Ekle` (BtnCariEkle) — secili istasyona `FrmMikroCariListesi` ile cari ekler (`RecetelstasyonCari`). - `Cari Sil` (BtnCariSil) — secili cariyi cikarir. - `Kapat` — kapat. **Cagirdigi katmanlar:** - Manager: `ReceteOperasyonManager.GetOperasyonKayitEdit(rcAId)` — recete + operasyonlar + istasyonlar + cariler modelini doldurur. - Manager: `ReceteOperasyonManager.OperasyonKaydet(model, yenikayit)` — transaction sil-yaz + tablo adi guncelleme proclari. - Manager: `ReceteOperasyonManager.OperasyonSil/OperasyonSilKontrol/IstasyonSilKontrol` — silme ve bagli hareket kontrolleri. - Service: `IIstasyonKartiService.SelectFirst/SelectList(...)` — operasyon karti varsayilan istasyonu ve bagli istasyonlari. - Service: `IGenelService.GrupListesi("ReceteAna","Grubu")` — grup combobox. - SQL/Prosedur: `Operasyon_Sil_Kontrol`, `Istasyon_Sil_Kontrol` — bagli hareket adedi kontrolu. `ReceteOperasyon_Tablo_AdGuncelle`, `ReceteIstasyon_Tablo_AdGuncelle` — kayit sonrası ad/iliski guncelleme. **Istasyon sirasiyla iliskisi:** DOGRUDAN. Operasyon.Sira burada girilir/kontrol edilir (1'den başlayan, tekil, ardisik); bu deger uretim girisinde `Uretim_PlanlananGuncelle` prosedurunun "operasyon Sira N uretimini Sira N+1 planlanana tasi" mantiginin temelidir. `IstasyonGruplamaKullan=0` senaryosunda `Recetelstasyon` (operasyon-istasyon) eslesmeleri kullanilir. **Notlar:** `OperasyonSiraKontrol Sira=1` zorunlulugu ve tekil ardisik sıra sartini denetler. Operasyon ekleme, operasyon kartinin VarsayilanIstasyonKodu ve operasyona bagli istasyonlari otomatik getirir.

Recete Aciklamalari (`FrmReceteAciklamalar.cs` / `FrmReceteAciklamalar.Designer.cs`)

Ne ise yarar: Bir receteye serbest aciklama satirlari (`AciklamaDeger`) atar; "ReceteAciklama" modulune tanimli aciklama kod sablonlarini (`AciklamaKod`) baz alir. **Once ne olmalı (onkosul):** Kayitli bir recete (`Recete.Id` dolu) gerekir; `FrmReceteED` -> Aciklamalar butonundan acilir. **Sonra ne olur:** Kaydet -> mevcut receteye ait `AciklamaDeger` kayitlari silinip yeniden yazilir (`degerService.Delete + InsertOrUpdate`). Sil -> receteye bagli aciklamalari siler. **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — degerleri sil-yeniden yaz. - `Sil` (BtnSil) — receteye bagli aciklamalari sil. - Grid (`myView1`) — Sira ve Deger1 kolonlari duzenlenebilir. **Cagirdigi katmanlar:** - Service: `IAciklamaDegerService.SelectListWhere(where)` / `Delete` / `InsertOrUpdate` — receteye ait aciklama degerleri. - Service: `IAciklamaKodService.SelectListWhere(where)` — "ReceteAciklama" modulu kod sablonlari (deger yoksa sablondan uretir). **Istasyon sirasiyla iliskisi:** - **Notlar:** Deger yoksa `AciklamaKod` listesinden satirlar uretilip `EntId=Recete.Id` ile baglanir. Kaydette null `Deger1/2/3` bos string'e cevrilir.

Recete Maliyet (Genel) (FrmReceteMaliyetGenel.cs / FrmReceteMaliyetGenel.Designer.cs)

Ne ise yarar: Recetenin stok maliyeti (varsayılan stokların Mikro standart maliyeti x miktar) + operasyon maliyeti toplamını hesaplar ve genel maliyeti gösterir; yazdırılabilir. **Once ne olmalı (onkosul):** FrmReceteED - > Stok Maliyet ile acilir; Model (ReceteKayitModel klonu) set edilmiş olmalı. **Sonra ne olur:** Bagla -> stok ve operasyon maliyetleri hesaplanır, ekranda gösterilir (DB'ye yazma yok). Yazdır -> DataSet'e çevirip rapor basar. **Butonlar & kısayollar:** - Bagla (BtnBagla) — maliyetleri yeniden hesaplar. - Yazdır (BtnYazdır) — "ReceteMaliyet" raporunu (StokMaliyet/OperasyonMaliyet/ToplamMaliyet tabloları) basar. - Kapat (BtnKapat) — kapatır. **Çagırdığı katmanlar:** - Manager: MikroReceteManager.GetStokStandartMaliyetler(stokKodları) — Mikro STOKLAR.sto_standartmaliyet değerleri. - Service: _db.ReceteOperasyon.SelectList(c => c.RcAId == id) — operasyon maliyetleri (MaliyetFiyat toplamı). - Yazdırma: DataSet.Yaz("ReceteMaliyet", false) — rapor çıktı. **İstasyon sırasıyla ilişkisi:** - **Notlar:** Sadece VarsayılanStokKodu dolu detaylar maliyete dahil edilir.

Recete Maliyet (Stoklar) (FrmReceteMaliyetStoklar.cs / FrmReceteMaliyetStoklar.Designer.cs)

Ne ise yarar: Recetede stokların farklı maliyet bazlarını (Son Alış, Son 5 Alış Ortalama, Standart Maliyet, Son Sayım Giriş, Devir Giriş) miktar x fiyat olarak ayrı ayrı hesaplar ve toplamalarını gösterir; yazdırılabilir. **Once ne olmalı (onkosul):** FrmReceteListesi -> Stok Maliyet veya FrmReceteED ile acilir; IdGuid (recete) set edilmiş olmalı (model DB'den taze çekilir). **Sonra ne olur:** Bagla -> recete detayları + Mikro fiyatlar birleştirilip maliyet kolonları hesaplanır (DB'ye yazma yok). Yazdır -> rapor basar. **Butonlar & kısayollar:** - Bagla (BtnBagla) — receteyi çekip maliyetleri hesaplar. - Yazdır (BtnYazdır) — "ReceteStoklarMaliyet" raporunu (StokMaliyet/ToplamMaliyet) basar. - Kapat (BtnKapat) — kapatır. **Çagırdığı katmanlar:** - Manager: ReceteManager.GetReceteKayit(IdGuid) — recete modelini getirir. - Manager: MikroReceteManager.GetStokSonAlışSatisFiyatlar(stokKodları) — Mikro fn_by_Stok_Son5_Giris_Fiyati ve sto_standartmaliyet üzerinden 5 maliyet bazı. - Yazdırma: DataSet.Yaz("ReceteStoklarMaliyet", false). **İstasyon sırasıyla ilişkisi:** - **Notlar:** Maliyet bazları fn_by_Stok_Son5_Giris_Fiyati(sto_kod, NULL, n) fonksiyonunun n=0..3 parametreleriyle gelir.

Recete Sec (FrmReceteSec.cs / FrmReceteSec.Designer.cs)

Ne ise yarar: Sade recete seçim/arama diyalogu (tek grid). Baska ekranlarda recete seçmek için kullanılır; seçim modunda seçili receteyi dondurur, normal acilista FrmReceteED ile düzenleme yapar. **Once ne olmalı (onkosul):** Seçim modunda SecimIcinAcildi=true ile acilir. Form acilisinda filtresiz arar. **Sonra ne olur:** Çift-tik/Enter -> seçim modunda SecilenRow/SecilenKod/SecilenId set edilip Secildi=true ile kapanır; değilse FrmReceteED acilir. **Butonlar & kısayollar:** - Ara (BtnAra) — Kodu/Adi/Grup/Ara filtreleriyle yeniden listeler. - Temizle (BtnTemizle) — filtreleri bosaltır. - Grid çift-tik / Enter (myView1) — sec veya duzenle. - Kapat — kapat. **Çagırdığı katmanlar:** - Service: IReceteAnaService.SelectListWhere(where) — receteleri listeler. - Service: IGenelService.GrupListesi("ReceteAna", "Grubu") — grup combobox. **İstasyon sırasıyla ilişkisi:** - **Notlar:** FrmReceteListesi 'nin sadeleştirilmiş muadili; detay/operasyon gridleri yoktur.

Recete Grup Listesi (FrmReceteGrupListesi.cs / FrmReceteGrupListesi.Designer.cs)

Ne ise yarar: Recete gruplarını (ReceteGrup = birden çok receteyi takım halinde toplayan üst kayıt) listeler; seçili grubun içindeki receteleri alt gridda gösterir. Seçim modunda grup dondurur. **Once ne olmalı (onkosul):** Acilista grup filtreleri ve grup comboboxu yüklenir, liste çekilir. **Sonra ne olur:** Çift-tik/Enter -> seçim modunda grubu dondurur; değilse FrmReceteGrupED (grup kart) acilir. Grup degisiminde alt grid grubun receteleriyle dolar. **Butonlar & kısayollar:** - Ara (BtnAra) — Kodu/Aciklama/Grup filtreleriyle listeler. - Temizle

(BtnTemizle) — filtreleri bosaltir. - `Grup/Takim Ekle` (BtnGrupTakimEkle) — bos `FrmReceteGrupED` acar. - Grid cift-tik / Enter (myView1) — sec veya `FrmReceteGrupED` ile duzenle. - `Kapat` — kapat. **Cagirdigi katmanlar:** - Service: `IReceteGrupService.SelectListWhere(where)` — grup listesi. - Service: `IReceteGrupDetayService.SelectList(c => c.RcGId == id)` — gruptaki recete id'leri. - Service: `IReceteAnaService.SelectListWhere(" where Id IN(...)" )` — gruba bagli recete basliklari. - Service: `IGenelService.GrupListesi("ReceteAna","Grubu")` — grup combobox. **Istasyon sirasiyla iliskisi:** - **Notlar:** Grup, recetelerin uzerinde bir "takim" katmanidir; uretim akisini dogrudan etkilemez.

Recete Grup Ekle / Duzenle (`FrmReceteGrupED.cs` / `FrmReceteGrupED.Designer.cs`)

Ne ise yarar: Bir recete grubunun kodu, grubu, aciklamasi ile gruba dahil receteleri (ReceteGrupDetay: recete + miktar + aciklama) tanimlar. **Once ne olmalı (onkosul):** `FrmReceteGrupListesi` -> Grup/Takim Ekle (yeni) veya satira cift-tik (IdGuid dolu -> duzenleme). **Sonra ne olur:** Kaydet ->

`ReceteGrupManager.ReceteGrupKaydet` transaction ile ReceteGrup upsert + ReceteGrupDetay sil-yeniden yaz; KayitEdildi=true ile kapanir, liste yenilenir. Sil -> grup ve detaylarini siler. **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — TextLeriKontrolEt (kod bossa GetEvrakNo) + AktarModele + ReceteGrupKaydet. - `Sil` (BtnSil) — onayla sonra grubu sil. - `Detay Ekle` (BtnDetayEkle) — `FrmReceteListesi` secimiyle gruba recete ekler (Miktar=1). - `Detay Sil` (BtnDetaySil) — secili recete detayini cikarir. - TxtReceteKodu buton — `GetEvrakNo("Recete")` ile grup kodu uretir. - Grid (myView1) — Miktar ve Aciklama kolonlari duzenlenebilir. - `Kapat` — kapat. **Cagirdigi katmanlar:** - Manager: `ReceteGrupManager.GetReceteKayit(id)` — grup + detaylar modeli. - Manager: `ReceteGrupManager.ReceteGrupKaydet(model, yenikayit)` / `ReceteGrupSil(model)` — transaction sil-yaz / sil. - Service: `IGenelService.GrupListesi("ReceteGrup","Grubu")` — grup combobox; `GetEvrakNo("Recete")` — kod. **Istasyon sirasiyla iliskisi:** - **Notlar:** ReceteGrupDetay her satiri bir receteyi ve takimdaki miktarini tutar.

Yardimci dosya (`ReceteStokRenkBeden.cs`)

Ne ise yarar: `MyUI.ReceteModule` namespace altinda bos govdeli dosya (UI mantigi icermez). Renk/beden entity'si (`My.Entities.ReceteStoklar.ReceteStokRenkBeden`) `FrmReceteStokSec` 'te kullanilir; bu dosyanin kendisi aktif kod barindirmaz. **Once ne olmalı (onkosul):** - **Sonra ne olur:** - **Butonlar & kisayollar:** - **Cagirdigi katmanlar:** - **Istasyon sirasiyla iliskisi:** - **Notlar:** Pratikte islevsiz/placeholder dosya.

Modul: ReceteIstasyonGrupModul

Bu modul, "istasyon gruplama" akisinin tanim/eslestirme altyapisini yonetir. Uretimde her operasyonun hangi istasyonda yapilacagini "grup kodu" üzerinden tanimlamaya yarar. Uc ekrandan olusur: (1) grup kodlarinin sozluk kaydi (`FrmReceteIstasyonGrupKodlari`), (2) bir grup kodu altinda hangi operasyonun hangi istasyona gidecegini esleyen tablo (`FrmReceteIstasyonGrupOperasyonEslestir`), ve (3) belirli bir receteye bir veya daha fazla grup kodu baglamak (`FrmReceteIstasyonGrupIstasyonEslestir`). Bu tanimlar dogrudan veri girisi yapmaz; uretim motorunda `Uretim_SonrakiIstasyonaGonder` prosedurunun

`ReceteAna.IstasyonGruplamaKullan = 1` iken hangi operasyona hangi istasyonun (TEK `UretimIstasyon` kaydi) otomatik olusturulacagini belirleyen referans veridir. Eslesme `ReceteIstasyonGrupOperasyon` tablosu üzerinden `GrupKodu = UretimEmri.IstasyonGrupKodu AND OperasyonKodu = UretimOperasyon.OperasyonKodu` JOIN'i ile kurulur.

Uc ekran da `MyFrmKayit` (harici `My.Kontrol.Formlar` DLL'i) base formundan turer. Base form standart alt buton seridini (BtnKaydet, BtnSil, BtnDuzenle, BtnYeni, BtnKapat, BtnYazdir, navigasyon: BtnIlk/BtnOnceki/BtnSonraki/BtnSon) ve secim modu altyapisini (`SecimIcinAcildi` , `Secildi` , `SecilenKod` , `SecilenRow` , `SecilenId` , `IdGuid` , `YeniKayit` , `KayitEdildi` , `AcilisBittimi`) saglar. Buton kisayol tuslari (ShortcutKeys) Designer dosyalarinda tanimli degildir; butonlar Click event'leriyle calisir. Grid icinde cift tiklama/Enter (`MyEventDoubleClickEnter`) secim modunda kaydi secip kapatir, normal modda Duzenle'yi tetikler. Hicbir ekranda BtnYazdir icin handler baglanmamistir (pasif).

Istasyon Grup Kodlari (`FrmReceteIstasyonGrupKodlari.cs`)

Ne ise yarar: Istasyon grup kodlarinin ana sozluk/tanim ekranidir (Kodu, Adi, Aciklama). Diger iki ekran ve recete bu kodlari referans alir. Hem standalone kayit ekrani hem de baska ekranlardan "kod sec" amacli (SecimIcinAcildi) acilan secim ekrani olarak calisir. **Once ne olmalı (onkosul):** Yok. Bagimsiz bir tanim ekranidir; FrmAna ribbon menüsünden ("Istasyon-Grup Kodlar") veya diger eslestirme ekranlarindaki grup kodu butonundan acilir. **Sonra ne olur:** Kaydet -> `ReceteIstasyonGrupKod` tablosuna insert/update; liste yeniden baglanir. Sil -> ilgili satir silinir. Secim modunda cift tik/Enter ile `SecilenKod` / `SecilenRow` / `SecilenId` doldurulur, `Secildi=true` yapilip form kapanir (cagiran ekran kodu alir). **Butonlar & kisayollar:** - `Kaydet` (`BtnKaydet`) — `Kaydet()` cagirir: zorunlu alan (Kodu) kontrolu + `KodVarmi(...)` benzersizlik kontrolu, sonra `InsertOrUpdate` , ardindan `Bagla()` . - `Sil` (`BtnSil`) — once `MesajSor("Kaydi silmek istiyormusunuz..")` onayi; secili kayit yoksa uyari; `Sil()` -> `Delete` . - `Duzenle` (`BtnDuzenle`) — secili grid satirini klonlayip text kutularina aktarir (`AktarTextlere`) . - `Yeni` (`BtnYeni`) — alanlari temizler (`TemizleText`) , `YeniKayit=true` . - `Kapat` (`BtnKapat`) — base form formu kapatir. - Grid cift tik / Enter (`MyView1.MyEventDoubleClickEnter`) — secim modunda kodu secip kapatir; degilse Duzenle'yi tetikler (`BtnDuzenle.PerformClick()`). **Cagirdigi katmanlar:** - Manager/Service: `IReceteIstasyonGrupKodService` (Ortak.DbPro.ReceteIstasyonGrupKodlar) — `SelectListWhere(" Order By Kodu ")` liste, `InsertOrUpdate(_mdl)` kayit, `Delete(_mdl)` sil. - Service: `ReceteIstasyonGrupKodService.KodVarmi<ReceteIstasyonGrupKod>(_mdl,"Kodu",YeniKayit)` — ayni Kodu ile baska kayit var mi diye `Select count(*)` ile kontrol eder; varsa hata. - SQL/Prosedur: `ReceteIstasyonGrupKod` tablosu (CRUD). Dolayli olarak `Uretim_SonrakiIstasyonaGonder` bu kodlari (UretimEmri.IstasyonGrupKodu üzerinden) tüketir. - API: - **Istasyon sirasiyla iliskisi:** Dolayli. Burada tanimlanan grup kodu, operasyon-istasyon eslestirmesinin (sira) ust anahtaridir; sirayi belirlemez ama hangi istasyon setinin kullanilacagini belirler. **Notlar:** Entity alanlari: Id (Guid PK), Kodu, Adi, Aciklama. Grid'de Id kolonu gizli.

`myGrid1.MyGridKayitAdi = "IstasyonKartlariListesi"` (grid yerlesim kayit adi). Ribbon caption "Istasyon-Grup Kodlar", form basligi "Istasyon Grup Kodlari".

Recete Istasyon-Operasyon Eslestir (`FrmReceteIstasyonGrupOperasyonEslestir.cs`)

Ne ise yarar: Bir grup kodu altında, her operasyon kodunun hangi istasyon koduna/adina karşılık geldiğini esleyen tablodur (GrupKodu + OperasyonKodu -> IstasyonKodu/IstasyonAdi). Uretim gruplama motorunun (`Uretim_SonrakiIstasyonaGonder`) operasyon basına TEK `UretimIstasyon` kaydı oluşturmak için okuduğu ana referanstır. **Once ne olmalı (onkosul):** En az bir grup kodu (`FrmReceteIstasyonGrupKodlari` 'nda) tanımlı olmalı; eslenecek operasyon kodları (Operasyon Kartları) ve istasyon kodları (Istasyon Kartları) tanımlı olmalı. Ekran `FrmAna` ribbon ("Istasyon-Grup Operasyonlar") veya `FrmReceteED`'deki "Istasyon-Grup Ayarlar" butonundan açılır. **Sonra ne olur:** Once grup kodu seçilir (`TxtGrupKodu` butonu), liste o gruba filtrelenir. Yeni/Düzenle ile operasyon ve istasyon seçilip Kaydet -> `ReceteIstasyonGrupOperasyon` tablosuna insert/update; liste `Bagla()` ile o grup koduna göre yeniden yüklenir, text alanları kilitlenir (`TxtLeriKapat`). Sil -> satır silinir. **Butonlar & kisayollar:** - `Kaydet (BtnKaydet)` — `Kaydet()` : GrupKodu/OperasyonKodu/IstasyonKodu zorunlu kontrolü, `InsertOrUpdate` , `Bagla()` , `TxtLeriKapat()` . Acilista pasiftir, Yeni/Düzenle ile aktiflesir. - `Sil (BtnSil)` — `MesajSor` onayı + seçim kontrolü, `Sil()` -> `Delete` . Acilista pasif. - `Duzenle (BtnDuzenle)` — seçili satırı klonlar, alanlara aktarır (`AktarTextlere`), alanları acar (`TxtLeriAc`). - `Yeni (BtnYeni)` — `TemizleText` (GrupKodu hariç temizler), alanları acar, boş model oluşturur. - `TxtGrupKodu (ButtonEdit butonu)` — `TxtGrupKodu_ButtonClick` : `FrmReceteIstasyonGrupKodlari` 'ni seçim modunda acar, seçilen kodu yazıp listeyi yeniler. - `TxtOperasyonKodu / TxtOperasyonAdi (ButtonEdit butonu)` — `TxtOperasyonKodu_ButtonClick` : `_mdl` null ise işlem yapmaz; `FrmOperasyonKartlari` 'ni seçim modunda acar, seçilen `OperasyonKodu / OperasyonAdi` 'ni yazar. - `TxtIstasyonKodu / TxtIstasyonAdi (ButtonEdit butonu)` — `TxtIstasyonKodu_ButtonClick` : `_mdl` null ise işlem yapmaz; `FrmIstasyonKartList` 'i seçim modunda acar, seçilen `IstasyonKodu / IstasyonAdi` 'ni yazar. - `Kapat (BtnKapat)` — formu kapatır. - Grid çift tik / Enter — seçim modunda GrupKodu seçip kapatır; değilse Düzenle. **Cagirdigi katmanlar:** - Manager/Service: `IReceteIstasyonGrupOperasyonService` (Ortak.DbPro.ReceteIstasyonGrupOperasyonlar) — `SelectListWhere(" where GrupKodu='...' Order By GrupKodu ")` liste, `InsertOrUpdate` , `Delete` . Diğer formlar: `FrmReceteIstasyonGrupKodlari` (grup kodu seçimi), `FrmOperasyonKartlari` (MyUI.UretimOperasyonModule, operasyon seçimi), `FrmIstasyonKartList` (MyUI.IstasyonModul, istasyon seçimi). - SQL/Prosedur: `ReceteIstasyonGrupOperasyon` tablosu (CRUD). `Uretim_SonrakiIstasyonaGonder` bu tabloyu `LEFT OUTER JOIN ReceteIstasyonGrupOperasyon GROUP ON GROUP.GrupKodu = UR.IstasyonGrupKodu AND GROUP.OperasyonKodu = URO.OperasyonKodu` ile okuyup her operasyon için `UretimIstasyon` (IstasyonKodu/IstasyonAdi) kaydı acar. - API: - **Istasyon sirasiyla iliskisi:** Dogrudan ilişkili ama sirayı KENDISI tutmaz. Designer'daki uyarı etiketi: "Sıralamayı Operasyondan Alır. Burda eşleşenleri operasyondan kontrol eder." Yani operasyon sirası `UretimOperasyon / ReceteOperasyon` tarafından belirlenir; bu ekran sadece her operasyona istasyon esler. **Notlar:** Entity alanları: Id (PK), GrupKodu, OperasyonKodu, OperasyonAdi, IstasyonKodu, IstasyonAdi. Grid'de Id/OprId/IstId kolonları gizlenir (gizlenen OprId/IstId kolonları entity'de yok; SütunGizle çağırısı varsa-yoksa güvenli çalışır). `myGrid1.MyGridKayitAdi = "IstasyonOperasyonEslestirListesi"` . Form başlığı "Recete Istasyon-Operasyon Eşleştir".

Receteye Istasyon Grup Sec (`FrmReceteIstasyonGrupIstasyonEslestir.cs`)

Ne ise yarar: Belirli bir receteye (RcAId) bir veya daha fazla istasyon grup kodu bağlar. `ReceteIstasyonGrupIstasyon` tablosuna (RcAId + GrupKodu) kayıt ekler. Böylece o recete üretildiğinde hangi grup kodunun (dolayısıyla operasyon-istasyon eşleştirmesinin) geçerli olacağı belirlenir. **Once ne olmalı (onkosul):** Recete kayıtlı olmalı. `FrmReceteED`'de "Istasyon Gruplar" butonu, recete kayıtlı değilse ("Recete Kayıt Edilmeden Istasyon Grup Eklenemiyor.") engeller; aksi halde `f.RcAId = _mdl.Recete.Id` atayarak bu formu

acar. Eslenecek grup kodunun operasyonlari (`FrmReceteIstasyonGrupOperasyonEslestir` 'de) tanimli olmalı.

Sonra ne olur: Acilista recete bilgisi (`ReceteBagla`) ve mevcut esleme listesi (RcAld'ye gore) yuklenir, text/butonlar kilitlidir. Grup kodu secildiginde `ReceteEslesiyormu(kod)` ile recetenin operasyonlari ile grup operasyonlarinin tam eslesip eslesmedigi dogrulanir; eslesmiyorsa kayıt engellenir. Kaydet ->

`ReceteIstasyonGrupIstasyon` tablosuna insert/update, liste yeniden baglanir, alanlar kilitlenir. Sil -> esleme satiri silinir. **Butonlar & kisayollar:** - `Kaydet (BtnKaydet)` — `Kaydet()` : GrupKodu ve ReceteKodu zorunlu kontrolü, `InsertOrUpdate` (RcAld + GrupKodu), `Bagla()` , `TxtLeriKapat()` . Acilista pasiftir. - `Sil (BtnSil)` — `MesajSor` onayi + secim kontrolü, `Sil()` -> `Delete` . Acilista pasif. - `Duzenle (BtnDuzenle)` — secili satiri klonlar, GrupKodu'nu aktarir, alanlari acar (`TxtLeriAc`). - `Yeni (BtnYeni)` — `TemizleText` , alanlari acar, bos model. - `TxtGrupKodu (ButtonEdit butonu)` — `TxtGrupKodu_ButtonClick` :

`FrmReceteIstasyonGrupKodlari` 'ni secim modunda acar; secilen kod için `ReceteEslesiyormu(kod)` kontrolü yapilir, gecerse kod yazilip liste yenilenir, degilse uyarı verip iptal eder. - `Kapat (BtnKapat)` — formu kapatir. - Grid çift tik / Enter — secim modunda GrupKodu secip kapatir; degilse Duzenle. - `TxtReceteKodu` ,

`TxtReceteAdi` — salt okunur (Enabled=false), acilista receteden doldurulur. **Çagirdigi katmanlar:** -

Manager/Service: `IReceteIstasyonGrupIstasyonService` (Ortak.DbPro.ReceteIstasyonGrupIstasyonlar) —

`SelectListWhere(" where RcAld='...' Order By GrupKodu ")` , `InsertOrUpdate` , `Delete` . -

Manager/Service: `IReceteAnaService` (Ortak.DbPro.ReceteAna) — `SelectFind(RcAld)` ile recete bilgisini

(ReceteKodu/ReceteAdi) getirir. - Manager/Service: `IReceteOperasyonService`

(Ortak.DbPro.ReceteOperasyon) — `SelectList(c => c.RcAld==RcAld)` recetenin operasyonlari getirir

(eslesme dogrulaması için). - Manager/Service: `IReceteIstasyonGrupOperasyonService`

(Ortak.DbPro.ReceteIstasyonGrupOperasyonlar) — `SelectList(c => c.GrupKodu==kod)` secilen grubun

operasyonlari getirir (eslesme dogrulaması için). - Service: `ReceteEslesiyormu(string kod)` — recetenin her operasyon kodunun, secilen grubun operasyonlari içinde bulunup bulunmadigini kontrol eder; bulunamayan operasyon varsa "Secilen Grup ile Recete Operasyonları eşleşmiyor" hatası. - SQL/Prosedur:

`ReceteIstasyonGrupIstasyon` tablosu (CRUD). Bu eslesme üretim asamasında recete bazlı grup kodu secimini saglar (UretimEmri.IstasyonGrupKodu uretildiginde bu tanım baz alinir). - API: - **Istasyon sirasiyla iliskisi:**

Dolayli. Receteyi bir grup koduna bagladigi için, o recete uretildiginde operasyon-istasyon eslestirmesini (ve dolayisiyla `Uretim_SonrakiIstasyonaGonder` 'in istasyon olusturmasini) etkiler. Sirayi tutmaz. **Notlar:** Entity

alanlari: Id (PK), RcAld, GrupKodu. Grid'de Id/RcAld gizli. `myGrid1.MyGridKayitAdi =`

`"IstasyonReceteEslestirListesi"` . Form basligi "Receteye Istasyon Grup Seç" (lblBaslik arka plan Teal). Tek receteye birden fazla grup kodu eklenebilir (liste). Recete-grup operasyon eslesme dogrulaması, hatalı grup secimini engelleyen önemli iş kuralıdır.

Modul: MalKabul

MalKabul (Mal Kabul) modulu, bir uretim emrine (UretimEmri) bagli olarak uretilen/giren mamul-malzeme stok fislerini (UretimStokFis) listeler ve görüntüler. Fis basligi uretim emri + 1. operasyonun istasyon bilgisiyle (UretimEmri.Sira=1 -> UretimOperasyon -> UretimIstasyon) eslestirilir; fis satirlari ise UretimStokFisHareket tablosundan gelir. Modul iki formdan olusur: liste formu (`FrmMalKabulListe`) ve fis görüntuleme/edit formu (`FrmMalKabulED`). Bu modulun ekranlarinda fis ASLA bu formlardan **kaydedilmez** (FrmMalKabulED.Kaydet()) govdesi yorum satiri, BtnKaydet `Visible=false` , AktarModele() bos); fisler tablet/uretim akisi (IstasyonTakip + IstasyonSevk) tarafından olusturulur. Bu modul sadece okuma, yazdirma ve **silme** (hem UretimStokFis/UretimStokFisHareket/DepoStokHareket hem de Mikro'ya aktarilan fisin geri silinmesi) islemlerini yapar. Fis ayrica Siparis listesinden sag-tik ("Mal Kabul Fisi Ac") ile SipId üzerinden de acilabilir.

Mal Kabul Liste (`MyUI/MalKabul/FrmMalKabulListe.cs` , `FrmMalKabulListe.Designer.cs`)

Ne ise yarar: Uretim stok fislerini (UretimStokFis) tarih araligi ve istasyon kodu/adi filtresiyle listeler. Bir satira çift tiklayinca o fisin detayini (FrmMalKabulED) acar. Form, FrmAna ana menüsünden (`BarBtnMalKabulListe_ItemClick`) MDI child olarak acilir; ayrica baska formlardan "secim modu" (SecimIcinAcildi) ile cagrilabilir. **Once ne olmalı (onkosul):** Sistemde uretim akisi tarafından olusturulmus UretimStokFis kayitlari bulunmalı (uretim girisi -> istasyon sevk akisi fisi yaratir). Form acilir acilmaz Frk_Load icinde tarih filtresi 01.01. olarak set edilir ve `BtnAra.PerformClick()` ile otomatik arama yapilir; ek bir on kosul yoktur. **Sonra ne olur:** Kayit/silme yapmaz; sadece okuma. Çift tik -> normal modda `FrmMalKabulED` (IdGuid=secili fis) dialog acar; secim modunda (SecimIcinAcildi) secilen fisin EvrakNo'sunu SecilenKod'a, satiri SecilenRow'a yazip Secildi=true ile formu kapatir (cagiran forma fis secimi doner). Yazdir butonu liste verisini DataSet'e ceviriip "MalKabul" yazdirma sablonuyla yazdirir. **Butonlar & kisayollar:** - `BtnAra` — `BtnAra_Click` -> `Bagla()` ; SorguAyarla()+SorguAyarlaTrh() ile WHERE kurar, `_srv.GetFisList(sor)` cagirir, gridi doldurur. (Form acilista PerformClick ile otomatik tetiklenir.) - `BtnTemizle` — `BtnTemizle_Click` ; TxtAra'yi bosaltir, TxtTarihi1'i 01.01. yapar, TxtTarihi2'yi bosaltir (arama filtrelerini sifirlar). - `BtnYazdir` — `BtnYazdir_Click` -> `Yazdir()` ; secili satir varsa `list` 'i "Hareketler" DataTable'ina ceviriip "MalKabul" sablonuyla `ds.Yaz("MalKabul", false)` ile yazdirir. - `BtnDizayn` — base liste formundan gelir (yazdirma dizayn duzenleme); MalKabul kodunda ozel Click handler bagli degil. - `BtnKapat` — base form kapatma butonu (My.Kontrol.Formlar / MyFrmListe). - Grid: `myView1.MyEventDoubleClickEnter` (çift tik / Enter) -> `MyView1.MyEventDoubleClickEnter` -> secim modunda fis sec, normal modda FrmMalKabulED ac. - `TxtAra` (Ara), `TxtTarihi1` (>= tarih), `TxtTarihi2` (<= tarih) — filtre giris alanlari; EnterMoveNextControl=true (Enter ile sonraki kontrole gecis). - Not: BtnAra/BtnTemizle/BtnYazdir/BtnDizayn/BtnKapat MyFrmListe base formundan gelir; Text ve ShortcutKeys degerleri proje kaynakinda/resx'te tanimli degil (DevExpress image'li, base formda yönetilir). **Cagirdigi katmanlar:** - Service: `IUretimStokFisService.GetFisList(string sor)` (`UretimStokFisService.GetFisList`) — `UretimStokFisModel.GetSelectSqlCode(sor)` SQL'ini calistirir, List doner. - SQL: `UretimStokFisModel.GetSelectSqlCode` — `UretimStokFis UF LEFT JOIN UretimEmri UR (UF.UrId=UR.Id) LEFT JOIN UretimOperasyon UrO (UrO.UrId=Ur.Id AND UrO.Sira=1) LEFT JOIN UretimIstasyon UrI (UrI.UrOId=UrO.Id)` ile fis + IsEmriNo + SiparisKodu(IsEmriKodu) + 1. operasyon istasyon kodu/adi getirir. Filtre: istasyon kodu/adi LIKE (TxtAra), `UF.Durumu` (durumualani, varsayilani bos), `UF.Tarih` araligi. - Yazdirma: `DataSet.Yaz("MalKabul", false)` (My.Kontrol.Yazdirma uzantisi) — "MalKabul" yazdirma sablonuyla rapor uretir. - API: - **Istasyon sirasiyla iliskisi:** Fis basligindaki istasyon kodu/adi, uretim emrinin **1. operasyonunun** (UretimOperasyon.Sira=1) istasyonu (UretimIstasyon) üzerinden eslestirilir; SorguAyarla'daki arama da bu istasyon kodu/adina gore filtreler. SutunCaptionDegistir ile "SiparisKodu" basligi

"IsEmriKodu" olarak gosterilir. **Notlar:** durumu alani sabit "" (bos) — durum filtresi pratikte calismaz (UF.Durumu filtresi hep atlanir). Arama girdileri SQL'e dogrudan string interpolasyon ile gomulu (parametresiz; SQL injection riski). SutunGizle ile Id ve Urd kolonlari gizlenir, Tarih dd.MM.yyyy formatlanir.

Mal Kabul Fisi (MyUI/MalKabul/FrmMalKabulED.cs , FrmMalKabulED.Designer.cs)

Ne ise yarar: Tek bir uretim stok fisini (UretimStokFis) ve satirlarini (UretimStokFisHareket) salt-okunur görüntüler; fisi (ve Mikro'ya aktarilan karsiligini) silmeyi saglar. Baslik alanlari: Is Emri No/Kodu, Tarih, Istasyon Kodu/Adi, EvrakNo, BelgeNo, Durumu. Form basligi "Mal Kabul Fisi". Iki sekilde acilir: (1) liste cift tikindan IdGuid ile, (2) Siparis listesinden SiparisIdDenAra=true + SipId ile. **Once ne olmalı (onkosul):** Görüntülenecek fisin var olmasi gerekir. IdGuid (liste cift tik) ya da SipId (Siparis listesi "Mal Kabul Fisi Ac") set edilmiş olmalı. SiparisIdDenAra=true ise `GetFisFirst(" where Ur.SipId='...' ")` ile siparis id'sine bagli ilk fis bulunur ve IdGuid ona set edilir. **Sonra ne olur:** Kaydetme yapmaz (Kaydet() govdesi yorumda, BtnKaydet Visible=false). Silme: BtnSil -> `Sil()` ; once `_srv.FisSil(IdGuid)` ile UretimStokFis + UretimStokFisHareket + DepoStokHareket satirlarini (FisId üzerinden, tek transaction) siler, sonra

`_mngMikroKayit.DeleteMikroAktarilanFisByBelgeNo(_mdl.EvrakNo)` ile Mikro tarafinda ayni EvrakNo'lu STOK_HAREKETLERI + BEDEN_HAREKETLERI (ve aktarilan fis kaydi) silinir, KayitEdildi=true yapilip form kapatilir.

Butonlar & kisayollar: - `BtnSil` — `BtnSil_Click` -> `Sil()` ; "Kaydi silmek istiyormusunuz.." onayi (MesajSor) sonrasi yerel fisi + Mikro aktarilan fisi siler, formu kapatir. - `BtnKaydet` — base form kaydet butonu, ANCAK `Visible=false` (kaydetme bu formda devre disi); Kaydet() metodu bos/yorumda. - `BtnYeni` / `BtnDuzenle` / `BtnYazdir` / `BtnKapat` — base kayit formundan (MyFrmKayit) gelir; MalKabul kodunda özel Click handler bagli degil (BtnYazdir bu formda kodla bos). - `BtnIlk` / `BtnOnceki` / `BtnSonraki` / `BtnSon` — base form kayit navigasyon butonlari (ilk/onceki/sonraki/son kayda gecis); MyFrmKayit base davranisi. - Baslik/satir alanlari: TxtIsEmriNo, TxtIsEmriKodu, TxtIstasyonKodu, TxtIstasyonAdi, TxtEvrakNo, TxtBelgeNo, TxtDurumu, TxtTarih — AktarTextlere() ile doldurulur (görüntüleme; AktarModele() bos oldugu icin geri yazma yok). - Not: BtnSil/BtnKaydet/BtnYeni/BtnDuzenle/BtnYazdir/BtnKapat ve navigasyon butonlari MyFrmKayit base formundan gelir; Text ve ShortcutKeys degerleri proje kaynakinda/resx'te yok (base formdayonetilir). Enter ile alanlar arasi gecis (EnterMoveNextControl) aktiftir. **Çagirdigi katmanlar:** - Service:

`IUretimStokFisService.GetFis(Guid id)` — `UretimStokFisModel.GetSelectSqlCode(" where UF.Id='...' ")` ile tek fis getirir. - Service: `IUretimStokFisService.GetFisFirst(string sor)` — SipId'ye bagli ilk fisi getirir (SiparisIdDenAra modunda). - Service: `IUretimStokFisService.GetStokHareketByFisId(Guid? fisId)` — `UretimStokFisHareket.GetSelectSqlCodeByFisId(fisId)` ile fis satirlarini getirir, gride baglar. - Service: `IUretimStokFisService.FisSil(Guid? fisId)` — tek transaction icinde `DELETE FROM UretimStokFis WHERE Id=...` , `DELETE FROM UretimStokFisHareket WHERE FisId=...` , `DELETE FROM DepoStokHareket WHERE FisId=...` . - Manager: `MikroKayitManager.DeleteMikroAktarilanFisByBelgeNo(string belgeNo)` — Mikro DB'de ayni belge no'lu BEDEN_HAREKETLERI ve aktarilan STOK_HAREKETLERI fisini

(MikroStokHareketAktarilanFisler.GetDeleteSqlCodeByBelgeNo) tek transaction'da siler. - SQL/Prosedur: dogrudan stored procedure cagrilmaz; islemler entity'lerin GetSelectSqlCode / DELETE SQL'leri ile yapilir. (Uretim_MiktarGuncelle vb. prosedurler bu modulden tetiklenmez.) - API: - **Istasyon sirasiyla iliskisi:**

Görüntülenen fis basligindaki Istasyon Kodu/Adi, uretim emrinin 1. operasyonunun (UretimOperasyon.Sira=1) UretimIstasyon kaydindan gelir (GetSelectSqlCode JOIN'i). Grid satirlarinda GiriCikis/Sira alanlari gizlenir; satir akisi/istasyon sevkiyle dogrudan etkilesim yoktur (sadece okuma+silme). **Notlar:** Sil() icinde Mikro silme sonucu kontrolu ters mantikla yazilmis: `if (rs2.Success) { MesajHata(rs2.Message); return; }` — Mikro silme BASARILI olunca hata mesajı gosterip return ediyor (form kapanmadan once); bu olasi bir bug. AktarModele() ve TextLeriKontrolEt() govdeleri tamamen yorum/bos; Kaydet() bu formda hicbir sey yapmaz. Grid `MyGridKayitAdi="SiparisKayit1"` (Siparis formundan kopyalanmis adlandirma). Form Designer'da Text="FrmMalKabulED" olsa da lblBaslik.Text="Mal Kabul Fisi" gosterilir.

Modul: UretimKontroller

UretimV4 (CepPatronERP masaustu ERP) icindeki UretimKontroller modulu, saha operatorlerinin tablet üzerinden girdigi olcum/kalite kontrol kayitlarini ofis tarafinda goruntulemeye ve raporlamaya yarayan tek bir salt-okunur liste ekranindan olusur. Olcum kayitlari (UretimKontrol tablosu, Turu='OlcumGiris') asil olarak WebUretim TabletV2 projesindeki istasyon takip akisinda (Pop_IstasyonAktifIslemOlcumGiris.razor -> UretimKontrolService.SaveUretimKontrol) olusturulur; bu modulde herhangi bir kayit ekleme/silme/duzeltme yoktur. Ekran, ortak iki SQL veritabanindan UretimV3_FEZA icindeki UretimKontrol tablosunu (UretimEmri ve Personel ile JOIN ederek) okur. Modulun ana islevi: tarih ve serbest metin filtresiyle olcum kontrol kayitlarini listelemek, tolerans disi (HataliGiris) kayitlari gormek ve listeyi yazdirmek.

Olcu Kontrol Listesi (`FrmOlcuKontrollListesi.cs` / `FrmOlcuKontrollListesi.Designer.cs`)

Ne ise yarar: Uretim sirasinda girilen olcum kontrol kayitlarini (Turu='OlcumGiris') bir grid uzerinde listeler. Her satirda istasyon, stok, olcum degeri (OlcumDegeri/2/3), tolerans araliklari (Min/Max), hatali giris bayragi, is emri no/kodu, kaydeden personel ve tarih gosterilir. Salt-okunur bir izleme/rapor ekranidir; veri girisi yapilmaz. **Once ne olmalı (onkosul):** Olcum kayitlarinin onceden olusmus olmasi gerekir. Bu kayitlar UretimV4'te degil, WebUretim TabletV2 saha akisinda olusur: operator istasyonu baslatip aktif islemdede Olcum Girisini popup'ini (Pop_IstasyonAktifIslemOlcumGiris.razor) acar, olculen degerleri girer ve kaydeder. Kayit UretimKontrolService.SaveUretimKontrol ile UretimKontrol tablosuna Turu='OlcumGiris', UrlId/UrlId/IstHrId, IstasyonKodu/Adi, StokKodu/Adi, OlcumDegeri ve tolerans araliklari (OlcumDegeriMin/Max...) ile yazilir; deger tolerans disindaysa HataliGiris=true isaretlenir. Ayrica ilgili is emrinin (UretimEmri) ve kaydeden personelin (Personel) DB'de bulunmasi gerekir (JOIN icin). **Sonra ne olur:** Ekran salt-okunurdur; hicbir tabloyu yazmaz/degistirmez ve hicbir stored procedure cagirmaz. Yapilabilecek tek "sonra" islemi: secili kayit varken Yazdir ile listenin DevExpress rapor cikti uretimine gonderilmesidir (ds.Yaz("UretimKontrol", false)). Cift tiklama yalnızca form baska bir formdan "secim icin" acildiyse (SecimIcinAcildi=true) secilen kaydi geri dondurup formu kapatir; normal acilista cift tiklamanin etkisi yoktur. **Butonlar & kisayollar:** - **Ara (BtnAra)** — Filtreleri uygulayip gridi yeniden doldurur (BtnAra_Click -> Bagla()). Form acilisinda otomatik bir kez tetiklenir (Frm_Load icinde BtnAra.PerformClick()). - **Temizle (BtnTemizle)** — Arama kutusunu bosaltir, baslangic tarihini (TxtTarihi1) yilbasina (01.01.) ceker, bitis tarihini (TxtTarihi2) bosaltir (BtnTemizle_Click). Not: temizledikten sonra otomatik arama yapmaz; yeniden Ara'ya basmak gerekir. - **Yazdir (BtnYazdir)** — Secili kayit varsa listeyi (list) "Hareketler" tablosu olarak bir DataSet'e koyup "UretimKontrol" rapor sablonuyla yazdirir (BtnYazdir_Click -> Yazdir()). - **Dizayn (BtnDizayn)** — Base form (MyFrmListe) uzerinden gelen rapor/grid dizayn butonu; bu formda ozel bir Click handler baglanmamistir (base davranis). - **Kapat (BtnKapat)** — Base form (MyFrmListe) uzerinden gelen formu kapatma butonu; bu formda ozel handler yok (base davranis). - **Cift tiklama / Enter (myView1.MyEventDoubleClickEnter)** — Secim modunda (SecimIcinAcildi) secili UretimKontrol satirini SecilenRow'a atar, Secildi=true yapar ve formu kapatir; normal modda islem yapmaz. - Arama alanlari: **TxtAra** (serbest metin: IstasyonKodu/IstasyonAdi/IsEmriNo/SiparisKodu icinde LIKE), **TxtTarihi1** (baslangic tarihi >=), **TxtTarihi2** (bitis tarihi <=). - Not: Buton Text/ShortcutKeys (Enter=Kaydet, Esc=Kapat vb.) tanimlari bu formda degil, harici base form MyFrmListe (My.Kontrol.Formlar, DLL) icindedir; .Designer.cs'de yalnızca gorunum/reng ve Click baglamalari vardir. **Cagirdigi katmanlar:** - Service: **IUretimKontrolService.GetViewListWhere(string whereSql)** (UretimKontrolService) — UretimKontrol tablosunu UretimEmri (UrlId) ve Personel (KayitEden=Personel.Kodu) ile LEFT JOIN ederek IsEmriNo, IsEmriKodu (SiparisKodu) ve Kullanici (Adi+Soyadi) ek alanlariyla birlikte verilen WHERE kosuluna gore okur; Dapper Query ile calisir, sonucu SuccessDataResult/ErrorDataResult olarak doner. - Service (base): **IGenelService** (Ortak.DbPro.GenelServis) — formda alani tanimli ancak bu ekranda dogrudan cagrilmaz (genel yardimci servis

referansi). - DataAccess: `UretimKontrolDal : BaseDal<UretimKontrol>` — tablo erisimi icin Dapper tabanlı DAL; bu ekranda yalnızca okuma (Query) yolu kullanılır. - SQL/Prosedur: Yok. Ekran herhangi bir stored procedure çağırır (Uretim_MiktarGuncelle / Uretim_PlanlananGuncelle / Uretim_SonrakistasyonaGonder / Uretim_DurumGuncelle ile ilişkisi yoktur). Tek SQL, GetViewListWhere içindeki dinamik SELECT + WHERE sorgusudur. - Yazdırma: `ds.Yaz("UretimKontrol", false)` (My.Kontrol.Yazdırma) — list -> DataTable("Hareketler") -> DataSet("UretimKontrolDS") rapor çıktı motoru. - API: Yok (UretimV4 masaüstü, DB'ye doğrudan bağlanır; API kullanmaz). **Istasyon sirasiyla ilişkisi:** Doğrudan miktar/akış motoruna katkı vermez; istasyon sırasını (operasyon Sıra N -> N+1) etkilemez. Yalnızca kayıtlı istasyon bazında gösterir (IstasyonKodu/IstasyonAdı) ve her ölçüm satırı bir UretimIstasyonHareket'e (IstHrld) ile UretimIstasyon'a (Urld) ve is emrine (Urld) bağlıdır. Tolerans dışı (HatalıGiris) ölçümler bu ekranda görülsede istasyon sevkini/durumunu burada değiştirmez; o akış TabletV2 tarafında yürütülür. **Notlar:** - Filtre sabit olarak `UK.Turu='OlcumGiris'` ile düşünülmüş olsa da (SorguAyarla içinde t1 değişkenine atanıp hemen TxtAra.Text ile ezilmesi nedeniyle) bu koşul sorguya EKLENMEZ; pratikte tüm UretimKontrol türleri listelenir. Bu, koddaki bir hata/olu satırdır (string t1 = "...OlcumGiris..."; hemen ardından t1 = TxtAra.Text.Trim()). - Form açılışında başlangıç tarihi her zaman içinde bulunulan yılın 01 Ocak'ına set edilir; bitiş tarihi boş gelir (üst sınır yok). - Arama sorguları parametresiz string interpolasyonu ile kurulur (LIKE '%...%' ve CAST tarih koşulları) - SQL injection açısından korunmasızdır; analiz notu olarak belirtilmiştir. - Gizlenen kolonlar: Id, Urld, Urld, IstHrld (SutunGizle); Tarih kolonu "dd.MM.yyyy HH:mm" formatlıdır. Grid yerleşimi MyGridKayıtAdi="OlcuKontrolListesi" / MyGridKayıtAdi GridYerleşimYükle() ile kullanıcıya özel saklanıp yüklenir. - Entity (UretimKontrol) 5 ölçüm değeri (OlcumDeğeri..5) ve 5 tolerans çifti (Min/Max..5) tasır; .cs entity'sinde OlcumDeğeri4/5 alanları yokken (Ignore'lu IsEmriNo/Kullanici/IsEmriKodu var) tablo create dosyasında (UretimKontrolCreates.cs) OlcumDeğeri4/OlcumDeğeri5 kolonları da oluşturulur.

Modul: UretimTalepler

Uretim Talepleri modulu, ofis/yonetim tarafından bir istasyona ureilmesi istenen mamul/recete kalemlerini "uretim talebi" evraki olarak girmek ve listelemek icin kullanilir. Modul iki formdan olusur: liste ekranı (`FrmUretimTalepList`) ve kayıt/duzenleme ekranı (`FrmUretimTalepED`). Veriler `UretimTalep` (evrak basligi) ve `UretimTalepHareket` (kalem satirlari) tablolarında tutulur; her satirda hedef istasyon (IstasyonKodu/IstasyonAdi), stok/recete (StokKodu/StokAdi), miktar, birim, parti ve lot bilgileri yer alır. Stok kalemleri Recete Listesi (`FrmReceteListesi`) üzerinden secilir. Modul, bu sahadaki diger uretim/akis motoruna (Uretim_MiktarGuncelle, Uretim_PlanlananGuncelle vb. prosedurler) DOGRUDAN baglanmaz; sadece bir talep/evrak kaydi olusturur. Kayit, `UretimTalepManager` üzerinden tek transaction icinde yapilir (basligi InsertOrUpdate, hareketleri sil-yeniden yaz). Formlar `My.Kontrol.Formlar` icindeki ortak temel siniflardan turer (liste: `MyFrmListe` , kayıt: `MyFrmKayit`) ve bu temel sinifların standart butonlarını (Kaydet/Kapat/Sil/Yazdir/navigasyon) miras alır.

Uretim Talep Liste (`MyUI/UretimTalepler/FrmUretimTalepList.cs` + `.Designer.cs`)

Ne ise yarar: Girilmis uretim talep evraklarını tarih araligina gore listeler (ust grid: `myGrid1 / myView1`) ve secili evrakin kalem satirlarını alt gride (`myGrid2 / myView2`) gosterir. Buradan yeni talep eklenir veya mevcut talep çift tıklayarak/Enter ile duzenlenir. Form ayrıca baska bir ekrandan "secim icin" acildiginda (SecimIcinAcildi) talep evrakını secip geri dondurebilir. **Once ne olmalı (onkosul):** Uygulamaya giris yapilmis ve veritabanı baglantısı (`Ortak.DbPro`) hazır olmalı. Listede evrak gorunmesi icin onceden en az bir uretim talebi kaydedilmis olmalı (yoksa liste bos gelir). **Sonra ne olur:** Bu ekran salt okuma/listelemedir; dogrudan tablo degistirmez. "Ur.Talep Ekle" veya çift tiklama ile `FrmUretimTalepED` acilir; orada yapılan kaydet/sil islemi `UretimTalep` ve `UretimTalepHareket` tablolarını gunceller, sonra `ActionAktar = Bagla` callback'i ile liste yeniden yuklenir. Secim modunda (`SecimIcinAcildi`) çift tik/Enter -> `SecilenKod = EvrakNo` , `SecilenRow` , `Secildi=true` set edilip form kapanir. **Butonlar & kisayollar:** - `BtnAra` ("Ara") — `BtnAra_Click` -> `Bagla()` : tarih filtreleriyle (`TxtTarihi1 / TxtTarihi2`) sorgu kurup `UretimTalep` listesini yeniden ceker. - `BtnTemizle` ("Temizle") — `BtnTemizle_Click` : `TxtTarihi1` ve `TxtTarihi2` tarih filtre kutularını bosaltir (listeyi otomatik yenilemez). - `BtnEkleUretimTalep` (Text: "Ur.Talep Ekle") — `BtnEkleUretimTalep_Click` : yeni kayıt icin `FrmUretimTalepED` acar (`ActionAktar = Bagla`), `ShowDialog()` . - `BtnKapat` (temel `MyFrmListe` butonu) — formu kapatir. - `BtnYazdir` / `BtnDizayn` (temel `MyFrmListe` butonları, Designer'da tanimli) — bu formda özel click handler baglanmamis; grid yazdirma/dizayn temel davranisi. - Grid çift tik / Enter — `MyView1_MyEventDoubleClickEnter` : secim modunda evraki dondurur, normal modda `FrmUretimTalepED` (duzenleme) acar. - Ust gride satir degisimi — `MyView1_FocusedRowChanged` -> `BaglaHareket(itm.UrtTlpId)` : secili evrakin hareketlerini alt gride yukler. - `TxtTarihi1 / TxtTarihi2` (MyDateEdit) — tarih araligi filtresi; `Frm_Load` 'da `TxtTarihi1` bugun-7 gune set edilir. **Cagirdigi katmanlar:** - Manager/Service: `IUretimTalepService.SelectListWhere(string)` (`_srv` , = `Ortak.DbPro.UretimTalep`) — `where 1=1` + tarih sartiyla `UretimTalep` kayitlarını ceker. - Manager/Service: `IUretimTalepHareketService.SelectList(c => c.UrtTlpId == urtTlpId)` (`_srvHareket`) — secili evrakin `UretimTalepHareket` satirlarını ceker. - Manager/Service: `UretimTalepManager` (`_mng` , `Frm_Load` 'da kurulur) — bu formda olusturulur ancak metodları dogrudan cagrilmaz (kayıt/sil islemleri ED formuna devredilir). - Manager/Service: `IGenelService` (`_srvGenel`) — alanı tanimli, bu formda aktif kullanilmiyor. - SQL/Prosedur: dogrudan stored procedure cagrilmaz; sorgu BaseService/DAL üzerinden parametrik `WHERE` ile uretilir (filtre: `CAST(coalesce(Tarih,'1901-01-01') AS DATE)` araligi). - API: - **Istasyon sirasiyla iliskisi:** Dolayli. Hareket satirlarında hedef `IstasyonKodu / IstasyonAdi` gorunur; ancak bu liste ekranı operasyon-sıra/akis motorunu

(Uretim_MiktarGuncelle, Uretim_SonrakilistasyonaGonder) tetiklemez, yalnızca talep evraklarını gösterir. **Notlar:** `SorguAyarla()` içindeki kod/cari/durum filtreleri tamamen yorum satırı; aktif filtre yalnızca tarih aralığı (`SorguAyarlaTrh`). `SutunGizle` / `SutunGizle2` ile teknik kolonlar (UrtTlpId, Ent, EntId, EntKodu, EntKodu2, EntTarih, UrtTlpHrId) gizlenir. Grid yerleşimleri `MyGridKayitAdi` ile saklanır ("UretimTalepListesi", "UretimTalepListesiDetaylar"). `Turu = "Siparis"` alanı tanımlı fakat aktif sorguda kullanılmıyor.

Uretim Talep Kayıt (`MyUI/UretimTalepler/FrmUretimTalepED.cs` + `.Designer.cs`)

Ne ise yarar: Tek bir üretim talep evrakını oluşturmak/düzenlemek için kullanılır. Üst kısımda evrak başlığı (Tarih, Evrak No, Açıklama); alt kısımda kalem grid'i (`myGrid1` / `myView1`) ile her satır için hedef istasyon (lookup), stok/recete, miktar, birim, açıklama, parti, lot girilir. Kalemler Recete Listesinden seçilerek eklenir. **Once ne olmalı (onkosul):** `FrmUretimTalepList` üzerinden "Ur.Talep Ekle" (yeni) veya çift tik/Enter (`IdGuid` dolu = düzenleme) ile açılmalı. İstasyon lookup'ları için `IstasyonKarti` kayıtları (`Ortak.DbPro.IstasyonKarti`) tanımlı olmalı. Recete seçimi için `ReceteAna` kayıtları (Recete modulu) tanımlı olmalı. Kaydet/Sil butonlarının aktif olması için lisans aktif olmalı (`Ortak.LisansAktif`). **Sonra ne olur:** "Kaydet" -> doğrulama (`TextLeriKontrolEt`) + modele aktarım (`AktarModele`) sonrası `UretimTalepManager.Kaydet(_mdl, Hareketler)` ile TEK transaction içinde: `UretimTalep` başlığı InsertOrUpdate edilir, ardından o evraka ait tüm `UretimTalepHareket` satırları `Delete` edilip yeniden `InsertOrUpdate` yapılır (sil-yaz deseni). Başarılı olursa `KayitEdildi=true` , `ActionAktar?.Invoke()` (liste yenilenir) ve form kapanır. "Sil" -> `UretimTalepManager.Sil(_mdl)` ile başlık ve tüm hareketler tek transaction'da silinir. Bu modul, kayıttan sonra herhangi bir akis prosedürünü (Uretim_MiktarGuncelle vb.) çağırılmaz. **Butonlar & kısayollar:** - `BtnKaydet` (temel `MyFrmKayit` butonu) — `BtnKaydet_Click` -> `Kaydet()` : doğrula, aktar, `_mng.Kaydet(...)` . - `BtnSil` (temel `MyFrmKayit` butonu) — `BtnSil_Click` -> `Sil()` : onay sorar, `_mng.Sil(...)` . - `BtnStokSec` (Text: "F5") — `BtnStokSec_Click` -> `StokSecReceteden()` : Recete Listesinden kalem seçip yeni `UretimTalepHareket` ekler. Klavye kısayolu: **F5** (`Frm_KeyDown` -> `BtnStokSec.PerformClick()`). - `BtnStokSil` (Text: "F8") — `BtnStokSil_Click` -> `StokSil()` : seçili kalem satırını listeden çıkarır (onay sorar). Klavye kısayolu: **F8** (`Frm_KeyDown` -> `BtnStokSil.PerformClick()`). - `TxtSiparisKodu` (Evrak No) editor butonu — `TxtSiparisKodu_ButtonClick` : dolu ise onay sorduktan sonra `EvrakNoAl()` ile yeni evrak no üretir. - `BtnKapat` (temel `MyFrmKayit` butonu) — formu kapatır. - `BtnYazdir` / `BtnYeni` / `BtnDuzenle` / navigasyon butonları `BtnIlk` / `BtnOnceki` / `BtnSonraki` / `BtnSon` (temel `MyFrmKayit` , Designer'da tanımlı) — bu formda özel click handler bağlanmamış; temel form davranışları. - Grid hücre editörü açılınca — `MyView1_ShownEditor` : TextEdit ise mevcut metni otomatik seçer (SelectAll). - İstasyon lookup değişince — `colCmbIstasyonKodu_EditValueChanged` / `colCmbIstasyonAdi_EditValueChanged` : seçilen `IstasyonKarti` 'na göre satırın İstasyonKodu ve İstasyonAdi hücrelerini eşleştirir. **Çağırdığı katmanlar:** - Manager/Service: `UretimTalepManager.GetTalepNew()` — boş `UretimTalep` dondurur (yeni kayıt). - Manager/Service: `UretimTalepManager.GetTalep(Guid?)` -> `UretimTalepService.SelectFirst(c => c.UrtTlpId == id)` — düzenlemede evrak başlığını çeker. - Manager/Service: `UretimTalepManager.GetTalepHareketler(Guid?)` -> `UretimTalepHareketService.SelectList(c => c.UrtTlpId == ...)` — evrakin kalem satırlarını çeker. - Manager/Service: `UretimTalepManager.Kaydet(UretimTalep, List<UretimTalepHareket>)` — tek transaction: `UretimTalep.InsertOrUpdate` + `UretimTalepHareket.Delete(by UrtTlpId)` + `UretimTalepHareket.InsertOrUpdate(list)` ; hata olursa rollback. - Manager/Service: `UretimTalepManager.Sil(UretimTalep)` — tek transaction: `UretimTalep.Delete` + `UretimTalepHareket.Delete(by UrtTlpId)` . - Manager/Service: `IstasyonKartiService.SelectListWhere("")` (`_srvIstasyon`) — istasyon lookup veri kaynağını (`bsCari`) doldurur. - Manager/Service: `IGenelService.GetEvrakNo("UretimTalep")` (`_srvGenel`) -> `GenelDal.GetEvrakNo` — `AyarSayac` tablosundan "UretimTalep" sayacını okuyup formatlar (BasinaEkle + sıfır dolgu/BasamakSayisi) ve sayacı 1 artırır. - Manager/Service: `IMikroStokService` (`_srvCari` =

`Ortak.DbMikro.Stoklar`) — alanı tanımlı; kalem secimi recete üzerinden yapıldığı için aktif yol kullanmıyor (`StokTekSec__` metodu `FrmMikroStokListesi` ile stok secer ama hiçbir butona bağlı değil = olu kod). - UI çağırışı: `FrmReceteListesi` (SecimIcinAcildi) — `StokSecReceteden()` içinde recete secilir; secilen `ReceteAna` 'dan `ReceteKodu->StokKodu`, `ReceteAdi->StokAdi`, `EntegreBirim->Birimi` ile yeni hareket oluşturulur. - SQL/Prosedur: Doğrudan stored procedure çağrılmaz. `GetEvrakNo` içindeki inline SQL (AyarSayac okuma/guncelleme) tek SQL etkilesimidir. - API: - **Istasyon sirasiyla iliskisi:** Her kalem satirinda hedef `IstasyonKodu` / `IstasyonAdi` lookup ile secilir; bu, talebin hangi istasyonda uretilecegini belirtir. Ancak modul operasyon-sira tabanlı akis motorunu (`Uretim_PlanlananGuncelle` / `Uretim_SonrakIstasyonaGonder`) calistirmaz; sadece istasyon bazlı talep kaydi tutar. **Notlar:** Dogrulama (`TextLeriKontrolEt`): Evrak No bossa otomatik `EvrakNoAl()` cagrilir, Tarih bossa simdiki zaman atanir; her kalemde IstasyonKodu bos olamaz ve Miktar > 0 olmalı, aksi halde hata mesajı. `AktarModele` içinde Id'ler `MyGuid.NewGuid()` ile uretilir ve audit alanlari (Kullanici, KayitEden= `Ortak.KullaniciAdi` , KayitTarihi) doldurulur; ayrıca baslik IstasyonKodu/IstasyonAdi tum satirlara kopyalanir (NOT: baslikta IstasyonKodu/IstasyonAdi'yi text'lere set eden bir kontrol yok, bu yuzden `itm.IstasyonKodu = _mdl.IstasyonKodu` satir bazında lookup ile girilen degeri baslik degeriyle (genelde bos) ezme riski tasir - bilinen dikkat noktası). Grid kolonlarından yalnızca Miktar, Aciklama, Parti, Lot editlenebilir (yesil baslikli); IstasyonKodu/IstasyonAdi lookup ile editlenir; StokKodu/StokAdi/Birimi salt okunur. `colMiktar` formatı "n4" (4 ondalik). `bs` (BaseForm BindingSource) Designer'da grid DataSource olarak atanmis gorunse de kodda grid `Hareketler` listesine baglaniyor.

Modul: MikroModul

MikroModul, UretimV4 (CepPatronERP) masaustu uygulamasinin Mikro ERP (MikroDB_V16_FEZA24) ile entegrasyon koprusudur. Iki yonlu calisir: (1) Mikro tarafından veri okuma/secme — Cari, Stok, Siparis ve Recete listeleri (cogu zaman baska formlara "secim penceresi" olarak acilir, **SecimIcinAcildi** bayragiyla); (2) Uretim sonuclarini Mikro'ya geri yazma — is emri uretim/fire/sarf hareketlerini Mikro **STOK_HAREKETLERI** (+ **BEDEN_HAREKETLERI**, **PARTILOT**) tablolarina fis olarak kaydetme, kaydedilen fisleri listeleme/silme. Tum formlar **My.Kontrol** kutuphanesindeki base formlardan turer: liste formlari **MyFrmListe** (arama paneli + grid + alt buton seridi **BtnAra** / **BtnTemizle** / **BtnYazdir** / **BtnDizayn** / **BtnKapat**), kayit/popup formlari **MyFrmKayit** (**BtnKaydet** / **BtnKapat** / **BtnSil** + navigasyon butonlari). Veri erisimi iki ayri DatabaseFactory uzerinden yapilir: **Ortak.DbMikro** (Mikro ERP) ve **Ortak.DbPro** (UretimV3_FEZA uretim DB). Mikro'ya yazma mantigi **MikroKayitManager** (transaction + parti/lot/renk/beden hesaplama) ve **MikroConvertManager** (StokHareketleriModel -> MikroStokHareketleri fis turune gore donusum, evrak seri/sira atama) siniflarinda toplanir; fis turleri kullanıcı tanımlı **MikroEntegre** ayarlarından (**Ortak.MikroEntAyarlar**) okunur.

Mikro Cari Listesi (**FrmMikroCariListesi.cs** / **.Designer.cs**)

Ne ise yarar: Mikro ERP'deki cari (musteri/tedarikci) kartlarini arar ve listeler. Cogunlukla baska ekranlardan "cari sec" diyalogu olarak acilir (FrmUretimEmriED, FrmIstasyonKartlari, FrmReceteOperasyonED, OperasyonIstasyonControlV2 vb.). **Once ne olmalı (onkosul):** Mikro DB baglantisi (**Ortak.DbMikro**) hazır olmalı. Secim modunda acan formun **SecimIcinAcildi = true** set etmesi gerekir. **Sonra ne olur:** Salt-okunur listeleme; Mikro'da degisiklik yapmaz. Secim modunda satira çift tiklayınca **SecilenKod** (cari_kod), **SecilenRow** (MikroCari) doldurulur, **Secildi=true** yapilip form kapanir; cagiran form bu degerleri okur. **Butonlar & kisayollar:** - **Ara** (BtnAra) — arama kriterleriyle listeyi yeniden yukler (**Bagla()**). - **Temizle** (BtnTemizle) — Kodu/Unvani/Ara metin kutularini temizler. - **Yazdir** / **Dizayn** (BtnYazdir/BtnDizayn) — grid yazdirma/kolon dizayni (base form ozelligi). - **Kapat** (BtnKapat) — formu kapatir. - Grid çift tik / Enter — secim (MyEventDoubleClickEnter). - Arama kutulari: **Kodu**, **Ünvanı**, **Ara** (kod+unvan birlikte). **Cagirdigi katmanlar:** - Service: **IMikroCariService.GetViewListWhere(where)** (**Ortak.DbMikro.Cariler**) — verilen WHERE ile cari view listesini ceker (cari_kod / cari_unvan1 / cari_unvan2 LIKE filtreleri SorguAyarla ile uretilir). **Istasyon sirasiyla iliskisi:** - **Notlar:** Filtreler string birlestirme ile uretilir (parametrik degil). **CrGuid** kolonu gizlenir.

Mikro Stok Listesi (**FrmMikroStokListesi.cs** / **.Designer.cs**)

Ne ise yarar: Mikro ERP stok kartlarini ana grup, stok cinsi (Mamul/Hammadde/Yari Mamul vb.) ve kod/ad ile arar ve listeler. Recete tasarim ekranlarinda (FrmReceteED, FrmReceteDetayED, FrmUretimTalepED) stok secimi icin acilir. **Once ne olmalı (onkosul):** **Ortak.DbMikro** ve **Ortak.MikroStokGrubu** ayari hazır olmalı. **TumStoklar=true** ise acilista otomatik yukleme yapilmaz (kullanici filtreleyip arar). Secim modunda **SecimIcinAcildi=true**. **Sonra ne olur:** Salt-okunur listeleme. Secim modunda çift tik/Enter ile **SecilenKod** =StokKodu, **SecilenRow** =MikroStok doldurulur, form kapanir. **Butonlar & kisayollar:** - **Ara** (BtnAra) — **Bagla()** ile filtreli listeyi yukler. - **Temizle** (BtnTemizle) — StokKodu/StokAdi/AnaGrubu/Ara alanlarini temizler. - **Kapat** / **Yazdir** / **Dizayn** — base form butonlari. - Grid çift tik / Enter — secim. - Filtreler: **StokKodu**, **StokAdi**, **AnaGrubu** (combo), **StokCinsi** (combo), **Ara**. **Cagirdigi katmanlar:** - Service: **IMikroStokService.GetViewListWhere(where, Ortak.MikroStokGrubu)** (**Ortak.DbMikro.Stoklar**) — stok view listesi (sto_kod/sto_isim/anagrup/sto_cins filtreli). - Service: **IMikroGenelService.GrupListesi("STOKLAR", Ortak.MikroStokGrubu)** — ana grup combosu icin distinct grup listesi. - Manager: **MikroStokCinsiManager.GetCinsListFull()** — stok cinsi (kod+ad) listesini doldurur; **GetCinsiKodu(ad)** ile

secilen cinsin sto_cins kodu bulunur. **Istasyon sirasiyla iliskisi:** - **Notlar:** Stok cinsi secilmezse (-2) cins filtresi uygulanmaz.

Mikro Siparis Listesi (FrmMikroSiparisListesi.cs / .Designer.cs)

Ne ise yarar: Mikro ERP'deki acik/kapali satis siparislerini tarih ve cari ile listeler; secilen siparisin satir hareketlerini alt gridde gosterir. Sag-tik menüsünden secili Mikro siparisinden UretimV4 is emri (Siparis/MikroSiparis) olusturulur. UretimV4 tarafında zaten aktarilmis siparisler yesil/menekse renkle isaretlenir. **Once ne olmalı (onkosul):** Ortak.DbMikro ve Ortak.DbPro hazir olmalı. Is emri olusturmak için ilgili Mikro stok kodlarına karsilik UretimV4'te ReceteAna.EntegreStokKodu eslesen receteler tanimli olmalı. **Sonra ne olur:** Listeleme salt-okunur. Sag-tik "Uretim EmriOlustur/Guncelle" -> FrmMikroUretimEmriOlustur acilir; kayıt sonrası UretimV4 Siparis / SiparisHareket / SiparisHareketDetay tablolarına yazilir ve liste Aktarildi olarak isaretlenir (geri cagrim Action=Bagla). **Butonlar & kisayollar:** - Ara (BtnAra) — Bagla() filtreli liste + ilk siparisin hareketlerini yukler. - Temizle (BtnTemizle) — kod/ad/ara/tarih filtrelerini temizler. - Kapali Siparisler (ChcSiparisAcikKapali) — isaretli=Kapali, degil=Acik siparisler. - Tarih filtreleri: TxtTarihi1 (varsayilan: bugun-1 yıl), TxtTarihi2. - Sag-tik menu: Uretim EmriOlustur/Guncelle — FrmMikroUretimEmriOlustur acar. - Grid çift tik/Enter (secim modunda) — SecilenKod =CariKodu ile kapanir. **Cagirdigi katmanlar:** - Service: IMikroSiparisService.GetViewListWhere(where) (Ortak.DbMikro.Siparisler) — siparis view (cari kodu/unvani, tarih, acik/kapali filtreli). - Service: IMikroSiparisHareketService.GetViewListSeriSira(seri, sira) (Ortak.DbMikro.SiparisHareketler) — secili siparisin satir hareketleri. - Service: ISiparisService.SelectListWhere(" where Turu ='MikroSiparis' ") (Ortak.DbPro.Siparis) — UretimV4'te aktarilmis siparisleri tespit edip Aktarildi=true isaretler (SiparisKodu = EvrakSeri+EvrakSira eslesmesi). **Istasyon sirasiyla iliskisi:** Bu ekrandan olusan is emri, FrmMikroUretimEmriOlustur -> FrmUretimEmriED akisina baglanir; istasyon/operasyon yapisi recete üzerinden ileride kurulu. **Notlar:** SipGuid gizlenir. Aktarilmis satirlar RowStyle ile yesil (myView1) gosterilir.

Mikro Recete Listesi (FrmMikroReceteListesi.cs / .Designer.cs)

Ne ise yarar: Mikro ERP urun recetelerini (URUN_RECETELERI) listeler; secilen recetenin UretimV4 (Pro) karsiligini ve operasyonlarini alt gridlerde gosterir. UretimV4'e aktarilmis receteler yesil isaretlenir. Sag-tik menu ile receteyi UretimV4'e iceri aktarma ve fire yuzdesi guncelleme ekranlari acilir. **Once ne olmalı (onkosul):** Ortak.DbPro ve Ortak.DbMikro hazir olmalı; MikroReceteManager (her ikisi ile) kurulu olmalı. **Sonra ne olur:** Listeleme salt-okunur. Sag-tik "Receteyi İceri Aktar" -> FrmReceteED (MikrodanAktar=true) ile UretimV4 recetesi olusturulur; "Recete Fire Yüzde Guncelle" -> FrmMikroReceteFireGuncelle acilir. **Butonlar & kisayollar:** - Ara (BtnAra) — Bagla() ile listeyi yukler (rec iptal=0 + filtre). - Temizle (BtnTemizle) — Ara alanini temizler. - comboBox1 durum filtresi: Aktif (varsayilan) / Pasif / Tüm (sto_pasif_fl üzerinden). - Sag-tik menu: Receteyi İceri Aktar , Recete Fire Yüzde Guncelle . - Grid çift tik/Enter (secim modunda) — SecilenKod =ReceteKodu ile kapanir. **Cagirdigi katmanlar:** - Manager: MikroReceteManager.GetMikroReceteList(where) — URUN_RECETELERI'den (NOLOCK) recete basliklarini cekerek (rec_anakod, fn_Stoklsmi, fn_StokBirimi, rec_anamiktar, sto_pasif_fl). - Service: IReceteAnaService.SelectListWhere() (Ortak.DbPro.ReceteAna) — UretimV4 recetelerini cekip eslesenleri Aktarildi isaretler; secilen recetenin Pro karsiligini gosterir. - Service: IReceteOperasyonService.SelectList(c => c.RcAId == rcaid) (Ortak.DbPro.ReceteOperasyon) — secilen Pro recetesinin operasyonlari. **Istasyon sirasiyla iliskisi:** Pro recetesinin operasyonlari (Sira bazli) bu ekranda goruntulenir; uretim akisinin temelini olusturan operasyon sirasi burada izlenebilir. **Notlar:** Aktarilmis receteler GridView_RowStyle ile yesil. Pro grid kolonlari (Id, RcAGuid, RcAId, RcOGuid) gizlenir.

Mikro Uretim Emri Olustur (`FrmMikroUretimEmriOlustur.cs` / `.Designer.cs`)

Ne ise yarar: Secilen bir Mikro satis siparisinden, satirlardaki stok kodlarına karsilik gelen UretimV4 recetelerini bularak otomatik bir UretimV4 is emri (Siparis turu="MikroSiparis") olusturur. Ust gridde Mikro siparis basligi, alt gridde Mikro siparis hareketleri gosterilir. **Once ne olmalı (onkosul):** `FrmMikroSiparisListesi`'nden bir siparis secilip `SipGuid` set edilmiş olmalı. Her Mikro stok kodu için `ReceteAna.EntegreStokKodu` eslesen bir UretimV4 recetesi tanımlı olmalı — yoksa "Ürüne ait Reçete Bulunamadı" hatası verir ve form kapanır. **Sonra ne olur:** `Kaydet` ile UretimV4 `Siparis` + `SiparisHareket` + `SiparisHareketDetay` tablolarına yazılır (SiparisManager.SiparisKaydet transaction içinde eski hareket/detayları silip yenisini insert eder). Ardından `Action?.Invoke()` (cagiran liste yenilenir) ve `FrmUretimEmriED` (UretimTuru="MikroSiparis") acilir; bu form kapanır. **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — `SiparisManager.SiparisKaydet(_mdl, YeniKayit)` cagirir; basari olursa is emri duzenleme formuna gecer. - `Kapat` (BtnKapat) — base form ile kapatir. **Cagirdigi katmanlar:** - Service: `IMikroSiparisService.GetViewListWhere(" where Sip.SipGuid =...")` — Mikro siparis basligini ceker. - Service: `IMikroSiparisHareketService.GetViewListSeriSira(seri, sir)` — Mikro siparis satirlarini ceker. - Manager: `SiparisManager.GetSiparis()` — bos SiparisKayitModel; `SiparisManager.SiparisKaydet(mdl, yenikayit)` — UretimV4 siparisini transaction'la kaydeder. - Manager: `ReceteManager.GetReceteKayit(rcAId)` — eslesen recetenin tam kaydini (detaylar dahil) getirir; ilk detaydan hareket stok bilgisi, tum detaylardan SiparisHareketDetay olusturulur. - DAL: `_dbPro.ReceteAna.SelectFirst(c => c.EntegreStokKodu == sipH.StokKodu)` — stok koduna gore recete eslestirme. **Istasyon sirasiyla iliskisi:** Olusan is emri sonradan `FrmUretimEmriED`'de operasyon/istasyon yapisiyla devam eder; recetenin operasyon sirasi uretim akisinin temelidir. **Notlar:** Her Mikro siparis satiri için tek recete secilir (`ReceteTekSec`). Hareketler `EntKayitSeri/Sira/Guid` ile Mikro siparis satirina baglanir (geri izlenebilirlik).

Mikro Recete Fire Yüzde Güncelle (`FrmMikroReceteFireGuncelle.cs` / `.Designer.cs`)

Ne ise yarar: Mikro recetesindeki tüketim kalemlerinin fire yuzdelerini, UretimV4'e aktarilmis recetenin detaylarına (ReceteDetay.FireYuzde) toplu olarak gunceller. Ust gridde Mikro recete detayları (referans), alt gridde duzenlenebilir Pro recete detayları gosterilir. **Once ne olmalı (onkosul):** `FrmMikroReceteListesi` sag-tik menüsünden acilmis ve `MikroReceteKodu` + `MikrodanAktar=true` set edilmiş olmalı. Recete daha önce UretimV4'e aktarilmis olmalı (ReceteDetay kayitları için). **Sonra ne olur:** `Kaydet` ile `ReceteManager.ReceteDetayFireYuzdeGuncelle(receteDetaylar)` cagrilir; UretimV4 `ReceteDetay` tablosundaki fire yuzdeleri guncellenir, mesaj verilip form kapanir. **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — fire yuzde guncellemeyi yazar. - Alt grid duzenlenebilir kolonlar: `ReceteSira`, `Miktar`, `Ebat`, `Gram`, `Olcu`, `FireYuzde` (yesil baslikli). - `Stok Cinsi` combo, `Recete Grubu` combo, entegre stok kodu/adi/birim/model kodu alanları (cogu salt bilgi). **Cagirdigi katmanlar:** - Manager: `ReceteManager.GetReceteKayit()` — bos recete kayit modeli; `GetReceteDetayByReceteKodu(receteKodu)` — Pro recete detayları; `ReceteDetayFireYuzdeGuncelle(list)` — fire yuzdelerini gunceller. - Manager: `MikroReceteManager.GetMikroReceteList(where)` — Mikro recete basligi; `GetMikroReceteHareketler(receteKodu)` — Mikro tüketim kalemleri (rec_fireyuzde dahil). - Service: `IMikroStokService.GetViewListWhere(...)` — entegre stok bilgisi/cinsi; `IGenelService.GrupListesi("ReceteAna","Grubu")` — recete grup combosu. **Istasyon sirasiyla iliskisi:** - **Notlar:** Mikro detay sirasi (ReceteSira) ile Pro detay sirasi eslestirilerek fire yuzdesi tasinir. EvrakNoAl ile gerekirse yeni recete kodu uretilir.

Mikroya Uretim Kaydet V2 (`FrmMikroyaUretimKaydetV2.cs` / `.Designer.cs`)

Ne ise yarar: Modulün ana yazma ekrani. Tamamlanmis bir UretimV4 is emrinin sonuclarini (uretilen urun + fire + sarf edilen stoklar + stok fiyesi) Mikro ERP'ye stok hareketi fişleri olarak kaydeder. Ust gridde is emri

hareketleri (uretim/fire miktarlari), alt gridde kullanılan/cikan stoklar gosterilir. **Once ne olmalı (onkosul):** Is emri uretim girisleri tamamlanmis olmalı (UretimIstasyonHareket -> ... miktarlar toplanmis).

FrmSiparisListesi'nden `SipId` set edilerek acilir. `Ortak.MikroEntAyarlar` (MikroEntegre fis turu/maliyet ayarlari) ve `Ortak.IstasyonAyarlarBagla()`, `Ortak.MalKabulKullan`, `Ortak.PlKapat` ayarlari yuklenir. Daha once aktarilmis (`Siparis.Ent`) kullaniciya eski fisleri silmesi gerektiği uyarisi cıkar. **Sonra ne olur:** `Kaydet` butonu, ayarlara gore urun girisi / urun fire cikisi / stok cikisi / stok fire cikisi listelerini StokHareketleriModel olarak olusturur, maliyet hesaplar (standart/recete maliyet), fis turune gore (StokVirman/UretimHareket/UretimdenGiris/SayimDepoGiris/UretimeCikis/SarfDepoCikis/FireCikis) `MikroConvertManager.Convert...` ile MikroStokHareketleri'ne donusturur ve `MikroKayitManager.StokHareketKaydet` ile Mikro `STOK_HAREKETLERI` (+ `BEDEN_HAREKETLERI` + `PARTILOT` + `StokDepoRaf`) tablolarina transaction icinde yazar. Basariliysa `SiparisManager.SiparisEntGuncelle` ile UretimV4 `Siparis` / `SiparisHareket` tablolarina `Ent=1` + evrak seri/sira islenir; `Kaydet` butonu pasiflesir.

Butonlar & kisayollar: - `Kaydet` (BtnKaydet) — tum fisleri Mikro'ya yazar (yukaridaki akis). - `Yazdir` (BtnYazdir) — `BaglaKaliteYazdir()` ile kalite raporunu (KaliteRapor REPX) yazdirir. - `myButton1` (gizli/yardimci) — mal kabul hesaplama onizleme formunu (`FrmMikroMalKabulHesaplama`) acar. - `Kapat` (BtnKapat) — formu kapatir. **Agirdigi katmanlar:** - Manager: `SiparisManager.GetSiparis(SipId)` — is emri (Siparis + hareketler); `SiparisEntGuncelle(...)` — `Ent` + evrak seri/sira yazar; `GetKaliteYazdir(SipId)` — kalite rapor datasi. - Manager: `ReceteManager.GetRafOmru(rcAId)` — urun raf omru (parti son kullanma tarihi icin). - Manager: `MikroConvertManager.SetUretimUrunGirisFisiAyar` / `SetUretimUrunFireCikisFisiAyar` / `SetUretimStokCikisFisiAyar` / `SetUretimStokFireCikisFisiAyar` (depo/fis ayarlari) ve `ConvertStokVirmanFisi` / `ConvertUretimHareketFisi` / `ConvertUretimdenGirisFisi` / `ConvertSayimDepoGiris` / `ConvertUretimeCikisFisi` / `ConvertSarfDepoCikis` / `ConvertFireCikis` (fis turune gore MikroStokHareketleri + evrak seri/sira uretimi). - Manager: `MikroKayitManager.GetMikroStokMaliyetListWhere(where)` — stok standart/recete maliyeti; `StokHareketKaydet(lisMikro, depoRaf)` — Mikro fis kaydi (transaction; parti/lot/renk-beden hesaplama dahil). - Service: `IstasyonTakipStokHareketService.GetViewListKullanimWherePartiLot` / `GetViewListKullanimWhereMalKabul` / `GetViewListKullanimMalKabulFis` — kullanılan/cikan stoklar; `IstasyonTakipHareketDetayService.GetViewListStokFire(...)` — stok fireleri; `IstasyonTakipStokHareketDetayService.DetaylarGuncelleBySipId(SipId)` — kayit oncesi detay senkronu; `ITempMikroStokService.SelectListWhere(...)` — birim katsayilari (Birim2/3/4, Katsayi) ile birim donusumu. - Yardimci sinif: `MikroyaKaydetMalKabulHesaplama` / `MikroyaKaydetMalKabulFireHesaplama` — `Ortak.MalKabulKullan` aktifse parti/lot bazinda mal kabul fisine gore stok/fire dagitimi yapar. - SQL/Prosedur: dolayli olarak fis kaydi sirasinda STOKLAR (sto_detay_takip/renk/beden), `PARTILOT`, `BEDEN_HAREKETLERI`; maliyet icin `fn_by_Stok_Son5_Giris_Fiyati`, `fn_StokIsmi`, `fn_StokBirimi` Mikro fonksiyonlari. **Istasyon sirasiyla iliskisi:** Bu ekran uretim akisinin SONUNDADIR — istasyon takip hareketleriyle (uretim girisi, sarf, fire, mal kabul) biriken miktarlar burada Mikro'ya stok hareketi olarak aktarilir. Olcum/akis motoru (Uretim_MiktarGuncelle/PlanlananGuncelle/SonrakIstasyonaGonder) is emri tarafinda calismis, miktarlar UretimIstasyonHareket->...->UrO seviyesine toplanmistir; burada yalnizca okunup fise donusturulur. **Notlar:** Urun fiyati = (stok cikis tutar + fire cikis tutar) / toplam urun miktarı olarak hesaplanip urun giris/fire fislerine yazilir. Birim donusumu TempMikroStok katsayilariyla yapilir (birimpntr 1-4). `.txt` uzantili eski V1 ayni isimli formun yedegidir (aktif degil; bu V2 dosyasi kullanilir).

Mikroya Sarf Fire Kaydet (`FrmMikroyaSarfFireKaydet.cs` / `.Designer.cs`)

Ne ise yarar: Istasyon takip hareket detaylarından dogan ara sarf cikis ve fire giris fislerini (IstasyonTakipHareketDetay) Mikro ERP'ye stok hareketi olarak kaydeder. Tam is emri kapatmadan, istasyon bazinda sarf/fire aktarimi icin kullanilir. **Once ne olmalı (onkosul):** FrmIstasyonFisList ekranından aktarılacak

FisList (List) doldurularak acilmis olmalı. Acilista **DahaOnceKayitEdilmismi()** ile EntCode'u olan kayitların Mikro'da zaten var olup olmadığı kontrol edilir — varsa Kaydet butonu gizlenir (mukerrer aktarım engeli). **Sonra ne olur:** **Kaydet** ile her detay, turune göre (SarfCikisFisi / FireGirisFisi) ve ayarlardaki fis turune göre (StokVirman / SarfDepoCikis / FireCikis) StokHareketleriModel'e cevrilir, **MikroConvertManager.Convert...** ile MikroStokHareketleri oluşturulur, **MikroKayitManager.StokHareketKaydet(lisMikro)** ile Mikro'ya yazilir. Basariliysa her FisList kaydına **Ent=true**, **EntSeri/EntSira/EntDate** yazilip **IIstasyonTakupHareketDetayService.InsertOrUpdate(FisList)** ile UretimV4'e geri kaydedilir. **Butonlar & kisayollar:** - **Kaydet** (BtnKaydet) — sarf/fire fislerini Mikro'ya aktarir ve UretimV4 detaylarını Ent isaretler. - **Kapat** (BtnKapat) — formu kapatir. **Agirdigi katmanlar:** - Manager: **MikroConvertManager.SetSarfCikisFisiTuruAyar / SetFireGirisFisiTuruAyar** (depo/ayar); **ConvertStokVirmanFisi / ConvertSarfDepoCikis** (fis donusumu). - Manager: **MikroKayitManager.StokHareketIdKayitEdilmismi(id)** (mukerrer kontrol); **StokHareketKaydet(lisMikro)** (Mikro kayit). - Service: **IIstasyonTakupHareketDetayService.InsertOrUpdate(FisList)** (**Ortak.DbPro.IstasyonTakupHareketDetay**) — Ent bilgilerini geri yazar. **Istasyon sirasiyla iliskisi:** Saha akisinda istasyon bazli sarf/fire kayitlari (IstasyonTakupHareketDetay, Turu=SarfCikisFisi/FireGirisFisi) bu ekranda Mikro'ya aktarilir; is emri tam kapanmadan ara aktarim saglar. **Notlar:** Her detaya benzersiz **EntCode** (Guid) atanip Mikro fis **sth_Guid** olarak kullanilir; bu sayede mukerrer aktarim tespiti yapilir.

Mikro Mal Kabul Hesaplama (**FrmMikroMalKabulHesaplama.cs** / **.Designer.cs**)

Ne ise yarar: Mal kabul fisleri ile istasyon kullanım hareketlerini eslestirip, hangi stok/parti/lot'tan ne kadar dusulecegini hesaplayan onizleme/dogrulama ekranidir (FrmMikroyaUretimKaydetV2 icindeki MalKabul hesaplamasinin gorsel kontrolu). 4 grid: mal kabul fisi, istasyon hareket, hesaplanan, kalan. **Once ne olmalı (onkosul):** Acan form (FrmMikroyaUretimKaydetV2.myButton1) **MalKabulFis**, **IstasyonHareket**, **StokFireListPartili** listelerini doldurmali. **Ortak.MalKabulKullan** senaryosu icin anlamlidir. **Sonra ne olur:** Salt hesaplama/onizleme; DB'ye yazmaz. **myButton1** -> **hsp.Convert()** (hesaplanan dagitim) ve **hsp.GetKalanList()** (artan mal kabul miktarlari) gridlere basilir. **Butonlar & kisayollar:** - **myButton1** — hesaplamayi calistirip hesaplanan + kalan listeleri gosterir. **Agirdigi katmanlar:** - Yardimci sinif: **MikroyaKaydetMalKabulHesaplama.Convert()** — partili fireleri dusup mal kabul fisine gore istasyon hareketlerini parti/lot bazinda dagitir; **GetKalanList()** — kullanilmayan mal kabul kalanlari. **Istasyon sirasiyla iliskisi:** - **Notlar:** Bu ekran dogrudan menude degil, Mikroya Uretim Kaydet V2 icinden yardimci olarak acilir. Asil kayit FrmMikroyaUretimKaydetV2'de yapilir.

Mikro Uretim Kaydedilen Fişler (**FrmMikroUretimKaydedilenFisler.cs** / **.Designer.cs**)

Ne ise yarar: Bir is emri (BelgeNo/SipId) icin Mikro'ya daha once aktarilmis stok hareketi fislerini listeler ve secilenleri Mikro'dan silmeyi saglar. Yanlis/tekrar aktarim sonrasi temizlik ve yeniden aktarim icin kullanilir. **Once ne olmalı (onkosul):** FrmSiparisListesi'nden **BelgeNo** ve **SipId** set edilerek acilmis olmalı; o is emrine ait Mikro fisleri var olmalı. **Sonra ne olur:** **Sil** ile secili (**Sec=true**) fisler **MikroKayitManager.DeleteMikroAktarilanFisBySeriSira** ile Mikro **STOK_HAREKETLERI** (+ **BEDEN_HAREKETLERI** + **PARTILOT**) tablolarindan silinir. Tum fisler silindiyse (**fisKaldimi=false**) **SiparisManager.SiparisEntGuncelle(SipId,"","","","",0,0)** ile UretimV4'teki entegrasyon bayraklari (Ent=0, Kapandi=0) temizlenir; liste yeniden yuklenir. **Butonlar & kisayollar:** - **Sil** (BtnSil) — secili fisleri Mikro'dan siler (onay sorar). - **Kapat** (BtnKapat) — formu kapatir (**BtnKapat_Click_1**). - Grid **Sec** kolonu — silinecek fisleri isaretlemek icin duzenlenebilir checkbox. **Agirdigi katmanlar:** - Manager: **MikroKayitManager.GetMikroAktarilanFisByBelgeNo(belgeNo)** — belge no'ya gore aktarilmis fisleri ceker; **DeleteMikroAktarilanFisBySeriSira(seri, sira)** — fisi (ve bagli beden/parti hareketlerini) transaction'la siler. - Manager: **SiparisManager.SiparisEntGuncelle(...)** — fis kalmayinca UretimV4 entegrasyon

bayraklarini sifirlar. **Istasyon sirasiyla iliskisi:** - **Notlar:** Mal kabul fisi turu (sth_tip=2, sth_cins=6, sth_evraktip=2) listeden haric tutulur. Silme parti/lot ozelkod3='AKT' ve belge_no eslesmesine dayanir (yalnizca bu uygulamanin actigi fisler).

Modul: SiparisModule

UretimV4 (CepPatronERP.exe, WinForms) masaustu uygulamasinin "Siparis / Recete Uretim" giris ve takip modulu. Bu modul, hem musteriler SIPARISI hem de dahili RECETE URETIM emirlerinin (her ikisi de aynı **Siparis** tablosunda **Turu** alanıyla ayrıştırılır) baslangic noktasidir: siparis basligi (cari, tarih, teslim) + receteler (**SiparisHareket**) + recete malzeme detaylari (**SiparisHareketDetay**) burada olusturulur. Liste ekranı üzerinden bu siparisten URETIM EMRI olusturulur/baslatilir (**FrmUretimEmriED**), ilk istasyonlara gonderilir, uretilen miktarlar geri toplanir (TempSiparisUretimMiktar), Mikro ERP'ye uretim fisi gonderilir ve iptal edilen miktarlar depoya iade edilir. Yani saha akisinin (TabletV2'deki IstasyonTakip) "kaynak" tarafidir; istasyon-sira motoru bu modulun urettigi **UretimEmri/UretimOperasyon/UretimIstasyon** kayitlari üzerinde calisir.

Tum formlar harici **My.Kontrol** kutuphanesindeki base formlardan turer: liste formlari **MyFrmListe** (BtnAra/BtnTemizle/BtnYazdir/BtnDizayn/BtnKapat + **SecimIcinAcildi/Secildi/SecilenRow** secim altyapisi + grid cift-tik = **MyEventDoubleClickEnter**), kayit formlari **MyFrmKayit** / **MyFrmKayitFull** (BtnKaydet/BtnSil/BtnYazdir/BtnKapat + **YeniKayit/KayitEdildi/IdGuid/AcilisBittimi**). Base form davranisi geneldir: kapat butonu/Esc formu kapatir, kaydet butonu kaydeder; bu modulde ayrica F5/F6/F7/F8 recete kisayollari forma ozel eklenmistir.

Siparis / Recete Uretim Listesi (**FrmSiparisListesi.cs** , **FrmSiparisListesi.Designer.cs**)

Ne ise yarar: Siparis ve Recete uretim emirlerinin ana liste ekranidir. Ust gridde siparisler (**Siparis**), alt gridde secili siparisin hareketleri (**SiparisHareket** , kalan miktarli view) listelenir. **Turu** alanı ("Siparis" / "Recete" / "MikroSiparis") forma disaridan set edilerek aynı form 3 farklı baslik/davranisla acilir (Recete ise baslik "Recete Üretim Listesi", buton "Üretim Ekle"; kolon basliklari "İş Emri Tarihi/Kodu" olur). Sag-tik context menu su üzerinden uretim emri olusturma/baslatma, ilk istasyona gonderme, miktar guncelleme, Mikroya uretim gonderme, iptal iadesi gibi tum orkestrasyon islemleri buradan tetiklenir. **Once ne olmalı (onkosul):** Lisans/kullanici girisi yapilmis (**Ortak.KullaniciAdi** , **Ortak.DbPro** , **Ortak.DbMikro** dolu), Mikro stok grubu ayarli (**Ortak.MikroStokGrubu**). Listede islem yapmak icin once **FrmSiparisEd** / **FrmSiparisReceteEd** ile siparis kayitli olmalı. **Sonra ne olur:** Form acilisinda otomatik **BtnAra.PerformClick()** ile veri yuklenir. Secilen islemlere gore: yeni **UretimEmri/UretimOperasyon/UretimIstasyon** kayitlari olusur (**FrmUretimEmriED**), **IstasyonTakipHareket** ilk istasyonlara dusurulur (**FrmOperasyonTakipDetaylar**), **SiparisHareket.UretimMiktari/FireMiktari/IptalMiktari** guncellenir (TempSiparisUretimMiktar MERGE), Mikro ERP'ye uretim fisi yazilir, ya da iptal iadesi depoya islenir. Cift-tik secimde (**SecimIcinAcildi**) **SecilenKod=SiparisKodu** doner. **Butonlar & kisayollar:** - **BtnAra** ("Ara") — **Bagla()** ; filtre + tarih kriterlerine gore **Siparis** listesini yeniler. - **BtnTemizle** ("Temizle") — filtre kutularini temizler (tarih1 = 01.01.). - **BtnEkleSiparis** ("Siparis Ekle" / Recete'de "Üretim Ekle") — Turu'ye gore **FrmSiparisEd** veya **FrmSiparisReceteEd** yeni kayit acar. - **BtnUretimEmriOlustur** ("Üretime Emri") — secili siparis icin **UretimEmriEmriOlusturBaslat(itm, false)** . - **BtnYazdir** ("Yazdir") — **Yazdir()** ; secili siparisi tek DataSet'te (Siparis+Hareketler+Detaylar) "Siparis" dizaynina basar. - **BtnDizayn** ("YazdirTek / Ctrl+D") — **Yazdir2()** ; her hareketi ayrı sayfa olarak yazdirir (YaziciAyar ile adet/onizleme). - **BtnKapat** (base / Esc) — formu kapatir. - Durum filtre butonlari: **BtnDurumuHepsi** , **BtnDurumuYeniKayit** , **BtnDurumuBeklemede** , **BtnDurumuUretimde** , **BtnDurumuHazir** — **DurumuAyarla(...)** + **BtnAra.PerformClick()** (secili buton renkleri **FilterButonRenklendir**). - Kapandi filtre butonlari: **BtnKapandiTumu** , **BtnKapandiAcik** , **BtnKapandiKapandi** — **KapandiAyarla(...)** + arama (Açık = **coalesce(Kapandi,0)=0**). - Grid cift-tik / Enter (**MyView1_MyEventDoubleClickEnter**) — secim modunda secip kapatir; degilse Turu'ye gore **FrmSiparisEd** / **FrmSiparisReceteEd** duzenleme acar. - Sag-tik context menu (**ContextMenuyeEkle**) ogeleri:

"Uretimi Emri (Olustur / Oto Baslat)" -> `ConIsEmriOlusturBaslat` (OtoBaslat=true); "Uretim Emri (Olustur / Guncelle)" -> `ConIsEmriOlusturV2` (OtoBaslat=false); "Operasyon (İlk İstasyonlara Gönder)" -> `ConOperasyonIstasyonBaslat`; "Üretilen Adetleri Güncelle" -> `ConUretimAdetleriGuncelle`; "Mikroya Uretim Gonder" -> `ConMikroyaUretimGonderYeni`; "Mikroya Aktarilan Uretim Fisleri" -> `ConMikroyaGonderilenFisler`; "Mal Kabul Fisi" -> `ConMalKabulFisAc`; "İptal Edilenleri Depo Kontrole GeriAktar" -> `ConIptalFisiniDepoyaAktar`; "Git" alt menu -> Üretim Emri / Üretim Operasyonlar / İstasyon Hareketler / İstasyon Bekleyenler / İstasyon Hareket Detaylar formlarini SiparisKodu ile filtreli acar. **Cagirdigi katmanlar:** - Service: `ISiparisService.SelectListWhere(where)` — siparis listesi. - Service: `ISiparisHareketService.GetViewListKalanMiktarliWhere(" where SH.SipId='...'")` — secili siparisin kalan-miktarli hareket gorunumu (alt grid). - Service: `IGenelService.GrupListesi("Siparis", "Durumu")` — durum combosu icin grup degerleri. - Service: `ITempMikroStokService.SelectFirst(StokKodu)` — parti takipli urun kontrolu (TakipTip 1/2). - Manager: `SiparisManager.GetSiparis(id)` — yazdirma icin Siparis+Hareketler+Detaylar getirir. - Manager: `SiparisManager.GetTempSiparisUretimMiktarBySipId(sipId, kullanici)` — uretilen adetleri TempSiparisUretimMiktar üzerinden hesaplar ve `SiparisHareket` e MERGE eder (adet guncelleme + Mikroya/iptale gonderme onkosulu). - Manager: `SiparisManager.GetOperasyonKoduByRcAId(rcaId)` — RcAId icin Sira=1 ilk operasyon kodu (ilk istasyona gonderme icin). - Acilan formlar: `FrmUretimEmriED` (uretim emri olustur/baslat), `FrmOperasyonTakipDetaylar` (ilk istasyona gonder, Durumu="Beklemede"), `FrmMikroyaUretimKaydetV2` (Mikroya uretim fisi), `FrmMikroUretimKaydedilenFisler`, `FrmMalKabuled`, `FrmUretimIptalleriDepoyaAktar`, ve Git menu form lari (`FrmUretimEmriListesi`, `FrmUretimOperasyonList`, `FrmIstasyonHareketler`, `FrmIstasyonBekleyenler`, `FrmIstasyonHareketDetaylar`). - SQL/Prosedur: dolayli olarak TempSiparisUretimMiktar inline T-SQL (IstasyonTakipHareket -> UretimIstasyon -> UretimOperasyon -> UretimEmri toplama + SiparisHareket MERGE). **İstasyon sirasiyla iliskisi:** Modulun istasyon-sira motoruyla ana baglanti noktasidir. "Operasyon (İlk İstasyonlara Gönder)" her benzersiz operasyon (RcAId -> Sira=1 OperasyonKodu) icin `FrmOperasyonTakipDetaylar` acararak isi ilk istasyonun bekleyenler kuyruguna (`IstasyonTakipHareket` Beklemede) koyar. "Üretilen Adetleri Güncelle" ise istasyon hareketlerini operasyon-sira bazinda yukari toplayip siparise yansitir. Mikroya gonderme ve iptal iadesi yalnızca Durumu="Hazir" (tum siralar bitmis) siparislerde calisir. **Notlar:** `Ortak.PlKapat` true ise tum gridlerde Parti/Lot kolonlari gizlenir ve parti-takip zorunlulugu atlanir. SorguAyarla string interpolasyonla WHERE kurar (parametrik degil). `Yazdir2/Yazdir` yazdirma adi sabit "Siparis"tir.

Siparis Kayıt (`FrmSiparisEd.cs` , `FrmSiparisEd.Designer.cs`)

Ne ise yarar: Musteri SIPARISI basligi ve recete satirlarinin girildigi/duzenlendiği kayıt formu (`MyFrmKayitFull`). Cari secimi (kod/unvan otomatik doldurma, Mikro cari listesinden), siparis kodu otomatik evrak no, tarih/teslim tarihi, kargo, e-posta, aciklama girilir; grid'e recete grubu veya tekil recete eklenir, her recete satirinin malzeme detaylari (`SiparisHareketDetay`) `SiparisPanelControl` ile ayarlanir. **Once ne olmalı (onkosul):** Receteler (`ReceteAna` / `ReceteGrup`) onceden tanimlanmis olmalı. Mikro stoklari (`Ortak.MikroStokGrubu`) ve cari listesi erisilebilir olmalı. Duzenleme icin `IdGuid` set edilir (yeni kayıt icin bos). **Sonra ne olur:** Kaydette `SiparisManager.SiparisKaydet(md1, yeniKayit)` cagilir; transaction icinde `Siparis` upsert + ilgili `SiparisHareket` ve `SiparisHareketDetay` once silinip yeniden yazilir. Durumu bos ise "YeniKayit" set edilir, `Miktar` (hareket toplami) ve `Notu` (recete adlari) hesaplanir. Kayit sonrasi `ActionAktar` (liste yenileme) tetiklenir; e-posta varsa sorulup mail gonderilir. Sipariste uretim baslatilmis (`SipariseBagliUretimVar`) form salt-okunur olur. **Butonlar & kisayollar:** - `BtnKaydet` ("Kaydet") — parti-takip kontrolu sonrasi `Kaydet()` . - `BtnSil` ("Sil") — `SiparisSilKontrol` sonrasi `Sil()` . - `BtnReceteGrupSec` ("Recete Grup Seç" / F5) — `ReceteGrupSec()` ; cari+siparis kodu kontrolu ile recete grubu satirlarini ekler. - `BtnReceteTekSec` / `BtnReceteSec` ("Recete Seç" / F6) — `ReceteTekSec()` ; tek recete ekler. -

BtnReceteDegistir ("Recete Değiştir" / F7) — **ReceteDegistir()** ; secili satirin recetesini degistirir. - **BtnReceteSil** ("Recete Sil" / F8) — **ReceteSil()** ; secili recete satirini ve detaylarini listeden cikarir. - **BtnYazdir** ("Yazdir") — **Yazdir()** (DataSet -> "Siparis" dizayni). - **BtnYazdirTek** — **YazdirTek()** (hareket basina ayri yazdirma). - **BtnMailPdfDesing** — **YazdirMail()** ; "SiparisMail" dizaynini hazirlar. - **BtnMailGonder** — **MailGonder(false)** ; **FrmFisMailGonder** ile e-posta gonderir. - **TxtSiparisKodu** buton (**TxtSiparisKodu_ButtonClick**) — **EvrakNoAl()** ile yeni siparis kodu uretir. - Cari kutulari **Leave** — secilen Mikro cariden Unvan/Kargo/Email otomatik doldurur. - Grid cift-tik (**MyView1_MyEventDoubleClickEnter**) — **RecetePanelBagla()** ile secili recete satirinin malzeme paneli acilir. - **BtnKapat** / Esc — kapatir. **Cagirdigi katmanlar:** - Manager: **SiparisManager.GetSiparis()/GetSiparis(id)** — bos/dolu kayit modeli. - Manager: **SiparisManager.SiparisKaydet(md1, yeniKayit)** — transaction'li upsert (Siparis + Hareket + Detay sil/ekle). - Manager: **SiparisManager.SiparisSil(md1) / SiparisSilKontrol(id)** (proc **Siparis_Sil_Kontrol**) — bagli hareket kontrolu + silme. - Manager: **SiparisManager.SipariseBagliUretimVarmi(sipid)** — **UretimEmri** de Sipid var mi (salt-okunur kararına). - Manager: **ReceteManager.GetReceteKayit(id)** ve **ReceteGrupManager.GetReceteKayit(id)** — recete/recete grubu detaylari. - Service: **IGenelService.GetEvrakNo("Siparis")** — siparis kodu uretimi. - Service: **IMikroCariService.GetViewListWhere(...)** — cari lookup datasi. - Service: **ITempMikroStokService.SelectFirst(StokKodu)** — parti takip kontrolu. - Service: **Ortak.DbMikro.Stoklar.GetViewListWhere/GetRenkListWhere** — stok/renk listeleri (panel icin). - Acilan formlar: **FrmReceteGrupListesi** , **FrmReceteSec** , **FrmFisMailGonder** , **SiparisPanelControl** . **Istasyon sirasiyla iliskisi:** Dolayli. Burada secilen recete (**RcAid**) ileride uretim emri ve operasyon-sira zincirini belirler; ancak bu form yalnızca siparis/recete verisini hazirlar, istasyona is dusurmez. Uretim baslamissa form kilitletir (degistirilemez). **Notlar:** **MyFrmKayitFull** turevi. Parti takipli (**TempMikroStok.TakipTip** 1/2) urunde parti no zorunlu (PIKapat haricinde). Recete satirinda **SiparisdeGosterme=false** detay varsa satir "YeniKayit" olarak isaretlenir ve panelle ayarlanmadan kayit engellenir.

Recete Uretim Kayit (**FrmSiparisReceteEd.cs** , **FrmSiparisReceteEd.Designer.cs**)

Ne ise yarar: Dahili RECETE URETİM emri (Turu="Recete") kayit formu. **FrmSiparisEd** ile cok benzer ama: evrak no "Uretim" serisinden alinir, cari opsiyonel (buton ile Mikro cari listesinden secilir / temizlenir), gridde Beden duzenlenebilir ve kayit oncesi parti/lot/beden takip kurallari daha siki uygulanir (parti-lot daha once girilmis mi kontrolu dahil). **Once ne olmalı (onkosul):** Receteler tanimli, Mikro stok/renk listeleri erisilebilir. Duzenleme icin **IdGuid** . Uretim baslamamis olmalı (baslamissa salt-okunur). **Sonra ne olur:** **SiparisManager.SiparisKaydet** ile ayni **Siparis/SiparisHareket/SiparisHareketDetay** tablolarina Turu="Recete" olarak yazar. Kayit oncesi her hareket icin parti takibi (TakipTip 1/2), beden takibi (RbTakipTip=2 veya TakipTip=3) ve **MikroKayitManager.PartiLotDataOnceGirilmismi** ile parti-lot tekrar kontrolu yapilir; Lot bos/0 ise 1'e cekilir. Kayit sonrasi **ActionAktar** ile liste yenilenir. **Butonlar & kisayollar:** - **BtnKaydet** ("Kaydet") — parti/beden/parti-lot kontrolleri sonrasi **Kaydet()** . - **BtnSil** ("Sil") — **SiparisSilKontrol** sonrasi **Sil()** . - **BtnReceteGrupSec** (F5) — **ReceteGrupSec()** . - **BtnReceteTekSec** / **BtnReceteSec** (F6) — **ReceteTekSec()** . - **BtnReceteDegistir** (F7) — **ReceteDegistir()** (mevcut miktari korur). - **BtnReceteSil** (F8) — **ReceteSil()** . - **BtnYazdir** / **BtnYazdirTek** — **Yazdir()** / **YazdirTek()** ("Siparis" dizayni). - **TxtCariKodu** buton (**TxtCariKodu_ButtonClick**) — **FrmMikroCariListesi** secimi. - **BtnCariTemizle** — cari kod/unvan temizler. - **TxtSiparisKodu** buton — **EvrakNoAl()** ("Uretim" serisi). - Grid cift-tik — **RecetePanelBagla()** (malzeme paneli). - **BtnKapat** / Esc — kapatir. **Cagirdigi katmanlar:** - Manager: **SiparisManager.GetSiparis/SiparisKaydet/SiparisSil/SiparisSilKontrol/SipariseBagliUretimVarmi** (FrmSiparisEd ile ayni). - Manager: **ReceteManager.GetReceteKayit / ReceteGrupManager.GetReceteKayit** — recete(grup) detaylari. - Manager: **MikroKayitManager.PartiLotDataOnceGirilmismi(stokKodu, parti, lot)** — parti-lot daha once kayitli mi. - Service: **IGenelService.GetEvrakNo("Uretim")** — uretim kodu. - Service:

`ITempMikroStokService.SelectFirst(StokKodu)` — TakipTip/RbTakipTip okuma. - Service: `Ortak.DbMikro.Stoklar.GetViewListWhere/GetRenkListWhere` . - Acilan formlar: `FrmReceteSec` , `FrmReceteGrupListesi` , `FrmMikroCarilListesi` , `SiparisPanelControl` . **Istasyon sirasiyla iliskisi:** `FrmSiparisEd` ile ayni; recete secimi ileri uretim operasyon-sira zincirini tanimlar, form dogrudan istasyona is dusurmez. **Notlar:** `CheckForIllegalCrossThreadCalls=false` set edilir. Beden kolonu duzenlenebilir acilir (`FrmSiparisEd`'de Beden duzenlenmez). Hareket yoksa kayit engellenir ("Hareket Bulunamadi").

Siparis Hareket Seç (`FrmSiparisHareketSec.cs` , `FrmSiparisHareketSec.Designer.cs`)

Ne ise yarar: Baska ekranlardan cagrilan SECIM (lookup) listesi: siparis hareketlerini (`SiparisHareketModel` view) filtreleyip tekini secip dondurur. Tek grid uzerinde cari kodu/unvan/siparis kodu/durum/kapandi/tarih filtreleriyle calisir. **Once ne olmalı (onkosul):** `SecimIcinAcildi=true` ile acilmis olmalı (secim modu). Cagiran ekran dondurulen `SecilenRow` u (`SiparisHareketModel`) kullanir. **Sonra ne olur:** Cift-tik / Enter ile secili hareket `SecilenRow` a atanir, `Secildi=true` yapilip form kapanir (tablo degistirme, salt okuma). Secim modu degilse hareket cift-tikinda islem yapilmaz. **Butonlar & kisayollar:** - `BtnAra` ("Ara") — `Bagla()` ; filtreyle hareket listesini yeniler. - `BtnTemizle` ("Temizle") — filtre kutularini sifirlar. - `CmbDurumu` / `CmbKapandi` — durum ve acik/kapandi filtresi. - Grid cift-tik / Enter (`MyView1_MyEventDoubleClickEnter`) — secip dondurur. - `BtnKapat` / Esc — secmeden kapatir. **Cagirdigi katmanlar:** - Service: `ISiparisHareketService.GetViewListWhere(where)` — hareket view listesi (Siparis JOIN'li, alias S.). - Service: `IGenelService.GrupListesi("Siparis","Durumu")` — durum combosu. - Manager: `SiparisManager` (`Frm_Load`'da olusturulur, dogrudan metot cagrisi yok). **Istasyon sirasiyla iliskisi:** - **Notlar:** Salt secim formudur; veri yazmaz. Filtreler string interpolasyonludur (parametrik degil). Tarih1 varsayilani bir ay once.

Uretim Iptalleri Depoya Aktar (`FrmUretimIptalleriDepoyaAktar.cs` , `FrmUretimIptalleriDepoyaAktar.Designer.cs`)

Ne ise yarar: Uretim sirasinda IPTAL edilen miktarlara karsilik gelen tuketilmis hammadde/malzemeyi yerel depoya geri (iade) fisi olarak aktaran kayit formu (`MyFrmKayit`). Ust kisimda siparis bilgisi + hareketler, ortada kullanılan stok hareketleri, altta otomatik hesaplanan iade satirlari (`DepoStokHareket`) gosterilir; kullanıcı parti/lot/miktar/tarihleri duzenleyip kaydeder. **Once ne olmalı (onkosul):** Liste ekranindan "Iptal Edilenleri Depo Kontrole GeriAktar" ile `SipId` set edilerek acilir; siparis Durumu="Hazir" olmalı (liste once `GetTempSiparisUretimMiktarBySipId` ile miktarlari gunceller). Hareketlerde `IptalMiktari>0` ve `Miktar > (UretimMiktari+FireMiktari)` olan satirlar bulunmalı, aksi halde "Aktarılacak Kayıt Bulunamadı" uyarisi cikar. **Sonra ne olur:** `AktarilacakAyarla()` iade miktarini `Carpan * (Miktar-(Uretim+Fire))` formuluyla hesaplar; her satir depoda kayitli mi (`StGuid`) dogrulani. Kaydette FisSeri="IA", FisTuru=IslemTuru="UretimIade" olan `DepoStokFis` + `DepoStokHareket` listesi `DepoStokFisManager.KaydetDepoStokFis` ile yazilir; ardindan `SiparisManager.SiparisIptalEntGuncelle(sipId, seri, sira)` ile `Siparis.EntIptal=1`, `EntIptalSeri/Sira` set edilir. Kaydet butonu sonrasi devre disi birakilir. **Butonlar & kisayollar:** - `BtnKaydet` ("Kaydet") — depoda kayitli olmayan stok kontrolu sonrasi `Kaydet()` (grid editorunu kapatip fisi yazar). - `BtnYazdir` (base) — yazdirma (form ozelinde handler bagli degil). - `BtnKapat` / Esc — kapatir. - Alt grid (`myView3`) duzenlenebilir kolonlar: GirişMiktar, PartiNo, LotNo, UretimTarihi, SonKulTarihi (yesil baslikli). **Cagirdigi katmanlar:** - Manager: `SiparisManager.GetSiparis(SipId)` — siparis+hareket basligi. - Manager: `SiparisManager.SiparisIptalEntGuncelle(sipId, seri, sira)` — `Siparis.EntIptal/EntIptalSeri/EntIptalSira` set eder. - Manager: `DepoStokFisManager.GetDepoStok(list)` — stokların depo karsiligi (StGuid/Birim) eslenir. - Manager: `DepoStokFisManager.FisNoAl("UretimIade","IA")` — yeni iade fis no. - Manager: `DepoStokFisManager.KaydetDepoStokFis(fis, list, yenikayit, false)` —

`DepoStokFis + DepoStokHareket` kaydeder (giris fisi, cikis=false). - Service:

`IIstasyonTakipStokHareketService.GetViewListKullanimWherePartiLot(" and HRD.SipId='...'" )` — bu

siparise ait kullanılan stok (parti/lot/carpan) hareketleri. **Istasyon sirasiyla iliskisi:** Akisin SON tarafindadir: istasyon hareketlerinde olusan iptal miktarlarinin hammadde karsiligini depoya iade eder; istasyon-sira motorunu calistirmaz, sadece sonuclarini (IstasyonTakipStokHareketDetay/kullanilan stok) okur. **Notlar:** Yalnizca Durumu="Hazir" siparislerde anlamlidir (liste tarafinda zorlanir). Depoda olmayan stok varsa hata mesajı verilir ama akis devam edebilir (return etmeden Kaydet cagrilir).

Yardimci UserControl'ler (`SiparisControl.cs` , `SiparisPanelControl.cs`)

Ne ise yarar: `SiparisPanelControl` (namespace MyUI.MyControl) bir recete satirinin malzeme detaylarini duzenlemek icin dinamik panel olusturur; her `ReceteDetay` (SiparideGosterme=false) icin bir `SiparisControl` (stok kodu/adi/renk/beden/miktar/aciklama editorleri) uretir. `SiparisControl` stok seciminde renk/beden listelerini recete kisitina veya Mikro stok renk/beden tanimina gore doldurur. Bu panel `FrmSiparisEd` ve `FrmSiparisReceteEd` icinde `PanelDetay` e gomulu calisir. **Once ne olmalı (onkosul):** Cagiran kayıt formu `Hareket` , `Detaylar` , `Recete` (ReceteKayitModel), `StoklarAll` , `StokRenklerAll` ve `KayitAction` set etmis olmalı. **Sonra ne olur:** "Tamam" (`BtnTamam_Click`) -> `Aktar()` her `SiparisControl` u `SiparisHareketDetay` modeline yazar (stok kodu bos ise hata), ilk satirdan `Hareket.Renk/Beden` alir, sonra `KayitAction` (formun `PanelGeriAktar` i) cagrilir; bu da degisiklikleri `_mdl.Hareketler/_mdl.Detaylar` a geri isler ve gridi yeniler. **Butonlar & kisayollar:** - `BtnTamam` ("Tamam") — `Aktar()` + `KayitAction.Invoke()` . - Stok kodu/adi combolari `Leave` — secilen Mikro stoktan birim/renk/beden otomatik doldurma. **Cagirdigi katmanlar:** - Service: `Ortak.DbMikro.Stoklar.GetViewListWhere(" where S.sto_kod in (...)" )` / `GetRenkListWhere` / `GetBedenListWhere` — panel acilisinda yalnizca recetedeki stok kodlarini ceker. - Veri: `Recete.ReceteStoklar` , `Recete.ReceteStokRenkBedenler` , `Recete.ReceteDetaylar` — renk/beden kisitlamalari. **Istasyon sirasiyla iliskisi:** - **Notlar:** Saf UI yardimcilaridir, DB'ye yazmaz; sadece okuma yapip kayıt formunun in-memory modelini gunceller. `SiparisControl` te kullanilmayan `RenkBagla___/BedenBagla___` eski metotlar mevcut.

Modul: KullaniciModul

UretimV4 (CepPatronERP) masaustu uygulamasinin kullanıcı yönetim modulu. Tek bir formdan (`FrmKullaniciKayit`) oluşur ve uygulamaya giriş yapacak kullanıcıların tanımlandığı yer/silindiği/duzenlendiği CRUD ekranıdır. Yönettiği kayıtlar `UretimV3_FEZA` veritabanındaki `Kullanici` tablosuna yazılır (`[Table("Kullanici")]`). Bu kayıtlar doğrudan üretim/istasyon akisini etkilemez; uygulama açılısındaki `FrmLogin` ekranında kullanıcı adı/sifre doğrulaması ve `Admin` yetkisi için kullanılır. Sifreler veritabanında düz metin değil, AES anahtarı (`Ortak.GetKey()`) -> "OzelAnahtar1234%&" ile şifrelenmiş olarak (`string.Sifrele/SifreCoz`) saklanır.

Form `MyFrmKayit` (My.Kontrol.Formlar kutuphanesindeki base liste+kayıt formu) türüdür; standart alt bar butonları (Kaydet, Yeni, Düzenle, Sil, Yazdır, Kapat ve kayıt gezinme butonları İlk/Önceki/Sonraki/Son) base'den gelir. Form `SecimIcinAcildi` bayrağı ile "secim modu"nda da açılabilir; bu modda grid'de bir satıra çift tıklayınca/Enter'a basınca kullanıcı seçilir ve çağırılan forma `SecilenKod` (KullaniciAdi) + `SecilenRow` (Kullanici nesnesi) döndürülür.

Kullanici Kaydi (`FrmKullaniciKayit.cs` / `FrmKullaniciKayit.Designer.cs`)

Ne ise yarar: Uygulamaya giriş yapacak kullanıcıları listeler ve ekleme/duzenleme/silme işlemlerini yapar. Üst kısımda form alanları (KullaniciAdi, Sifre, Adı, Soyadı, Admin onay kutusu), alt kısımda mevcut kullanıcıların grid listesi bulunur. **Önce ne olmalı (onkosul):** Uygulamaya giriş yapılmış olmalı (`FrmLogin` -> `Ortak.DbPro` bağlantı fabrikası kurulmuş olmalı). Form, ana ekrandan (`FrmAna`) "Kullanıcılar" menu/bar butonu (`BarBtnKullanicilar_ItemClick`) tiklanınca `new FrmKullaniciKayit().ShowDialog()` ile açılır. Pratikte yalnızca Admin kullanıcıların erişmesi beklenir (yeni kullanıcı/yetki tanımı). **Sonra ne olur:** - Kaydet -> `Kullanici` tablosuna INSERT (yeni Id ile) veya UPDATE (mevcut Id) yapılır; grid yeniden yüklenir (`Bagla()`) ve form alanları temizlenir (`Temizle()`). - Sil -> secili satır `Kullanici` tablosundan DELETE edilir; grid yeniden yüklenir, alanlar temizlenir. - Bu kayıtlar daha sonra `FrmLogin.Kontrol()` tarafından giriş doğrulamasında okunur (KullaniciAdi + şifrelenmiş Sifre karşılaştırması). Baska bir ekrana otomatik geçiş yoktur; işlem aynı form içinde kalır. - Secim modunda (`SecimIcinAcildi=true`): kayıt yapılmaz; satır seçilince `Secildi=true` set edilip form kapanır, çağırılan forma değer döner. **Butonlar & kısayollar:** - `BtnKaydet` ("Kaydet", alt bar) — `BtnKaydet_Click` : form alanlarını `_mdl` Kullanici nesnesine yazar, şifreyi `TxtSifre.Text.Sifrele(Ortak.GetKey())` ile şifreler, `_srv.InsertOrUpdate(_mdl)` çağırır. Base formda genelde Enter/F2 ile tetiklenir. - `BtnSil` ("Sil", alt bar) — `BtnSil_Click` : "Kaydı silmek istiyormusunuz" onayı (`MesajSor`) sonrası grid'deki secili kullanıcıyı `_srv.Delete(data)` ile siler. - `BtnDuzenle` ("Düzenle", alt bar) — `BtnDegistir_Click` : grid'deki secili kullanıcıyı `_mdl` 'e klonlar, form alanlarına doldurur (sifre çözülerek `SifreCoz` gösterilir). - `BtnYeni` ("Yeni", alt bar) — `BtnYeni_Click` : `Temizle()` çağırır; yeni boş `Kullanici` oluşturur, tüm alanları sıfırlar. - `BtnKapat` ("Kapat", alt bar) — base form: formu kapatır (genelde Esc). - `BtnYazdir` ("Yazdır", alt bar) — base form yazdırma (bu formda özel kod yok; grid yazdırma). - `BtnIlk` / `BtnOnceki` / `BtnSonraki` / `BtnSon` — base form kayıt gezinme butonları. - Grid satırına çift tıklama / Enter (`myView1.MyEventDoubleClickEnter` -> `MyView1.MyEventDoubleClickEnter`): secim modundaysa kaydı secip kapatır; değilse secili kaydı form alanlarına doldurur (düzenleme). - `ChcAdmin` ("Admin" onay kutusu) — kullanıcının admin (tam yetki) olup olmadığını belirler. **Çağırdığı katmanlar:** - Service: `IKullaniciService` `_srv = Ortak.DbPro.Kullanicilar` — `KullaniciService : BaseService<Kullanici>` (özel metodu yoktur, tüm davranış base'den gelir). - `_srv.SelectListWhere()` — tüm kullanıcıları getirir (grid kaynağı, `Bagla()` içinde). - `_srv.InsertOrUpdate(_mdl)` — Id varsa UPDATE yoksa INSERT (Kaydet). - `_srv.Delete(data)` — secili kullanıcıyı siler. - Service (kullanılmıyor ama enjekte edilmiş): `IGenelService _srvGenel =`

Ortak.DbPro.GenelServis . - DataAccess: **KullaniciDal : BaseDal<Kullanici>** (Dapper, özel sorgu yok; CRUD base'den). - Entity: **My.Entities.Kullanici** — alanlar: **Id (Guid, PK)** , **Adi** , **Soyadi** , **KullaniciAdi** , **Sifre (sifrelenmis)** , **Admin (bool)** ; **Clone()** metodu (MemberwiseClone). - Yardimci: **Ortak.GetKey()** AES anahtari; **string.Sifrele(key)** / **string.SifreCoz(key)** uzanti metotlari (My.Core) ile sifre sifreleme/cozme. - SQL/Prosedur: yok (saf CRUD; stored procedure cagrilmaz). - API: yok (UretimV4 DB'ye dogrudan baglanir, API kullanmaz). **Istasyon sirasiyla iliskisi:** - (Kullanici tablosu uretim miktar/akis motoruyla, operasyon sirasiyla veya istasyon gruplamasiyla iliskili degildir; yalnızca uygulama girisi/yetkilendirme icindir.) **Notlar:** - Grid'de **Id** ve **Sifre** kolonlari gizlenir (**SutunGizle("Id")** , **SutunGizle("Sifre")**); sifre listede gosterilmez. - Form alanlarina kayit yuklenirken sifre cozulur (**SifreCoz**), kaydedilirken yeniden sifrelenir (**Sifrele**); böylece duzenlemede gercek sifre gosterilir. - İlk kullanıcı otomatik olusturma KullaniciModul'de degil, **FrmLogin.Kontrol()** icinde yapilir: **Kullanici** tablosu bossa otomatik "Admin" / sifre "1" (sifrelenmis), Admin=true bir kayit eklenir. - Giris dogrulaması (**FrmLogin**) kullanıcı adini ve sifreyi **ToUpper()** yaparak karsilastirir; bu nedenle kullanıcı adi/sifre büyük-kucuk harf duyarsizdir. - Grid yerlesimi **myGrid1.MyGridKayitAdi = "KullanciKayitListesi"** adiyla saklanir/yuklenir (**GridYerlesimYukle**). - Tum alanlar **MyMaxLength = 75** (max 75 karakter), **EnterMoveNextControl = true** (Enter ile sonraki alana gecis). - Hata durumlarında kullanıcıya **MesajHata(rs.Message)** ile mesaj gosterilir; islem yapilmaz.

Modul: PersonelModul

PersonelModul, üretim sahasında ve ofiste çalışan personel kartlarının tek bir ekrandan yönetildiği (CRUD) moduldur. Tek bir form içerir: `FrmPersonelKartlari`. Burada tanımlanan personel; istasyon takibi (IstasyonPersoneli + IstasyonKodu eşleştirmesi), yetki/admin bayrakları, görev bazlı SMS gönderimi (BAKIM / PLANLAMA görevleri) ve şifre (MD5 hash) yönetimi için kullanılır. Kayıtlar doğrudan `UretimV3_FEZA` veritabanındaki `Personel` tablosuna Dapper ile yazılır. Kullanıcı giriş/şifre yönetiminden (`FrmKullaniciKayit`) ayrı bir kavramdır; bu modul daha çok saha personeli ve görev/SMS eşleştirmesi içindir. Modulün istasyon-iliskili tarafı: bir personel `IstasyonPersoneli=1` ve `IstasyonKodu` ile bir istasyona bağlandığında, tablet (`TabletV2`) istasyon takip akışında o istasyonda kimlerin çalışabileceğini/etiketleneceğini belirler.

Personel Kaydı (`FrmPersonelKartlari.cs` / `FrmPersonelKartlari.Designer.cs`)

Ne ise yarar: Personel kartları listesini gösterir ve yeni personel ekleme, var olanı düzenleme, silme işlemlerini yapar. Bir personelin Kodu, Adı, Soyadı, Grup, Görevi, CepTel, Şifre alanları ile Yetkili / Admin / IstasyonPersoneli / SmsGonder bayraklarını ve istasyon eşleştirmesini (IstasyonKodu) yönetir. **Once ne olmalı (onkosul):** `FrmAna` (ana ekran) açık olmalı; Ribbon -> "Personel Kartları" (`BarBtnPersonelListesi`, `Id=118`) butonuna tıklanmalı. Form `ShowDialog()` ile modal açılır. Sayac üretmek için `AyarSayac` tablosunda "Personel" kodlu satır, grup/görev comboları için `Personel` tablosunda Grup/Görevi değerleri, istasyon combosu için `IstasyonKarti` kayıtları hazır olmalıdır (bunlar form yüklenirken otomatik çekilir). **Sonra ne olur:** Kaydet sonrası `Personel` tablosuna INSERT (yeni) ya da UPDATE (mevcut) yapılır; şifre girilirse `GetInsOrUpdCode()` SQL'i (şifre dahil), girilmemişse `GetUpdCodeSifresiz()` SQL'i (şifre hariç) çalıştırılır. İşlem sonrası grid `Bagla()` ile yeniden yüklenir, form alanları `Temizle()` ile sıfırlanır ve kayıt grubu (`GrpKayit`) tekrar pasifleşir. Silme sonrası ilgili satır `Personel` tablosundan kaldırılır. Ayrı bir ekrana gidilmez; aynı form açık kalır. Form seçim modunda açılmissa (`SecimIcinAcildi`) çift tıklama/Enter ile seçilen personel geri dondurulup form kapanır. **Butonlar & kısayollar:** - `BtnYeni` (Yeni) — `BtnYeni_Click` : `Temizle()` çağırıp kayıt grubunu (`GrpKayit`) aktifleştirir, yeni boş kayıt girişi hazırlar (`YeniKayit=true`). - `BtnKaydet` (Kaydet) — `BtnKaydet_Click` : Form alanlarını `_mdl` (Personel) nesnesine aktarır; Kodu bossa `EvrakNoAl()` ile otomatik kod üretir; CepTel maskesini ("(000) 000-0000") temizler; `KodVarMi()` ile aynı Kodda kayıt kontrolü yapar; yeni kayıta şifre zorunluluğunu denetler; sonra `Kaydet()` çalışır. - `BtnSil` (Sil) — `BtnSil_Click` : "Kaydı silmek istiyormusunuz" onayı (`MesajSor`) sonrası seçili gridteki personeli `_srv.Delete(data)` ile siler. - `BtnDuzenle` (Düzenle) — `BtnDegistir_Click` : Gridte seçili personeli `AktarTextlere(data)` ile form alanlarına doldurur, düzenleme moduna geçer (`GrpKayit` aktif, `YeniKayit=false`). - `BtnKapat` (Kapat) — base form (`MyFrmKayit`) davranışı; formu kapatır (genel `Esc=Kapat` kalibi). Bu modulde özel event bağlanmamış. - `BtnYazdir` (Yazdır) — Designer'da tanımlı ancak bu formda Click event'i bağlanmamış (kullanılmıyor). - Navigasyon butonları `BtnIlk` / `BtnOnceki` / `BtnSonraki` / `BtnSon` — base form (`MyFrmKayit`) gezinme butonları; bu formda özel event bağlanmamış. - Grid çift tıklama / Enter — `myView1_MyEventDoubleClickEnter` : Seçim modunda (`SecimIcinAcildi`) satırı seçip formu kapatır; normal modda `AktarTextlere()` ile satırı forma doldurur (düzenleme). - Form kontrol kısayolları: `CepTel` alanında maske "(000) 000-0000"; `Şifre` alanı `PasswordChar=''`; tüm kayıt alanlarında `EnterMoveNextControl=true` (Enter ile bir sonraki kontrole geçiş); gride `EnterMoveNextColumn=true`. Base `MyFrmKayit` kalibi gereği `Enter=Kaydet` / `Esc=Kapat` genel davranışları uygulanır (özel `ShortcutKeys/ToolStrip` menu ögesi bu formda tanımlı değil). Çağırdığı katmanlar: - Manager/Service: `IPersonelService.SelectListWhere()` (`PersonelService -> BaseService`) — `Personel` tablosundaki tüm personeli listeler, grid'e bağlar (`Bagla()`). - Manager/Service: `IPersonelService.PersonelKaydet(Personel, yenikayitmi)` — önce `KodVarMi(Kodu)` ile mukerrer kod denetimi, sonra `Personel.GetInsOrUpdCode()`

SQL'ini (Id varsa UPDATE / yoksa INSERT, sifre dahil) Dapper Execute ile calistirir. - Manager/Service: `IPersonelService.PersonelKaydetSifresiz(Personel, yenikayitmi)` — sifre bos birakildiginda `Personel.GetUpdCodeSifresiz()` SQL'i ile sadece UPDATE yapar, mevcut sifreyi degistirmez. - Manager/Service: `IPersonelService.SifreLe64(metin)` — sifreyi "mikroconnect" tuzu ile birlestirip MD5 hash uredir; kaydetmeden once `_mdl.Sifre` bu degerle doldurulur. - Manager/Service: `IPersonelService.Delete(Personel)` (BaseService) — secili personeli `Personel` tablosundan siler. - Manager/Service: `IGenelService.GetEvrakNo("Personel")` (GenelService -> GenelDal.GetEvrakNo) — `AyarSayac` tablosundan "Personel" sayacini okuyup `BasinaEkle + BasamakSayisi` ile formatlanmis yeni kod uredir ve Verilecek degerini +1 artirir (Kodu bossa cagrilir). - Manager/Service: `IGenelService.GrupListesi("Personel", "Grup")` ve `GrupListesi("Personel", "Gorevi")` (GenelDal.GrupListesi) — `Select Grup/Gorevi as Kodu From Personel group by ...` ile mevcut benzersiz Grup ve Gorevi degerlerini cekip `CmbGrup / CmbGorevi` combolarina baglar (basa bos satir eklenir). - Manager/Service: `IstasyonKartiService.SelectListWhere(" Order By IstasyonKodu ")` — istasyon kartlarini cekip `CmbIstasyonKodu` (MyLookupEdit) lookup'ina `IstasyonKodu` degeriyle baglar. - Manager/Service: `IGenelService.Query<int>(sql, entity)` — formdaki `KodVarmi()` mukerrer kod kontrolu icin `Select count(*) From Personel where Kodu=@Kodu` benzeri sorguyu calistirir (ClassExtensions.GetClassTableName / GetClassColumnNameKey ile tablo ve PK adi dinamik alinir). - SQL/Prosedur: `Personel.GetInsOrUpdCode()` — IF EXISTS(Id) UPDATE ELSE INSERT; tum alanlari (Kodu, Adi, Soyadi, Grup, Gorevi, Yetkili, Admin, Sifre, IstasyonPersoneli, IstasyonKodu, CepTel, SmsGonder) yazar. - SQL/Prosedur: `Personel.GetUpdCodeSifresiz()` — Sifre haric tum alanlari UPDATE eder (WHERE Id=@Id). - SQL/Prosedur: `GenelDal.GetEvrakNo` (inline T-SQL, AyarSayac uzerinde) — yeni Personel kodu uredir. - API: - (UretimV4 masaustu; bu form dogrudan DB'ye baglanir, API kullanmaz) Istasyon sirasiyla iliskisi: Dogrudan operasyon-Sira akis motoruyla (Uretim_MiktarGuncelle / Uretim_PlanlananGuncelle / Uretim_SonrakilstasyonaGonder) bu form etkilesmez. Ancak burada `IstasyonPersoneli=1` ve `IstasyonKodu` ile bir istasyona baglanan personel, tablet (TabletV2) istasyon takip akisinda (IstasyonTakipPage baslat/durdur/bitir) o istasyonda calisan personeli temsil eder; istasyon eslestirmesi bu formdaki `CmbIstasyonKodu` (IstasyonKarti listesi) ile yapilir. Notlar:* - Grid'de `Id` ve `Sifre` kolonlari gizlenir (`SutunGizle`). Grid yerlesimi `MyGridKayitAdi="PersonelKartlariListesi"` adıyla kaydedilir/yuklenir (`GridYerlesimYukle`). - Kodu "Admin"/"admin" olan kayitta `TxtKodu` ve `TxtAdi` alanlari salt-okunur yapilir (admin kullanicisinin kodu/adi degistirilemez). - Sifre kurali: Guncellemede sifrealani bos birakilirsa sifre degismez (form uzerindeki label1: "Guncellemede Sifre Girilmezse Sifreyi Degistirmez"); yeni kayitta sifre zorunludur ("Lutfen sifre Giriniz"). - SMS kurali (form uzerindeki label2): SMS gonderilmesi icin Gorevi alanina Bakimci icin "BAKIM", Planlamaci icin "PLANLAMA" yazilmali; ChcSmsGonder isaretili olmalı. CepTel maske disindaki "-" ve bosluklar kaydedilmeden once temizlenir. - Form base sinifi `MyFrmKayit` (My.Kontrol.Formlar, derlenmis DLL) — `pnlAltBtn / BtnKaydet / BtnSil / BtnYeni / BtnDuzenle / BtnKapat / navigasyon butonlari` ve genel Enter/Esc kalibi base'den gelir; bu form sadece ilgili Click event'lerini `EventlerBagla()` icinde baglar. - Formun bilinen bir bug'i: form icindeki `KodVarmi()` mukerrer kod bulundugunda `new ErrorResult(...)` nesnesini olusturur ama `return` etmez (satir 245), bu yuzden formdaki kontrol mukerrer kodu engellemez; gercek engelleme servisteki `PersonelService.KodVarmi` ile `PersonelKaydet/PersonelKaydetSifresiz` icinde yapilir.

Modul: AyarVeGenelModul

Bu modul, UretimV4 (CepPatronERP / WinForms) masaustu uygulamasinin "cekirdek/altyapi" katmanidir. Iceriginde uygulamaya giris (login), donanim bazli lisans kontrolu, kullanicinin kendi olusturdugu favori dugmelerden olusan masaustu paneli ve uygulamanin tum ayar ekranlari yer alır. Ayar ekranlarinin hemen hepsi tek bir **Ayar** tablosu (Modul + Grup + Kodu + Deger + Aciklama) üzerinde çalışır; her ekran sadece **Modul** filtresiyle ayırır (GenelAyarlar -> **Modul='Genel'** , MikroEntAyarlari -> **Modul='MikroEntegre'** , IstasyonUretimAyarlari -> **Modul='IstasyonUretim'**). Mikro ERP entegrasyonu için hangi üretim/stok/fire/sarf hareketinin Mikro'da hangi fis turune yazilacagi da bu moduldeki **FrmMikroEntAyarlari** ekranında belirlenir. Ayrica Mikro'da stok kodu degisince UretimV3 DB'sindeki tum ilgili tablolari toplu guncelleyen **FrmStokKodGuncelle** ile, acil duyuru mesajini tutan **FrmMesajGenel** de burada bulunur.

Modulun cogu ayar formu **My.Kontrol.Formlar.MyFrmKayit** (harici DLL) tabanlıdır; ortak alt buton seridi base formdan gelir: **BtnKaydet** , **BtnSil** , **BtnYeni** , **BtnDuzenle** , **BtnYazdir** , **BtnKapat** ve navigasyon dugmeleri **BtnIlk** / **BtnOnceki** / **BtnSonraki** / **BtnSon** . Base form ayrica **MesajHata** / **MesajBilgi** / **MesajSor** , **IdGuid** , **YeniKayit** , **KayitEdildi** , **AcilisBittimi** , **SecimIcinAcildi** / **Secildi** / **SecilenKod** / **SecilenId** / **SecilenRow** (secim modu) ve grid bileşenleri **myGrid1** / **myView1** / **bs** (BindingSource) uyelerini sağlar. Login ekranı ise **MyFrmLoginPaneli** tabanlıdır ve kendine özel kisayollar tasir.

Giris / Login (**FrmLogin.cs** , **MyFrmLoginPaneli.cs**)

Ne ise yarar: Uygulamaya kullanıcı adı + sifre ile giriş yapılan ekran. Hic kullanıcı yoksa otomatik olarak "Admin / 1" admin kullanicisini olusturur. Basarili girese kadar ana ekran (FrmAna) acilmez. **Once ne olmalı (onkosul):** **FrmAna_Load** içinde önce **UpdateProgram.VersiyonKontrol()** (guncelleme kontrolu) gecmeli; sonra **Ortak.GetKey()** ile AES anahtari alinip **Ortak.DbPro** (DatabaseFactoryPro) ve **Ortak.DbMikro** (DatabaseFactoryMikro) baglantilari kurulur. Kullanici adi **Ortak.AyarIni** 'den ("AYAR/KULLANICI") on-doldurulur. **Sonra ne olur:** Giriş dogrulaniursa **GirisYapildi=true** , **Ortak.KullaniciAdi** set edilir, kullanıcı adı tekrar ini dosyasina yazilir ve form kapanir. FrmAna devaminda skin yukler, masaustunu acar, **OrtakLis.SistemLisansKontrolu()** ile lisansi kontrol eder (lisans pasifse Recete/ReceteGrup/Kullaniciilar dugmeleri devre disi), ardından **MikroEntAyarlarBagla** / **GenelAyarlarBagla** / **IstasyonAyarlarBagla** ve **TempGuncelle** (Mikro stok temp tablolari) çalışır. **Butonlar & kisayollar:** - **Giris (F2)** / **BtnGiris** — **Kontrol()** dogrulamasini calistirir; dogruysa giriş yapar (Enter sifre kutusunda da ayni isi yapar). - **Kapat (Esc)** / **BtnKapat** — formu kapatir (giris yapilmamis sayilir, FrmAna kendini kapatir). - **Enter** (Kullanici kutusunda) — odagi Sifre kutusuna tasir; **Enter** (Sifre kutusunda) — girişi dener. - **Pro** / **BtnDbPro** , **Mik** / **BtnDbMikro** — DB ayar panelini acmak için tasarlanmis; kod yorum satirinda, ayrica **Visible=false** (kullanilmiyor). **Cagirdigi katmanlar:** - Service: **Ortak.DbPro.Kullaniciilar.Query<int>("select count(*) ... Kullanicisi")** — kullanıcı sayisini sayar; 0 ise admin ekler (**...Kullaniciilar.Insert(new Kullanicisi{...})**). - Service: **Ortak.DbPro.Kullaniciilar.SelectFirst(k => k.KullaniciAdi == ka)** — kullanıcıyi getirir; sifre **ps.Sifrele(_aesMasterKey)** ile karsilastirilir. - Helper: **Ortak.GetKey()** — AES master anahtari; **Ortak.AyarIni.Oku/Yaz** — ini ayar dosyasi. - SQL/Prosedur: - ; API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Sifre karsilastirmasi buyuk harfe cevrilerek (**ToUpper()**) yapilir, yani kullanıcı adı/sifre buyuk-kucuk harf duyarsizdir. **KeyPreview=true** olduğu için Esc/F2 kisayollari her kontrolde çalışır.

Lisans Kaydi (FrmLisansGiris.cs , OrtakLis.cs)

Ne ise yarar: Donanima (islemci ID'si) bagli lisans anahtarini olusturup `lisans.lic` dosyasina yazan ekran. Key bilgisayara ozeldir; uretici tarafından uretilen "Karsilik" degeri girilince lisans aktiflesir. **Once ne olmalı (onkosul):** `Ortak.LisansKayit()` cagrildiginda (lisans pasifken) acilir. Acilista `OrtakLis.SistemLisansKontrolKey()` ile makinenin islemci seri numarasindan uretilen Key gosterilir; varsa mevcut `lisans.lic` icerigi okunur. **Sonra ne olur:** Girilen karsilik, `OrtakLis.SistemLisansKeyKontrolu(Key)` ciktisi ile esitse `lisans.lic` dosyasi guncellenir ve `Application.Restart()` ile uygulama yeniden baslatilir. Esit degilse "Lisans Kodu Gecersiz" mesajı. **Butonlar & kisayollar:** - `Kaydet` / `button2` — `button2_Click` : girilen karsiligi dogrular, uyarsa dosyaya yazip uygulamayi restart eder. **Cagirdigi katmanlar:** - Helper: `OrtakLis.SistemLisansKontrolKey()` — `GetBaseCpuInfo()` (WMI `win32_processor.processorID`) -> `Kisa()` (MD5 + ozel sayisal donusum) ile makine anahtari uretir. - Helper: `OrtakLis.SistemLisansKeyKontrolu(data)` = `Kisa(data)` — beklenen karsiligi hesaplar. - Helper: `OrtakLis.SistemLisansKontrolu()` — `lisans.lic` ile makine anahtarini karsilastirip bool doner (FrmAna acilista bunu kullanir). - SQL/Prosedur: - ; API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Lisans makine basnadir (islemci ID). Lisans pasifken FrmAna'da Recete Listesi, Recete Grup/Takimler ve Kullanicilar dugmeleri kapali kalir.

Masaustu (Favori Dugmeler Paneli) (FrmMasaustu.cs , MasaUstuButonlar.cs)

Ne ise yarar: Ana ekran (MDI) icinde acilan, kullanicinin kendi favori kisayol dugmelerini tasarlayip yerlestirdigi panel. Dugmeler `FavorilerSettings.json` dosyasinda saklanir (konum, boyut, renk, isim). Dugmeye tiklayınca ilgili modul formu acilir. **Once ne olmalı (onkosul):** Login basarili olduktan sonra `FrmAna.MasaUstuAc()` tek instance halinde acar (`Name="Masaustu"` , `MdiParent=FrmAna`). Acilista `ButonBagla()` JSON'dan dugme listesini okur, yoksa bos liste ile dosyayi olusturur. **Sonra ne olur:** Duzenleme modunda yapilan degisiklikler (tasima, isim, renk, ekleme/silme) `AyarKaydet()` ile `FavorilerSettings.json` 'a serialize edilir. Normal modda dugmeye tiklanınca `FormAc.FormSec(btn.Tag)` ile ilgili form acilir. **Butonlar & kisayollar (sag tik / ContextMenuStrip btnContext):** - `Duzenle` — `DuzenleAc()` : duzenleme modunu acar (panel arka plani DarkCyan, dugmeler surukle-birak ile tasinabilir). - `Kaydet` — `DuzenleKapat()` : duzenleme modunu kapatir ve `AyarKaydet()` ile JSON'a yazar. - `Iptal` — `DuzenleKapat()` : duzenleme modunu kapatir. - `YenidenIsimver` — `Interaction.InputBox` ile secili dugmeye yeni isim verir. - `Renk` — `ColorDialog` ile secili dugmenin arka plan rengini degistirir. - `Butonu Kaldir` — onay sonrasi secili dugmeyi listeden ve panelden siler. **Cagirdigi katmanlar:** - UI yonlendirme: `FormAc.FormSec(tag)` — Tag string'ine gore ilgili modul formunu acar (Siparisler, ReceteUretimEkle, UretimEmirleri, OperasyonTakip, IstasyonTakip, MikroStokListesi, IstasyonRaporu vb.; FrmAna ribbon dugmeleri de ayni metodu kullanir). - Dosya: `JsonConvert` ile `FavorilerSettings.json` oku/yaz (Newtonsoft). - SQL/Prosedur: - ; API: - **Istasyon sirasiyla iliskisi:** Dolayli — masaustu dugmesi `IstasyonTakip` ile saha takip ekranini (FrmUretimIstasyonTakip) acabilir; ancak akis mantigini iceren ekranlar UretimIstasyon modulundedir. **Notlar:** Yeni favori dugme `FrmMasaustu.ButonEkle(Tag)` ile (disaridan, FrmAna ribbon'undan) eklenir. Dugmeler hareket eşiği 5px ile koruma altinda (kayma olunca tiklama iptal). Sinif adi kodda `FrmMasaustu1`.

Genel Ayarlar (FrmGenelAyarlar.cs)

Ne ise yarar: `Modul='Genel'` kapsamindaki uygulama genel ayarlarini listeleyip degerlerini duzenler (orn. `PLKapat` plasiyer/PL kapanma davranisi). Sadece mevcut kayitlarin Deger/Aciklamaalani guncellenir; yeni kod ekleme/silme aktif degildir. **Once ne olmalı (onkosul):** Veritabanı guncellemesi (`Ortak.DatabaseGuncelleUretim` / "DB Guncelle") ile `Ayar` tablosundaki Genel kayitlari olusturulmus olmalı.

Ribbon: `BarBtnGenelAyarlar` -> `FrmGenelAyarlar.ShowDialog()` . **Sonra ne olur:** Kaydet sonrası `Ayar` tablosuna `InsertOrUpdate` yazilir, grid yeniden baglanir ve `Ortak.GenelAyarlarBagla()` (ek olarak `Ortak.MikroEntAyarlarBagla()`) ile RAM'deki ayar cache'i tazelenir (`Ortak.GenelAyarlar` , `Ortak.PlKapat`).

Butonlar & kisayollar: - `BtnKaydet` — `Kaydet()` -> `AktarModele()` -> `_srv.InsertOrUpdate(_mdl)` , sonra `Bagla()` + `Ortak.GenelAyarlarBagla()` + `Ortak.MikroEntAyarlarBagla()` . - `BtnDuzenle` — `Duzenle()` : gridde secili satiri text kutularina aktarir (kayda çift tik = ayni is). - `BtnYeni` — `TemizleText()` : text kutularini bosaltir. - `BtnSil` — onay sorar; ancak `Sil()` govdesi yorumlu (silme fiilen pasif). - Çift tik / Enter (grid) — secim modunda satiri secip kapatir; degilse Duzenle. **Cagirdigi katmanlar:** - Service: `IAyarService` (`Ortak.DbPro.Ayarlar` , BaseService) — `SelectListWhere("where Modul='Genel' Order By Kodu")` , `InsertOrUpdate(Ayar)` . - Helper: `Ortak.GenelAyarlarBagla()` — `Modul='Genel'` ayarlarini cache'e alir, `PlKapat` flag'ini set eder. - SQL/Prosedur: dogrudan SP yok (generic DAL). ; API: - **Istasyon sirasiyla iliskisi:** -

Notlar: `TxtKodu` salt-okunur (Enabled=false); Kodu yeni olusturulamaz, sadece var olan ayarin Degeri/Aciklamasi guncellenir.

Mikro Entegre Ayarlari (`FrmMikroEntAyarlari.cs`)

Ne ise yarar: Mikro ERP entegrasyonunun beyni. Iki bolumden olusur: (1) ust "Fisler Genel" panelinde Mikro Firma No, Kullanici Kodu ve her uretim hareketinin (urun girisi, stok cikisi, urun/stok fire, sarf cikisi, fire girisi, hizli uretim) Mikro'da hangi fis turune yazilacaginin secimi; (2) sekmeli alt grid ile `Modul='MikroEntegre'` altindaki tum ayar detaylarinin grup grup duzenlenmesi. **Once ne olmalı (onkosul):** DB Guncelle ile `Ayar` tablosunda `Modul='MikroEntegre'` (GENEL, FisTuru ve diger grup) kayitlari olusturulmus olmalı. Ribbon: `BarBtnMikroEntAyarlari` -> `FrmMikroEntAyarlari.ShowDialog()` . Acilista `GenelBagla()` combo'lari `MikroKayitFisTurleri` listeleriyle, sekmeleri `GetAyarFisTurleriList()` grup adlariyla doldurur. **Sonra ne olur:** "Genel Kaydet" tum genel/fis-turu degerlerini topluca `InsertOrUpdate(_listGenelVeTur)` yapar; alt grid "Kaydet" tek detay kaydini yazar. Her ikisinde de `Ortak.MikroEntAyarlarBagla()` cache'i tazelenir (`Ortak.MikroEntAyarlar`). Bu ayarlar daha sonra Mikro'ya kayit ekranlarinda (FrmMikroyaUretimKaydetV2, FrmMikroyaSarfFireKaydet, FrmHizliUretimEG) `MikroConvertManager` tarafindan okunur. **Butonlar & kisayollar:** - `Genel Kaydet` / `BtnGenelKaydet` — `GenelAktar()` ile Firma No/Kullanici Kodu ve tum fis-turu combo degerlerini modele aktarip topluca kaydeder + cache tazeler. - `BtnKaydet` (alt buton serisi) — secili detay ayar satirini kaydeder + `Ortak.MikroEntAyarlarBagla()` . - `BtnDuzenle` — gridde secili `Ayar` satirini Kodu/Aciklama/Deger kutularina aktarir. - `BtnYeni` — text kutularini temizler. - `Ayarlari Sil` / `BtnAyarlarSil` — onay sonrası `_srv.Delete(c => c.Modul=='MikroEntegre')` ile tum MikroEntegre ayarlarini sifirlar; sonra DB Guncelle yeniden cagirmali. - `BtnSil` — gizli (Visible=false), kullanilmiyor. - Sekmeler (`TabGrupAdlari`) — sekme degisince `DetayBagla(grup)` ile o grubun ayar satirlari gridde gosterilir. - Çift tik / Enter (grid) — secim modunda secip kapatir; degilse `BtnDuzenle.PerformClick()` . **Cagirdigi katmanlar:** - Service: `IAyarService` — `SelectListWhere("where Modul='MikroEntegre' Order By Modul,Grup,Kodu")` , `InsertOrUpdate(Ayar / List<Ayar>)` , `Delete(c => c.Modul=='MikroEntegre')` . - Manager: `MikroKayitFisTurleri` — `GetAyarFisTurleriList()` (sekme grup adlari) ve `GetUretimUrunGirisFisiTuruList` / `GetUretimStokCikisFisiTuruList` / `GetUretimUrunFireCikisFisiTuruList` / `GetUretimStokFireCikisFisiTuruList` / `GetSarfCikisFisiTuruList` / `GetFireGirisFisiTuruList` / `GetHizliUretimFisiTuruList` (combo secenekleri); `MikroAyarFisTurleri` enum'u Kodu eslesmesinde kullanilir. - Helper: `Ortak.MikroEntAyarlarBagla()` — `Modul='MikroEntegre'` ayarlarini RAM cache'e (`Ortak.MikroEntAyarlar`) alir. - SQL/Prosedur: dogrudan SP yok. ; API: - **Istasyon sirasiyla iliskisi:** Dolayli — buradaki fis turu ayarlari, istasyon/operasyon sonunda Mikro'ya uretim/stok/fire fisi yazilirken kullanilir (MikroConvertManager). **Notlar:** Combo'lar `Grup='GENEL'` (FirmaNo, KullaniciKodu) ve

`Grup='FisTuru'` kayitlariyla iki yonlu eslestirilir. `TxtKodu` salt-okunur. Tasarimda 10 sabit XtraTabPage var ama sekmeler calistirmada `_listGrupAd` (8 ayar fis turu) ile dinamik doldurulur.

Istasyon Uretim Ayarlari (`FrmIstasyonUretimAyarlari.cs`)

Ne ise yarar: `Modul='IstasyonUretim'` kapsamindaki saha/istasyon uretim davranis ayarlarini listeleyip duzenler (orn. `Grup='Istasyon'` , `Kodu='MalKabulKullan'` -> mal kabul ozelliginin acik/kapali olmasi). **Once ne olmalı (onkosul):** DB Guncelle ile `Ayar` tablosunda IstasyonUretim kayitlari olusmus olmalı. Ribbon: `BarBtnIstasyonUretimAyarlari` -> `FrmIstasyonUretimAyarlari.ShowDialog()` . **Sonra ne olur:** Kaydet -> `Ayar.InsertOrUpdate` ardından grid yenilenir. (Bu ekran kaydetten sonra `Ortak.MikroEntAyarlarBagla()` cagirir; istasyon cache'i `Ortak.IstasyonAyarlarBagla()` ise FrmAna acilista calisir ve `Ortak.MalKabulKullan` flag'ini set eder.) **Butonlar & kisayollar:** - `BtnKaydet` — `Kaydet()` -> `_srv.InsertOrUpdate(_mdl)` , sonra `Bagla()` + `Ortak.MikroEntAyarlarBagla()` . - `BtnDuzenle` — secili satiri kutulara aktarir. - `BtnYeni` — kutulari temizler. - `BtnSil` — onay sorar; `Sil()` govdesi yorumlu (silme pasif). - Cift tik / Enter (grid) — secim modunda secer; degilse Duzenle. **Cagirdigi katmanlar:** - Service: `IAyarService` — `SelectListWhere("where Modul='IstasyonUretim' Order By Kodu")` , `InsertOrUpdate(Ayar)` . - Helper: `Ortak.IstasyonAyarlarBagla()` — `Modul='IstasyonUretim'` ayarlarini cache'ler, `MalKabulKullan` flag'ini set eder (FrmAna acilista). - SQL/Prosedur: dogrudan SP yok. ; API: - **Istasyon sirasiyla iliskisi:** Dogrudan — buradaki ayarlar (orn. `MalKabulKullan`) saha istasyon takip akisini etkiler. **Notlar:** Kod yapisi GenelAyarlar ile birebir ayni (sadece `ayarModul` farkli). `TxtKodu` salt-okunur degil ama Kodu degeri modele aktarilmiyor (sadece Aciklama+Deger guncellenir).

SMS Ayarlari (`FrmSmsAyarlari.cs`)

Ne ise yarar: SMS gonderim saglayicisi bilgilerini (Kullanici Kodu, Sifre, Baslik/gonderici adi, Gonderim URL, Rapor URL) tutar. Diger ayar ekranlarından farklı olarak ayrı bir `SmsAyar` tablosu/servisi kullanir ve tam CRUD (silme dahil) aktiftir. **Once ne olmalı (onkosul):** DB Guncelle ile `SmsAyar` tablosu olusmus olmalı. Ribbon: `BarBtnSmsAyarlari` -> `FrmSmsAyarlari.ShowDialog()` . **Sonra ne olur:** Kaydet -> `SmsAyar.InsertOrUpdate` ; bu degerler SMS gonderimi yapan modul (orn. SMS Rapor / sevkiyat bildirimleri) tarafından kullanilir. **Butonlar & kisayollar:** - `BtnKaydet` — `Kaydet()` : `TextLeriKontrolEt()` (5 alan da zorunlu) -> `_srv.InsertOrUpdate(_mdl)` -> grid yenilenir. - `BtnDuzenle` — secili satiri kutulara aktarir. - `BtnYeni` — kutulari temizler. - `BtnSil` — onay sonrasi `Sil()` -> `_srv.Delete(_mdl)` (gercek silme yapar). - `myButton1` — Gonderim URL kutusunu temizler; `myButton2` — Rapor URL kutusunu temizler. - Cift tik / Enter (grid) — secim modunda secer; degilse `BtnDuzenle.PerformClick()` . **Cagirdigi katmanlar:** - Service: `ISmsAyarService` (`Ortak.DbPro.SmsAyar`) — `SelectListWhere("")` , `InsertOrUpdate(SmsAyar)` , `Delete(SmsAyar)` . - SQL/Prosedur: dogrudan SP yok. ; API: SMS gonderim URL'leri (GunderimUrl/RaporUrl) ayar olarak saklanir; cagrilma SMS modulunde. **Istasyon sirasiyla iliskisi:** - **Notlar:** Bu modulde silmenin gercekten calistigi tek ayar ekranı; zorunlu alan validasyonu da burada yapilir.

Acil Mesaj (`FrmMesajGenel.cs`)

Ne ise yarar: Uygulama genelinde gosterilecek tek bir "Acil" duyuru mesajini (`Mesajlar` tablosu, `Modul='Genel'` , `Kodu='Acil'`) zengin metin (RichTextBox) olarak duzenler. **Once ne olmalı (onkosul):** DB Guncelle ile `Mesajlar` tablosunda Genel/Acil kaydi mevcut olmalı. Ribbon: `BarBtnAcilMesaj` -> `FrmMesajGenel.ShowDialog()` . Acilista `MesajBagla()` ile mevcut mesaj yuklenir. **Sonra ne olur:** Kaydet -> `Mesajlar.InsertOrUpdate(msj)` ile guncellenir ve form kapanir. Mesaj, uygulamada acil duyuru gosteren

yerlerde okunur. **Butonlar & kisayollar:** - `BtnKaydet` — `BtnKaydet_Click` : RichTextBox metnini `msj.Mesaj` 'a aktarir, `_srv.InsertOrUpdate(msj)` ve formu kapatir. - `BtnSil` — gizli (Visible=false). **Cagirdigi katmanlar:** - Service: `IMesajlarService` (`Ortak.DbPro.Mesajlar`) — `SelectFirstWhere("where Modul='Genel' and Kodu='Acil'")` , `InsertOrUpdate(Mesajlar)` . - SQL/Prosedur: dogrudan SP yok. ; API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Tek kayıt üzerinde çalışır; yeni mesaj oluşturmaz, var olan Genel/Acil kaydini gunceller.

Stok Kodu Guncelleme (`FrmStokKodGuncelle.cs`)

Ne ise yarar: Bir stok kodunun (eski) sistem genelinde baska bir koda (yeni) toplu olarak degistirilmesini saglar. Mikro tarafında stok kodu degistiginde UretimV3 (uretim) veritabanındaki tum ilgili tablolarda referansi tek transaction icinde gunceller. **Once ne olmalı (onkosul):** Ribbon: `btnStokGuncelle` ->

`FrmStokKodGuncelle.ShowDialog()` . Kullanici eski ve yeni stok kodlarini girer. **Sonra ne olur:** Onay sonrası `IUretimStokFisService.StokKoduGuncelle(eski, yeni)` çalışır; tek transaction icinde su tablolar UPDATE edilir: `DepoStokHareket` , `UretimStok` , `DepoStok` , `DepoKabulIrsaliyeHareket` , `DepoStokBarkod` , `ReceteDetay` (VarsayılanStokKodu) , `SiparisHareket` , `SiparisHareketDetay` , `UretimStokFisHareket` . Hata olursa rollback yapılır. **Butonlar & kisayollar:** - `Kaydet` / `btnKaydet` — `btnKaydet_Click` : "X kodu Y ile degistirilecek, devam?" onayi -> `_srv.StokKoduGuncelle(...)` -> basariyliysa "Basariyla Guncellendi".

Cagirdigi katmanlar: - Service: `IUretimStokFisService` (`Ortak.DbPro.UretimStokFis`) — `StokKoduGuncelle(string eskiKod, string yeniKod)` : 9 tabloda transactional UPDATE. - SQL/Prosedur: inline UPDATE'ler (SP degil), `_dal.Execute(...)` ile tek transaction. ; API: - **Istasyon sirasiyla iliskisi:** Dolayli — uretim/recete/siparis hareketlerindeki stok kodu referanslarini gunceller; akis sirasini degistirmez. **Notlar:** Dikkat: stok kodu string interpolation ile sorguya gomulur (SQL injection riski); kullanıcı tarafından dikkatli girilmeli. Plain WinForms `Form` tabanlı (MyFrmKayit degil).

Yardimci/Altyapi dosyalari (form degil)

- `MasaUstuButonlar.cs` — Masaustu favori dugmesinin POCO modeli (Text, Name, LocationX/Y, SizeX/Y, Icon64, Renk). `FavorilerSettings.json` 'a serialize edilir.
- `MyExtentionsObject.cs` (`MyExtentionObject`) — string uzanti metotlari `MesajHata` / `MesajBilgi` / `MesajSor` (DevExpress XtraMessageBox), `IsNullOrEmpty` (Guid?/string) ve `ToDataTable<T>` cevirci uzantilari.
- `OrtakLis.cs` — donanim bazli lisans uretim/kontrol yardimcisi (MD5 + WMI islemci ID); `FrmLisansGiris` ve `FrmAna` acilis lisans kontrolu bunu kullanir.
- `MyFrmLoginPaneli.cs/.Designer.cs` — `FrmLogin`'in base formu; Esc=Kapat, F2=Giris kisayollari, surukle-ile-tasi ve giris kontrol kutulari (`TxtKullanici` , `TxtSifre` , `BtnGiris` , `BtnKapat`) burada tanimli.

Modul: MailModul

UretimV4 (CepPatronERP WinForms) icindeki mail/e-posta modulu. Uretim DB'sindeki (`UretimV3_FEZA`) `MailSettings` tablosunda saklanan SMTP gonderici hesaplarini yonetir ve bu hesaplar üzerinden e-posta gonderir. Uc formdan olusur: (1) SMTP hesap ayarlarinin CRUD'unu yapan `FrmMailSettings` , (2) ayarlarin gercekten calistigini denemek icin manuel test maili gonderen `FrmMailGonderOrnek` , (3) bir siparis fisini PDF/XLSX olarak uretip ekli e-posta ile cariyeye gonderen `FrmFisMailGonder` . Tum formlar `MailManager(Ortak.DbPro, Ortak.DbMikro)` üzerinden calisir; ayarlar `_dbPro.MailSettings` (Pro/Uretim DB) tablosundan okunur/yazilir, gercek gonderim `System.Net.Mail.SmtpClient` ile yapilir. SMTP sifresi DB'de simetrik sifreli tutulur (`Sifrele` / `SifreCoz` + `Ortak.GetKey()`). Modul uretim miktar/akis motoruyla (operasyon-sira, istasyon prosedurleri) ilgili degildir; tamamen yordimci/altyapi moduludur.

Mail Ayarlari (`FrmMailSettings.cs` / `FrmMailSettings.Designer.cs`)

Ne ise yarar: SMTP gonderici hesaplarinin (Host, Port, MailAdres, Pass, DisplayName, EnableSsl, varsayilan Konu/Body) listelendigi ve eklenip/guncellenip/silindigi ayar ekranidir. Ust panelde giris alanlari, altta kayitli ayar listesi (grid) vardir. **Once ne olmalı (onkosul):** Uygulamaya giris yapilmis ve `Ortak.DbPro` / `Ortak.DbMikro` baglanti factory'leri hazir olmalı. Form, ana menuden `FrmAna.BarBtnMailAyarlari_ItemClick` ile `new FrmMailSettings().ShowDialog()` cagirlarak acilir. `MailSettings` tablosunun olmasi gerekir. **Sonra ne olur:** Kaydet -> `MailSettings` tablosuna INSERT veya UPDATE (`MailManager.MailSettingsKaydet` -> `_dbPro.MailSettings.InsertOrUpdate`). Sil -> `MailSettings` 'ten ilgili Id silinir (`MailManager.MailSettingsSil` -> `_dbPro.MailSettings.Delete`). Her iki islemden sonra grid yeniden baglanir (`Bagla`) ve form alanlari temizlenir (`TemizleText`). Ekran kapaninca cagiran ana menuye donulur. **Butonlar & kisayollar:** - **Yeni** (BtnYeni) — `TemizleText()` : form alanlarini bosaltir, yeni bos `MailAyar` olusturur (yeni kayit girisine hazirlar). - **Kaydet** (BtnKaydet) — `Kaydet()` : zorunlu alan kontrolu (`TextleriKontrolEt` : Host/Port/MailAdres/Pass dolu mu), `AktarRowa()` ile alanlari modele aktarir (sifre `TxtPass.Text.Sifrele(Ortak.GetKey())` ile sifrelenir), `_mng.MailSettingsKaydet` cagirir. - **Sil** (BtnSil) — once "Kaydi Silmek Istiyormusunuz" onayi (`MesajSor`), onaylanirsa `Sil()` -> `await _mng.MailSettingsSil(_fis.Id)` . - **Kapat** (BtnKapat) — `this.Close()` . - **Test Mail** (BtnTestMail) — `new FrmMailGonderOrnek().ShowDialog()` : test maili gonderme formunu acar. - Grid satirina cift tik / Enter (`MyView1_MyEventDoubleClickEnter`) — secili `MailAyar` satirini klonlar, `AktarTextlere()` ile form alanlarina doldurur (sifre `SifreCoz` ile cozulup gosterilir). Grid tum kolonlari ReadOnly'dir; duzenleme yalnızca ust paneldeki textbox'lardan yapilir. - Genel kisayol: `.Designer.cs` 'de buton ShortcutKeys / form-level Enter=Kaydet, Esc=Kapat tanimi YOK; butonlar yalnızca Click ile calisir. MyTextEdit'lerde `EnterMoveNextControl=true` (Enter ile sonraki alana gecis). **Cagirdigi katmanlar:** - Manager/Service: `MailManager.GetMailSettings()` — `_dbPro.MailSettings.SelectListWhere("")` ile tum ayarlari listeler (grid kaynagi). - Manager/Service: `MailManager.MailSettingsKaydet(MailAyar)` — `_dbPro.MailSettings.InsertOrUpdate(fis)` (Id varsa UPDATE, yoksa INSERT; `MailAyar.GetInsertCode` / `GetUpdateCode` SQL'leri). - Manager/Service: `MailManager.MailSettingsSil(Guid?id)` — `_dbPro.MailSettings.Delete(c => c.Id == id)` , async. - Service/DAL: `MailAyarService` : `BaseService<MailAyar>` / `MailAyarDal` : `BaseDal<MailAyar>` — `MailSettings` tablosu icin temel CRUD (Manager dogrudan `_dbPro.MailSettings` DAL'i üzerinden cagirir). - Yardimci: `string.Sifrele(key)` / `string.SifreCoz(key)` (sifre sifrele/coz), `Ortak.GetKey()` (master anahtar), `MesajHata` / `MesajSor` (toast/diyalog uzantilari). - SQL/Prosedur: Stored procedure kullanmaz; `MailAyar.GetInsertCode()` / `GetUpdateCode()` parametrelili SQL stringleri üzerinden Dapper ile calisir. Tablo: `MailSettings` . - API: -

Istasyon sirasiyla iliskisi: - **Notlar:** Form veri kaynagi `Ortak.DbPro` (Uretim/Pro DB), `MailManager` ctor'unda `Ortak.DbMikro` da gecilir ama bu ekranda Mikro DB kullanilmaz. Sifre gridde maskelenmez ama `TxtPass'ta PasswordChar='*'`. `MailKodu` (orn. "Şirket Maili") bilgilendirme amaclı; gönderimde kullanılan tek/ilk ayar genelde `FirstOrDefault()` ile secilir (diger formlarda).

Mail Gonder Ornek / Test Mail (`FrmMailGonderOrnek.cs` / `FrmMailGonderOrnek.Designer.cs`)

Ne ise yarar: Kayitli SMTP ayarinin gercekten mail gonderebildigini dogrulamak icin manuel/test e-postasi gonderme ekranidir. Alici mail adresi, konu ve mesaj govdesi girilip tek tikla deneme maili atilir (ek dosya yok).

Once ne olmalı (onkosul): En az bir gecerli `MailSettings` kaydi olmalı (Host/Port/MailAdres/Pass/EnableSsl dogru). Form `FrmMailSettings` icindeki `Test Mail` butonundan `ShowDialog` ile acilir. Yuklenince `Bagla()` ayarlari `_list` 'e cekeir. **Sonra ne olur:** Gonder -> listenin ilk ayari (`_list.FirstOrDefault()`) ile

`MailManager.GonderTek(...)` cagirilir; basariliysa "Mail Gonderildi" bilgi mesaji, hata varsa hata mesaji gosterilir. Veritabaninda degisiklik YAPMAZ (sadece SMTP gonderimi). Kapat ile cagiran ayar ekranina donulur.

Butonlar & kisayollar: - `Gönder` (BtnGonder) — `BtnGonder_Click` : `_mdl = _list.FirstOrDefault()` ; ayar yoksa "mail Ayarlari bulunamadı" hatasi; varsa `MailManager.GonderTek(_mdl, TxtMail.Text, TxtKonu.Text, TxtBody.Text, null, Ortak.GetKey())` (dosya=null, sadece duz mail). Sonuc toast ile bildirilir. - `Kapat` (BtnKapat) — `this.Close()` . - Kisayol: `.Designer.cs` 'de ShortcutKeys/F-tusu tanimi YOK. Alanlarda `EnterMoveNextControl=true` . **Cagirdigi katmanlar:** - Manager/Service: `MailManager.GetMailSettings()` —

yuklemede ayar listesini cekeir (`_dbPro.MailSettings.SelectListWhere("")`). - Manager/Service:

`MailManager.GonderTek(MailAyar ayar, string alicimail, string konu, string body, byte[] dosya, string masterKey, string dosyaUzanti="pdf")` (static) — `SmtpClient` ile (Host/Port/EnableSsl,

`NetworkCredential(MailAdres, Pass.SifreCoz(masterKey))`) tek alici maili gonderir; `dosya` null oldugundan eksiz gonderim. `SuccessResult` / `ErrorResult` doner. - Yardimci: `Ortak.GetKey()` , `MesajBilgi` / `MesajHata` .

- SQL/Prosedur: Yalnizca `MailSettings` SELECT (gonderimde DB yazimi yok). Prosedur yok. - API: - **Istasyon**

sirasiyla iliskisi: - **Notlar:** Her zaman ilk ayar (`FirstOrDefault`) kullanilir; birden fazla ayar varsa hangisinin secilecegi UI'dan secilemez. Mail body `IsBodyHtml=true` olarak gonderilir.

Sipariş Mail Gonder (`FrmFisMailGonder.cs` / `FrmFisMailGonder.Designer.cs`)

Ne ise yarar: Bir siparis fisini (SiparisKayitModel) DevExpress dizayni ("SiparisMail" REPX) ile PDF veya XLSX'e ceviriip, secilen mail adresine ek olarak gonderen ekrandır. Ekran govdesi bir log/RichTextBox'tir; her adim (dosya olusturma, gonderim, sonuc) buraya yazilir. Otomatik gonderim modunu da destekler. **Once ne olmalı**

(onkosul): Gonderilecek siparis once hazirlanmis/secilmis olmalı. Form siparis ekranından (`FrmSiparisEd.MailGonder` -> `BtnMailGonder_Click`) doldurulup acilir: cagiran taraf `f.Mdl = _mdl`

(siparis+hareket+detay modeli), `f.MailAdres = TxtEmail.Text` , `f.YazdirmaAdi = "SiparisMail"` , `f.OtoGonder` set eder, sonra `ShowDialog()` . Gecerli bir `MailSettings` kaydi (`_setting = ilk ayar`) ve "SiparisMail" dizayni mevcut olmalı. **Sonra ne olur:** Gonderim oncesi mail adresi `MailAddress` ile dogrulanir (`Mailkontrol`). Dosya, `Mdl` 'den uretilen 3 tablolulu `DataSet` ("Siparis", "Hareketler", "Detaylar") üzerinden

`YazdirPdf / YazdirXlsx(YazdirmaAdi)` ile byte[]'a render edilir; `MailManager.GonderTek` ile ekli olarak gonderilir. Basari/hata her durumda log'a yazilir; ayrıca toast gosterilir. `OtoGonder=true` ise yuklenir yuklenmez

PDF gonderilir ve basariliysa `TmrKapat` (2 sn) ile form otomatik kapanir. Bu islem veritabanında kayit DEGISTIRMEZ (sadece SMTP). Kapaninca cagiran siparis ekranina donulur. **Butonlar & kisayollar:** - `Gönder Pdf` (Varsayılan) (BtnGonderPdf) — `BtnGonderPdf_Click` -> `GonderPdf()` : mail kontrol ->

`DatasetOlustur()` -> `ds.YazdirPdf(YazdirmaAdi)` -> `MailManager.GonderTek(_setting, mailAdres, _setting.Konu, _setting.Body, dosya, Ortak.GetKey(), "PDF")` . - `Gönder Xlsx` (BtnGonderXlsx) —

`BtnGonderXlsx_Click` -> `GonderXlsx()` : ayni akis ama `ds.YazdirXlsx(YazdirmaAdi)` + ek uzantisi "xlsx". -

`Kapat` (BtnKapat) — `this.Close()` . - `TmrKapat` (Timer, Interval=2000ms) — `OtoGonder` basarili oldugunda

Start edilir; Tick'te `Stop()` + `this.Close()` (formu otomatik kapatir). - Kisayol: `.Designer.cs` 'de
ShortcutKeys/F-tusu tanimi YOK; buton metinlerinde "(Varsayilan)" PDF'in onerilen secenek oldugunu belirtir.
Cagirdigi katmanlar: - Manager/Service: `MailManager.GetMailSettings()` — yuklemede ayarlari ceker,
`_setting = rs.Data.FirstOrDefault()` . - Manager/Service: `MailManager.GonderTek(...)` (static) — siparis
dosyasini ek olarak SMTP ile gonderir (PDF/XLSX uzantisiyla). - Yazdirma/Rapor: `DataSet.YazdirPdf(string)`
ve `DataSet.YazdirXlsx(string)` (`My.Kontrol.Yazdirma` namespace, harici DevExpress yazdirma
kutuphanesi) — verilen dizayn adina ("SiparisMail") gore DataSet'i PDF/XLSX byte[]'a render eder. -
Model/Yardimci: `SiparisKayitModel` (Siparis + List + List); `Mdl.X.ToDataTable(...)` ile DataSet kurulur
(`DatasetOlustur`). `Mailkontrol()` / `GetMailAdres()` mail dogrulama; `Logyaz()` RichTextBox log. -
SQL/Prosedur: Yalnizca `MailSettings` SELECT (`GetMailSettings`). Gonderimde DB yazimi/prosedur YOK. -
API: - **Istasyon sirasiyla iliskisi:** - **Notlar:** Cagiran taraf `FrmSiparisEd` (SiparisModule). `_setting` her zaman
ilk ayardir. Hata mesajlarında `Mdl.Siparis.CariKodu` log'a eklenir. PDF varsayilan secenektir. Bu form siparis
fisi e-posta gonderimine ozeldir; genel mail gonderimi degildir.

Modul: SmsModule

UretimV4 (CepPatronERP.exe) masaustu uygulamasinda NetGSM SMS gonderim raporlarini tarih araliginda listeleyen tek formdan olusan kucuk bir modul. Modulun amaci, NetGSM uzerinden gonderilen SMS'lerin teslim/durum raporlarini (NetGSM rapor servisinden) cekip DevExpress grid'de gosterme. SMS gonderme arayuzu (telefon + mesaj alanlari, "Gonder" butonu) form uzerinde mevcut fakat tasarimda gizlenmis (`Visible=false`) ve ilgili click handler'i (`myButton2_Click`) tamamen yorum satiri oldugu icin SMS gonderme islevi su an pasiftir; form pratikte sadece RAPOR (sorgulama/listeleme) amaclı calisir.

Modul, NetGSM kimlik bilgilerini ve servis URL'lerini `SmsAyar` tablosundan okur. Bu ayarlar ayri bir form olan `FrmSmsAyarlari` (AyarVeGenelModul klasoru) uzerinden girilir; bu yuzden onkosul olarak SMS ayarlarinin daha once kaydedilmis olmasi gerekir. SMS gonderim/rapor mantigi `My.Sms.dll` adli harici referansta (`My.Sms / My.Sms.NetGsm` namespace'leri; `NetGsmSmsManager` , `SmsSettings` siniflari) bulunur ve kaynak kodu projede yer almaz; davranislar form kullanimindan cikarilmistir.

Onemli not: Bu modul UretimV3_FEZA uretim akisinin (operasyon/istasyon/recete) hicbir parçasi degildir. Uretim miktar/akis motoru (Uretim_MiktarGuncelle, Uretim_PlanlananGuncelle, Uretim_SonrakilstasyonaGonder) ile hicbir iliskisi yoktur. Sadece `SmsAyar` tablosunu okur ve dis NetGSM HTTP API'sine baglanir; uretim tablolarina dokunmaz.

SMS Raporu (`MyUI\SmsModule\FrmSmsRapor.cs` + `FrmSmsRapor.Designer.cs`)

Ne ise yarar: Iki tarih (baslangic/bitis) arasinda NetGSM uzerinden gonderilmis SMS'lerin durum/teslim raporunu NetGSM rapor servisinden cekip grid'de listeler. (Tasarimda gizli alanlar nedeniyle tek aktif islevi raporlamadir; manuel SMS gonderme ekrandadir ama pasiftir.) **Once ne olmalı (onkosul):** - SMS ayarlari onceden girilmis olmalı: `SmsAyar` tablosunda en az bir kayit bulunmalı (KullaniciKodu, Sifre, Baslik, GunderimUrl, RaporUrl). Bu kayit `FrmSmsAyarlari` formundan girilir. Kayit yoksa form acilisinda "Sms Ayarlari Yapilmamis." bilgi mesajı cikar ve `settings` null kalir (rapor cekilemez). - Form, ana ekrandan (`FrmAna`) ribbon ogesi `BarBtnSmsRapor` tiklanarak MDI alt penceresi olarak acilir (`f.MdiParent = this; f.Show();`). **Sonra ne olur:** - Hicbir veritabani tablosu YAZILMAZ/degismez; islem salt-okunurdur. Sadece dis NetGSM rapor HTTP servisine (RaporUrl) istek atilir ve donen rapor satirlari `myGrid1` grid'ine baglanir (`myGrid1.DataSource = rs.Data`). - Hata olursa `MesajHata(rs.Message)` ile kullaniciya bildirilir; basari/hata disinda baska ekrana yonlendirme yoktur (form acik kalir). - Uretim akisi tabloları (UretimIstasyonHareket, UretimIstasyon, ReceteAna vb.) etkilenmez. **Butonlar & kisayollar:** - `BtnAra` (taban form butonu, `MyFrmListe`) — Click `BtnAra_Click` - `> GetRapor()` cagirir: `myDateEdit1` (BasTarih) ve `myDateEdit2` (BitTarih) degerleriyle NetGSM rapor servisinden tarih araliginda rapor ceker, grid'e basar. - `myDateEdit1` (etiket `BasTarih`) — rapor baslangic tarihi (maske `dd.MM.yyyy` , `EnterMoveNextControl=true` -> Enter ile sonraki kontrole gecer). - `myDateEdit2` (etiket `BitTarih`) — rapor bitis tarihi (maske `dd.MM.yyyy`). - `myButton2` (Text "Gonder", `Visible=false`) — Click `myButton2_Click` PASIF: handler govdesi tamamen yorum satiri (`smsManager.SmsSend(...)` cagrisi devre disi). Manuel SMS gonderme islevi aktif degil. - `myTextEdit1` (etiket `Tel` , `Visible=false` , `MaxLength=75`) — manuel SMS icin telefon alanı (gizli/pasif). - `richTextBox2` (Text "Selam", `Visible=false`) — manuel SMS mesaj govdesi (gizli/pasif). - Taban form butonlari `BtnYazdir` , `BtnDizayn` , `BtnTemizle` , `BtnKapat` (`MyFrmListe` 'den miras) — bu formda OZEL Click handler'i baglanmamistir; yalnizca `BtnAra.Click` Designer'da bu forma ozel olarak wire edilmistir. `BtnKapat` taban formun standart pencere kapatma davranisini saglar; `BtnYazdir` / `BtnDizayn` grid yazdirma/kolon dizayni icin taban davranisina baglidir. - Kisayollar: Designer'da bu forma ozel ShortcutKeys/F-tusu tanimi YOK. Tarih kontrollerinde `EnterMoveNextControl=true` , grid view'da `EnterMoveNextColumn=true` (Enter -> sonraki kolon) ayarlidir.

Enter=Kaydet / Esc=Kapat gibi davranislar varsa taban sinif `MyFrmListe` 'den (harici DLL) gelir; kaynakta dogrulanamadi. **Cagirdigi katmanlar:** - Service: `ISmsAyarService.SelectFirstWhere("")` — `Ortak.DbPro.SmsAyar` üzerinden cagilir; `SmsAyar` tablosundan ilk kaydi getirir (KullaniciKodu, Sifre, Baslik, GunderimUrl, RaporUrl). `BaglaSmsAyar()` icinde bu degerlerle `SmsSettings` nesnesi olusturulur. (`SmsAyarService : BaseService<SmsAyar>`, DAL: `SmsAyarDal : BaseDal<SmsAyar>`.) - Manager (harici DLL `My.Sms.dll`): `NetGsmSmsManager.GetRaporByTarih(DateTime bas, DateTime son)` — NetGSM rapor servisine (RaporUrl) HTTP istegi atip tarih araligindaki SMS durum raporunu `IDataResult` olarak dondurur (`rs.Data` grid'e baglanir). Kaynak kodu projede yok; imza ve davranis form kullanimindan cikarilmistir. - Manager (harici DLL): `NetGsmSmsManager.SmsSend(string tel, string mesaj)` — manuel SMS gonderir; ANCAK cagrisi formda yorum satirinda oldugu icin kullanilmiyor. - Manager kurucusu (harici DLL): `new NetGsmSmsManager(SmsSettings settings), new SmsSettings(KullaniciKodu, Sifre, Baslik, GunderimUrl, RaporUrl)`. - SQL/Prosedur: Stored procedure CAGRILMAZ. Yalnizca `SmsAyar` tablosu okunur (SELECT, BaseDal/Dapper üzerinden). Tablo semasi: `My.DataBaseSettings\DatabaseCreate\UretimCreateModule\SmsAyarCreates.cs` (kolonlar: Id PK, KullaniciKodu varchar(50), Sifre varchar(250), Baslik varchar(50), GunderimUrl varchar(250), RaporUrl varchar(250)). - API: Dis NetGSM REST servisi — varsayilan rapor URL'si `https://api.netgsm.com.tr/sms/report`, gonderim URL'si `https://api.netgsm.com.tr/sms/send/xml` (`SmsAyar` entity varsayilanlari). Bunlar UretimV4 projesinin kendi API'si degil, ucuncu taraf NetGSM HTTP API'sidir. **Istasyon sirasiyla iliskisi:** - (Yok. Modul uretim operasyon/istasyon akisindan tamamen bagimsizdir.) **Notlar:** - Form acilisi (`FrmSmsRapor_Load`): `BaglaSmsAyar()` -> `new NetGsmSmsManager(settings)` -> `GetRapor()` -> `myGrid1.GridYerlesimYukle()`. Yani form aciliminda otomatik olarak (varsayilan tarihlerle) bir rapor cekme denemesi yapilir. - `BaglaSmsAyar()` icinde ayar kaydi yoksa "Sms Ayarlari Yapilmamis." (kod icinde yazim hatasi mevcut) mesajı gosterilir; ardindan `smsManager = new NetGsmSmsManager(settings)` settings null iken olusturulur ve `GetRapor()` cagrisi NetGSM'e bos kimlikle gidip hata dondurebilir. - Designer'da tarih kontrollerinin sabit varsayilan degeri `"26.05.2023"` olarak gomulu (dinamik DateTime.Now ile doldurulmaz); kullanıcı Ara'dan once tarihleri elle ayarlamalidir. - `myGrid1.MyGridKayitAdi = "GridAdi_6C9DAB4298944D2BACB8A0C3323FED47"` ile grid kolon yerlesimi kullaniciya ozel saklanir/yuklenir (`GridYerlesimYukle()`). - Iliskili ayar formu: `MyUI\AyarVeGenelModul\FrmSmsAyarlari.cs` (`MyFrmKayit` turevi) — `SmsAyar` CRUD (BtnKaydet/BtnSil/BtnDuzenle/BtnYeni); `ISmsAyarService.InsertOrUpdate/Delete/SelectListWhere` cagilir. Bu form `FrmAna.BarBtnSmsAyarlari_ItemClick` -> `f.ShowDialog()` ile acilir. Bu modulun (SmsModule klasoru) parçasi degildir ama onkosulu olusturur.

Modul: Aciklamalar

Aciklamalar modulu, uretim genelinde kullanılan **kodlu serbest aciklama/etiket listelerini** (kod + deger çiftleri) tek bir merkezi tabloda (**AciklamaKod** , Uretim DB / UretimV3_FEZA) yönetir. Klasorde tek bir form vardır: **FrmAciklamaKodlar** . Bu form generic bir CRUD ekranidir; hangi amac için acildigini disaridan set edilen **AciklamaModulTuru** enum'u belirler (**ReceteAciklama** , **OperasyonAciklama**). Form acilince yalnızca o modul turune ait kayitlari listeler (**where Modul='<tur>' Order By Sira,Kodu**), kod/deger ekle-duzenle-sil islemleri yapar ve **Sira** alanıyla sıralama tutar. Bu kodlar dogrudan uretim akisini veya istasyon hareketlerini DEGISTIRMEZ; tanım/sozluk niteligindedir. Uretildikleri yerlerde (örn. recete bazinda aciklama secimi yapan **FrmReceteAciklamalar**) acilir liste kaynagi olarak okunurlar. Form **My.Kontrol.Formlar.MyFrmKayit** base sinifindan turer (ust GroupControl form bolgesi + alt grid + base alt buton seridi Kaydet/Sil/Yeni/Duzenle/Yazdir/Kapat ve gezinme butonlari). Grid'de çift tiklama/Enter ortak **MyEventDoubleClickEnter** olayini tetikler: form secim modunda (**SecimIcinAcildi**) acildiysa satiri secip kapatir, aksi halde Duzenle butonunu calistirir.

Aciklama Kodlar (**FrmAciklamaKodlar.cs** / **FrmAciklamaKodlar.Designer.cs**)

Ne ise yarar: **AciklamaKod** tablosundaki kodlu aciklama tanimlarinin tam CRUD ekranidir (Kodu + Deger/ **Deger1** + **Sira**). Tek form, **AciklamaModulTuru** enum degerine gore farkli amaclarla (su an menuden yalnızca **ReceteAciklama**; enum'da ayrıca **OperasyonAciklama** tanimli) acilir ve sadece o modul turune ait kayitlari yönetir. Pencere basligi ve ust baslik etiketi (**lblBaslik**) enum'un string karsiligi ile doldurulur. **Once ne olmalı (onkosul):** Form **AciklamaModulTuru** set edilerek acilmali. Ana menude (**FrmAna**) "Recete Aciklamalar" (**BarBtnReceteAciklama** , Caption="Recete Aciklamalar", Id=137) tiklanir; handler **BarBtnReceteAciklama_ItemClick** formu olusturup **AciklamaModulTuru = AciklamaModulTuru.ReceteAciklama** atar ve **ShowDialog()** ile acar. **AciklamaKod** tablosu DB'de mevcut olmalı (sema **AciklamaKodCreates.AciklamaKodCreate** ile uretilir). **Sonra ne olur:** Kaydet -> secili modul turu **Modul** alanina string olarak yazilarak **IAciklamaKodService.InsertOrUpdate(_mdl)** cagrilir; basarili ise **KayitEdildi=true** set edilip liste **Bagla()** ile yeniden cekilir (form acik kalir). Sil -> **IAciklamaKodService.Delete(_mdl)** ; basarili ise liste yenilenir. Degisen tablo: **AciklamaKod** . Bu kayitlar daha sonra tuketici ekranlarda (örn. **MyUI\ReceteModul\FrmReceteAciklamalar.cs**) icinde **kodService.SelectListWhere(" where Modul='ReceteAciklama' Order By Sira,Kodu")**) acilir aciklama listesi kaynagi olarak okunur. Hicbir stored procedure veya uretim/istasyon tablosu tetiklenmez. **Butonlar & kisayollar:** - **Kaydet** (**BtnKaydet** , base) — **BtnKaydet_Click** -> **Kaydet()** : **TextLeriKontrolEt()** (Kodu zorunlu) -> **AktarModele()** (Id bos ise yeni Guid, **Modul** =enum.ToString()) -> **_srv.InsertOrUpdate(_mdl)** -> **Bagla()** . (ardindan tekrar **Bagla()**). - **Sil** (**BtnSil** , base) — **BtnSil_Click** : "Kaydı silmek istiyormusunuz.." onayi sorar; secili kayit yoksa (**_mdl.Id** bos) bilgi mesaji verir; **Sil()** -> **_srv.Delete(_mdl)** -> **Bagla()** . - **Yeni** (**BtnYeni** , base) — **BtnYeni_Click** -> **TemizleText()** : yeni bos **AciklamaKod** , **IdGuid=Guid.Empty** , Kodu/Adi kutularini bosaltir. - **Düzenle** (**BtnDuzenle** , base) — **BtnDegistir_Click** : secili grid satirini (**myView1.MyGetCurrentItem<AciklamaKod>()**) klonlayip (**_mdl = itm.Clone()**) forma yukler (**AktarTextlere**); klonlama acilis sirasinda **AcilisBittimi=false/true** ile cevrelenir. - Grid çift tiklama / Enter (**myView1.MyEventDoubleClickEnter** -> **MyView1.MyEventDoubleClickEnter**) — **SecimIcinAcildi** ise satiri secip (**SecilenKod** , **SecilenRow** , **SecilenId** , **Secildi=true**) formu kapatir; degilse **BtnDuzenle.PerformClick()** . - **Kodu** (**TxtKodu** , **MyTextEdit** , **MaxLength 75**) — kod girisi; bos olamaz (**TextLeriKontrolEt**). - **Deger** (**TxtAdi** , **MyTextEdit** , **MaxLength 75**) — **Deger1** alanina karsilik gelir. - Grid'de **Sira** sutunu duzenlenebilir (**SutunEditAc("Sira")**)

+ `SutunReadOnlyKapat("Sira")`); listeleme `Sira, Kodu` sirasina gore yapilir. - `Yazdir` (`BtnYazdir` , base) — base form butonu, bu formda ozel olay baglanmamis (varsayilan base davranis). - `Kapat` (`BtnKapat` , base) — formu kapatir (base davranis). - Gezinme butonlari `BtnIlk` / `BtnOnceki` / `BtnSonraki` / `BtnSon` (base) — base kayit gezinme seridi; bu formda ozel olay baglanmamis. **Cagirdigi katmanlar:** - Manager/Service: `IAciklamaKodService` (`Ortak.DbPro.AciklamaKod` üzerinden) — `SelectListWhere($"where Modul='{AciklamaModulTuru}' Order By Sira,Kodu ")` , `InsertOrUpdate(AciklamaKod)` , `Delete(AciklamaKod)` . Servis kendi metodu eklemez; `BaseService<AciklamaKod>` ve `IBaseService<AciklamaKod>` temel CRUD'unu kullanir. - DataAccess: `AciklamaKodDal` : `BaseDal<AciklamaKod>` — Dapper tabanlı generic DAL (`AciklamaKod` tablosu, `[Table("AciklamaKod")]` , PK `Id` Guid). - Entity: `AciklamaKod` (alanlar: `Id` , `Modul` , `Kodu` , `Deger1` , `Deger2` , `Deger3` , `Sira` ; `Clone()` metodu). Grid'de `Id` , `Modul` , `Deger2` , `Deger3` sutunlari gizlenir (`SutunGizle`). - Enum: `AciklamaModulTuru` { `ReceteAciklama` , `OperasyonAciklama` } (My.Entities.UretimAciklamalar). - SQL/Prosedur: Stored procedure cagrilmaz. Sema `AciklamaKodCreates.AciklamaKodCreate` (My.DataBaseSettings) ile olusturulur. Tek SQL, generic DAL'in urettigi SELECT/INSERT/UPDATE/DELETE. - API: - (UretimV4 masaustu; API kullanmaz). **Istasyon sirasiyla iliskisi:** - (Dogrudan iliskisi yoktur. Kayitlar miktar/akis motorunu, operasyon Sira'sini veya UretimIstasyon olusturmayi etkilemez; yalnızca tanim/sozluk verisidir ve tuketici ekranlarda acilir liste olarak okunur.) **Notlar:** Namespace `MyUI.Aciklamalar` . Grid duzeni `AciklamalariKodListesi1` adiyla saklanir (`MyGridKayitAdi`), `myGrid1.GridYerlesimYukle()` ile yuklenir. Event'ler Designer yerine kod ile baglanir (`EventlerBagla()`). `Kaydet()` icinde zaten `Bagla()` cagrilirken `BtnKaydet_Click` bir kez daha `Bagla()` cagirir (cift yenileme). Designer'da base butonlar icin ShortcutKeys/F-tuslari atanmamistir; Enter=Kaydet / Esc=Kapat gibi kisayollar varsa `MyFrmKayit` base sinifindan gelir (base kaynak bu repoda yok, harici My.Kontrol DLL'inde). `AciklamaModulTuru` enum'unda `OperasyonAciklama` degeri tanimli olsa da `FrmAna` menusunda bu turu acan bir buton bulunamadi (su an yalnızca `ReceteAciklama` baglanmis). Bu form ile `MyUI\IstasyonModul\FrmIstasyonAciklamalari` (farkli tablo/enum: `IstasyonAciklama` + `IstasyonAciklamaModulTuru`) karistirmamak gerekir; isim benzer, veri kaynagi farklidir.

Modul: Raporlar

UretimV4 (CepPatronERP.exe) masaustu uygulamasinin "Raporlar" klasoru, uretim/recete/istasyon verilerini salt-okunur listeleyen 5 form icerir. Uc alt klasore ayrilir: **IstasyonRaporlari** (operator/istasyon bazli uretim ozeti), **ReceteRaporlari** (recetelerde kullanan stoklarin listesi + recete genel raporu) ve **UretimRaporlari** (uretimde tuketilen stoklarin toplu/detayli raporu, Mikro stok kategorileriyle birlikte). Tum formlar **My.Kontrol.Formlar.MyFrmListe** taban sinifindan turer; bu sinif ortak liste duzenini (sol filtre tab paneli **tabAra1**, orta **myGrid1** + DevExpress **myView1**, alt buton paneli) ve standart butonlari (**BtnAra**, **BtnTemizle**, **BtnYazdir**, **BtnDizayn**, **BtnKapat**) saglar. Formlarin tamamı acilista otomatik **BtnAra.PerformClick()** cagirir; UretimRaporlari formlari ek olarak acilista Mikro stok/kategori temp tablolarini ve uretim stok-hareket detay merge tablosunu gunceller. Hicbiri veri yazmaz/silmez (yalnizca rapor okur); bu yuzden uretim akis prosedurleri (Uretim_MiktarGuncelle / Uretim_PlanlananGuncelle / Uretim_SonrakilstasyonaGonder) bu modulde cagirlmaz. Raporlar uretim akisinin SONUCUNU goruntuleyen ekranlardir.

Istasyon Raporu (**MyUI/Raporlar/IstasyonRaporlari/FrmIstasyonRaporu.cs**)

Ne ise yarar: Belirli tarih/saat araliginda, istasyon ve personel filtresiyle gerceklesen uretim hareketlerini (mamul giris, fire mamul giris, uretim bitis, uretim iptal) iki gridde gosterir: ust grid gun+stok+istasyon+personel kirilimli hareketler (**myGrid1**), alt grid sadece stok bazli toplamlar (**myGrid2**).

Once ne olmalı (onkosul): Saha akisinda (TabletV2) istasyon takip hareketleri girilmis ve **IstasyonTakipHareketDetay** tablosuna **Turu** = MamulGiris/FireMamulGiris/UretimBitis/UretimIptal kayitlari islenmis olmalı. Personel/Istasyon kartlari tanimli olmalı (combo'lari doldurmak icin). **Sonra ne olur:** Salt-okunur rapor; tabloya/proseure yazma yok. "Ara" sonrasi gridler doldurulur. Satira cift tiklaninca (yalnizca form **SecimIcinAcildi** ile bir secim diyalogu olarak acildiyse) secilen **ReceteKodu / Id** geri dondurulup form kapanir; normal acilista cift tik bir sey yapmaz. **Butonlar & kisayollar:** - **Ara** (**BtnAra**) — filtreleri uygulayip **Bagla()** ile her iki gridi yeniden doldurur (Designer'da **BtnAra.Click -> BtnAra_Click**). Acilista da otomatik tetiklenir. - **Temizle** (**BtnTemizle**) — taban formdan gelen filtre temizleme butonu. - **Yazdir** (**BtnYazdir**) — taban formdan gelen grid yazdirma. - **Dizayn** (**BtnDizayn**) — grid kolon/yerlesim dizayni. - **Kapat** (**BtnKapat**) — formu kapatir. - Tarih1/Tarih2 (**TxtTarihi1 / TxtTarihi2**), Saat1/Saat2 (**TxtSaat1** vars. "00:00", **TxtSaat2** vars. "23:59"), Istasyon combo (**CmbIstasyon**), Personel combo (**CmbPersonel**) — filtre alanlari (kisayol tanimli degil; Enter ile sonraki kontrole gecir, EnterMoveNextControl=true). **Cagirdigi katmanlar:** - Manager/Service: **IstasyonRaporManager.GetIstasyonRapor(string andSorgu)** — hem hareket hem toplam SQL'ini calistirip **IstasyonRaporModel** (Hareketler + Toplamlar) dondurur. - Service: **IIstasyonKartiService.SelectListWhere("")** — **CmbIstasyon** combo'sunu istasyon kodlariyla doldurur. - Service: **IPersonelService.SelectListWhere("")** — **CmbPersonel** combo'sunu personel kodlariyla doldurur. - SQL: **IstasyonRaporHareketModel.GetSelectSqlCode()** — **IstasyonTakipHareketDetay HR** + **IstasyonTakipHareket IST** + **Personel Prs** JOIN; **Turu IN** (MamulGiris,FireMamulGiris,UretimBitis,UretimIptal) ; gun/stok/istasyon/personel GROUP BY; UretimMiktar/FireMiktar/IptalMiktar SUM. - SQL: **IstasyonRaporToplamModel.GetSelectSqlCode()** — ayni filtre, sadece stok bazinda SUM (alt toplam gridi). - API: - **Istasyon sirasiyla iliskisi:** Dogrudan istasyon bazli; **CmbIstasyon** ile tek istasyona daraltilabilir. Filtre SQL'i **Ist.IstasyonKodu** ve **HR.KayitEden** (personel) uzerinden calisir. Operasyon-Sira mantigi yok; hareket kaydinin baglandigi istasyon koduna gore raporlar. **Notlar:** Filtre stringleri parametresiz string-interpolation ile kuruluyor (SQL injection riski). Saat alanlari tarih ile

birlestirilerek `CAST(... AS datetime)` araligi olusturur. `myGrid1` kayit adi "IstasyonRaporuHareketList1", `myGrid2` "IstasyonRaporuToplamList1".

Recete Kullanilan Stok Listesi (`MyUI/Raporlar/ReceteRaporlari/FrmReceteKullanilanStokList.cs`)

Ne ise yarar: Tum recete detay satirlarini (her recetede kullanilan stoklar) tek duz listede gosterir; recete kodu/adi veya varsayilan stok kodu/adi ile aranabilir. **Once ne olmalı (onkosul):** Receteler ve recete detaylari (`ReceteAna` / `ReceteDetay`) tanimlanmis olmalı. **Sonra ne olur:** Salt-okunur liste. Satira cift tiklaninca ilgili recete `FrmReceteED` (Recete Edit/Detay formu) ile `IdGuid = itm.RcAId` olarak acilir; orada kayit edilirse (`f.KayitEdildi`) liste tekrar yenilenir (`BtnAra.PerformClick()`) ve odak ayni satira geri konur. Form secim modunda acildiysa (`SecimIcinAcildi`) secilen `ReceteKodu` dondurulup kapanir. **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — `Bagla()` ile listeyi `TxtAra` metnine gore filtreler (`EventlerBagla` icinde `BtnAra.Click += BtnAra_Click`). - `Temizle` / `Yazdir` / `Dizayn` / `Kapat` — taban form (`MyFrmListe`) standart butonlari. - Cift tik / Enter (grid) — `MyView1_MyEventDoubleClickEnter` : receteyi `FrmReceteED` ile acar veya secim modunda secip kapatir. - `Ara` filtre kutusu (`TxtAra`) — recete kodu/adi/varsayilan stok kodu/adi icinde LIKE arama. **Agirdigi katmanlar:** - Manager/Service: `ReceteManager.GetReceteKullanilanStokList(string wheresql)` — `ReceteDetay rd LEFT JOIN ReceteAna ra` sorgusuyla `ReceteKullanilanStokModel` listesi dondurur. - SQL: yukaridaki inline SELECT (`ReceteDetay + ReceteAna`); filtre `ra.ReceteKodu / ra.ReceteAdi / VarsayilanStokKodu / VarsayilanStokAdi` uzerinden LIKE. - Form: `FrmReceteED { IdGuid = RcAId }` — cift tikta recete duzenleme formu acar. - API: - **Istasyon sirasiyla iliskisi:** - (recete tanim raporu; uretim/istasyon akisi yok) **Notlar:** Filtre verildiginde sorgu basina `where 1 = 1` eklenir. `myGrid1` kayit adi "ReceteKullanilanStokList1".

Recete Raporu (`MyUI/Raporlar/ReceteRaporlari/FrmReceteRaporu.cs`)

Ne ise yarar: Recetelerin genel raporu: her recete icin once recete ust satiri, ardindan o recetenin detay (kullanilan stok) satirlari fis-sira/recete-sira duzeninde listelenir; fireli miktar dahil hesaplanir. Entegre (Mikro) stok kodu/adi ile aranabilir. **Once ne olmalı (onkosul):** `ReceteAna` (EntegreStokKodu/Adi dahil) ve `ReceteDetay` kayitlari olmalı. Detay satirlarinin cins bilgisi icin `TempMikroStok` temp tablosu dolu olmalı (LEFT JOIN; bos olursa cins bos gelir). **Sonra ne olur:** Salt-okunur. Cift tik davranisi Kullanilan Stok Listesi ile ayni (recete `FrmReceteED` ile acilir veya secim modunda secilir). `RcAId` ve `RcDId` kolonlari gridde gizlenir. **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — `Bagla()` ile genel rapor sorgusunu calistirir (Designer'da event yok; `EventlerBagla` icinde `BtnAra.Click += BtnAra_Click`). - `Temizle` / `Yazdir` / `Dizayn` / `Kapat` — taban form standart butonlari. - Cift tik / Enter (grid) — `MyView1_MyEventDoubleClickEnter` : `FrmReceteED` acar veya secimi dondurur. - `Ara` filtre kutusu (`TxtAra`) — `EntegreStokKodu / EntegreStokAdi` icinde LIKE arama. **Agirdigi katmanlar:** - Manager/Service: `ReceteManager.GetReceteGenelRaporuList(string wheresql)` — `#TempReceteRaporu` gecici tablosu + `ReceteRaporCursor` cursor'u ile her receteyi gezip ust+detay satirlarini uretir; `FireliMiktar = Miktar * (1 + FireYuzde/100)` hesaplar. - SQL: cursor tabanlı rapor; `ReceteAna` (cursor kaynagi) + `ReceteDetay RD LEFT JOIN TempMikroStok TmpST` (cins bilgisi); sonuc `ReceteRaporuModel`. - Form: `FrmReceteED { IdGuid = RcAId }` — cift tikta recete duzenleme. - API: - **Istasyon sirasiyla iliskisi:** - (recete tanim/maliyet raporu) **Notlar:** `myView1.MyKurusHane = 4` (miktar 4 ondalik gosterilir). Filtre verildiginde `where 1 = 1` eklenir. `myGrid1` kayit adi "ReceteRaporu1". Cursor + temp table kullandigindan buyuk recete sayisinda yavaslayabilir.

Uretim Stok Tuketim Raporu (`MyUI/Raporlar/UretimRaporlari/FrmUretimStokTuketimRaporu.cs`)

Ne ise yarar: Uretimde tuketilen/giren stoklari TOPLU (stok bazli, miktarlari SUM'lanmis) raporu. Her stok icin uretim/fire/iptal miktarlari ve recete carpanıyla hesaplanan stok miktarlari; Mikro stok kategori/kalite

kontrol/reyon bilgileriyle zenginlestirilmis. Tarih-saat araligi ve cok secimli kategori/kalitekontrol/reyon filtreleri + serbest metin arama. **Once ne olmalı (onkosul):** Saha (TabletV2) uretim akisinda mamul/fire/stok hareketleri girilmis olmalı. ONEMLI: form acilista `TempGuncelle()` ile Mikro stok ve kategori temp tablolarini ve `IstasyonTakipStokHareketDetay` merge tablosunu gunceller; bu yuzden Mikro DB (`Ortak.DbMikro`) erisilebilir olmalı. Mikro stok kategorileri (kategori/kalitekontrol/reyon) combo'lari bu temp tablodan beslenir. **Sonra ne olur:** Salt-okunur rapor (uretim akisini degistirmez). Acilis sirasinda yan etki olarak: `TempMikroStokKategori` , `TempMikroStok` , `TempSonGuncelleme` ve `IstasyonTakipStokHareketDetay` tablolari guncellenir/merge edilir (TempGuncelle). "Ara"da grid doldurulur, kayıt sayisi status bar'da (`lblKayitSayisi`) gosterilir. Gridde cift tik/odak event'leri devre disi (yorum satiri). **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — `Bagla()` ile toplu raporu calistirir (Designer'da `BtnAra.Click -> BtnAra_Click` , `TabIndex=0`). - `Temizle` (`BtnTemizle`) / `Yazdir` / `Dizayn` / `Kapat` — taban form standart butonlari. - Tarih1/2 (`TxtTarihi1` / `TxtTarihi2`), Saat1/2 (`TxtSaat1` = "00:00", `TxtSaat2` = "23:59") — tarih-saat araligi filtresi. - Kategori (`CmbKategori`), Kalite.Kont. (`CmbKaliteKontrol`), Reyon (`CmbReyon`) — `MyComboBoxCheck` cok secimli (; ayraç) filtreler. - `Ara` metin kutusu (`TxtAra`) — stok kodu/adi LIKE arama. **Cagirdigi katmanlar:** - Service: `IstasyonTakipStokHareketDetayService.GetListViewInKategoriToplu(sor, sor2)` — iki kollu UNION ALL sorgu: (1) `IstasyonTakipStokHareketDetay` ITSHD LEFT JOIN `TempMikroStok` stok bazli SUM; (2) `IstasyonTakipHareketDetay` ITHD `Turu='FireStokGiris'` fire stok girisleri. Kategori/kalitekontrol/reyon TMPS üzerinden filtrelendir. - Service: `IstasyonTakipStokHareketDetayService.DetaylarGuncelleToplu()` — acilista (TempGuncelle) `IstasyonTakipStokHareketDetay` tablosunu MERGE ile yeniden hesaplar (UrO.Sira=1, Turu UretimBitis/MamulGiris/FireMamulGiris; recete planlanan miktar / siparis miktar carpanıyla `StokMiktar/StokFireMiktar/StoklptalMiktar` uretir). - Service: `ITempMikroStokService.MikroStokKategoriGuncelle(mikroDb)` — Mikro STOK_KATEGORILERI/KALITE_KONTROL/REYONLARI -> `TempMikroStokKategori` merge. - Service: `ITempMikroStokService.MikroStokGuncelle(mikroDb)` — Mikro STOKLAR (+kategori/kalite/reyon/birim) -> `TempMikroStok` merge (3 dakikada bir; TempSonGuncelleme ile kontrol). - Service: `ITempMikroStokService.GetStokKategoriListStokKategori() / GetStokKategoriListKaliteKontrol() / GetStokReyonList()` — combo'lari `TempMikroStokKategori` 'den (Turu'ye gore) doldurur. - SQL/Prosedur: yukaridaki inline MERGE ve UNION ALL sorgulari (stored procedure degil, servis ici dinamik SQL). - API: - **Istasyon sirasiyla iliskisi:** Dolayli: `DetaylarGuncelleToplu` ve stok miktar carpani yalnızca operasyon `UrO.Sira=1` (ilk operasyon) icin hesaplanır — yani stok tüketimi ilk istasyon/operasyon hareketlerine baglanır. Raporun kendisinde istasyon kilimi yok (toplu). **Notlar:** `myView1.MyKurusHane = 2` . Filtre stringleri parametresiz interpolation (injection riski). Acilistaki TempGuncelle Mikro DB'ye baglanamazsa hata mesajı gosterir ve rapor verisi gelmez. `myGrid1` kayıt adi "UretimStokTuketimRaporuList1".

Uretim Stok Tuketim Raporu Detayli

(`MyUI/Raporlar/UretimRaporlari/FrmUretimStokTuketimRaporuDetayli.cs`)

Ne ise yarar: Yukaridaki raporun DETAYLI versiyonu: is emri (siparis) kodu, uretilen mamul stok kodu/adi ve tuketilen stok kilimiyla gosterir. Toplu rapordan farki, satirlarin is emri + mamul + stok bazinda gruplanmasidir (hangi is emrinde hangi stoktan ne kadar harcandi). **Once ne olmalı (onkosul):** Toplu rapor ile ayni: saha uretim hareketleri girilmis ve Mikro DB erisilebilir olmalı. Acilista `TempGuncelle()` ayni sekilde temp ve detay merge tablolarini gunceller. **Sonra ne olur:** Salt-okunur. Acilista ayni yan etkiler (TempMikroStok/Kategori + IstasyonTakipStokHareketDetay merge). "Ara"da grid doldurulur, kayıt sayisi `lblKayitSayisi` 'de. Cift tik/odak event'leri devre disi (yorum). **Butonlar & kisayollar:** - `Ara` (`BtnAra`) — `Bagla()` ile detayli raporu calistirir (Designer'da `BtnAra.Click -> BtnAra_Click` , `TabIndex=0`). - `Temizle` (`BtnTemizle`) / `Yazdir` / `Dizayn` / `Kapat` — taban form standart butonlari. - Tarih1/2, Saat1/2, Kategori (`CmbKategori`), Kalite.Kont. (`CmbKaliteKontrol`), Reyon (`CmbReyon`) cok secimli filtreler, `Ara` (`TxtAra`) serbest metin. - `Ara` metni —

ana kolda is emri kodu / uretilen stok kodu-adi / stok kodu-adi LIKE; fire (alt) kolda siparis kodu / recete kodu-adi / stok kodu-adi LIKE. **Cagirdigi katmanlar:** - Service:

`IIstasyonTakipStokHareketDetayService.GetListViewInKategoriDetayli(sor, sor2)` — UNION ALL: (1) `IstasyonTakipStokHareketDetay ITSHD` + `UretimEmri UR` + `TempMikroStok` (is emri/mamul/stok detayli, Tip='Fis'); (2) `IstasyonTakipHareketDetay ITHD` `Turu='FireStokGiris'` (Tip='Fire'). Is emri+mamul+stok+kategori bazinda GROUP BY. - Service:

`IIstasyonTakipStokHareketDetayService.DetaylarGuncelleToplu()` — acilista detay merge tablosunu yeniden hesaplar (toplu rapor ile ayni metot). - Service: `ITempMikroStokService.MikroStokKategoriGuncelle / MikroStokGuncelle(mikroDb)` — Mikro temp tablolari guncelle. - Service:

`ITempMikroStokService.GetStokKategoriListStokKategori / GetStokKategoriListKaliteKontrol / GetStokReyonList()` — combo doldurma. - SQL/Prosedur: servis ici inline UNION ALL + MERGE sorgulari (stored procedure degil). - API: - **Istasyon sirasiyla iliskisi:** Toplu rapor ile ayni — stok tuketim carpani

`Ur0.Sira=1` operasyonuna bagli hesaplanır; raporda is emri/mamul kirilimi var ama istasyon-sira kirilimi yok.

Notlar: `myView1.MyKurusHane = 2` . `myGrid1` kayit adi "UretimStokTuketimRaporuListDetayli1". Toplu rapordan tek farki cagirilan servis metodu (`GetListViewInKategoriDetayli`) ve arama kapsamı; filtre/temp guncelleme mantigi aynidir.

Modul: HizliUretimModule

HizliUretimModule, UretimV4 (CepPatronERP) masaustu uygulamasinda tek bir ekrandan "tek tikla" uretim girisi yapmak icin kullanilir. Klasorde tek form vardir: `FrmHizliUretimEG`. Normal uretim akisi (Siparis -> Uretim Emri -> Istasyon/Operasyon hareketleri -> Mikro'ya kaydetme, bkz. SiparisModule / UretimModule / MikroModul) yerine bu ekran, secilen bir recetenin tek seferde hem urun girisini hem de recete detaylarindaki (hammadde/sarf) stoklarin cikisini Mikro ERP'ye yazar. Yani is emri/istasyon zinciri olmadan, recete + miktar bilgisiyle dogrudan Mikro `STOK_HAREKETLERI` 'ne bir fis uretir. Form `My.Kontrol.Formlar` kutuphanesindeki `MyFrmKayit` base formundan turer; alt buton seridi (Kaydet/Yeni/Duzenle/Sil/Yazdir/Kapat + Ilk/Onceki/Sonraki/Son gezinme butonlari) base'den gelir, ancak bu ekranda fiilen yalnızca `BtnKaydet` ve `BtnKapat` islevseldir (`BtnSil.Visible=false`). Veri iki ayri DatabaseFactory ile islenir: `Ortak.DbPro` (UretimV3_FEZA uretim DB — recete okuma) ve `Ortak.DbMikro` (Mikro ERP — depo listesi + fis yazma). Fis turu ve depo/seri/gider/proje gibi tum parametreler kullanıcı tanımlı `MikroEntegre` ayarlarından (`Ortak.MikroEntAyarlar` , Grup = `HizliUretimFisi`) okunur.

Hizli Uretim Kayit (`FrmHizliUretimEG.cs` / `.Designer.cs`)

Ne ise yarar: Bir recete (UretimV4 `ReceteAna`) ve miktar secilerek tek seferde Mikro ERP'ye uretim fisi yazar: secilen recete kodunun mamul `giris` hareketi + recetenin tum detaylarindaki (`ReceteDetay.VarsayilanStokKodu`) hammadde/sarf stoklarin `cikis` hareketi olusturulur. Cikis miktarlari recete detay miktarı x girilen mamul miktarı olarak hesaplanır; her detayda `FireYuzde` varsa fire payi eklenir. Istasyon/operasyon adimlari, is emri ve siparis baglantisi yoktur — adi ustunde "hizli" tek-ekran giristir. **Once ne olmalı (onkosul):** - Mikro entegrasyon ayarlari tanımlı olmalı (`FrmMikroEntAyarlari` ekranında Modul= `MikroEntegre` , Grup= `HizliUretimFisi` için `EvrakSeri` , `GirisDepoKodu` , `CikisDepoKodu` , `GirisGiderKodu` , `CikisGiderKodu` vb. ve Grup= `FisTuru` için `HizliUretimFisiTuru` = `StokVirmanFisi` veya `UretimHareketFisi` ; ayrıca Grup= `GENEL` için `FirmaNo` / `KullaniciKodu`). Bunlar `Ortak.MikroEntAyarlar` cache'inden okunur. - Mikro DB (`Ortak.DbMikro`) ve uretim DB (`Ortak.DbPro`) baglantilari hazir olmalı. - Stok kodu alanından bir recete secilmis olmalı (`rcaId` dolu olmalı); aksi halde Kaydet'te recete bulunamaz. - Form acilista, ayarlarda tanımlı `GirisDepoKodu` 'na karsilik gelen depo Giris Depo alanina otomatik secilir; depo listesi Mikro `DEPOLAR` tablosundan yuklenir. - `Ortak.PIKapat` ayarina gore PartiNo/LotNo alanlari gizlenir/gosterilir (PIKapat aktifse parti/lot gizlenir). **Sonra ne olur:** - Kaydet'te recete `Ortak.DbPro` (UretimV3_FEZA) `ReceteAna` / `ReceteDetay` tablolarından okunur (yazma yapılmaz; bu DB salt okunur kullanilir). - Olusturulan `StokHareketleriModel` listesi secili fis turune gore `MikroStokHareketleri` kayitlarına cevrilir ve Mikro ERP `STOK_HAREKETLERI` tablosuna yazilir (parti/lot takipli stoklarda `PARTILOT` , renk/beden takipli stoklarda `BEDEN_HAREKETLERI` kayitlari da otomatik eklenir). Tum yazma tek transaction icinde yapilir. - `HizliUretimFisiTuru = StokVirmanFisi` ise virman fisi (sth_cins=3, sth_evraktip=6), `UretimHareketFisi` ise uretim hareket fisi (sth_cins=7, sth_evraktip=7) olarak yazilir; tanimsiz/baska deger gelirse varsayilan olarak StokVirmanFisi kullanilir. - Basarili kayitta "KayıtEdildi" bilgi mesaji gosterilir ve `BtnKaydet` pasiflestirilir (ayni fisin tekrar yazilmasini engellemek icin). Hata olursa `MesajHata` ile uyarı verilir, kayıt yapılmaz. - Not: Kayit sonrasi UretimV4 tarafında hiçbir tabloya yazma/guncelleme yoktur (kodda siparis/evrak guncelleme bloklari yorum satiri olarak birakilmis). **Butonlar & kisayollar:** - `Kaydet` (BtnKaydet) — `Kaydet()` metodunu cagirir; fisi olusturup Mikro'ya yazar. (base'de Enter = Kaydet davranisi gelir.) - `Kapat` (BtnKapat) — formu kapatir. (base'de Esc = Kapat.) - `Sil` (BtnSil) — gizli (`Visible=false`), bu ekranda kullanilmaz. - `Yeni` / `Duzenle` / `Yazdir` ve gezinme butonlari (Ilk/Onceki/Sonraki/Son) — base `MyFrmKayit` 'ten gelir; bu ekranda anlamlı bir islevi yoktur (kayıt gezme/listeleme mantigi bu form icin

kurulmamis). - **StokKodu** alani button-click (TxtStokKodu butonu) — **FrmReceteListesi** 'ni secim modunda (Maximized) acar; recete secilince StokKodu/StokAdi/Birim doldurulur ve **rcaId** set edilir. - **StokAdi** alani button-click (TxtStokAdi butonu) — yukaridakiyle ayni davranis (recete secim penceresini acar). - Alanlar: **StokKodu**, **StokAdi**, **Birim** (TxtBirim1, secilen recetenin **EntegreBirim** degeri), **Miktar** (TxtMiktar, >0 zorunlu), **Giris Depo** (TxtDepoNoGiris lookup), **PartiNo** (TxtPartiNo, PKapat'a gore gizli), **Lot** (TxtLotNo, PKapat'a gore gizli). **Cagirdigi katmanlar:** - Manager: **ReceteManager.GetReceteKayit(Guid? rcaId)** (**Ortak.DbPro**) — secili recetenin ana + detay (+ stok/istasyon/renk-beden) kayitlarini **ReceteKayitModel** olarak getirir; Kaydet, bunun **ReceteDetaylar** listesini cikis hareketleri icin kullanir. - Manager: **MikroConvertManager.SetHizliUretimFisiUrunGirisAyar(sth, ayarlar, depoGir)** — mamul giris hareketine ayarlardan giris depo (ekrandan secilen depo override eder), evrak seri, gider/proje/sorumluluk merkezi alanlarini set eder (**GirisDepoKodu** / **GirisGiderKodu**). - Manager: **MikroConvertManager.SetHizliUretimFisiStokCikisAyar(sth2, ayarlar)** — her recete detayindan olusan cikis hareketine ayarlardan cikis depo/gider (**CikisDepoKodu** / **CikisGiderKodu**) ve seri bilgisini set eder. - Manager: **MikroConvertManager.ConvertStokVirmanFisi(lisStokFisi, ayarlar)** — StokHareketleriModel listesini virman fisine (sth_cins=3, sth_evraktip=6) ceviriir; **GetStokVirmanEvrakSira(seri)** ile siradaki evrak no'yu hesaplar, firma/kullanici kodunu ayarlardan alir, depo 0 ise 1'e ceviriir. - Manager: **MikroConvertManager.ConvertUretimHareketFisi(lisStokFisi, ayarlar)** — alternatif fis turu; uretim hareket fisine (sth_cins=7, sth_evraktip=7) ceviriir; **GetUretimHareketFisiEvrakSira(seri)** ile evrak sirasi hesaplar. - Manager: **MikroKayitManager.StokHareketKaydet(lisMikro)** — Mikro **STOK_HAREKETLERI** 'ne yazar (transaction). Stok kartinin **sto_detay_takip** / **sto_renkDetayli** / **sto_bedenli_takip** degerlerine bakarak gerektiginde **PARTILOT** (parti/lot) ve **BEDEN_HAREKETLERI** (renk/beden) kayitlarini da uretir; parti no daha once girilmisse hata doner. - Helper: **MikroKayitFisTurleri.GetHizliUretimFisiTuru(ayarDegeri)** — **FisTuru/HizliUretimFisiTuru** ayar metnini **MikroFisGirisTurleri** enum'una ceviriir (yalnizca **StokVirmanFisi** veya **UretimHareketFisi**). - Service: **IMikroGenelService.GetMikroDepoListesi(" order by dep_no")** (**Ortak.DbMikro.GenelServis**) — Giris Depo lookup'i icin Mikro **DEPOLAR** tablosundan (**dep_no** , **dep_adi**) depo listesini ceker. - SQL/Prosedur: Dogrudan sakli prosedur cagrilmaz. Kullanilan sorgular ham SQL'dir: recete okuma **ReceteAna** / **ReceteDetay** üzerinden (QueryBuilder SelectFirst/SelectList); evrak sıra hesaplama **SELECT ISNULL(MAX(sth_evrakno_sira),0) ... FROM STOK_HAREKETLERI** ; stok takip tipi **SELECT sto_detay_takip, sto_renkDetayli, sto_bedenli_takip ... FROM STOKLAR** ; yazma **STOK_HAREKETLERI** / **PARTILOT** / **BEDEN_HAREKETLERI** insert. - API: - **Istasyon sirasiyla iliskisi:** Yoktur. Bu ekran istasyon/operasyon zincirini (UretimIstasyon, IstasyonHareketler, ReceteyeBagliIstasyon) tamamen atlar; secilen recetenin **GetReceteKayit** ile gelen **ReceteyeBagliIstasyonlar** koleksiyonu okunsa da Kaydet mantiginda kullanilmaz. Yalnizca recetenin malzeme listesi (**ReceteDetaylar**) reçeteden uretim sarfini hesaplamak icin kullanilir. Yani "hizli uretim" = istasyon bazli takip yapmadan, recete + miktardan dogrudan stok giris/cikis fisi. **Notlar:** - Form ana menuden **FrmAna.BarBtnHizliUretim_ItemClick** ile modal (**ShowDialog**) acilir; parametre/onkosul gerektirmez. - **Miktar <= 0** ise kayit engellenir ("Lütfen Miktar Giriniz"). - Cikis miktarı hesabı: **gec_miktar = ReceteDetay.Miktar + (Miktar * FireYuzde/100)** , sonra **sth_miktar = gec_miktar * mamulMiktari** . (Fire yuzdesi recete detay birim miktarina uygulanir, sonra mamul miktarıyla carpilir.) - Tum hareketlerde **sth_special2 = "HZL"** (hizli uretim isareti); **StokHareketKaydet** ayrica **sth_special3 = "AKT"** (aktarildi) damgasini ekler. Giris hareketi **sth_tip=0** , cikis hareketleri **sth_tip=1** . - PartiNo/Lot yalnizca mamul giris hareketine (**sth_parti_kodu** , **sth_lot_no**) yazilir; recete detay (cikis) hareketlerinde parti/lot bos birakilir. Parti/lot takipli stokta **StokHareketKaydet** lot no bos ise 1'e tamamlar ve mukerrer parti kontrolu yapar. - **_mikroStokService** ve **_mngMikroKayit** alanlari ile renk/beden bagla metotlari (RenkBagla/BedenBagla) tanimli ama tamamen yorum satirinda; ekranda renk/beden secimi suanda devre disi (renk/beden hareketi yine de stok kartinin takip tipine gore StokHareketKaydet icinde otomatik islenir — fakat bu ekrandan Renk/Beden

degeri gonderilmedigi icin pratikte renk/beden hareketi olusmaz). - Belge no bos (`sth_belge_no = ""`) ve tarih alanlari `DateTime.Now` olarak set edilir; tutar/fiyat bilgisi gonderilmez ($sth_tutar = Fiyat(0) * miktar = 0$).

Modul: Ana Kabuk, Login, Yetki ve DB Yonetimi

UretimV4 (CepPatronERP) iki ayrı çalıştırılabilir uygulamadan oluşur: (1) ana WinForms ERP uygulaması **MyUI** (**CepPatronERP.exe**) ve (2) yardımcı DB/yetki yönetim aracı **My.DataBaseSettings** (**DatabaseKontrol.exe**). Bu bölüm, bu iki uygulamanın kabuk (shell), giriş (login), kullanıcı/yetki yönetimi ve veritabanı bağlantı/sema yönetimi ekranlarını belgelemektedir. Ana uygulamada **FrmAna** ribbon + MDI ana penceresidir; açılırken önce otomatik güncelleme kontrolü (**UpdateProgram** -> FTP **update.ceppatron.com**), ardından **FrmLogin** ile kullanıcı doğrulaması, sonra donanım tabanlı sistem lisans kontrolü (**OrtakLis.SistemLisansKontrolu** -> **lisans.lic** dosyası + işlemci ID) yapılır; başarılıysa masaüstü açılır, **Ortak.DbPro** / **Ortak.DbMikro** bağlantı fabrikaları kurulur, genel/istasyon/mikro ayarlar belgeye bağlanır ve Mikro stok temp tabloları güncellenir. Tüm bağlantı ayarları INI dosyalarında AES anahtarıyla ("ÖzelAnahtar1234%&", **Ortak.GetKey()**) şifreli saklanır; uygulama verileri **UretimV3_FEZA** (programconnection / DbPro), Mikro ERP verileri **MikroDB_V16_FEZA24** (mikroconnection / DbMikro) veritabanındadır. Yardımcı **My.DataBaseSettings** uygulaması ise tarih-tabanlı günlük parola (**FrmLogin** "GSYA_2") ile korunur ve DB bağlantı stringi tanımlama, sema oluşturma/güncelleme, kullanıcı CRUD ve modul bazlı yetki matrisi yönetimini sağlar.

FrmAna - Ana Kabuk (**MyUI\FrmAna.cs** + **FrmAna.Designer.cs**)

Ne ise yarar: Ana ERP uygulamasının ribbon menülü MDI ana penceresidir. Tüm modul ekranlarını (sipariş, reçete, üretim, istasyon, mikro, raporlar, ayarlar, kullanıcı/personel vb.) ribbon bar butonları üzerinden açan kabuk formudur. Alt durum çubuğunda (status bar) kullanıcı adı, lisans durumu, bağlı firma DB adı ve versiyon bilgisini gösterir. **MyFrmAna** (My.Kontrol kutuphanesindeki base ribbon form) türüdür. **Once ne olmalı (onkosul):** Uygulama exe'si çalıştırılır; bağlantı ayar INI dosyaları (**DbPro** / **DbMikro**) önceden tanımlanmış olmalı (yoksa **My.DataBaseSettings** aracıyla girilir). Form yüklenirken sırayla: güncelleme kontrolü -> login -> lisans kontrolü yapılır. **Sonra ne olur:** - **FrmAna_Load** önce **UpdateKontrol()** çağırır; güncelleme indirildiyse **return** ile uygulamadan çıkar (yeni sürüm kurulacaktır). - Ardından **FrmLogin** (modal) açılır. Giriş başarısızsa **this.Close()** ile uygulama kapanır. - Giriş başarılıysa: skin yüklenir (**SkinAdd**), masaüstü MDI child açılır (**MasaUstuAc** -> **FrmMasaustu1**), opacity fade-in timer başlar, alt barda kullanıcı adı yazılır. - **OrtakLis.SistemLisansKontrolu()** ile lisans doğrulanır: aktifse **Ortak.LisansAktif=true** , alt bar "Lisans = Aktif", Reçete Listesi / Reçete Grup Takimler / Kullanıcılar butonları **Enabled=true** ; pasifse bu butonlar devre dışı bırakılır ("Lisans = Pasif"). - **Ortak.DbPro** ve **Ortak.DbMikro** bağlantı fabrikaları yeniden oluşturulur, pencere maximize edilir, alt barda firma DB adı gösterilir. - **Ortak.MikroEntAyarlarBagla()** , **GenelAyarlarBagla()** , **IstasyonAyarlarBagla()** ile ayar tabloları belgeye yüklenir; versiyon yazılır. - **TempGuncelle()** çağırılır: Mikro stok kategori + stok temp tabloları (**MikroStokKategoriGuncelle** , **MikroStokGuncelle**) Mikro DB'den senkronlanır. - Form kapatılırken (**FrmAnaV2_FormClosing**) seçili skin/palet INI'ye yazılır ve "Programı Kapatmak İstiyormusunuz" onayı sorulur. **Butonlar & kısayollar:** (Ribbon bar butonları; her biri ilgili ekranı açar) - Modul ekranlarını açan generic buton handler **BarBtnlar_Click** — buton **Hint** değerini tag olarak alıp **FormAc.FormSec(tag)** çağırır (Siparişler, ReçeteÜretimler, MikroStokListesi, ÜretimEmirleri, OperasyonTakip, İstasyonTakip, İstasyonRaporu vb.). - **BarBtnKullanıcılar** — **FrmKullanıcıKayıt** (kullanıcı CRUD) açar (lisans aktifse). - **BarBtnPersonelListesi** — **FrmPersonelKartlari** açar. - **BtnLisansKayıt** — **Ortak.LisansKayıt()** -> **FrmLisansGiris** (lisans anahtarı girişi). - **BarBtnMailAyarlari** — **FrmMailSettings** ; **BarBtnSmsAyarlari** — **FrmSmsAyarlari** ; **BarBtnSmsRapor** — **FrmSmsRapor** . - **BarBtnMikroEntAyarlari** — **FrmMikroEntAyarlari** ; **BarBtnGenelAyarlar** — **FrmGenelAyarlar** ; **BarBtnIstasyonUretimAyarlari** — **FrmIstasyonUretimAyarlari** . -

`BarBtnIstasyonBaslatmaHatalari` / `BarBtnIstasyonDurdurmaKodlari` / `BarBtnIstasyonFireSebepleri` — `FrmIstasyonAciklamalari` (modul turune gore). - `BarBtnReceteAciklama` — `FrmAciklamaKodlar` (ReceteAciklama). - `BarBtnReceteIstasyonGruplar` / `BarBtnReceteIstasyonGrupOperasyonlar` — recete-istasyon grup tanim ekranlari. - `BarBtnDbGuncelle` — `Ortak.DatabaseGuncelleUretim()` (sema guncelleme). - `BarBtnIstasyonHareketLog`, `BarBtnIstasyonBakimList`, `BarBtnMalKabulListe`, `BarBtnOlcuKontrol`, `BarBtnHizliUretim`, `BarBtnStokTuketimRaporu(Detayli)`, `BarBtnReceteKullanilanStok`, `BarBtnReceteGenelRaporu` — ilgili liste/rapor ekranlari (MDI child olarak `.Show()`). - `btnStokGuncelle` — `FrmStokKodGuncelle` acar. - `BarBtnAcilMesaj` — `FrmMesajGenel` (acil mesaj). - Tum Sekmeleri Kapat (`BarUstTumSekmeleriKapat` / sag tik menu) — Masaustu disindaki tum MDI child'lari onayla kapatir. - Hizli Baslangica Ekle (ribbon sag tik) — aktif ribbon butonunu masaustune kisayol olarak ekler (`frmmasaUstu.ButonEkle`). - Ribbon QAT'a ekleme menuyu gizlenir, yerine "Hizli Baslangica Ekle" ve "Tum Sekmeleri Kapat" eklenir (`RibbonControl1_ShowCustomizationMenu`). **Cagirdigi katmanlar:** - Manager/Service: `OrtakLis.SistemLisansKontrolu()` — `lisans.lic` dosyasini islemci ID hash'i (`GetBaseCpuInfo` /MD5 tabanlı `Kisa()`) ile karsilastirir; donanim kilitli lisans dogrulaması. - Manager/Service: `UpdateProgram.VersiyonKontrol()` — FTP (`update.ceppatron.com/update_uretimv1/`) üzerinden uzak `versiyon.txt` ile yerel surumu kiyaslar; yeni surum varsa `download.zip` indirir (`FtpManager`). - Manager/Service: `Ortak.DbPro` (`DatabaseFactoryPro`) / `Ortak.DbMikro` (`DatabaseFactoryMikro`) — Unity DI container'la tum servisleri (Ayarlar, Siparis, Recete, Uretim, Istasyon, Kullanicilar, TempMikroStok vb.) cozer. - Manager/Service: `Ortak.MikroEntAyarlarBagla/GenelAyarlarBagla/IstasyonAyarlarBagla` — `IAyarService.SelectListWhere(" where Modul='...' ")` ile ayar tablolarini yukler; `PlKapat`, `MalKabulKullan` gibi global bayraklari set eder. - Manager/Service: `ITempMikroStokService.MikroStokKategoriGuncelle` / `MikroStokGuncelle` — Mikro DB'den stok/kategori temp senkron (MERGE). - Manager/Service: `FormAc.FormSec(tag)` — string tag'e gore ilgili modul formunu olusturup MDI child olarak acan dispatcher (switch-case). - SQL/Prosedur: dogrudan prosedur cagrilmaz; temp guncelleme SQL'i Mikro fonksiyonlari `fn_StokCins` kullanir, STOKLAR/STOK_KATEGORILERI/STOK_REYONLARI/STOK_KALITE_KONTROL_TANIMLARI tablolarini NOLOCK ile okur. - API: yok (FTP guncelleme haricinde dogrudan DB). **Istasyon sirasiyla iliskisi:** - (Kabuk/menu formu; uretim akisi veya istasyon sirasiyla dogrudan iliskisi yoktur, sadece ilgili ekranlari acar.) **Notlar:** - Lisans pasifken yalnızca Recete Listesi/Grup ve Kullanicilar butonlari kapatilir; diger moduller acik kalir. - Skin/palet secimleri INI `[Theme]` bolumunde `ApplicationSkinName` / `ApplicationSkinPaletteName` olarak saklanir. - `AktifButon` alanı, ribbon highlight degisiminde guncellenir ve "Hizli Baslangica Ekle" islevinde kullanilir. - Versiyon `versiyon.txt` dosyasindan okunur; FTP guncelleme bu dosyayi karsilastirir.

MyFrmLoginPaneli - Login Panel Taban (`MyUI\AyarVeGenelModul\MyFrmLoginPaneli.cs`)

Ne ise yarar: Ana uygulamanin login ekranı (`MyUI.FrmLogin`) icin gorsel/davranis taban sinifidir (base form). Kenarlıksız pencerenin baslik/etiket alanlarından fareyle suruklenip tasinmasini, ESC ile kapatma / F2 ile giris kisayollarini ve ortak mesaj kutusu yardimcilarini (`MesajHata`, `MesajBilgi`, `MesajSor`, `GetKontrol`) saglar. Kendi basina veri islemi yapmaz; gorsel cerceve ve etkilesim altyapisidir. **Once ne olmalı (onkosul):** Dogrudan acilmaz; `MyUI.FrmLogin` bu siniftan tureyerek kullanir. `FormOlusturuldu` bayragi form `Shown` olunca true olur. **Sonra ne olur:** Kullanici basligi suruklerse pencere konumu degisir; BtnKapat veya ESC ile form kapanir; F2 ile turetilen formdaki `BtnGiris` tetiklenir. **Butonlar & kisayollar:** - `BtnKapat` / `Esc` — formu kapatir (`BtnKapat_Click` -> `this.Close()`). - `F2` — `BtnGiris.PerformClick()` (turetilen FrmLogin'deki giris butonunu tetikler). - Baslik/etiket/panel alanlari fare suruklemesi — pencere tasima (`Lbl_Baslik_MouseDown/Move/Up`). **Cagirdigi katmanlar:** - Manager/Service: yok (saf UI taban sinifi). - SQL/Prosedur: yok. - API: yok. **Istasyon sirasiyla iliskisi:** - **Notlar:** - `MesajHata/MesajBilgi/MesajSor` static

yardimcileri ve `GetKontrol(Action)` try/catch sarmalayicisi tum login akisinda kullanilir. - Asagidaki `MyUI.FrmLogin` bu sinifin turevidir (gercek dogrulama mantigi orada).

FrmLogin (MyUI) - Uygulama Girisi (`MyUI\AyarVeGenelModul\FrmLogin.cs` + `FrmLogin.Designer.cs`)

Ne ise yarar: Ana ERP uygulamasinin gercek kullanıcı giriş ekranidir (`MyFrmLoginPaneli` turevi). Kullanıcı adı/sifre doğrulaması yapar ve basarılıysa global `Ortak.DbPro` / `Ortak.DbMikro` bağlantı fabrikalarını kurar. `FrmAna_Load` içinde modal acilir. **Once ne olmalı (onkosul):** Bağlantı ayar INI dosyaları tanımlı olmalı (form load'da `DatabaseFactoryPro/Mikro` kurulur; ayar yoksa hata `GetKontrol` ile yakalanır). Son kullanılan kullanıcı adı INI `[AYAR] KULLANICI` 'dan okunup doldurulur. **Sonra ne olur:** - Giriş basarılıysa `GirisYapildi=true`, kullanıcı adı INI'ye yazilir, `Ortak.KullaniciAdi` set edilir, form kapanır; kontrol `FrmAna_Load` 'a doner (lisans kontrolu + masaustu acma). - Basarisizsa "Kullanıcı Adı Veya Şifre Geçersiz" uyarisi gosterilir, form acik kalir. - Özel durum: `Kullanici` tablosu bossa otomatik `Admin` / sifre "1" (AES sifreli), `Admin=true` ilk kullanıcı INSERT edilir (`Kontrol()` içinde). **Butonlar & kisayollar:** - `BtnGiris` / `F2` — `BtnGiris_Click` -> `Kontrol()` ile dogrulama. - `BtnKapat` / `LblClose` / `Esc` — formu kapatir (giris yapilmadan kapanirsa uygulama sonlanir). - `TxtKullanici` Enter — fokusu sifre alanina tasir; `TxtSifre` Enter — `Kontrol()` calistirir. - `BtnDbPro` / `BtnDbMikro` — DB ayar butonlari (tasarimda gizli, `Visible=false`; icerik comment-out edilmis, kullanilmiyor). **Cagirdigi katmanlar:** - Manager/Service: `Ortak.DbPro.Kullanicilar` (`IKullaniciService`) — `Query<int>("select count(*) ...")` (kullanici sayisi), `Insert(...)` (ilk admin), `SelectFirst(k => k.KullaniciAdi == ka)` (dogrulama). - Manager/Service: `DatabaseFactoryPro` / `DatabaseFactoryMikro` (form load'da kurulur). - Yardimci: `Ortak.GetKey()` AES anahtari; `string.Sifrele(key)` ile parola sifreleme (DB'deki sifre ile karsilastirma). - SQL/Prosedur: dogrudan inline `select count(*) as Adet from Kullanici` + servis tabanlı CRUD; stored procedure yok. - API: yok. **Istasyon sirasiyla iliskisi:** - **Notlar:** - Kullanıcı adı ve sifre `ToUpper()` ile karsilastirilir -> giris buyuk/kucuk harf duyersiz. - Sifreler DB'de duz metin degil AES sifreli ("OzelAnahtar1234%&") saklanir. - Bu form modul bazli yetki (Kullanicilzinler) kontrolu YAPMAZ; sadece kullanıcı/sifre + `Admin` bayragi anlamlidir. Modul bazli yetki matrisi yardimci `My.DataBaseSettings` araciyla tanimlanir (asagiya bkz). - Tasarimda `BtnDbPro` / `BtnDbMikro` butonlarnin DB ayar acma kodu yorum satirina alinmis (login ekranindan DB ayari degistirilemez).

FrmLogin (My.DataBaseSettings) - Yardimci Arac Girisi (`My.DataBaseSettings\FrmLogin.cs` + `Designer`)

Ne ise yarar: Yardimci `DatabaseKontrol.exe` (DB/yetki yonetim araci) ve DB paneli (`FrmDataPanel`) acilirken gosterilen koruma ekranidir. Veritabani sema/baglanti islemleri gibi hassas islemlere yetkisiz erisimi engellemek icin tarih tabanlı (gunluk degisen) bir parola sorar. **Once ne olmalı (onkosul):** `FrmDataPanel.Frm_Load` (veya araci dogrudan calistirma) bu formu modal acar. Kullanıcının gunun gecerli parolasini bilmesi gerekir. **Sonra ne olur:** - Dogru parola girilirse `GirisYapildi=true`, form kapanır; cagiran `FrmDataPanel` `Ortak.GetSettings()` ile DB ayarlarini yukler (panel acilir). - Yanlissa "Şifre Geçersiz" uyarisi; tekrar denenebilir. - Cagiran panelde giris yapilmazsa `Application.Exit()` ile arac kapanır. **Butonlar & kisayollar:** - `button1` ("Giris") — `Kontrol()` calistirir; `textBox1` Enter ile de tetiklenir. - `button2` ("Kapat") — formu kapatir (giris yapilmamis sayilir). **Cagirdigi katmanlar:** - Manager/Service: yok (parola tamamen istemci tarafinda, koddaki algoritma ile uretilir). - SQL/Prosedur: yok. - API: yok. **Istasyon sirasiyla iliskisi:** - **Notlar:** - Parola algoritmasi (`// GSYA_2`): `gun(2hane) + saat(2hane) + (yil-2000)(2hane) + ay(2hane)` formatinda gunluk/saatlik degisen bir kod. Yani parola servis/personel tarafından bilinen ve gun-saate gore degisen statik bir formül; veritabaninda

saklanmaz. - Bu form ana uygulamadaki `FrmLogin` 'den tamamen ayridir (farkli namespace `My.DatabaseSettings` , farkli mantik); karistirmamak gerekir.

FrmKullaniciAyar - Kullanici Ayarlari/CRUD (`My.DataBaseSettings\FrmKullaniciAyar.cs` + Designer)

Ne ise yarar: Yardimci aracta uygulamaya giris yapacak kullanicilari (Kullanici tablosu) listeleyen ve ekleme/duzenleme/silme yapan CRUD ekranidir. Her kullanıcı için modul bazli yetkileri tanımlamak üzere `FrmKullaniciYetkiler` ekranina kapi acar. (Ana uygulamadaki `FrmKullaniciKayit` ile ayni tabloyu yönetir; bu, yönetici araci muadilidir.) **Once ne olmalı (onkosul):** `FrmDataPanel` -> "Kullanıcı Ayar Ve Yetkiler" butonundan acilir; `Ortak.GetSettings()` ile `Ortak.Connection` (programconnection) kurulmus olmalı. **Sonra ne olur:** - Kaydet -> `Kullanici` tablosuna IF EXISTS bazli UPDATE/INSERT; grid yenilenir (`Bagla`), alanlar temizlenir. - Sil -> önce `KullaniciIzinler` (yetki kayitlari), sonra `Kullanici` satiri DELETE edilir (cascade el ile); grid yenilenir. - Yetkiler -> secili kullanıcı için `FrmKullaniciYetkiler` modal acilir (modul bazli yetki matrisi). **Butonlar & kisayollar:** - `BtnKaydet` — `BtnKaydet_Click` : `RowaAktar()` ile form alanlarini `_mdl` 'e yazar, sifreyi `Sifrele(Ortak.ANA_KEY)` ile sifreler, UPDATE/INSERT calistirir. - `BtnYeni` — `BtnYeni_Click` : yeni bos `Kullanici` olusturur, alanlara aktarir. - `BtnDegistir` ("Duzenle") — `BtnDegistir_Click` / `Duzenle()` : grid'deki secili kaydi `_mdl` 'e klonlar, sifreyi `SifreCoz` ile cozup gosterir. - `BtnSil` — `BtnSil_Click` : onayla `KullaniciIzinler` + `Kullanici` siler. - `BtnYetkiler` — `BtnYetkiler_Click` : secili kullanıcı için `FrmKullaniciYetkiler` acar (Kullanici + KulIdSi gecirilir). - `BtnKapat` — formu kapatir. - Grid çift tik (`dataGridView1_DoubleClick`) — secili kaydi duzenlemeye alir (`Duzenle`). **Cagirdigi katmanlar:** - Manager/Service: `Ortak.Connection` (Dapper `IDbConnection`) üzerinden dogrudan inline SQL — `Query<Kullanici>("Select * from Kullanici")` , IF EXISTS UPDATE/INSERT, DELETE. - Yardimci: `Sifrele(Ortak.ANA_KEY)` / `SifreCoz(Ortak.ANA_KEY)` — parola sifreleme/cozme (ANA_KEY = "OzelAnahtar1234%&"). - Entity: `My.DatabaseSettings.Entites.Kullanici` (Id Guid PK, Adi, Soyadi, KullaniciAdi, Sifre, Admin) + `Clone()` . - SQL/Prosedur: dogrudan inline SQL (`Kullanici` , `KullaniciIzinler` tablolari); stored procedure yok. - API: yok. **Isstasyon sirasiyla iliskisi:** - **Notlar:** - Grid'de `Id` ve `Sifre` kolonlari gizlenir; tum kolonlar read-only (duzenleme form alanlarından yapilir). - Duzenlemede sifre cozulup gosterilir, kaydederken yeniden sifrelenir. - `ChcAdmin` admin (tam yetki) bayragini set eder; admin ise modul bazli yetki kontrolu pratikte atlanir. - Silme isleminde `KullaniciIzinler` önce silinir (yabanci anahtar/yetim kayit temizligi).

FrmKullaniciYetkiler - Modul Bazli Yetki Matrisi (`My.DataBaseSettings\FrmKullaniciYetkiler.cs` + Designer)

Ne ise yarar: Secili bir kullanıcı için modul/yetki bazli izinleri (acik/kapali) duzenleyen yetki matrisi ekranidir. `KullaniciYetkiler` (tum yetki tanımlari) ile `KullaniciIzinler` (kullaniciya verilmiş izinler) tablolarini birlestirerek bir grid'de gosterir; kullanıcı "Durum" onay kutularini isaretleyerek izin verir/alir. **Once ne olmalı (onkosul):** `FrmKullaniciAyar` -> "Yetkiler" butonundan, secili `Kullanici` ve `KulIdSi` (Guid) set edilerek acilir. Kullanici secilmemisse "Kullanıcı Seçilmemiş" uyarisi verip form kapanir. **Sonra ne olur:** - Kaydet -> grid'deki her satir için `KullaniciIzinler` tablosuna IF EXISTS UPDATE/INSERT (Durum, KulId, YetId) yapilir; grid yenilenir. - Sil -> secili kullanicinin ilgili izin kayitlari `KullaniciIzinler` 'den DELETE edilir. - Bu yetkiler, ilgili moduller calistirildiginda erisim kontrolu için okunur (modul bazli izin matrisi). **Butonlar & kisayollar:** - `BtnKaydet` — `BtnKaydet_Click` -> `Kaydet()` : `bs.EndEdit()` sonrasi listedeki tum satirlar için IF EXISTS UPDATE/INSERT (Durum). - `BtnSil` — `BtnSil_Click` -> `Sil()` : onayla izin kayitlarini siler. - `BtnKapat` — formu kapatir. - Grid "Durum" kolonu — tek duzenlenebilir kolon (onay kutusu); diger kolonlar (Modul, Yetki,

Aciklama) read-only. **Agirdigi katmanlar:** - Manager/Service: `Ortak.Connection` (Dapper) üzerinden inline SQL — `KullaniciYetkiler YT LEFT OUTER JOIN KullaniciIzinler IZ ON IZ.YetId=YT.Id AND IZ.KulId=@KulId` ile birlesik liste; IF EXISTS UPDATE/INSERT; DELETE. - Entity: `KullaniciYetkilerModel` (Durum bool, Modul, Yetki, Aciklama, YetId Guid, KulId Guid). - SQL/Prosedur: dogrudan inline SQL (`KullaniciYetkiler` , `KullaniciIzinler` tablolari); stored procedure yok. - API: yok. **Istasyon sirasiyla iliskisi:** - **Notlar:** - `Durum` COALESCE(IZ.Durum, 0) ile gelir: izin kaydi yoksa varsayilan 0 (kapali). - `KulId` / `YetId` kolonlari grid'de gizlenir; sadece `Durum` duzenlenebilir. - Bu matris (`KullaniciIzinler`), ana uygulamadaki `FrmLogin` tarafindan giris aninda OKUNMAZ; yetki kontrolu modulleri/ilgili formlari calistirirken yapilir (bu form sadece tanimlama yeridir). - Yetki tanimlari (`KullaniciYetkiler`) once seed/tanimli olmalı (aksi halde matris bos gelir).

FrmDataPanel (My.DataBaseSettings\FrmAna.cs) - DB Ayar Kabugu

(`My.DataBaseSettings\FrmAna.cs` + `FrmAna.Designer.cs`)

Ne ise yarar: Yardimci `DatabaseKontrol.exe` aracinin ana penceresidir (sinif adi `FrmDataPanel` , dosya `FrmAna.cs`). Veritabani baglanti ayarlari, sema olusturma/guncelleme/kontrol ve kullanıcı/yetki yönetimi işlemlerini gruplandırarak butonlu bir kabuk formudur. **Once ne olmalı (onkosul):** Arac çalıştırılır; `Frm_Load` önce `FrmLogin` (tarih tabanlı parola) açar. Giriş yapılmazsa `Application.Exit()` ; yapılırsa `Ortak.GetSettings()` ile programconnection bağlantısı kurulur. **Sonra ne olur:** Kullanıcı ilgili butona basarak: DB bağlantı stringi tanımlar, sema oluşturma/guncelleme/kontrol eder veya kullanıcı/yetki ayarlarını açar. İşlemler sonucunda `MessageBox` ile bilgilendirme verilir. **Butonlar & kisayollar:** - `BtnBaglantiPro` ("Program Bağlantı Ayar") — `FrmDataPaneli(DbProAdi)` açar (programconnection ayar formu). - `BtnBaglantiMikro` ("Mikro Bağlantı Ayar") — `FrmDataPaneli(DbMikroAdi)` açar (mikroconnection ayar formu). - `BtnDatabaseKontrol` ("Database Kontrol") — `Ortak.DatabaseKontrol()` : DB var mı kontrol eder; yoksa onayla Genel + Uretim + DepoKabul sema tablolarını oluşturma. - `BtnDatabaseOlusturUretim` ("Üretim Tablolarını Oluştur") — `Ortak.DatabaseGuncelleUretim()` : Genel + Uretim sema guncelleme. - `BtnDatabaseOlusturDepoKabul` ("Depo Kabul Tablolarını Oluştur") — `Ortak.DatabaseGuncelleDepoKabul()` : Genel + DepoKabul sema guncelleme. - `BtnKullaniciAyar` ("Kullanıcı Ayar Ve Yetkiler") — `FrmKullaniciAyar` açar. - `BtnKapat` ("Kapat") — formu kapatır. **Agirdigi katmanlar:** - Manager/Service: `Ortak.GetSettings()` — programconnection ayarını AES anahtarıyla cozer, `IDbConnection` kurar. - Manager/Service: `Ortak.DatabaseKontrol/DatabaseGuncelleUretim/DatabaseGuncelleDepoKabul` — `DatabaseCreate.GenelCreate` , `UretimCreate` , `DepoKabulCreate` siniflerinin `DatabaseKontrol/DatabaseOlustur/DatabaseGuncelle` metotlarını çağırır (sema DDL). - SQL/Prosedur: sema oluşturma/guncelleme SQL'leri `DatabaseCreate` modulundedir (CREATE/ALTER TABLE vb.); stored procedure çağırışı değil DDL. - API: yok. **Istasyon sirasiyla iliskisi:** - **Notlar:** - Bu form sema/baglanti yönetimini ana uygulamadan ayırarak tutar; üretim ortamında hassas bir araçtır (bu yüzden tarih tabanlı parola ile korunur). - `DatabaseKontrol` DB yoksa oluşturma; `DatabaseGuncelle*` ise mevcut DB'ye eksik tablo/kolon ekler. - Ana uygulamadaki `BarBtnDbGuncelle` da `Ortak.DatabaseGuncelleUretim()` çağırır (aynı sema guncelleme işlemine ikinci giriş noktası).

FrmDataPaneli + MyFrmDataPaneli - DB Bağlantı Ayar Paneli

(`My.DataBaseSettings\DatabasePanel\FrmDataPaneli.cs` ve `MyFrmDataPaneli.cs`)

Ne ise yarar: Tek bir veritabanı bağlantısının (programconnection veya mikroconnection) ayarlarını (Database adı, Server/IP, Port, kullanıcı adı, şifre) gösterip düzenleyen ve şifreli olarak INI'ye kaydeden ayar panelidir. `MyFrmDataPaneli` görsel/davranış tabanlı (pencere tasımı, kısayollar, ortak mesaj kutuları), `FrmDataPaneli` ise

asil okuma/yazma mantigidir. **Once ne olmalı (onkosul):** `FrmDataPanel` 'den "Program Bağlantı Ayar" veya "Mikro Bağlantı Ayar" butonuyla, hedef DB adi (`DbProAdi` / `DbMikroAdi`) constructor'a gecirilerek acilir. Yetkili kullanicinin (tarih parolasıyla giris yapmis) erismesi gerekir. **Sonra ne olur:** - Form yuklenince ayar klasorleri olusturulur (`KlasorleriOlustur`), mevcut baglanti ayarlari INI'den okunup (`DbConnectionSettings.GetSetting` , AES "OzelAnahtar1234%&") alanlara doldurulur (`IniOku`). - Kaydet -> alanlardan `DatabaseModel` olusturulur ve `DbConnectionSettings.SaveSetting` ile sifreli INI'ye yazilir; `AyarDegisti=true` set edilir (cagiran formun baglantiyi yeniden kurmasi icin sinyal). **Butonlar & kisayollar:** - `BtnKaydet` / `F2` — `BtnKaydet_Click` -> `SaveSettings()` (sifreli kaydet) + `AyarDegisti=true` . - `BtnKapat` / `Esc` — formu kapatir. - `btnDatabase` ("Database") — `BtnDatabase_Click` : alanlara ornek/varsayilan deger doldurur (Veritabani, ServerAdi, port 0, sa, bos sifre). - Baslik/panel fare suruklemesi — pencere tasima. **Cagirdigi katmanlar:** - Manager/Service: `DbConnectionSettings.GetSetting(DbAdi, AnaKey)` / `SaveSetting(md1, AnaKey)` — baglanti ayarini AES anahtariyla INI'den okur/INI'ye sifreli yazar. - Entity: `DatabaseModel` (Database, Server, UserName, Password, ConnectionString) — `DatabaseModel(DbAdi)` . - Yardimci: `IniDosyasi` (ayar + database INI dosyalari), `ProgramYolAyarlari` (yol sabitleri). - SQL/Prosedur: yok (sadece ayar/INI islemleri). - API: yok. **Istasyon sirasiyla iliskisi:** - **Notlar:** - AES anahtari bu sinifta ayrıca sabit tanimli: `const string AnaKey = "OzelAnahtar1234%&"` (genel `Ortak.GetKey()` ile ayni deger). - `AnaKey` / `AyarIni` / `DatabaseIni` static alanlardir; iki kez instance acilsa bile tek ayar dosyasi kullanilir. - `MyFrmDataPaneli` parametresiz ve `string dbAdi` parametrelili iki constructor sunar; parametrelili olan event bagli, asil kullanım budur. - Sifre alani panelde duz gorunur (UI'da maskeli olabilir, koddan duz okunur); INI'ye sifreli yazilir.